

```
#include <iostream>
#include <vector>
#include "gen/core.h"
#include "gen/app.h"
```

```
int main() {
    gen::App app;
    app.setTitle("My App");
    app.init();
    app.run();
    return 0;
}
```



Introduction to

GEN FRAMEWORK

CROSS-PLATFORM APPLICATION
DEVELOPMENT IN C/C++



CROSS-PLATFORM



C/C++



CODE FAST,
RUN FASTER...

FAST DEVELOPMENT • EASY TO USE • POWERFUL AND SCALABLE

Introduction to GEN Framework

Cross-Platform Application Development in C/C++

Contents

Chapter I: C/C++, cross-platform and sense of GEN Framework.....	14
1. The starting point	14
2. The origin of the C language.....	15
3. C and C++ are not the same language	15
4. The evolution of C++.....	16
5. C/C++ versus other languages	16
6. The real problem: the platform.....	17
7. What GEN Framework.....	18
8. Cross-platform development without giving up performance.....	18
9. The absence of a unique library in C++.....	19
10. Single application, multiple platforms.....	19
11. The importance of an architecture of its own.....	20
12. C/C++ as a strategic decision	21
13. What can be learned in this book.....	21
Chapter II: Preparing the Development Environment.....	22
1. A practical chapter.....	22
2. The official framework page.....	23
3. The Projects folder as a workspace	24
4. Recommended organization in Windows.....	25
5. Recommended organization in Linux	26
6. Download via Git	26
7. Tools required.....	28
8. Why we use these tools.....	28
9. Windows as a development environment	29
10. Linux as a development environment	30
11. WSL2 as mixed environment Windows/Linux.....	30
12. Docker and Docker Compose	34
13. Common scripts as input point.....	35
14. Convention of names in scripts	35
15. Script Modifiers	36
16. Stage modifiers.....	36
17. Windows Platform Modifiers	37

18. Platform modifiers in Linux	38
19. Mode modifiers	38
20. Compiler modifiers	39
21. Memory control modifiers	39
22. Trace modifiers	40
23. Feedback modifiers	41
24. DOCKERS modifier	41
25. Combination of modifiers	42
26. Summary table of modifiers	42
27. CMake as internal infrastructure	43
28. Visual Studio and Visual Studio Code	43
29. Own projects at the same level as GEN_FrameWork	44
30. Usual errors during preparation	45
31. How to deal with a compilation error	46
32. The environment as part of learning	47
33. Quick compilation reference	49
Chapter III: Overview and Framework Architecture	51
1. A global vision before entering into the detail	51
2. The modular essence of GEN Framework	51
3. The role of the factories	52
4. Plants and platform insulation	52
5. Confining the platform code	53
6. XFACTORY: eXtender Utilities	54
7. DIOFACTORY: Data Input/Output	54
8. GRPFACTORY: Graphic Factory	55
9. INPFACTORY: Input Factory	55
10. SNDFACTORY: Sound Factory	56
11. Additional groups and cross-cutting services	56
12. Compression	57
13. Cipher	57
14. AppFlow	57
15. Scripts	58
16. UserInterface	58
17. Databases	58

18. Relationship between modules	59
19. How the programmer should approach this architecture.....	59
20. Summary of architecture	59
Chapter IV: First application with GEN Framework	62
1. The objective of the chapter	62
2. The three main modalities of implementation.....	62
3. Why don't we write the main function	63
4. The life cycle of an application with GEN Framework.....	63
5. Base use without console or graphics AppFlow mode	64
6. Console application with AppFlow	64
7. Finite Status Machine with XFSMACHINE.....	65
8. Typical structure of a console application.....	66
9. XPATHMANAGER and folder assets.....	67
10. Main application configuration	68
11. Configuration variables mapping.....	68
12. Relationship between AppFlow, Configuration and Traces	70
13. CMakeLists.txt minimum for a console application	70
14. Graphic application with AppFlowGraphics	71
15. Add UserInterface to a graphical application.....	71
16. Typical structure of a graphical application.....	72
17. Recommended Skeleton of a First Application	73
18. Why this skeleton matters.....	73
Chapter V: XUtils, the extended utilities of the framework	75
1. The Origin of XUtils.....	75
2. XUses as common framework basis	75
3. XFACTORY and platform insulation	75
4. General Map of XUtils.....	76
5. Basic types, chains, buffers and routes	77
6. Data collections and structures	78
7. Time, system and platform-dependent objects	79
8. Files, directories and XPATHSMANAGER.....	80
9. File formats supported in XUtils	80
10. Configuration with XFILECFG	81
11. Events, observers and publication.....	82

12. Finite State Machine	83
13. Traces, logs, memory and feedback	83
14. Concurrency, threads and planning	84
15. Serialization, coding and binary structures	84
16. Internationalization, translation and texts	85
17. Licenses, firmware, storage and special resources	86
18. Auxiliary and integration modules	86
19. How to study XUtils from DevTestsConsole	87
20. Criteria for use in an application with GEN Framework	87
21. Practical summary	87
Chapter VI: DataIO, data input and output in GEN Framework.....	90
1. The origin of DataIO.....	90
2. DataIO as second block of GEN Framework	90
3. DIOFACTORY and platform insulation	90
4. General Map of DataIO.....	91
5. Addresses, identifiers and network utilities	92
6. Streams: the main abstraction of communication	92
7. Listing of devices and descriptors.....	93
8. TLS and streams secure	93
9. High-level protocols.....	94
10. DIOCoreProtocol and DIOProtocol	94
11. Web, HTTP client and embedded server	94
12. ScraperWeb and obtaining external information.....	95
13. Nodes: nodes, items, values and handlers	95
14. Drivers, I2C, SPI and physical devices	95
15. Cameras, notifications, alerts and updates	96
16. Diagnosis, connectivity and operational security	96
17. Relationship with XUtils, AppFlow and other factories.....	97
18. Catalogue of DataIO classes and modules.....	97
19. How to study DataIO from DevTestsConsole	104
20. Criteria for use in an application with GEN Framework	104
21. Practical summary	105
Chapter VII: Graphic, the graphical system of GEN Framework	107
1. The third block that shaped GEN Framework	107

2. GRPFACTORY and platform insulation.....	107
3. Graphic within an AppFlowGraphics application	108
4. Displays, views and graphic properties	108
5. Canvas 2D, colors, paths and reconstruction areas.....	109
6. Bitmaps, sequences and raster formats	110
7. Vector files: common base, SVG and DXF	110
8. Vector render over canvas.....	111
9. OpenGL ES, EGL and accelerated submission.....	111
10. Video and extraction of frames	112
11. Statistical Charts	112
12. Graphic and UserInterface.....	113
13. General Map of Graphic	113
14. Catalogue of Graphic classes and modules	113
15. How to study Graphic from examples	117
16. Criteria for use in an application with GEN Framework	117
17. Practical summary	118
Chapter VIII: Input, input devices and interaction abstraction.....	119
1. The Input block inside GEN Framework	119
2. A block born after the graphical system.....	119
3. The key idea: break down devices in cursors and buttons.....	119
4. INPFACTORY and platform insulation.....	120
5. INPMANAGER: the device access point.....	120
6. INPDEVICE: a device reduced to common elements.....	121
7. INPBUTTON: pulse, status and transition	121
8. INPCURSOR: position, change and selection	122
9. INPCURSORMOTIONPOINT and INPCURSORMOTION	122
10. The pattern of use in graphic applications	123
11. Input and UserInterface.....	123
12. INPCAPTURE and INPCAPTURE_XEVENT	124
13. INPSIMULATE: Software-generated input	124
14. Platform classes	125
15. Relationship between classes.....	125
16. Example of input skeleton in a graphical application.....	126
17. Input in applications without AppFlow	127

18. Common errors when using Input.....	127
19. Summary class catalogue of the Input block.....	128
20. Practical summary	128
Chapter IX: Sound, sound and playback in GEN Framework	129
1. The last factory block of GEN Framework	129
2. SNDFACTORY and platform insulation.....	129
3. General Map of Sound Block	130
4. SNDITEM: the reproducible unit.....	130
5. SNDPLAYCFG: volume and pitch per playback	131
6. Sounds from file: SNDFILE, OGG and WAV.....	131
7. SNDNOTE: sounds generated by frequency and duration	132
8. Sound events	132
9. SNDOPENALFACTORY: playback implementation	132
10. Buffer, source and play item	133
11. The reproduction state machine	133
12. Sound within a graphical application	134
13. Synchronization of Footsteps in Canvas2D.....	134
14. assets/sounds and organization of resources	135
15. Closing and cleaning pattern	135
16. Relationship with other modules of the framework	136
17. Summary class catalog of the Sound block	136
18. Criteria for use in an application with GEN Framework.....	137
19. Practical summary	137
Chapter X: Cipher, encryption and hashes in GEN Framework	139
1. The objective of the chapter	139
2. Hash, checksum, encryption and signature are not the same	139
3. General map of the Cipher block.....	139
4. The HASH class.....	140
5. Checksums and hashes fast	141
6. General cryptographic Hashes.....	141
7. HASHCOMP128V1	142
8. The CIPHER class.....	142
9. Symmetrical Encryption.....	143
10. Keys and internal representation	144

11. GKF and PEM key files	145
12. RSA and CIPHERRSA	145
13. Curve25519 / X25519	146
14. X.509 certificates and trusted roots	146
15. Relationship with assets and XPATHMANAGER	147
16. Block activation in CMake	147
17. Class catalogue of the Cipher block	148
18. How to study Cipher from DevTestsConsole	149
19. Criteria for use in an application with GEN Framework	149
20. Practical summary	149
Chapter XI: Compression of data and files in GEN Framework	152
1. The objective of the chapter	152
2. The Compress block inside GEN Framework	152
3. Data compression versus file compression	152
4. General map of classes	153
5. COMPRESSBASE: the common interface	153
6. COMPRESSMANAGER: creating algorithms	154
7. COMPRESS_ZIP: deflate compression over buffers	154
8. COMPRESS_GZ: GZip compatible compression	155
9. COMPRESS_LZW: LZW compression and GIF type of support	155
10. COMPRESS_LZRW1KH: quick and light compression	156
11. XFILEZIP: ZIP packages and files	156
12. Relationship with assets and XPATHMANAGER	157
13. Complete example: compress and decompress a text	157
14. Compression, encryption and order of operations	158
15. Activation using macros and compilation	158
16. Usual errors when using Compress	159
17. Compress block summary catalogue	159
18. Criteria for use in an application with GEN Framework	159
19. Practical summary	160
Chapter XII: Databases, SQL databases in GEN Framework	161
1. The role of Databases within the framework	161
2. Why a common connector is important	161
3. DB_SQL_FACTORY and engine selection	162

4. SQL Block Organization.....	162
5. DB_SQL_DATABASE: database abstraction	162
6. DB_SQL_CONNECTION: connection options	163
7. Database configuration in the application	164
8. Integration with assets and XPATHMANAGER	164
9. DB_SQL_QUERY: construction, binding and execution	164
10. Transactions, commit, rollback and savepoints	165
11. Creation and management of tables.....	166
12. DB_SQL_RESULT, DB_SQL_ROW and DB_SQL_ROWHEADER	166
13. DB_SQL_VARIANT: SQL data types in a common form	167
14. DB_SQL_ERROR and error management.....	167
15. SQLite: local base in file.....	168
16. MySQL and MariaDB: client/server connection	168
17. PostgreSQL: Advanced Relational Connector.....	168
18. Complete example of basic flow	169
19. Activation by CMake.....	170
20. Relationship with AppFlow, DIO, XUtilities and Cipher	170
21. Summary catalogue of the Databases block	170
22. Criteria for use in an application with GEN Framework.....	171
23. Usual errors when using Databases	171
24. Practical summary	172
Chapter XIII: Scripts, Dynamic Algorithms and Real-Time Plugins.....	173
1. The role of Scripts within GEN Framework.....	173
2. From compiled algorithms to loadable algorithms	173
3. Three supported languages	173
4. The SCRIPT class as a common point.....	174
5. Load, routes and assets/scripts	175
6. Configuration of the script list	175
7. SCRIPT_CACHE: script cache.....	176
8. Events and Script Errors.....	176
9. Function libraries: SCRIPT_LIB and SCRIPT_LIB_FUNCTION	177
10. Standard internal libraries	177
11. Optional libraries and integration with GEN Framework.....	178
12. SCRIPT_LNG_JAVASCRIPT	179

13. SCRIPT_LNG_LUA.....	179
14. SCRIPT_LNG_G: the language .GEN	180
15. Integration into ScriptsExample	180
16. Activation using CMake	181
17. Expand the system with its own functions	181
18. Examples of use	182
19. Relationship to other modules	182
20. Criteria for use	183
21. Usual errors when using Scripts	183
22. Summary catalogue of the Scripts block	183
23. Practical summary	183
Chapter XIV: UserInterface, widgets and multiplatform interface design.....	185
1. UserInterface location within GEN Framework.....	185
2. General architecture and execution flow	185
3. Load an interface and minimum cycle.....	186
4. UI_MANAGER and UI_LAYOUT: global management and composition.....	186
5. UI_ELEMENT: common basis for all widgets	187
6. Geometry, boundaries and alignment of coordinates	187
7. XML configuration and declarative styles.....	187
8. Colors, texts, animations and shared backgrounds.....	188
9. Skins and rendering on canvas	188
10. Reusable properties: selection, editing and scroll.....	189
11. Content Widgets: text, images and animation	189
12. Selection Widgets: Option, Button and CheckBox	190
13. MultiOption and State Representation	190
14. Containers, forms, menus and lists	191
15. Text editing and virtual keyboard.....	191
16. Linear, radial and image indicators of progress	192
17. Entry/exit events	192
18. Automatic texts, translation and dynamic data	193
19. Interfaces adaptable to different screens and displays.....	194
20. Update, reconstruction and performance areas	194
21. Integrated example based on UI_Options.....	194
22. UserInterface block complete catalog.....	195

23. Usual errors when working with UserInterface	195
24. Design criteria for an application with GEN Framework	196
25. Practical summary	196
Chapter XV: ThirdPartyLibraries, grow on open foundations	197
1. Selection and integration criteria	197
2. Scope of this review	197
3. General inventory	197
4. Graphics, images and typography	198
5. Sound and multimedia	200
6. C++ script languages and utilities	200
7. Compression and container formats	201
8. SQL databases	201
9. Network capture and specific functions of Windows	202
10. Embedded Hardware and SDK Platform	203
11. Unitary tests	204
12. Discrepancies identified in the inventory	204
13. Practical reading of licences	204
14. Recommended policy to maintain ThirdPartyLibraries	205
15. Practical summary	205
16. Sources of the review	205
Appendix I: C++ Coding Style Guide for GEN Framework	205
Overview of the GEN Framework	206
File names and modules	206
Header structure	206
Structure of a .cpp file	208
Section Separators	208
Indentation, keys and spacing	209
Conventions of names	209
Life cycle of a class	210
singleton pattern in GEN Framework	211
Documentation Doxygen	212
Public Types and API	212
Memory and ownership of objects	213
Including rules	213

Control flow	213
Platform and factory code.....	214
Traces and diagnosis.....	214
Styles to Avoid	214
Minimum module template	214
Checklist for new modules	215
Final style rule.....	216
Appendix II: Utilities — eXtended Trace Monitor.....	216
What a trace is and what problem it solves	216
Traces and logs are not the same.....	216
GEN Framework.....	217
Basic configuration from an application with GEN Framework	217
Emission of messages, colors and data blocks	218
Main Window Reading	218
Machine organization and application	220
Filters by IP, text, level, color and type	220
Important variables window	221
Pause, clean up and save a catch	221
Remote transport, UDP, pipeline and UART.....	221
Control of volume and cost of traces	222
Recommended use during a diagnostic session.....	222
Security and deployment.....	222
Classes and related components.....	222
Practical summary	223
Annex III: Frequently Asked Questions — Questions & Answers	224
1. Fundamental concepts	224
2. First steps.....	225
3. Architecture, modules and applications.....	225
4. Platforms, compilation and performance	226
5. Quality, diagnosis and safety	227
6. Third-party libraries, licences and maintenance	228
7. Adoption and criteria for use.....	229
Annex IV: Glossary of terms.....	231

Chapter I: C/C++, cross-platform and sense of GEN Framework

1. The starting point

Imagine an application that develops and debugs into a PC, runs also on a Raspberry Pi and must reach a device with much more limited resources. It changes the processor, changes the operating system — or even disappears —, changes tools and changes APIs. However, the idea of the application, its rules and most of its code should remain recognizable. That continuity is the problem that GEN Framework set out to solve.

It is not about obtaining the same binary for all destinations or pretending that all machines are the same. It is about getting something more valuable: that the same architecture can survive the change of platform; that the code that expresses what the application does is not buried under details of windows, files, threads, communications, graphics, devices or operating systems.

GEN Framework is a framework. This statement is essential. It is not a folder full of independent libraries or a collection of unrelated functions. A framework establishes a common language among its pieces: it defines how they are named, initialized, how they collaborate, what they depend on and how their implementations can be replaced. A basic function supports a class; several classes form a service; several services, coordinated, allow to build a complete application.

That architecture did not appear finished in a single moment. GEN Framework grew for years at the pace of concrete projects. First, utilities emerged to avoid repeating tasks; then it was necessary to sort them out, separate the common from the specifics of each platform and better control their relationships. When those pieces began to share types, conventions, life cycles and extension mechanisms, they ceased to be isolated solutions and became a system.

Each progress responded to a need that already existed outside the framework itself: reading a configuration, controlling a device, exchanging data, drawing an interface, recording an incidence, storing information or moving an application to another environment. In GEN Framework, a functionality is not incorporated just because it is interesting on paper. It is incorporated because it has been necessary to solve a real problem and because it can become a reusable piece for the next project.

That's why tests are needed, but they're not the only test. GEN Framework classes and modules have also gone through different compilations, hardware restrictions, installations, updates, imperfect communications and years of maintenance. There are framework-based applications working on real projects in different parts of the world. That accumulated experience does not eliminate all errors, but it does force design decisions to respond to reality and not just to an ideal example.

Three ideas have guided this evolution: order, control of dependencies and atomization of functionalities. Together they allow to develop in "Lego mode": to build small pieces, with a clear purpose and predictable connections, to combine them later in growing sets. GEN Framework itself is constructed like this, and the same way of working is available for applications that rely on it.

This chapter explains why this combination — C/C++, a modular architecture and a multiplatform layer born of real projects — offers a particularly powerful base. Before going through a very wide catalogue of classes and modules, it is important to understand the idea that holds them together. When that idea is understood, GEN Framework ceases to look like a large amount of code and begins to look like a box of pieces with which it is possible to build much more than each piece would do separately.

2. The origin of the C language

The C language was born in the early 1970s by Dennis Ritchie, in Bell laboratories and in close relationship with Unix. Its origin was profoundly practical: it required a language capable of writing systems with an efficiency close to that of the assembler, but whose code could be moved between machines with reasonable effort. That search for balance between control and portability changed computing.

The result was a small language in its fundamental mechanisms and enormous in its possibilities. The C language allows to describe algorithms and data structures and, at the same time, to work close to the memory, devices and calls of the system. It does not interpose a compulsory machinery between the program and the platform. That proximity demands responsibility, but also allows to decide precisely how the resources are used.

The portability of the C language never meant that any program could be compiled unchanged anywhere. It meant something more realistic and lasting: separating an important part of the program from the instructions of a particular machine and entrusting its translation to a compiler. Since then, the availability of a C language compiler has become one of the first signs that a platform can be opened to the development of its own software.

Decades later, the C language continues to be present in operating systems, firmware, controllers, libraries, toolchains and native APIs. It is also the meeting point for very different technologies: even when a component is written in another language, it is often exposed or consumed an interface compatible with the C language. That integration capability is crucial for a framework that must coexist with operating systems, SDK, graphic libraries, devices and existing code.

GEN Framework inherits from the C language that vocation to reach the real machine. It does not try to replace the hardware with an imaginary abstraction; it tries to isolate what changes and preserve what can be common. Where a platform offers a service, the framework can adapt it. Where it does not offer it, it can provide its own implementation or an appropriate alternative to the environment.

3. C and C++ are not the same language

In this book we use the expression C/C++ to refer to a shared technological base, but it is useful to be precise: C and C++ are different languages, with their own standards and models. They are closely related, share much of their ecosystem and can easily interoperate, but should not be confused.

The C language naturally organizes the program around functions, structures and modules. C++ expands that terrain with classes, templates, overload, polymorphism, lambdas, compilation time evaluation and other tools. It is a multiparadigm language: it allows you to choose between a processal style, object oriented, generic or functional according to the problem, and combine those styles when it brings clarity.

For GEN Framework, the decisive contribution of C++ is the possibility of creating abstractions without giving up control. A class can collect data, behavior and invariants; an interface can separate the service from its implementation; a factory can select the right variant for a platform; and RAII can link the life of a resource — memory, file, socket or blocking — to the life of an object.

This allows the complex code to be born of simple elements. A small class does not need to know the complete application. It only needs to comply well with its contract and depend on the essential parts. Another class can rely on it to add a level of behavior, and so on. Complexity does not disappear: it is distributed in places where it can be understood, tested and replaced.

GEN Framework uses C++ to express its architecture and maintains the ability to relate to the C language when required by the platform or library. This coexistence offers a bridge between two worlds: the stability of native interfaces and the power of modern abstractions. The programmer can work with high-level concepts without losing the possibility of descending to the system when necessary.

The choice does not eliminate the language requirement. C/C++ allows you to do much, including make costly mistakes. The role of the framework is to offer repeatable paths, clear life cycles and already contrasted components so that that freedom becomes building capacity, not permanent improvisation.

4. The evolution of C++

Talking about C++ today is not about the language of the nineties. Its modern revisions have incorporated movement of objects, smart pointers, lambdas, constexpr, ranges, concepts, modules and numerous improvements in the standard library. The goal remains unique: to offer high-level tools capable of producing effective native code.

A good abstraction in C++ can be solved to a large extent during the compilation. This makes it possible to write a clear interface without necessarily paying a relevant penalty in execution. It is a particularly valuable property in an architecture that should serve both teams with abundant resources and environments where every millisecond, every assignment and every kilobyte count.

Language evolves through international standards and multiple implementations. No company has either the language C or C++ or decides its own future. Manufacturers, open projects, specialists and users participate in an ecosystem where different interests coexist. This plurality does not prevent disagreements, but reduces the strategic dependence of a single provider.

At the same time, C++ retains unusual historical compatibility. Written code years ago can continue to be useful, while new parties adopt more expressive and secure mechanisms. For a framework that has evolved along with long-lived applications, that continuity matters as much as any recent feature.

GEN Framework takes advantage of language evolution with practical criteria: a feature is valuable when it improves clarity, security or performance and when it is available in compilers and platforms that must be supported. Modernity does not consist in using all newness, but in having more tools and choosing the one that best solves each problem.

5. C/C++ versus other languages

There is no optimal language for all projects. Python can be great for automation and prototyping; Java and C# offer very complete managed environments and business ecosystems; Kotlin and Swift are naturally integrated into their main platforms; Rust offers different guarantees for system software. Choosing a technology consists of consciously accepting their advantages and commitments.

GEN Framework focuses on applications where multiple needs match: native performance, resource control, low-level hardware or API integration, long service life, and the ability to reach very different platforms. In this field, C/C++ offer an exceptionally wide combination.

When a new architecture or system appears, it is very common for the first useful toolchain to include the C language and often C++. This happens because the systems themselves, SDK and controllers are largely built around them. The practical consequence is enormous coverage: from microcontrollers to servers, to personal computers, mobile devices and industrial systems.

This coverage does not depend on a single compiler. GEN Framework can be contrasted with different tool families, including GCC, Clang, Microsoft Visual C++ and Embarcadero C++ Builder, as well as manufacturer-specific toolchains. Compiling the same architecture with different implementations helps you discover non-portable assumptions, accidental extensions and errors that a single compiler might not reveal.

The diversity of compilers also offers operational freedom. A team can use the right tool for its destination without having to deliver control of the entire application to a single company or virtual machine. C and C++

languages are defined by standards; compilers compete in diagnostics, optimization, supported platforms and integration with their environments.

The native compilation allows generating highly optimized applications in speed and size. It does not itself guarantee a fast program — architecture and algorithms remain decisive — but it places the developer among the best possible conditions to achieve it. It does not impose a garbage collector, a virtual machine or a large runtime when the project does not need them.

C/C++ are also compatible with a huge variety of paradigms and technologies. They can expose a simple API to other languages, integrate procedural code, organize object-oriented systems, use generic programming and directly call system libraries. That ability to occupy the center of a heterogeneous architecture is difficult to match.

The price of that freedom is a greater responsibility. Memory, resources, dependencies and the compilation process must be handled rigorously. GEN Framework does not deny that cost; it organizes it. It provides a common basis for teams not to have to solve from scratch the same infrastructure problems in each application.

6. The real problem: the platform

In GEN Framework, a platform is not just an operating system or just a processor. It is the specific combination of hardware—based on a microcontroller or a microprocessor—and running environment on which the program should operate. It can be a PC x86-64 with Windows, a Raspberry Pi ARM with Linux, a Android device or a microcontroller with a real-time operating system. It can also be hardware without operating system, where the application is executed directly with minimal support available.

We call that concrete combination target or destination. The goal of GEN Framework is not to compile for a generic word such as "Linux" or "ARM", but to make the program work in the real destination: with its processor, its system — if it exists — its compiler, its libraries, its permissions and its devices.

While a program is small, the platform may seem invisible. As it grows, it appears when you manage routes and permissions, open a window, get precise time, create threads, draw, use a serial port, access a camera, network, upload resources, or respond to events. Each API introduces its own types, rules, and life cycles.

If all these details are directly entered into the logic of the application, the code is filled with local build conditions and exceptions. One block is used for Windows, another for Linux, another for Android and another for a specific plate. At first it looks like a shortcut; over time, each change forces reasoning on all destinations at once and the application stops clearly expressing its purpose.

The real problem is not that the platforms are different. They must be different: they have different capabilities, limits and security models. The problem is to allow those differences to spread unchecked throughout the program. A cross-platform architecture must concentrate on known points and offer stable contracts to the rest of the code.

GEN Framework places these decisions in specific layers, adapters and factories. The application requests a service using a common interface; the framework selects or builds the implementation corresponding to the target. So, what changes is localized and what defines the application can be preserved.

Carrying is not just about getting the compiler to finish without mistakes. It consists of preserving behavior, respecting the resources of the destination and being able to maintain the result. The precise definition of platform forces you to think from the beginning of the machine where the program will live, not only on the computer where it was written.

7. What GEN Framework

GEN Framework provides a common architecture between the application and the platform. Its modules are not limited to grouping utilities: they share conventions, types, creation patterns, traces, error management and dependency relationships. Learning a part prepares the programmer to understand the following.

The difference between a framework and a library collection is discovered by following an end-to-end operation. A complex functionality can start in a basic class of strings, buffers or time; then rely on files, streams or events; incorporate a protocol or a format; and end up being part of a visible service for the application. Each level uses the previous one instead of solving the same problem again.

This composition makes the scope grow cumulatively. Extended utilities support data input and output; data layers feed communications, configuration and storage; graphical systems and Input allow interaction; Sound, Cipher, Compress, Databases, Scripts and UserInterface add specialized capabilities. They are not islands: they form levels of the same construction.

The number of classes matters because it expands the field already resolved, but the relationship between them matters even more. An isolated class saves a few lines. A coherent set can save an entire architecture: initialization decisions, adaptation to platform, resource management, traceability and extension that, otherwise, each project would have to invent and maintain.

GEN Framework also offers a way to grow. An application can start using a few classes and add modules as new needs appear. It is not necessary to activate the entire framework or turn each program into a gigantic system. Atomization allows you to select pieces and preserve a common architectural direction.

The framework practices internally what it proposes to its users. Its complex functionalities are built on more basic components, and the applications can follow the same pattern. A class of the project can rely on GEN Frameworkservices, maintain a small task and become a piece for larger subsystems.

The result is a double acceleration. It advances faster because many needs already have a solution and because those solutions fit together. And it stays better because the application shares a recognizable vocabulary and structure with other projects developed on GEN Framework.

GEN Framework does not intend to hide everything or prevent the programmer from reaching the system. It pretends that this decision is explicit. The usual route must be comfortable, effective and portable; when an exceptional case requires native access, the architecture must allow to locate it without polluting the rest of the code.

8. Cross-platform development without giving up performance

In some environments, portability is obtained by placing the application on a virtual machine, an interpreter, or a full engine. It is a valid strategy, but it is not the only one. GEN Framework chooses to keep the native C/C++ compilation and perform the adaptation within its layers.

This allows each target to use the most appropriate compiler, optimizations and APIs. The same file interface can be solved with different calls; a canvas can be supported by different backends; a communication can be adapted to the available services. The application observes a common contract, while the framework leverages the actual capabilities of the platform.

Control also reaches size and consumption. In a PC, additional megabytes can be irrelevant; in an embedded system, they can prevent deployment. Modular architecture and conditional compilation by target help to include what is necessary without imposing components that the program will not use.

Abstraer does not mean ignoring the cost. A useful layer reduces complexity and retains predictable behavior. C++ allows many decisions to be resolved in compilation and that critical calls remain close to the system. When it is

necessary to optimize, the programmer can measure, identify the appropriate level and act without leaving the entire architecture.

The efficiency that GEN Framework is looking for has two dimensions. The application must be fast and adjusted in execution, but the equipment must also be effective when developing it. Reusing proven components, avoiding duplicating implementations and maintaining a single common logic reduces time to the result without converting performance into a subsequent concession.

There is no magic promise of speed: a bad algorithm is still a bad algorithm. What the technology base does offer is the possibility of achieving very high performance, controlling resources and maintaining that capability at different destinations. GEN Framework tries to extend portability to the scope of the program, not to lower its ambitions.

9. The absence of a unique library in C++

The standard C++ library provides containers, algorithms, memory, attendance, time and many other essential tools. Deliberately, it does not define a universal solution for windows, graphics, sound, devices, protocols, databases, user interfaces or all the services a full application needs.

The usual answer is to gather a library for every need. It can work, but soon there appear incompatible styles, duplicate types, different error models, transitive dependencies and compilation processes that do not match. The project ends up integrating excellent pieces that, without an architecture, do not necessarily form an excellent system.

GEN Framework does not reject third-party libraries. It uses them when they provide a mature solution and its open license allows it to do so safely. Whenever feasible, try to integrate them as source code to keep control of the multiplatform compilation. The selection considers simultaneously functionality, portability, maintenance and license; it is not enough for a library to resolve the immediate case.

Integration is the important word. The external library is located behind the framework conventions and abstractions, so that it does not impose its vocabulary on the entire application. If you change the backend, if a target needs another implementation or if the dependency needs to be updated, the impact remains contained.

In this way, GEN Framework occupies the space that the standard does not intend to cover: it offers a coherent environment to build applications. It does not compete with the standard library; it relies on it, complements it and organizes the additional capabilities that practice requires.

10. Single application, multiple platforms

The central promise can be expressed simply: to develop an application on a common basis and compile it for the supported targets without rewriting its main logic. It does not mean to produce the same executable, but to build a native executable for each destination from a shared architecture.

Business logic, models, own protocols, and application flows must remain at the common level. Window, device, storage, or system details are delegated to GEN Framework or encapsulated into project adapters. That border turns portability into a design property, not a final corrections campaign.

The benefit is multiplied over time. An improvement in the common code reaches all destinations. A corrected incidence in a shared service ceases to be repeated. A new platform can be added by implementing the necessary adaptation points, without cloning the entire application.

It also changes the way of testing. The team can develop comfortably in a main environment and periodically verify other compilers and targets. Each combination acts as a different perspective on the code: discover assumed sizes, byte order, accidental APIs, routes, encodings or indefinite behaviors that may remain hidden.

Naturally, a touch device is not used as a PC and a microcontroller does not offer the services of a server. The common part should not deny those differences. It must separate essential capabilities from presentation and hardware decisions, allowing each destination to adapt without breaking the core.

When that discipline is maintained, expanding the scope of an application ceases to be equivalent to keeping products disconnected. A family of executables born of the same design is maintained. That is one of the deepest advantages of GEN Framework: turning the variety of platforms into a controlled dimension of the project.

11. The importance of an architecture of its own

A framework needs an architecture because complexity is not overcome by accumulating functions, but by organizing relationships. In GEN Framework, that organization relies on three principles that reinforce each other and that can apply equally to the code of any application.

Command

Order means that each responsibility has a recognizable place. Names, directories, modules, layers and life cycles follow common criteria. Order reduces the number of improvised decisions and allows a programmer to be oriented to an unknown part of the system from what has been learned in another.

It is not an aesthetic preference. When a project grows, finding where a functionality should live and who should initiate or destroy it directly affects reliability. An orderly system makes its limits visible; a disorderly one forces to reconstruct them mentally every time it is modified.

Unit Control

Controlling dependencies means knowing what each component needs, preventing unnecessary relationships, and maintaining the right direction between layers. Basic parts should not depend on the most specialized ones. An application's logic should not know native details that the framework can solve. And an external library should not spread throughout the code if it can be left behind an interface.

An explicit dependency can be compiled, tested, replaced and revised. An accidental dependency is discovered late, often when carrying or updating. GEN Framework treats dependencies as part of the design because they are invisible lines that determine whether a class set can evolve or become immobilized.

Atomization of functionalities

To atomize consists in giving each class or function a reduced and complete purpose. A small piece is easier to understand, test and reuse. It can also be combined in ways that its author did not need in the original project. The key is not to divide by dividing, but to find units with clear responsibility and a stable interface.

Small classes allow you to create large sets because they limit the number of things to be understood at the same time. A buffer can power a stream; a stream can transport a protocol; a protocol client can participate in a service; and that service can be part of an application. Each level adds meaning without forcing the previous ones to know the end result.

Develop in Lego mode

Lego's metaphor does not mean that any piece fits anywhere. Blocks work because their connections have a definite shape. In software, those connections are interfaces, types, dependencies and life cycle contracts. The clearer they are, the greater the freedom to build.

GEN Framework applies this mode of development to itself: its major services rise on basic elements of the framework itself. And it extends it to the user: the classes of an application can become new blocks that combine existing services without breaking their separation. The result is not a monolithic program, but a structure capable of growing by composition.

Order without atomization produces compartments too large. Uncontrolled atomization of dependencies produces a tangle of tiny pieces. Unorderly controlled dependencies can remain difficult to locate. The three principles together create an architecture that can be expanded without losing legibility.

12. C/C++ as a strategic decision

Choosing C/C++ for GEN Framework was not a nostalgic decision, but a strategic one. They are languages available on an extraordinary variety of platforms, capable of producing very efficient native code and integrating with almost any relevant API. In addition, their standardization and the existence of independent compilers prevent the future of the framework from being linked to a single provider.

Compatibility with different compilers is part of that strategy. GCC, Clang, Microsoft Visual C++ and C++ Builder are not identical copies: they interpret diagnoses, extensions and optimizations from different implementations. Getting a code base to work with several of them increases their robustness and reduces involuntary dependencies.

However, a good technological base is not enough. GEN Framework is designed by programmers for programmers. Its utilities, functions and classes try to be effective and comfortable in daily work: solve complete tasks, maintain recognizable patterns and deliver large results with a reasonable amount of application code.

That comfort does not come from concealing important decisions, but from having made them many times. The framework pieces are born of concrete projects and refined when reused. When a need appears again, the aim is not to copy the previous solution, but to make it a better defined component so that the next project can start higher.

The tests unit and development check controlled behaviors. The deployed applications add another kind of evidence: prolonged operation, imperfect data, different machines, real users and maintenance. GEN Framework relies on both. A test protects one condition; experience reveals what conditions matter.

The complete strategy, therefore, is not simply "use C++". It is to combine a portable and efficient technology with an orderly architecture, controlled dependencies, small parts and production experience. C/C++ provide scope and control; GEN Framework converts those capabilities into a repeatable way to build applications.

13. What can be learned in this book

This book is not an inventory of thousands of names to memorize. It is a journey through the way GEN Framework transforms small pieces into complete applications. It presents the general structure, the compilation environment and the cycle of an application; then it travels through the modules that provide utilities, data, graphics, input, sound, encryption, compression, databases, scripts and user interfaces.

At each stage it will be necessary to ask three questions: what responsibility does this piece solve?, what does it depend on? and with what other pieces can it be combined? Those questions are more valuable than memory, because they allow to be oriented even to a class that is not yet known.

These pages explain how to separate the application logic from the platform, how to choose a target, how to take advantage of native implementations through a common interface and how to extend the architecture without converting it into a set of exceptions. It also shows where third-party libraries enter and why functionality, license and compilation possibilities form a single technical decision.

The practical goal is ambitious: to be able to build faster without giving up control; to reach more platforms without multiplying logic; to reuse without copying; and to obtain native applications, fast and adjusted to the environment where they should work.

GEN Framework does not promise that complex software will cease to be complex. It promises that complexity can be ordered. It offers many classes and a wide range, but its true power appears when you understand how they relate. Then a new need no longer forces you to start from scratch: it can be solved by combining existing blocks and adding only the missing piece.

That is the path that begins here. From the first basic type to the most specialized service; from the development computer to the real target; from a small class to an application capable of living for years. If the idea is stimulating, it is because the possibilities are also: to build serious, effective and portable software without losing sight of each of the pieces that make it possible.

Chapter II: Preparing the Development Environment

1. A practical chapter

In the previous chapter we saw why C and C++ remain fundamental languages for the development of cross-platform, efficient and close to the system software. We also present the *raison d'être* of GEN Framework: to provide an abstraction layer that allows to develop applications in C/C++ for different platforms without having to rewrite the logic of the application for each operating system.

Now we need to take a more practical step. Before we study the internal classes, modules, architecture or examples of the framework, we need to properly prepare the development environment. This chapter has precisely that objective: to explain what tools we need, how we should organize the repositories, where it is appropriate to place our own projects and what is the recommended procedure for making a first compilation.

In C/C++, the development environment is an essential part of the project. It is not enough to write source code. It is necessary to know which compiler is to be used, how headers are located, how libraries are linked, which construction system intervenes, what configuration is being generated, what external dependencies are needed and what differences exist between Windows, Linux, WSL2, Android or Docker.

In more managed languages, many of these decisions are hidden behind the execution environment, the package manager or the IDE. In C/C++, and especially in a multiplatform framework such as GEN Framework, these decisions are visible and are part of the knowledge that the developer must acquire.

This does not mean that the user has to master from day one Visual Studio, GCC, Clang, CMake, Docker, Docker Compose, Ninja, WSL2 and all the scripts in the framework. It means that you must understand what role each tool plays and what flow is recommended to start without getting lost in unnecessary details.

The most important idea of this chapter is simple: although GEN Framework uses CMake as an internal basis for organizing project construction, the first compilation should not be done by invoking CMake directly. The recommended entry point is the compilation scripts located in the common part of the framework, especially `compile.bat` in Windows and `compile.bash` in Linux.

These scripts exist to simplify work. They prepare paths, variables, options, configurations, platforms and internal commands. They allow you to compile all examples or select a specific one. They also allow you to work in stages, running only the configuration with CMake, only the compilation or phase of test.

Therefore, the reader should keep this rule from the beginning:

To start with GEN Framework, it is first compiled from the Common folder using `compile.bat` or `compile.bash`.

The detailed study of CMake appears at a later stage. The first practical contact is with the scripts of the framework itself.

2. The official framework page

The natural entry point to get GEN Framework is its official website:

```
https://www.genframework.com
```

From the web you can access the general information of the project and the repositories where the source code is maintained. GEN Framework should not be understood as a single folder with a few files, but as a set of repositories organized around a common architecture.

This separation is important. In small projects it may seem comfortable to have everything in a single folder. However, when a framework grows and must include kernel, common code, third-party libraries, examples, tests, resources, utilities and documentation, separation by repositories helps keep the system tidy.

A typical GEN Framework organization includes repositories such as:

```
GEN
Common
ThirdPartyLibraries
Examples
Tests
Utilities
Resources
Documents
```

Each has a specific function within the ecosystem.

The main repository of GEN Framework contains the main kernel of the framework. Here are the base classes, the main modules, the platform abstractions, the layers of graphics, communications, system, files, traces and other fundamental components.

Common contains common elements and plays a very important role in the compilation. For the starting user, this folder is key because from it the main construction scripts are released: `compile.bat` in Windows and `compile.bash` in Linux.

ThirdPartyLibraries contains third-party libraries used by GEN Framework. In C/C++ it is common to rely on external libraries for graphics, compression, image formats, parsers, rendering engines or auxiliary tools. These libraries may require platform-specific compilations, compiler and architecture.

Examples contain example applications. This repository is essential during learning, because it allows you to see real applications built on GEN Framework. Examples are not simple code fragments, but small applications that show how the framework is initialized, organized and used.

Tests contains tests aimed at validating specific functionalities. In an evolving framework, tests are essential for detecting regressions and checking that changes do not break previous behaviors.

Utilities contains auxiliary tools. Some can be used to generate resources, check configurations, debug modules, or perform support tasks during development.

Resources groups resources used by examples, applications or tools: images, fonts, configurations, templates or other elements external to the source code.

Documents may contain documentation, technical notes, references or additional project material.

3. The Projects folder as a workspace

An important decision when preparing the environment is where to place the framework and where to place our own projects.

The recommendation of this book is to use a general folder called Projects as a workspace. Inside it is placed, on the one hand, the complete folder of the framework, called GEN_FrameWork, and, on the other hand, the projects themselves.

It is important to understand this idea well: your own projects should not be placed inside the GEN_FrameWork folder. They should be at the same level as yours.

The recommended structure would be:

```

Projects/
|
|— GEN_FrameWork/
|   |— GEN/
|   |— Common/
|   |— ThirdPartyLibraries/
|   |— Examples/
|   |— Tests/
|   |— Utilities/
|   |— Resources/
|   |— Documents/
|
|— MiProyecto/
|— OtroProyecto/
|— PruebasGEN/

```

In this scheme, Projects is the general working root folder. Within it, GEN_FrameWork contains all the repositories that are part of the framework. Own projects, such as MiProyecto, OtroProyecto or TestaGEN, are sister folders of GEN_FrameWork.

This organization has several advantages.

The first is the clear separation between the framework and the applications. The framework is kept in its own folder, with its internal repositories organized. Own projects do not pollute that structure and can evolve independently.

The second is the ease of using relative routes. A project located in Projects/MyProject can access the framework by means of relative routes such as:

```

../GEN_FrameWork/GEN
../GEN_FrameWork/Common
../GEN_FrameWork/ThirdPartyLibraries
../GEN_FrameWork/Resources

```

This avoids relying on absolute routes of a particular machine. We do not want a project to depend on a route such as:

```

C:\Users\Usuario\Desktop\Temporal\GEN_FrameWork

```

This type of route makes the project only work on a particular computer. Instead, if we use relative paths from a common structure, the project can move to another disk, another user or another machine while maintaining the same relationship between folders.

The third advantage is that we can have several projects of our own using the same framework without duplicating it. For example:

```

Projects/
|
├── GEN_FrameWork/
├── AplicacionIndustrial/
├── HerramientaDiagnostico/
├── EjemploCliente/
└── PruebasGraficas/

```

All these projects can refer to the same GEN_FrameWork folder.

The fourth advantage is the cleaning of version control. The framework can be maintained with its own Git repositories, while each project can have its own independent repository. This is especially important in professional jobs, where an application can have its life cycle, branches, versions and separate deliveries from the framework.

Therefore, the recommended organizational rule is:

```

The GEN_FrameWork folder contains the framework.
Own projects are placed at the same level as GEN_FrameWork, within Projects.

```

4. Recommended organization in Windows

In Windows, a comfortable location could be:

```
E:\Projects\
```

Inside it we would place:

```

E:\Projects\GEN_FrameWork\
E:\Projects\MiProyecto\
E:\Projects\OtroProyecto\

```

The framework folder would contain the main repositories:

```

E:\Projects\GEN_FrameWork\GEN
E:\Projects\GEN_FrameWork\Common
E:\Projects\GEN_FrameWork\ThirdPartyLibraries
E:\Projects\GEN_FrameWork\Examples
E:\Projects\GEN_FrameWork\Tests
E:\Projects\GEN_FrameWork\Utilities
E:\Projects\GEN_FrameWork\Resources
E:\Projects\GEN_FrameWork\Documents

```

A project of its own would remain outside GEN_FrameWork, but within the same general project folder:

```
E:\Projects\MiProyecto
```

From MiProyecto, the relative routes to the framework would be:

```

..\GEN_FrameWork\GEN
..\GEN_FrameWork\Common
..\GEN_FrameWork\ThirdPartyLibraries
..\GEN_FrameWork\Resources

```

We could also use the system drive:

```

C:\Projects\GEN_FrameWork\
C:\Projects\MiProyecto\

```

The important thing is not the disk drive, but the relationship between folders.

An incorrect structure would be:

```
E:\Projects\GEN_FrameWork\Projects\MiProyecto
```

Although it could be run with manual settings, it is not the organization recommended in this book. It mixes your own developments within the framework folder and can complicate the updating, cleaning or maintenance of the tree.

The recommended structure is:

```
E:\Projects\GEN_Framework
E:\Projects\MiProyecto
```

5. Recommended organization in Linux

In Linux we can use an equivalent structure within the user's personal directory:

```
/home/usuario/Projects/
```

Inside that folder:

```
/home/usuario/Projects/GEN_Framework/
/home/usuario/Projects/MiProyecto/
/home/usuario/Projects/OtroProyecto/
```

The framework folder would contain:

```
/home/usuario/Projects/GEN_Framework/GEN
/home/usuario/Projects/GEN_Framework/Common
/home/usuario/Projects/GEN_Framework/ThirdPartyLibraries
/home/usuario/Projects/GEN_Framework/Examples
/home/usuario/Projects/GEN_Framework/Tests
/home/usuario/Projects/GEN_Framework/Utilities
/home/usuario/Projects/GEN_Framework/Resources
/home/usuario/Projects/GEN_Framework/Documents
```

From an own project located in:

```
/home/usuario/Projects/MiProyecto
```

we could access the framework by:

```
../GEN_Framework/GEN
../GEN_Framework/Common
../GEN_Framework/ThirdPartyLibraries
../GEN_Framework/Resources
```

This organization is clean, portable and easy to understand. If the project moves to another Linux machine, just keep the same relationship between GEN_Framework and your own projects.

6. Download via Git

The most recommended way to download GEN Framework is to use Git. Git allows you to clone repositories, update them, work with branches, review changes and keep a local copy synchronized.

In Windows, a download example could be:

```
cd /d E:\
mkdir Projects
cd Projects

mkdir GEN_Framework
cd GEN_Framework

git clone https://gitlab.genframework.com/gen-framework/gen.git GEN
```

```
git clone https://gitlab.genframework.com/gen-framework/common.git Common
git clone https://gitlab.genframework.com/gen-framework/thirdpartylibraries.git ThirdPartyLibraries
git clone https://gitlab.genframework.com/gen-framework/examples.git Examples
git clone https://gitlab.genframework.com/gen-framework/tests.git Tests
git clone https://gitlab.genframework.com/gen-framework/utilities.git Utilities
git clone https://gitlab.genframework.com/gen-framework/resources.git Resources
git clone https://gitlab.genframework.com/gen-framework/documents.git Documents
```

Then, if we want to create a project of our own, we would return to the General Projects folder:

```
cd /d E:\Projects
mkdir MiProyecto
```

The result would be:

```
E:\Projects\GEN_Framework
E:\Projects\MiProyecto
```

In Linux:

```
mkdir -p ~/Projects
cd ~/Projects

mkdir GEN_Framework
cd GEN_Framework

git clone https://gitlab.genframework.com/gen-framework/gen.git GEN
git clone https://gitlab.genframework.com/gen-framework/common.git Common
git clone https://gitlab.genframework.com/gen-framework/thirdpartylibraries.git ThirdPartyLibraries
git clone https://gitlab.genframework.com/gen-framework/examples.git Examples
git clone https://gitlab.genframework.com/gen-framework/tests.git Tests
git clone https://gitlab.genframework.com/gen-framework/utilities.git Utilities
git clone https://gitlab.genframework.com/gen-framework/resources.git Resources
git clone https://gitlab.genframework.com/gen-framework/documents.git Documents
```

And to create a project of your own:

```
cd ~/Projects
mkdir MiProyecto
```

The result:

```
~/Projects/GEN_Framework
~/Projects/MiProyecto
```

If the user only wants to download and compile, HTTPS is usually enough. If you are going to contribute to the project or work with write permissions, it may be preferable to use SSH.

To update a repository:

```
git pull
```

To update all the repositories in the framework, you can enter each GEN_Framework folder and run git pull. It is also possible to create auxiliary scripts that update the entire tree, but at first it is enough to understand that each folder is a separate repository.

During initial learning it is advisable not to directly modify the internal code of the framework unless you work on a specific branch. It is best to study the examples, create your own tests and start new applications as sister folders of GEN_Framework, within Projects.

As an alternative to the manual download of each repository, in the GEN Framework web download section (<https://www.genframework.com/#downloads>) two scripts are available: one for Windows and one for Linux.

Each script automatically downloads all GEN Framework repositories. It is only necessary to use the one corresponding to the operating system from which it is to be worked, which will serve as a basis for compilation.

7. Tools required

Several tools are needed to work with GEN Framework. Some are essential and others are recommended depending on the platform or type of compilation.

The first tool is a C/C++ compiler. In Windows Microsoft Visual C++, included with Visual Studio, is commonly used. Linux is used in GCC or Clang. Clang can also be used in Windows, although it is usually easier to work with the Microsoft compiler in the Visual Studio environment.

The second tool is CMake. GEN Framework uses CMake as an internal infrastructure to describe how projects are built. CMake allows you to generate configurations for different compilers and platforms.

The third tool is Ninja. Ninja is a fast construction system that is usually used together with CMake. CMake generates the required files and Ninja runs the compilation.

The fourth tool is Git, necessary to download and update the repositories.

The fifth tool is an editing or development environment. Here Visual Studio and Visual Studio Code stand out.

Visual Studio is a complete IDE for Windows. It includes debugger, integration with Microsoft compiler, solution management, analysis, memory inspection and many useful tools for C++.

Visual Studio Code is a lighter, extendable and multiplatform editor. It works very well with CMake, Git, WSL2, Docker and extensions for C/C++. It does not automatically replace the compiler; the compiler must be installed separately.

The sixth important tool is Docker. Docker allows you to create isolated environments using containers. In GEN Framework it can be useful for cross-building, for creating reproducible environments and for compiling third-party libraries in Linux when the main system does not have all the necessary dependencies installed.

Along with Docker, Docker Compose allows you to define and launch compilation environments more comfortably using configuration files.

8. Why we use these tools

The proposed tools are not arbitrary. They are part of the real ecosystem of modern C/C++ development.

Visual Studio is one of the best options to develop and debug C++ applications in Windows. Its debugger, its integration with MSVC and its maturity make it a very comfortable tool for this platform.

Visual Studio Code is especially useful when you want to work in a cross-platform way. It allows you to edit in Windows, Linux or WSL2 and does not impose a closed project structure. You can work on folders, integrate with Git, launch terminals, connect to WSL2 and use CMake or Docker extensions.

GCC and Clang are compilers widely used in Linux and in many development environments. GCC has a huge historical presence in the world GNU/Linux. Clang, based on LLVM, offers very clear diagnostics, good integration with modern tools and support for multiple platforms.

CMake allows the project not to depend on a single IDE. Instead of manually maintaining different projects for Visual Studio, Makefiles for Linux or separate configurations for each environment, CMake provides a common description of the construction system.

Ninja accelerates compilation and integrates very well with CMake. In large projects, the speed of the construction system can make a significant difference.

Git allows you to keep the code organized and updated. In a framework composed of several repositories, it is an essential tool.

Docker allows you to play environments. This is especially valuable when a compilation depends on specific versions of tools, system libraries or cross-compilation tools.

Docker Compose allows you to define those environments in a more orderly way, avoiding long and hard to remember commands.

All these tools also have a common feature: they allow automation. And automation is essential in a cross-platform framework. If a compilation depends on manual steps that each user runs differently, the result is fragile. If the compilation can be repeated through scripts, CMake and containers, the project becomes more stable and sustainable.

9. Windows as a development environment

Windows is a natural platform to start with GEN Framework if the user already uses Visual Studio.

A basic installation in Windows should include:

```
Visual Studio Community or higher
Desktop Development Workload with C++
Git
CMake
Ninja
Visual Studio Code, optional but recommended
Docker Desktop, optional to work with containers
```

During the installation of Visual Studio it is important to select the workload of C++. It is not enough to install Visual Studio as a general editor. GEN Framework requires the compiler of C/C++, the linker, SDK of Windows and the construction tools.

Once installed, you can check whether Git, CMake or Ninja are available:

```
git --version
cmake --version
ninja --version
```

To check the Microsoft compiler:

```
cl
```

If cl is not recognized, we may not be using a console with Visual Studio variables configured or C++ tools not installed.

However, although these checks are useful, the user should not start by manually compiling with CMake. The recommended stream in Windows is to go to the Common folder within GEN_FrameWork and use compile.bat.

For example:

```
cd /d E:\Projects\GEN_FrameWork\Common
compile.bat
```

The script compiles.bat is the recommended entry point for the compilation in Windows. From there you can launch compilations of examples, prepare the configuration and execute specific stages of the process.

10. Linux as a development environment

Linux is an essential platform for GEN Framework. Many embedded systems, servers, Raspberry Pi, industrial equipment and development environments use Linux. That's why it's important that the framework can properly compile and run on this system.

In a Debian-based distribution or Ubuntu, an initial installation could be:

```
sudo apt update
sudo apt install build-essential git cmake ninja-build clang
```

build-essential installs GCC, G++ and basic build tools. Clang is installed separately if you want to use LLVM as compiler.

Other system dependencies may also be needed, especially for graphic examples, communications, audio, third-party devices or libraries. These dependencies may vary depending on distribution and version.

The initial flow in Linux must also start from the Common folder inside GEN_Framework:

```
cd ~/Projects/GEN_Framework/Common
chmod +x compile.bash
./compile.bash
```

As in Windows, the script allows you to compile all examples or select a specific one. In addition, a particularly interesting option appears in Linux: to use Docker. In certain cases, it may be advisable or necessary to compile inside a container, especially when third-party libraries or cross-compilation environments are involved.

To do this, the script can accept the DOCKERS modifier, written in capital letters.

The user must understand that Docker is not mandatory for all cases. If the Linux environment has all the dependencies installed and the native compilation works, it can be used directly. But when working with third-party libraries, cross compilation or environments that must be reproducible, Docker becomes a very powerful tool.

11. WSL2 as mixed environment Windows/Linux

WSL2, the Windows Subsystem for Linux, allows us to run a Linux environment within Windows. For GEN Framework it is a very interesting tool because it allows us to work on the same computer with two worlds that would normally be separated: on the one hand Windows, Visual Studio, native graphics tools and debugging; on the other hand Linux, GCC, Clang, Bash, CMake, Docker and the framework build scripts themselves.

This combination is especially useful in a multiplatform framework. We can develop in Windows, compile with Microsoft tools, open the same code with Visual Studio Code and, at the same time, release Linux compilations from WSL2. This allows validation from early stages that the code does not accidentally depend on a single platform.

A basic installation of WSL2 can be done from PowerShell:

```
wsl --install
```

Once the Linux distribution is installed, as Ubuntu, the main tools are prepared:

```
sudo apt update
sudo apt install build-essential git cmake ninja-build clang
```

From there, you can compile GEN Framework from Linux using the scripts in the Common folder:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase
```

Or, when you want to use the Docker-based flow:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase DOCKERS
```

However, WSL2 introduces an important question that needs to be understood: the performance of the file system.

From Linux within WSL2 we can work in two main ways. The first is to save the fonts within the Linux file system itself, for example:

```
/home/usuario/Projects/GEN_Framework
/home/usuario/Projects/MiProyecto
```

The second is to work on a Windows unit mounted inside Linux:

```
/mnt/c/Projects/GEN_Framework
/mnt/c/Projects/MiProyecto
```

From the point of view of pure compilation performance in Linux, the first option is usually clearly faster. When fonts are inside Linux file system of WSL2, usually over the virtual disk distribution, the reading, writing, temporary file creation, object generation and directory routing operations are much more efficient.

This is especially relevant in C/C++. A compilation does not consist of reading a single large file. On the contrary, a real compilation can open thousands of small files: headers, fonts, objects, generated dependencies, intermediate files, scripts, caches and libraries. This intensive access pattern greatly penalizes any layer of slow translation between file systems.

The problem appears when WSL2 is used as a real mixed environment. If repositories are copied or cloned once in Windows and once inside Linux, the Linux compilation is faster, but introduces another problem: the sources are duplicated.

That duplication is not a good general solution. Having two separate code trees means having two places where modifications may exist. We can change a file in Windows and forget to replicate it in Linux. We can fix something in WSL2 and not have it in the copy of Windows. We can compile a version believing it corresponds to the last code, when it actually belongs to a outdated copy.

In small projects it may seem manageable; in a large framework or in a real application, it ends up being a source of errors. Therefore, while placing the fonts inside the Linux system of WSL2 is the fastest from the point of view of compilation, our recommendation for GEN Framework should be more balanced: avoid duplicating fonts whenever possible. It is preferable to have a single well-defined working tree and consciously assume the consequences of performance, rather than working with two copies that can diverge.

The performance problem when accessing WSL2 to Windows routes, such as `/mnt/c`, is known. WSL2 more clearly separates the Linux environment from the Windows environment than WSL1. That architecture has important advantages, because WSL2 runs a real Linux kernel inside a lightweight virtual machine, but cross-access between file systems has a cost.

In the interoperability between Windows and Linux within WSL2, 9P-based mechanisms are involved, the protocol associated with the system Plan 9. Plan 9 was a research operating system developed in Bell Labs. Its name comes from the film *Ed Wood Plan 9 from Outer Space*, a cult work famous precisely for its bad cinematic reputation. The anecdote may be curious, but in this context the important thing is not the origin of the name, but its practical consequences: for workloads with many small file accesses, such as a compilation C/C++, this layer of interoperability is not a good option from the point of view of performance.

In practice, this makes `/mnt/c` performance one of WSL2's endemic problems compared to WSL1. WSL1 had another architecture and, in some scenarios of access to Windows files from Linux, it could behave better. WSL2 improves many technical aspects, especially by having a real Linux kernel, but cross-access to Windows files can be clearly slower.

We therefore have three possible strategies.

The first strategy is to work within the Linux file system of WSL2. It is the fastest option to compile from Linux:

```
/home/usuario/Projects/GEN_Framework
/home/usuario/Projects/MiProyecto
```

Its advantage is performance. Its drawback is that if we also want to work with Windows tools directly on the same tree, we can find less comfortable flows. Visual Studio Code solves it quite well by integrating it with WSL, but other Windows tools can be less natural.

The second strategy is to work on a Windows drive from WSL2:

```
/mnt/c/Projects/GEN_Framework
/mnt/c/Projects/MiProyecto
```

Its advantage is that Windows and Linux see the same file tree directly. There is no duplication. The same folder can be opened from Windows, edit from Windows tools and compile from WSL2. Its downside is performance: compile from Linux over `/mnt/c` can be noticeably slower.

The third strategy, which is the one we recommend as a point of equilibrium for many GEN Framework scenarios, is to maintain a single working tree in Windows, preferably in a unit prepared for development, and to mitigate the cost of WSL2 with optimization techniques such as compilation cache and Dev Drive.

GEN Framework is prepared to take advantage of Compiler Cache techniques. The idea of a cache compiler is to avoid unnecessarily recompiling files that have not changed. In a large C/C++ project, many incremental compilations repeat work. If the system can detect that a compilation drive has already been processed with the same parameters, it can reuse the previous result. This reduces compilation time and decreases part of the impact of a slower file system.

The use of compilation cache does not eliminate the background problem of `/mnt/c`, because accesses to the file system still occur, but it can greatly reduce the cost in incremental compilations. In a real development stream, where a small part of the code is modified and recompiled many times, this improvement can be very significant.

In addition, in Windows 11 there is another interesting tool: Dev Drive. A Dev Drive is a storage volume designed for development workloads. It is based on ReFS and uses scenario-specific optimizations such as repositories, compilations, package restoration, temporary files and intensive development operations.

In practice, a Dev Drive can be a separate drive, for example:

```
D:\Projects\
```

or:

```
G:\Projects\
```

Within it we could maintain the recommended structure:

```
D:\Projects\
|
|— GEN_Framework\
|   |— GEN\
|   |— Common\
|   |— Examples\
|   |— ThirdPartyLibraries\
|   |— ...
|
|— MiProyecto\
```

From Windows we would work on:

```
D:\Projects\GEN_Framework
D:\Projects\MiProyecto
```

From WSL2 we would access the same structure by:

```
/mnt/d/Projects/GEN_Framework
/mnt/d/Projects/MiProyecto
```

This strategy maintains a single font tree. It does not require duplicate repositories. It allows you to use Windows tools naturally and, at the same time, launch compilations from WSL2. The cost of cross access still exists, but can be reduced using an optimized unit for development and based on compilation cache.

Therefore, a practical recommendation for GEN Framework in modern Windows equipment could be:

```
Use a Project folder on a Dev Drive.
Place GEN_Framework inside that folder.
Place your own projects at the same level as GEN_Framework.
Use Visual Studio or Visual Studio Code from Windows.
Use WSL2 for Linux compilations when necessary.
Enable Compiler Cache when the build stream allows.
Avoid duplicating sources except in very specific cases.
```

The final structure would be as follows:

```
D:\Projects\
|
├── GEN_Framework\
|   ├── GEN\
|   ├── Common\
|   ├── ThirdPartyLibraries\
|   ├── Examples\
|   ├── Tests\
|   ├── Utilities\
|   ├── Resources\
|   └── Documents\
|
├── MiProyecto\
└── OtroProyecto\
```

And from WSL2:

```
/mnt/d/Projects/
|
├── GEN_Framework/
└── MiProyecto/
```

To compile from WSL2 using that shared tree:

```
cd /mnt/d/Projects/GEN_Framework/Common
./compile.bash inibase INTEL64 DEBUG COMPILE
```

Or using Docker:

```
cd /mnt/d/Projects/GEN_Framework/Common
./compile.bash inibase INTEL64 DEBUG COMPILE DOCKERS
```

This is not the fastest option in absolute terms compared to keeping everything inside /home/usuario, but avoids the problem of managing two source trees. In real projects, this stability is usually more important than winning a few seconds at the expense of introducing risk of divergence.

If the target is an intensive, automated or continuous integration Linux compilation, then it can make sense to work within the Linux file system or within Docker containers, where the entire tree is mounted and compiled in a controlled manner. But for mixed daily development Windows/Linux, especially when alternating Visual Studio, Visual Studio Code, Bash scripts and Windows tools, the priority must be to maintain a single source of truth.

In short, WSL2 is a very valuable tool for GEN Framework, but you have to use it with criteria. Its integration of file systems is not perfect. Accessing Windows routes from Linux can be slow because of the interoperability layer.

Duplicating sources improves performance but worsens maintenance. That's why our recommendation is to use a single working tree, preferably on a Dev Drive when working on Windows 11, and rely on techniques such as Compiler Cache, Docker and a good folder structure to reduce performance impact without sacrificing project coherence.

12. Docker and Docker Compose

Docker allows you to create isolated execution and compilation environments using containers. In the context of GEN Framework, Docker can be used to create reproducible compilation environments, for cross-compilation and to build third-party libraries that require specific configurations.

In C/C++, the result of a compilation does not depend solely on the source code. It also depends on the compiler, the linker, the installed headers, the libraries available, the system version, the routes and the options used. This makes it possible for two computers to obtain different results if their environments are not the same.

Docker helps solve that problem. An image with the necessary tools provides a very similar environment to any computer that runs it. This reduces the errors difficult to play.

In GEN Framework, Docker can be especially useful in Linux for several reasons.

The first is the compilation of third-party libraries. Some external libraries may require specific packages, specific versions, or additional tools. Installing all that on the main system may not be desirable. A container allows you to encapsulate those dependencies.

The second is the cross compilation. If we need to generate binaries for a different architecture, we can prepare a container with the corresponding toolchain.

The third is continuous integration. The same containers that are used locally can be run on automatic servers to check that the framework compiles correctly.

The fourth is system cleaning. Instead of installing many dependencies on the main machine, they are kept inside the Docker image.

Docker Compose allows you to define these environments using configuration files. Instead of running long commands of Docker, we can describe services, volumes, images and commands in a Compose file.

A conceptual flow could be:

```
cd ~/Projects/GEN_Framework
docker compose build
docker compose run --rm gene-build
```

Inside the container, the flow should continue using GEN Frameworkscripts:

```
cd /workspace/Common
./compile.bash inibase
```

Or, if the script itself is prepared to invoke the container from outside:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase DOCKERS
```

The latter case is more comfortable for the user, because it maintains the same entry point: compile.bash.

The important thing is not to think of Docker as a substitute for the framework or as a substitute for CMake. Docker provides the environment. GEN Framework scripts organize the compilation. CMake generates the internal configuration. The compiler builds the binary.

Each tool fulfills its role.

13. Common scripts as input point

One of the most important aspects of GEN Framework is that the initial compilation should be done from the Common folder.

In Windows:

```
cd /d E:\Projects\GEN_Framework\Common
compile.bat
```

In Linux:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash
```

Or, in WSL2 using a single tree in a Windows drive:

```
cd /mnt/d/Projects/GEN_Framework/Common
./compile.bash
```

These scripts are the recommended entry point because they concentrate the logic needed to compile the framework and its examples. They allow to avoid errors derived from invoking CMake directly with incomplete parameters, incorrect paths or unprepared environment variables.

The starting user does not need to know yet what internal options CMake needs, where construction folders should be generated, which relative paths are used, which compiler is selected, or how the output binaries are structured. That information is managed by scripts.

An important feature is that scripts can be named after an example. This allows you to compile only that example.

For example, in Windows:

```
compile.bat inibase
```

In Linux:

```
./compile.bash inibase
```

And in Linux using Docker:

```
./compile.bash inibase DOCKERS
```

This mechanism is especially useful during learning. If we are studying inibase, we do not need to compile all the examples of the framework each time. We can focus on one, modify it, recompile it and execute it.

It is also useful for debugging. If a specific example fails, we can compile only that example and review the output more clearly.

14. Convention of names in scripts

Before using the build scripts it is important to know a basic convention of GEN Framework: the name of the program, application or example that is passed to the script must always be written in lower case.

For example, although the example may appear documented as IniBase, when calling the build script you should write:

```
compile.bat inibase
```

or in Linux:

```
./compile.bash inibase
```

The modifiers, on the other hand, are always written in capital letters. This makes it possible to distinguish clearly between the name of the program to be compiled and the special options that modify the behavior of the script.

The general criterion is:

```
compile.bat programa_en_minusculas MODIFICADOR_EN_MAYUSCULAS  
compile.bash programa_en_minusculas MODIFICADOR_EN_MAYUSCULAS
```

Correct examples:

```
compile.bat inibase INTEL64 DEBUG COMPILE  
./compile.bash inibase INTEL64 DEBUG COMPILE  
./compile.bash inibase ARM64 RELEASE CLANG DOCKERS
```

Wrong examples:

```
compile.bat IniBase intel64 debug compile  
./compile.bash IniBase docker
```

The difference is important. The program goes in lower case. The modifiers go in upper case.

15. Script Modifiers

GEN Framework build scripts allow you to modify the behavior of the construction process using arguments. These arguments are passed after the name of the program or example.

The general syntax can be understood as follows:

```
script program modifier modifier modifier ...
```

In Windows:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

Not all modifiers make sense on all platforms. Some belong to the Windows environment, others to the Linux environment, others to the compilation mode, others to the compiler, and others to the internal features of the framework.

It is also important to understand that scripts may have default values. If the user executes:

```
compile.bat inibase
```

or:

```
./compile.bash inibase
```

The script applies the default configuration defined by the framework. That configuration may be sufficient for a first test. When you need to accurately control the architecture, mode, compiler, trace or memory control, modifiers are used.

16. Stage modifiers

Stage modifiers indicate which part of the construction process we want to run.

The available values are:

```
CMAKE  
COMPILE  
TEST
```

CMAKE executes the configuration phase. At this stage build folders are prepared, compilers are detected, options are set, dependencies are located and the necessary files are generated so that the construction system can work.

Example in Windows:

```
compile.bat inibase CMAKE
```

Example in Linux:

```
./compile.bash inibase CMAKE
```

COMPILE executes the build phase. At this stage, source files are compiled, objects are generated, libraries are linked and the final binary is created.

Example in Windows:

```
compile.bat inibase COMPILE
```

Example in Linux:

```
./compile.bash inibase COMPILE
```

TEST executes the test phase, if available for the selected project or configuration. This phase allows validating components, checking behaviors or running automated tests.

Example in Windows:

```
compile.bat inibase TEST
```

Example in Linux:

```
./compile.bash inibase TEST
```

Splitting the process into stages has several advantages. If we have only changed configuration options, we can run CMAKE. If all is set up and we have changed source code, we can run COMPILE. If we want to check behavior after compiling, we can run TEST.

A complete compilation can run several phases internally, but stages allow you to control the process when you need more precision.

17. Windows Platform Modifiers

Platform modifiers indicate which architecture or destination we want to compile for. In Windows, the available values are:

```
INTEL32
INTEL64
ANDROID32
ANDROID64
```

INTEL32 selects a 32-bit Intel architecture build. This destination may be needed when we want to generate 32-bit system supported binaries or validate behavior in that architecture.

Example:

```
compile.bat inibase INTEL32 DEBUG COMPILE
```

INTEL64 selects a compilation for 64-bit Intel architecture. This is probably the most common destination in modern Windows.

Example:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

ANDROID32 selects a 32-bit compilation for Android, when the build environment and the necessary tools are configured.

Conceptual example:

```
compile.bat inibase ANDROID32 DEBUG COMPILE
```

ANDROID64 selects a compilation for 64-bit Android.

Conceptual example:

```
compile.bat inibase ANDROID64 DEBUG COMPILE
```

Android modifiers depend on the Android environment being properly prepared: SDK, NDK, toolchain, CMake paths and settings. They should not be confused with a normal Windows build. They allow you to select a build destination, but require specific tools to exist.

18. Platform modifiers in Linux

In Linux, the available values are:

```
INTEL64
ARM32
ARM64
RPI32
RPI64
```

INTEL64 selects a compilation for Intel architecture or 64-bit AMD. It is the usual destination for a modern PC Linux or a WSL2 machine over Windows.

Example:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

ARM32 selects a compilation for 32-bit ARM architecture. It can be useful in embedded systems, old plates or specific environments that still work in 32 bits.

Example:

```
./compile.bash inibase ARM32 DEBUG COMPILE
```

ARM64 selects a compilation for 64-bit ARM architecture. It is common in modern plates, recent embedded systems or Linux environments over ARM64.

Example:

```
./compile.bash inibase ARM64 DEBUG COMPILE
```

RPI32 selects a 32-bit Raspberry Pi-oriented compilation.

Example:

```
./compile.bash inibase RPI32 DEBUG COMPILE
```

RPI64 selects a 64-bit Raspberry Pi-oriented compilation.

Example:

```
./compile.bash inibase RPI64 DEBUG COMPILE
```

In these cases it is especially important to distinguish between native build and cross build. Running the script directly into a Raspberry Pi can correspond to a native build. Compiling from a PC Linux to generate binaries intended for Raspberry Pi is a cross build and usually requires toolchains, Docker or additional configuration.

19. Mode modifiers

Mode modifiers indicate whether we want to generate a development-oriented compilation or a distribution-oriented compilation.

The available values are:

```
DEBUG
RELEASE
```

DEBUG generates a compilation designed to develop. It typically includes debugging information, allows you to inspect variables, track step-by-step execution, and detect errors more easily. It may also include additional checks or less optimization.

Example in Windows:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

Example in Linux:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

RELEASE generates a compilation designed for final execution. It applies optimizations, reduces debugging information and generates binaries more suitable for distribution or performance testing.

Example in Windows:

```
compile.bat inibase INTEL64 RELEASE COMPILE
```

Example in Linux:

```
./compile.bash inibase INTEL64 RELEASE COMPILE
```

During learning it is recommended to start with DEBUG, because it facilitates debugging. When the application already compiles and works properly, RELEASE can be tested to measure performance or generate a version closer to distribution.

It is not advisable to measure real performance using a DEBUG build. It is also not comfortable to debug a complex problem using only RELEASE. Each mode has its purpose.

20. Compiler modifiers

Compiler modifiers indicate whether we want to use Clang or not.

The available values are:

```
CLANG
NOTCLANG
```

CLANG requests the use of Clang as compiler, as long as it is available for the selected platform and the environment is ready.

Example in Linux:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE
```

Example in Windows, if the environment supports it:

```
compile.bat inibase INTEL64 DEBUG CLANG COMPILE
```

NOTCLANG indicates that you do not want to use Clang. In Linux this can mean using GCC. In Windows it can mean using the Microsoft compiler, depending on the configured flow.

Example:

```
./compile.bash inibase INTEL64 DEBUG NOTCLANG COMPILE
```

The choice between CLANG and NOTCLANG may be important for detecting differences between compilers. A code that compiles with MSVC can display different warnings with Clang. A code that compiles with GCC can reveal other problems with Clang. So, in a multiplatform framework, testing multiple compilers is a very healthy practice.

Clang usually offers very clear diagnostic messages and is useful for finding compatibility problems. GCC is a key reference in Linux. MSVC is essential in Windows. GEN Framework must be able to live with these environments.

21. Memory control modifiers

Memory modifiers indicate whether or not the memory control system in the framework is activated.

The available values are:

```
MEMCTRL
NOTMEMCTRL
```

MEMCTRL activates memory control, if the configuration and framework support it. This mode can be useful during development to detect leaks, incorrect assignments, unrealized releases or behaviors related to the life cycle of objects and resources.

Example:

```
compile.bat inibase INTEL64 DEBUG MEMCTRL COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG MEMCTRL COMPILE
```

NOTMEMCTRL disables that memory control.

Example:

```
compile.bat inibase INTEL64 DEBUG NOTMEMCTRL COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG NOTMEMCTRL COMPILE
```

In C/C++, memory is a critical part of development. Although modern C++ offers tools such as RAII, standard containers and smart pointers, a low-level framework may need additional diagnostic mechanisms. Activated memory control during development can help find errors that would otherwise appear later and would be more difficult to locate.

However, these mechanisms can introduce additional cost or generate extra information. That is why it is useful to be able to activate or disable them through modifiers.

22. Trace modifiers

Trace modifiers control the generation or behavior of the trace system.

The available values are:

```
TRACE
TRACENOTINTER
NOTTRACE
```

TRACE activates the trace system. Traces are essential for diagnosing the behavior of an application, especially when working with various systems, threads, communications, graphical modules or different platforms.

Example:

```
compile.bat inibase INTEL64 DEBUG TRACE COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG TRACE COMPILE
```

TRACENOTINTER activates a trace configuration without the intermediate, interactive or interconnected part that may be associated with the framework trace system, depending on the specific script implementation. It is a useful option when you want to preserve traceability, but avoid certain additional communication, intermediation or output mechanisms.

Example:

```
compile.bat inibase INTEL64 DEBUG TRACENOTINTER COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG TRACENOTINTER COMPILE
```

NOTTRACE disables the trace system.

Example:

```
compile.bat inibase INTEL64 RELEASE NOTTRACE COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 RELEASE NOTTRACE COMPILE
```

During development, TRACE can be very useful. In distribution versions or performance tests, you may be interested in NOTTRACE to reduce noise, dependencies or running cost.

The TRACENOTINTER value allows an intermediate configuration: it is not the same to disable all traces that keep traces but without certain auxiliary mechanisms.

23. Feedback modifiers

Feedback modifiers control whether the process or application generates additional feedback information during compilation or execution, depending on how the framework uses it.

The available values are:

```
FEEDBACK
NOTFEEDBACK
```

FEEDBACK activates feedback generation. It can be useful during development, testing, script validation or behavior diagnosis.

Example:

```
compile.bat inibase INTEL64 DEBUG FEEDBACK COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 DEBUG FEEDBACK COMPILE
```

NOTFEEDBACK disables that additional feedback.

Example:

```
compile.bat inibase INTEL64 RELEASE NOTFEEDBACK COMPILE
```

In Linux:

```
./compile.bash inibase INTEL64 RELEASE NOTFEEDBACK COMPILE
```

The utility of this modifier depends on the context. In stages of development it may be interesting to have more information. In clean, automated or distribution compilations, it may be interesting to reduce output to what is essential.

24. DOCKERS modifier

In addition to the above groups, the modifier can be used in Linux:

```
DOCKERS
```

DOCKERS indicates that the compilation should be done using the Docker-based flow prepared by the framework.

Example:

```
./compile.bash inibase DOCKERS
```

It can also be combined with other modifiers:

```
./compile.bash inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE DOCKERS
```

Or to run only the CMake configuration within the Docker flow:

```
./compile.bash inibase INTEL64 DEBUG CMAKE DOCKERS
```

The use of DOCKERS may be especially important for cross-compilation or for compiling third-party libraries in Linux. If a dependency needs a particular environment, the container can provide that environment in a reproducible way.

Although the modifier is called DOCKERS, it conceptually refers to the use of Docker as a compilation environment.

25. Combination of modifiers

The modifiers allow you to build quite expressive commands. For example, if you want to compile inibase in Windows for Intel 64 bits, in Debug mode, using the build phase, with memory control and activated traces, you could write:

```
compile.bat inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

If you want to configure with CMake before compiling:

```
compile.bat inibase INTEL64 DEBUG CMAKE MEMCTRL TRACE
```

And then compile:

```
compile.bat inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

In Linux, an equivalent command could be:

```
./compile.bash inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

With Clang:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE MEMCTRL TRACE
```

With Docker:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE MEMCTRL TRACE DOCKERS
```

For a release compilation without memory control, without trace and without feedback:

```
./compile.bash inibase INTEL64 RELEASE NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE
```

The general idea is that the arguments describe the type of compilation we want to make:

```
programa      → inibase
platform → INTEL64
mode → DEBUG or RELEASE
compiler → CLANG or NOTCLANG
memory → MEMCTRL or NOTMEMCTRL
traces → TRACE, TRACENOTINTER or NOTTRACE
feedback → FEEDBACK or NOTFEEDBACK
stage → CMAKE, COMPILE or TEST
environment → DOCKERS, if Docker is used in Linux
```

The name of the program always remains lowercase. Modifiers always remain uppercase.

26. Summary table of modifiers

The following table summarizes the main modifiers:

Group	Values	Use
Stage	CMAKE, COMPILE, TEST	Selects the construction process phase
Windows Platform	INTEL32, INTEL64, ANDROID32, ANDROID64	Select build destination in Windows
Linux Platform	INTEL64, ARM32, ARM64, RPI32, RPI64	Select build destination in Linux
Mode	DEBUG, RELEASE	Select Development or Distribution Compilation

Builder	CLANG, NOTCLANG	Enables or disables use of Clang
Memory	MEMCTRL, NOTMEMCTRL	Enables or disables memory control
Traces	TRACE, TRACENOTINTER, NOTTRACE	Configure the trace system
Feedback	FEEDBACK, NOTFEEDBACK	Enables or disables additional information
Docker	DOCKERS	Use compilation flow with Docker in Linux

This table does not replace the help of the script itself. The specific version of the scripts can expand, limit or adjust combinations. Still, it summarizes the general philosophy: the scripts allow to describe the compilation using clear, readable modifiers and capital letters.

27. CMake as internal infrastructure

Although in this chapter we insist that the first compilation should be done with scripts, it is important to understand what role CMake plays.

CMake is the infrastructure that allows you to describe how the project is built portably. It is not a compiler. It does not replace Visual Studio, GCC or Clang. Your job is to generate the construction files needed for the chosen environment.

In Windows you can generate Visual Studio projects or configurations for Ninja. In Linux you can generate Makefiles or Ninja files. In more advanced environments you can work with specific toolchains for cross compilation.

GEN Framework uses CMake because it needs an orderly way to describe modules, dependencies, options, routes, configurations and platforms. Manually maintaining separate projects for each system would be difficult and error-prone.

However, CMake has many options. Summoning it directly without knowing the framework structure can cause problems:

```
Wrong paths.
Undefined variables.
Outbound folders misplaced.
Wrong builder.
Debug or Release settings incorrect.
Unlocated units.
Unprepared third-party booksellers.
```

That's why Common scripts are so important. They don't delete CMake, they encapsulate it. The user works with a simple entry and the script is responsible for calling CMake in the right way.

The study of CMake files is especially useful when the framework is better known. It allows you to understand how new projects are added, how modules are defined, how libraries are linked and how configurations are adapted for different platforms.

The recommended learning order is:

```
First use the scripts.
Then understand what scripts do.
Finally study CMake in detail.
```

28. Visual Studio and Visual Studio Code

Visual Studio and Visual Studio Code have similar names, but they are different tools.

Visual Studio is a complete IDE for Windows. It is especially development oriented with Microsoft compiler. It offers advanced debugging, memory inspection, analysis, integration with solutions and a very powerful environment for Windows applications.

Visual Studio Code is an extensible and multiplatform editor. It does not itself include a C++ language compiler, but it integrates very well with external compilers, CMake, Git, WSL2, Docker and terminals. It is especially useful when we want to work similarly on Windows and Linux.

For GEN Framework, both tools can live together perfectly.

A regular flow could be:

```
Use Visual Studio to debug Windows applications.
Use Visual Studio Code to edit code and work with scripts.
Use Visual Studio Code connected to WSL2 to compile in Linux.
Use Docker from the terminal when an isolated environment is needed.
```

The important thing is that the project does not depend exclusively on a IDE. GEN Framework must be able to be compiled using scripts and CMake, not just by pressing a button in Visual Studio. This independence is essential to maintain the cross-platform nature of the framework.

29. Own projects at the same level as GEN_FrameWork

When the user has already compiled official examples, the next natural step is to create a project of their own.

The recommendation is to place it within the General Projects folder, but not within GEN_FrameWork.

The structure would be:

```
Projects/
|
├── GEN_FrameWork/
|   ├── GEN/
|   ├── Common/
|   ├── Examples/
|   └── ...
└── MiProyecto/
```

This own application could have an internal structure such as:

```
MiProyecto/
|
├── CMakeLists.txt
├── Sources/
├── Headers/
├── Resources/
└── Scripts/
```

The exact structure depends on the type of application, but the important point is that the application is near the framework and can be accessed by relative routes.

For example, from Projects/MyProject we could refer to:

```
../GEN_FrameWork/GEN
../GEN_FrameWork/Common
../GEN_FrameWork/Resources
../GEN_FrameWork/ThirdPartyLibraries
```

This avoids relying on absolute paths of a particular machine. If another user has the entire tree on another disk or in another folder, the relative paths work while maintaining the general structure:

```
Projects/
├── GEN_FrameWork/
└── MiProyecto/
```

It also makes it easier for your own applications to follow the same style as the official examples. The reader can study an example in `GEN_FrameWork/Examples`, create an own version in `Projects/MyProject` and modify it without touching the original.

This separation is important. Official examples must be kept as a clean reference. Own projects must evolve independently.

In addition, this organization allows you to have multiple projects of your own using a single copy of the framework:

```
Projects/
├── GEN_FrameWork/
├── MiProyecto/
├── HerramientaInterna/
├── AplicacionCliente/
└── PruebasRender/
```

Each project can have its own Git repository, its own version cycle, and its own structure, but everyone can use the same `GEN_FrameWork` located in the Common Projects folder.

This organization also helps when you want to update the framework. We can update `GEN_FrameWork` without mixing those changes with our own projects. And we can update a project of our own without touching the repository of the framework.

The final recommendation is therefore:

```
Do not place your own projects within GEN_FrameWork.
Place them inside Projects, at the same level as GEN_FrameWork.
Use relative routes from the project to GEN_FrameWork.
```

30. Usual errors during preparation

The preparation of the environment may fail for many reasons. Some errors are very common.

One of the first is not having Git installed or not having it in the PATH. If `git --version` does not work, it is not possible to clone or update the repositories.

Another common error in Windows is to install Visual Studio without C++ tools. Visual Studio can be installed for many types of development, but GEN Framework needs the C++ compiler, SDK Windows and construction tools.

It is also common to install Visual Studio Code thinking it already includes compiler. Visual Studio Code is an editor. It needs GCC, Clang, MSVC or another external compiler.

Another common problem is having CMake installed, but not available in the PATH, or having an old version.

Ninja may also be missing if scripts use it as a generator.

In Linux, development packages may be missing. A graphical example may need headers from X11, OpenGL or other dependencies. A specific module may need additional libraries.

In Docker, the service may not be started, the user does not have permission to run containers, or Docker Compose does not find the configuration file.

In WSL2, a common error is not to correctly decide where the main font tree will be. Working within /home/usuario may be faster to compile from Linux, but it makes it difficult to maintain the same flow from Windows if you want to use Windows tools directly. Working from /mnt/c or /mnt/d allows for a single shared tree, but it can be slower. Duplicating fonts on both sides is usually a bad practice except in highly controlled cases.

The compilation may also fail if the repositories are not at the same level within GEN_FrameWork. For example, if the main repository is in a folder and Common in a completely different folder, scripts may not find what they expect.

Another common error is to place your own projects inside GEN_FrameWork and then mix your own files with the repository in the framework. Although this may work temporarily, it is not good practice. The recommended structure is that your own project is GEN_FrameWork's brother within Projects.

Useful commands to check the environment:

```
git --version
cmake --version
ninja --version
gcc --version
g++ --version
clang --version
docker --version
docker compose version
```

In Windows:

```
cl
```

To check Docker:

```
docker run hello-world
```

If these commands fail, the problem must be solved before blaming the framework. The first task is always to validate the basic tools.

31. How to deal with a compilation error

When a compilation error appears, the natural reaction of the beginner is to think that everything is wrong. In C/C++, however, errors must be analyzed by method.

First you have to locate the first real error. Many times a compilation produces tens or hundreds of messages, but they all stem from the first one. If a header is missing, then many additional errors appear because the compiler no longer knows types, classes or functions.

Second, check if the error appears during CMAKE, during COMPILE or during TEST. It is not the same as a configuration error as a code error or a test failure.

If CMAKE fails, the problem is usually in routes, dependencies, compiler or configuration.

If COMPILE fails, the problem is usually in source code, headers, compiler options or incompatibilities.

If TEST fails, the code may have compiled correctly, but some behavior is not expected.

Third, check if the error occurs in native compilation or with Docker. If it occurs only in native, a local dependency may be missing. If it occurs only in Docker, the container may not have correct access to the working tree or something is missing in the image.

Fourth, we have to reduce the case. If we are compiling all the examples, we can try to compile only one:

```
compile.bat inibase
```

or:

```
./compile.bash inibase
```

If that example compiles, the basic environment works and the problem may be in a specific dependency of another example.

Fifth, we need to check the folder structure. If scripts are designed to run from GEN_FrameWork/Common, we must respect that location. If a project itself uses relative paths to GEN_FrameWork, we must check that it is actually located at the same level as the folder in the framework.

For example, this structure is correct:

```
Projects/
├── GEN_FrameWork/
└── MiProyecto/
```

This other one is not recommended for own projects:

```
Projects/
├── GEN_FrameWork/
│   ├── GEN/
│   ├── Common/
│   └── MiProyecto/
```

This method of analysis avoids wasting time and helps to distinguish between installation errors, structure errors, configuration errors and actual code errors.

32. The environment as part of learning

In GEN Framework, preparing the environment is not a secondary process. It is part of real learning.

A programmer who works with C/C++ should know that the compiler matters, the linker matters, routes matter, libraries matter, architecture matters and configuration matters.

GEN Framework tries to reduce the amount of platform-specific code the user must write. But it cannot completely eliminate the reality of each system. Windows, Linux, Android, WSL2 or Docker have differences. The framework abstracts them as far as possible, but the construction environment must be well prepared.

Common scripts help make that preparation easier. Instead of starting by typing long and fragile CMake commands, the user has a controlled input:

```
compile.bat
```

or:

```
./compile.bash
```

Then you can specify:

```
compile.bat inibase
```

or:

```
./compile.bash inibase DOCKERS
```

With the already known environment, you can also use stages:

```
CMAKE
COMPILE
TEST
```

This design allows you to learn progressively. It is first compiled. Then you understand the script. Then you study CMake. Then you create your own projects.

The folder structure is also part of that learning. Keeping GEN_FrameWork as a framework folder and own projects as sister folders within Projects helps maintain a healthy separation between technology base and applications.

In WSL2, in addition, the environment forces you to make practical decisions about where sources are placed. The maximum performance option Linux is to use /home/usuario, but the most consistent option for mixed development Windows/Linux can be to keep a single tree in a Windows unit, preferably Dev Drive, and to offset the cost with Compiler Cache. This decision is not purely technical; it also affects the organization, the security of daily work, and the possibility of avoiding divergences between copies.

33. Quick compilation reference

The following tables summarize the most common commands to compile the example inibase. The name of the program is always written in lowercases and the modifiers always in uppercases. These references serve as a quick guide for the first steps; the actual script help should be viewed by running it without parameters.

Windows

Step	Objective	Command	Notes
1	Enter Common	cd /d E:\Projects\GEN_Framework\Common	Recommended entry point.
2	Show Script Options	Compile.bat	See the help and values available.
3	Compile Inibase with default values	compile.bat inibase	First test recommended.
4	Configure with CMake	compile.bat inibase INTEL64 DEBUG CMAKE	Just run the configuration phase.
5	Compile in Debug for Intel 64 bits	compile.bat inibase INTEL64 DEBUG COMPILE	Regular compilation during development.
6	Run test	compile.bat inibase INTEL64 DEBUG TEST	Validate the example if tests is available.
7	Compile in Release	compile.bat inibase INTEL64 RELEASE COMPILE	Optimized version.
8	Use Clang if available	compile.bat inibase INTEL64 DEBUG CLANG COMPILE	It requires environment prepared for Clang.
9	Complete development	compile.bat inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE	Clang, memory, trace and feedback.
10	Full distribution	compile.bat inibase INTEL64 RELEASE NOTCLANG NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE	Clean output for distribution.

Linux

Step	Objective	Command	Notes
1	Enter Common	cd ~/Projects/GEN_Framework/Common	Recommended entry point.
2	Give execution permissions	chmod +x compile.bash	Only if the script does not have permissions.
3	Show Script Options	./compile.bash	See the help and values available.
4	Compile Inibase with default values	./compile.bash inibase	First test recommended.
5	Configure with CMake	./compile.bash inibase INTEL64 DEBUG CMAKE	Just run the configuration phase.
6	Compile in Debug for Intel 64 bits	./compile.bash inibase INTEL64 DEBUG COMPILE	Usual use in PC Linux or WSL2.
7	Run test	./compile.bash inibase INTEL64 DEBUG TEST	Validate the example if tests is available.
8	Compile in Release	./compile.bash inibase INTEL64 RELEASE COMPILE	Optimized version.
9	Compile for ARM or Raspberry Pi	./compile.bash inibase ARM64 DEBUG COMPILE ./compile.bash inibase RPI64 DEBUG COMPILE	Examples of Linux platforms.
10	Compile using Docker	./compile.bash inibase DOCKERS	Reproducible environment or cross compilation.
11	Development with Docker	./compile.bash inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE DOCKERS	Clang, diagnostic and container.
12	Full distribution	./compile.bash inibase INTEL64 RELEASE NOTCLANG NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE	Clean output for distribution.

WSL2 with shared Windows drive

Step	Objective	Command or Resource	Notes
1	Use a single source of truth	D:\Projects\GEN_Framework\nD:\Projects\MiProyecto	Avoid duplicating sources between Windows and Linux.
2	Enter Common from WSL2	cd /mnt/d/Projects/GEN_Framework/Common	Linux equivalent of unit D: Windows.
3	Give execution permissions	chmod +x compile.bash	Only if the script does not have permissions.
4	Compile inbase by default	./compile.bash inibase	Validate the shared tree.
5	Compile in Debug	./compile.bash inibase INTEL64 DEBUG COMPILE	Usual use from WSL2.
6	Compile using Docker	./compile.bash inibase INTEL64 DEBUG COMPILE DOCKERS	Insulate the compilation environment.
7	Reduce the cost of /mnt/d	Compiler Cache Dev Drive	It mitigates the loss of Plan9/9P performance.

As a general rule, you should start with the simplest build and add modifiers only when they are needed. First it is validated to inibase compiles; then you test platform, mode, compiler, trace, memory control and Docker.

Chapter III: Overview and Framework Architecture

1. A global vision before entering into the detail

After preparing the development environment and compiling a first example, the next natural step is to understand how GEN Framework is organized. Before using specific classes or studying a module in depth, it is appropriate to have an overview of its architecture. A large framework is not understood only by the list of files it contains, but by the way in which it groups responsibilities, isolates dependencies and allows an application to be kept portable between platforms.

GEN Framework is born with a central idea: to develop applications in C/C++ while maintaining the performance of the native code, but preventing the logic of the application from being tied to a specific platform. To achieve this, it is not enough to create a few auxiliary functions. It is necessary to define a modular architecture, a set of factories, abstraction layers and conventions that clearly separate what belongs to the common core, what belongs to each functional module and what belongs to a given platform.

This chapter presents this overview. It is not intended to be an exhaustive reference of each class or a complete description of all the files in the framework. Its purpose is to offer the reader a mental map that facilitates the understanding of subsequent chapters, dedicated to basic types, utilities, input and output of data, graphics, user interfaces, communications, scripting, databases and other subsystems.

GEN Framework architecture can be understood as a combination of three ideas: modularity, factories and platform insulation. Modularity allows the framework to be divided into functional areas. Factories allow creating suitable objects without the application having to know the specific implementation. Platform insulation ensures that differences between Windows, Linux, Android, Raspberry Pi or other destinations are confined to specific areas of the code.

2. The modular essence of GEN Framework

Modularity is one of the most important properties of GEN Framework. A multiplatform framework cannot be a monolithic code mass where everything depends on everything. If each module knows internal details of everyone else, any change becomes dangerous. If each class calls directly to native APIs of the operating system, portability disappears. If each application has to decide which implementation to use for each platform, the framework ceases to fulfill its function.

This is why GEN Framework is organized into functional groups. Each group brings together classes that share a responsibility. Some areas provide basic utilities, others manage data input and output, others work with graphics, others with input devices, sound, compression, encryption, databases, scripting or application flow. This separation allows better reasoning about the system.

Modularity also facilitates growth. A living framework needs to incorporate new platforms, new drivers, new formats, new protocols or new rendering systems without breaking what already exists. If the architecture is well separated, a new module can be added in the right place and expose its functionality through existing interfaces. The application that uses the framework does not have to absorb all that detail.

Another important aspect is that modularity does not mean absolute isolation. Modules collaborate with each other. A graphical system may need to read resources from the file system. A user interface can be supported by graphics, keyboard and mouse input, timers and tracers. A communication module can use buffers, strings, threads and error control. The important thing is that those collaborations are produced through clear layers and not by chaotic dependencies.

At GEN Framework we can talk about main modules, represented by high-level factories, and additional or cross-sectional modules that provide more specific services. The main factories are XFACTORY, DIOFACTORY, GRPFACTORY, INPFACTORY and SNDFACTORY. Groups such as Compression, Cipher, AppFlow, Scripts, UserInterface or Databases, which expand the scope of the framework and allow you to build more complete applications.

3. The role of the factories

One of the essential concepts of GEN Framework is the use of factories. A factory is a component responsible for creating objects. This idea may seem simple, but it has very important architectural consequences. Instead of the application directly creating a specific class, it asks the factory for the object it needs. The factory decides which implementation corresponds.

In a multiplatform framework this is essential. An application may need to open a file, create a screen, start an input device, play sound or access a system resource. Conceptual operation may be the same on all platforms, but internal implementation is not. Windows has its own APIs. Linux has others. Android has its own model. Raspberry Pi can add particularities. The application should not have to know all that.

Factories act as a boundary between portable logic and concrete implementation. The application code requests an abstraction. The framework provides an appropriate instance. This reduces the need to use compilation conditions in the application code and centralizes the platform decision within the framework.

This pattern also improves maintenance. Incorporation of a new implementation for a particular platform does not force the application to change: the factory creates the right object when applicable. If a graphical backend is replaced, the internal part of the factory or its associated classes is modified, but the public interface can be maintained. Support for a new architecture is incorporated into the platform layer without polluting the rest of the code.

Therefore, a factory in GEN Framework is not just a convenience for creating objects. It is a key piece of platform insulation and design coherence. Each of the main factories is associated with a functional domain: general utilities, data input and output, graphics, user input and sound.

Factory	Main area	General responsibility
XFACTORY	eXtender Usefuls	Basic Utilities, auxiliary types, system, chains, buffers, time, threads, traces and common services.
DIOFACTORY	Data Input/Output	Data streams, communications, devices, protocols, files, URLs, network and access to external inputs/outputs.
GRPFACTORY	Graphic Factory	Displays, canvas, bitmaps, fonts, rendering 2D, graphics backends and visual resources.
INPFACTORY	Input Factory	Keyboard, mouse, tactile, input events and abstraction of interaction devices.
SNDFACTORY	Sound Factory	Sound, playback, buffer, streams audio devices and audio output abstraction.

4. Plants and platform insulation

Platform isolation is one of the most important objectives of the framework. The application must be able to express what it wants to do without relying directly on the API native to the operating system. This does not mean that the platform disappears, but that its differences are contained in specific places.

In GEN Framework, factories are directly involved in this isolation. When a module needs a system-dependent implementation, it should not be built directly from the application code. Creation is channeled through a factory. Thus, selecting the specific class can depend on the build platform, the enabled options, the available backend or the configuration of the framework.

For example, an application should not have to ask whether to create a screen should use a specific class of Windows, another class of Linux or a variant for Android. The application should work with a common graphical abstraction. Internally, GRPFACTORY and associated classes resolve which implementation corresponds. The same happens with input, sound, communications or certain system utilities.

This separation brings several advantages. The first is clarity. The application code is no longer full of build conditions. The second is maintainability. Differences are concentrated in the framework, not in each program that uses it. The third is portability. An application that respects framework abstractions can be compiled on several platforms with less changes. The fourth is ability to evolve. The framework can add platforms or backends without forcing to rewrite the application.

The general idea can be summarized as follows: the application works with common interfaces and services; the factories create specific objects; the platform dependent code is confined within the framework, especially in the platform folders and modules.

5. Confining the platform code

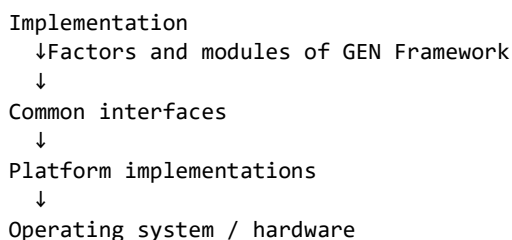
An essential principle of GEN Framework is that specific platform code should be confined. This means that native calls to Windows, Linux, Android or other systems should not be scattered throughout the framework much less by the applications that use it. They should concentrate on clearly identifiable modules, usually within the platform area.

The Platform folder or group contains the implementations that depend on a particular system. That's where the classes that know details of Windows, Linux, Android, Raspberry Pi or other destinations make sense. It's also where the inevitable differences appear: native types, system calls, window managers, graphical APIs, access to devices, permissions, routes, timers or specific synchronization mechanisms.

This organization allows the rest of the framework to remain as portable as possible. The upper layers define common concepts. The platform part implements those concepts using the tools of each system. If the specific code was mixed with the rest, each module would have to know all the systems and the architecture would become fragile.

Confining the platform does not mean ignoring it. On the contrary: it means recognizing that it exists and giving it a proper place. A serious multiplatform framework cannot be based on the illusion that all systems are the same. Windows, Linux and Android are not the same. What the framework can do is offer a coherent interface above those differences and limit the impact of each system on the final application.

The programmer using GEN Framework must learn to respect that border. If an application introduces direct calls to native APIs without need, it breaks part of the framework value. There may be cases where it is inevitable to access a very specific platform functionality, but the general rule must be clear: first, the abstraction of the framework is sought; only if it does not exist, a controlled extension is evaluated.



6. XFACTORY: eXtender Utilities

XFACTORY represents the base utility area of the framework, often identified with eXtender Utilities. It is one of the most fundamental parts because it provides services that end up being used by many other modules. In any large C/C++ architecture there is a layer of common utilities: strings, buffers, dates, times, routes, threads, synchronization, traces, configuration, auxiliary types and support structures.

The role of XFACTORY is to act as a point of creation and access to objects of that common layer. Although the user is not always aware of it, many high-level features depend on these utilities. A file system needs paths. A communication needs buffers. A graphical system needs to load resources. A trace module needs strings, time and data output. A configuration system needs serialization, files and basic types.

Within the scope of XFACTORY we can place functionalities such as chain management, dynamic buffers, routes, dates and times, timers, threads, mutexes, events, traces, data variants, lists, maps or auxiliary structures. Some of these classes can be completely portable. Others can be supported by platform implementations when they need access to operating system services.

String utilities are especially important in a multiplatform framework. Windows, Linux and Android may have differences in encoding, routes, separators, Unicode and internal representation. A framework that wants to avoid errors must provide classes and conventions of its own to treat text consistently.

Buffers are also a common base. Many areas of the framework work with memory blocks: communications, compression, encryption, file reading, images, audio or serialization. Having a uniform abstraction for buffers reduces errors and makes it easy to reuse code.

Traces are another essential component. In cross-platform applications, debugging only with the IDE is not always enough. There are scenarios where the application runs on a remote device, on Android, on a Raspberry Pi, on a service or on a system without a graphical interface. An integrated trace system allows you to observe the application behavior evenly.

In short, XFACTORY provides the soil on which many other parts of the framework are supported. It is not necessarily the most visible part for the end user, but it is one of the most important for the stability, coherence and portability of the whole.

7. DIOFACTORY: Data Input/Output

DIOFACTORY groups the input and output domain of data. In any real application, the data constantly enters and exits: files are read, logs is written, TCP or UDP communications are opened, serial ports are accessed, URLs are processed, protocols are worked, data is received from devices or interacts with external services.

The name DIO can be understood as Data Input/Output. Its primary responsibility is to provide a coherent way to work with data flows, communications and devices. This includes seemingly simple operations, such as opening a file, as well as more complex operations, such as managing a stream network, an industrial protocol or a communication with a sensor.

Within this group we can find abstractions for streams, files, sockets, TCP/UDP communications, serial ports, protocols, clients or servers, URLs, DNS, TLS, input/output devices and drivers. The presence of a factory allows the application to request a communication or a device without directly depending on the implementation of a specific platform.

The DIO area is especially sensitive to platform insulation. Network APIs, serial ports, permissions, device paths and system calls change between Windows, Linux and Android. If an application directly implements all those differences, the code becomes very difficult to maintain. DIOFACTORY concentrates that complexity and offers a more uniform layer.

It is also important to understand that DIO is not limited to network. Input and output includes any form of communication with the outside. It can be a local file, a remote connection, a device connected by serial port, a SPI interface or I2C on embedded systems, a specific protocol or an encrypted stream.

In industrial or embedded applications, this layer can be decisive. Many projects are not just a nice graphical interface: they need to talk with hardware, sensors, external devices, communication buses, network services or proprietary protocols. GEN Framework provides a basis for those communications not to contaminate the main logic of the application.

8. GRPFACTORY: Graphic Factory

GRPFACTORY groups the graphical part of the framework. Cross-platform graphic development is one of the areas where the need for abstraction is most clearly seen. Creating a window, obtaining a graphic context, drawing on screen, loading an image, representing text or managing a canvas is not done the same way on all platforms.

The goal of GRPFACTORY is to provide a point of creation for graphic objects and rendering backends. Within this scope we can place screens, canvas, bitmaps, fonts, colors, primitive drawing, images, rendering 2D, vector support, integration with engines such as AGG, OpenGL or OpenGL ES, and systems capable of representing graphic formats such as SVG or DXF when incorporated by the framework.

The graphic part has several layers. A basic layer can represent colors, dots, rectangles, regions, transformations or bitmaps. An upper layer can represent a canvas where lines, texts, polygons or images are drawn. Another layer can connect that canvas with a specific backend. Finally, a platform layer takes care that all this appears on a real screen or on a valid buffer for the system.

This separation allows an application to draw using common concepts, even if the backend internally is different. In Windows there may be a specific implementation. In Linux there may be another one. In Android another path can be used. If the framework exposes a coherent API, the application does not need to know all those details.

In addition, the graphic area is not only useful for user interfaces. It can also be useful to render reports, generate images, visualize data, represent vector graphics, load resources, draw over memory or prepare outputs for different devices.

GRPFACTORY is therefore one of the most visible factories for the user, because many applications show something on screen. But its value is not only in drawing. Its value is in allowing the visual logic to remain portable and the specific backend to be encapsulated.

9. INPFACTORY: Input Factory

INPFACTORY groups user input and interaction devices. In a graphical or interactive application, drawing is not enough. You also need to receive events: keyboard, mouse, touch screen, pointers, buttons, gestures or specific devices. Each platform has its own event system and its way of representing that input.

The goal of INPFACTORY is to abstract that diversity. The application should not depend directly on Windows window messages, X11 events, Android events or platform specific APIs. It should work with common events defined by the framework.

Within this group we can place keyboard, mouse, touch input, button states, pointer coordinates, movement events, pulses, releases, wheel, basic gestures or input adapters. In some environments it can also extend to specific physical devices, controls, sensors or platform inputs.

The entry is especially important because it connects the user to the application. If the abstraction is poor, each platform forces to write different code. If the abstraction is adequate, the application can react to user actions evenly.

INPFACTORY usually collaborates with the graphical part and the user interface. An interface button needs to know when the mouse is pressed or when the screen is touched. A text control needs to receive a keyboard. A display application needs to interpret movements. That collaboration must be done without breaking the separation of responsibilities: the input captures events, the interface interprets them and the graph represents them.

10. SNDFACTORY: Sound Factory

SNDFACTORY groups the sound area. Although not all applications need audio, having a sound abstraction can be important in multimedia applications, ad interfaces, industrial systems with alarms, games, interactive tools or environments where sound is part of the user experience.

Working with multiplatform sound can be complex. Each system has its own API s, preferred formats, output devices, buffers, latency and playback models. The goal of SNDFACTORY is to provide a common layer to create sound-related objects without the application relying on a specific API.

Within this group we can find sound devices, streams audio, buffers, playback, volume control, format loading, sound output or integration with specific backends. As with the rest of the factories, concrete implementation can vary by platform.

Sound can also be related to other modules. You can load data from files managed by DIO, use XUtilities buffers, apply decoding or compression, generate events for AppFlow or integrate with a user interface. Modularity allows these collaborations not to become messy dependencies.

SNDFACTORY thus completes the set of main factories, covering an area that does not always appear in the initial examples, but is part of a complete application architecture.

11. Additional groups and cross-cutting services

In addition to the main factories, GEN Framework includes or can integrate additional functional groups that extend its scope. These groups are not always presented as stand-alone first-level factories, but provide important services to build real applications. Among them we can highlight Compression, Cipher, AppFlow, Scripts, UserInterface and Databases.

These groups should be understood as transverse layers. Some are supported by XUses for strings, buffers or traces. Others rely on DIO to read or write data. Others interact with graphics, input or sound. Their goal is to solve frequent needs that appear when an application ceases to be a simple example and becomes a complete product.

Group	Main function	Relationship with architecture
Compression	Compression and decompression of data, files or buffers.	It supports buffers, streams and files to reduce size or package information.
Cipher	Encryption, hashes, signatures or cryptographic mechanisms.	It protects data and communications, usually together with DIO and buffers.
AppFlow	Organization of application flow, states, processes and execution control.	It helps to structure applications more complex than a linear main.
Scripts	Integration of embedded languages or script engines.	It allows to expand behavior without recompiling parts of the application.
UserInterface	Controls, windows, dialogues and interface elements.	It is supported by graphics, input and resources.

Databases	Access to databases and structured persistence.	Connect the application with storage and queries.
-----------	---	---

12. Compression

The compression group provides functionality to reduce data size, package information, or work with compressed formats. In actual applications, compression appears frequently: application resources, logs, exchange files, communications, backup or temporary storage.

Compression usually works with buffers and streams. This is why it is naturally related to XUtilities and DIO. A file can be read using DIO, stored in a common buffer, compressed and rewritten. A communication can send compressed data to reduce bandwidth. A resource package can be decompressed when you start the application.

From the point of view of architecture, Compression should not force the application to know the details of a specific external library. The framework can encapsulate the implementation and expose coherent operations. Thus it is possible to replace or expand algorithms without changing the logic of application.

13. Cipher

The Cipher group groups cryptography-related functionality. In a modern application, data protection may be essential. Hashes may be needed to verify integrity, symmetric encryption to protect local information, signature mechanisms, key generation, encoding or support secure communications.

Cryptography should be used with care. It is not about inventing own algorithms, but about properly encapsulating reliable operations and offering them consistently to the rest of the framework. Cipher can collaborate with DIO when transmitting data, with Compression when packaging resources, with Databases when storing sensitive data or with XUtilities when manipulating buffers and strings.

From the design point of view, Cipher is a good example of a cross-sectional group: it does not belong only to network, files, or databases. It can be used by all these modules. That is why it must be well separated and expose a clear interface.

14. AppFlow

AppFlow represents an important idea: an application is not just a collection of functions. It has a flow. It initiates resources, configures subsystems, changes status, processes events, executes tasks, responds to errors and ends in a controlled way. In small programs, all this can be within main. In real applications, that organization becomes insufficient.

The AppFlow group helps to structure the behavior of the application. It can include concepts such as states, transitions, processes, tasks, status machines, execution control or coordination between modules. Its aim is for the application to have a clear internal architecture, not an improvised call sequence.

AppFlow is related to many parts of the framework. You can start graphics, open communications, load settings, activate scripts, respond to input events or manage life cycles. Its utility increases as the application grows.

In the context of GEN Framework, AppFlow reinforces the idea of its own architecture. It is not just about having classes to open files or draw. It is about having an orderly way to build complete applications.

15. Scripts

The Scripts group allows you to integrate scripting languages or embedded engines into a C/C++ application. This possibility is very useful when we want part of the behavior to be configurable or extensible without recompiling the entire program.

A script engine can be used for automation, business rules, prototyping, advanced configuration, testing, content generation or customization by the user. Instead of encoding everything into C++, some parts may be delegated to scripts that the application loads and executes at runtime.

The scripting also helps to separate levels of responsibility. The high-performance core can remain in C/C++, while certain changing rules can be expressed in a more dynamic language. This combination can be very powerful if designed carefully.

From the framework point of view, Scripts needs to communicate with other modules. It must be able to read files, access data, produce traces, call functions of the framework or receive events. That is why it must be integrated without breaking the general architecture.

16. UserInterface

UserInterface groups the user interface elements. If GRPFACTORY provides the graphical base and INPFACTORY captures input events, UserInterface builds on those layers concepts closer to the user: windows, controls, buttons, lists, text boxes, dialogues, menus, panels, themes or screen structures.

A multiplatform user interface is one of the most complex challenges of a framework. It is not enough to draw shapes. It is necessary to manage events, visual states, focus, navigation, interaction, scaling, sources, resources, themes and behavior. In addition, each platform has its own conventions.

GEN Framework can address this complexity by creating a layer of its own interface supported by graphics and input. Thus, an application can build screens without relying directly on native controls of each system. This does not prevent specific integrations when necessary, but allows the interface logic to have a common basis.

UserInterface is a clear example of modular collaboration. It uses graphics to represent, input to interact, resources to load images or styles, traces to diagnose, AppFlow to coordinate states and possibly Scripts for customization or dynamic behavior.

17. Databases

The Databases group provides database-related functionality and structured persistence. Many applications need to store information, consult data, maintain complex configurations, record events, or work with persistent models.

A layer of databases may include connections, queries, results, transactions, adapters, specific engines or abstractions on SQL. As always, the key is to avoid the application unnecessarily relying on internal details of a specific library or engine when the framework can provide a common interface.

Databases is related to DIO, because a database is a form of data input and output. It also relates to Cipher if you need to protect information, to Compression if packages are stored, to Scripts if dynamic rules are executed, and to AppFlow if the application has processes that depend on persistent data.

Not all applications need databases, but when they need them, having an integrated layer in the framework helps maintain consistency with the rest of the system.

18. Relationship between modules

One useful way to understand GEN Framework is to imagine it as a series of layers. Common utilities and platform abstractions are found at the base. On them appear the input/output modules, graphics, input, sound and cross-sectional services. On top of everything is the application.

The application should try to rely on the top layers of the framework, not directly from the platform. When you need to draw, use the graphical layer. When you need to read data, use DIO. When you need user events, use Input. When you need common utilities, use XUtilities. When you need compression, encryption, scripts or databases, go to the corresponding group.

This organization not only improves portability. It also improves the design of the program itself. The developer learns to separate responsibilities. A business module should not directly open a window. An interface class should not implement a communication protocol. A driver should not know details of a screen. Each part should be in place.

The framework does not prevent the programmer from writing bad code, but it provides an architecture that invites you to organize it better. That is one of the reasons why GEN Framework should be studied as a complete system and not as a collection of loose functions.

19. How the programmer should approach this architecture

For a new user, the number of modules may seem great. The best way to learn GEN Framework is not to try to memorize the whole architecture from day one. It is appropriate to build a progressive vision.

It is first important to understand the base utilities and how the framework represents strings, buffers, routes, dates, traces and common types. Then you can study DIO, because almost any application needs to read or write data. Then it makes sense to enter graphics, input and user interface if you are going to create visual applications. Later you can incorporate sound, compression, encryption, scripts or databases according to the needs of the project.

It is also important to learn to recognize when a class belongs to the common world and when it belongs to the platform world. If a class is in Platform or depends on a particular system, it should not normally be used directly from an application unless there is a clear reason. The application should be based on the higher abstraction.

This criterion helps maintain the main objective: to write an application once and compile it on multiple platforms without touching the main logic. Every time an application introduces a direct call to a native API, that portability is reduced. Every time it uses a common factory or abstraction, it is reinforced.

20. Summary of architecture

GEN Framework is organized around a modular architecture designed to develop multiplatform C/C++ applications. Its core is not in a single class or in a single library, but in the combination of functional modules, factories and platform insulation.

XFACTORY provides base utilities. DIOFACTORY manages data input and output. GRPFACTORY organizes the graphical part. INPFACTORY abstracts the user input. SNDFACTORY covers sound. Around it, groups such as Compression, Cipher, AppFlow, Scripts, UserInterface and Databases expand the capabilities of the framework to build real applications.

Factories not only create objects. They also allow to isolate the application from the specific implementations of each platform. System-dependent code should remain confined to Platform, while the application works with common abstractions.

This architecture is the basis of the rest of the book. Each area is studied in more detail, starting with the fundamental utilities and moving towards complete applications.

Chapter IV: First application with GEN Framework

1. The objective of the chapter

After preparing the development environment and getting to know the general architecture of the framework, the next natural step is to build a first application. In GEN Framework you do not start by writing a main function of C++ or manually invoking a main loop. That part is embedded within the framework itself and is part of its boot, run and close infrastructure.

This decision is important. If the programmer decides to write his own main and call GEN Framework loose classes from it, technically he can do so, but in that case he would be using GEN Framework as a set of libraries, not as a framework. The difference is not only terminological. When GEN Framework is used as a framework, the application integrates into his life cycle, his route system, his configuration, his factories, his trace control, his platform model and his uniform way of organizing resources.

Therefore, this chapter is not part of a main written by the user. It describes the real skeleton that follows the examples of the repository and the recommended way to create a new application. The main practical reference is the inibase example, located within the console examples, because it contains a structure sufficiently complete to understand initialization, update, completion, configuration and state machine.

2. The three main modalities of implementation

GEN Framework allows you to work with different integration levels. This flexibility is important because not all applications need the same infrastructure. A minimal utility, a firmware for a microcontroller, a console tool, a graphical desktop application or an interface embedded on a small display do not have the same needs.

Practically, we can distinguish three main development modalities within GEN Framework: base use without the full AppFlow application modality, console application with AppFlow and graphical application with AppFlow. This classification matches the organization of the examples: the Base directory groups basic use examples from the framework, Console groups console applications based on AppFlow, and Graphics groups graphical applications based on AppFlowGraphics.

Modality	Sample directory	Standard class/focus	Typical use
Basic use of the framework	Examples/Base	Without relying on the console or graphics AppFlow mode; more direct use of GEN Frameworkservices.	Minimum examples, module testing, integration into small systems or environments where you want to control the flow more.
Console application	Examples/Console	APPFLOWCONSOLE and associated AppFlow modules.	Console tools, services, background processes, network utilities or applications without a graphical window.
Graphic application	Examples/Graphics	APPFLOWGRAPHICS, input, screen, canvas, views and optionally UserInterface.	Visual applications, user interfaces, graphic examples, embedded screens and cross-platform applications with graphical output.

This separation does not prevent more specific combinations. An AppFlow-free application can continue to be console or graphic if the environment requires it. For example, in microprocessors or embedded systems it may be interesting not to use the full application infrastructure and work directly with low-level modules, even if there is a graphical output to a small SPI or I2C display. In this case, attempts are made to reduce dependencies, memory and complexity.

The key idea is that the chosen modality determines how much of the life cycle assumes GEN Framework for us. The more we use AppFlow, the more integrated we are in the framework: configuration, traces, routes, resources, console or graphics are organized in a homogeneous way. The less we use AppFlow, the more responsibility remains in the application code.

3. Why don't we write the main function

In many C++ books the first application starts with `int main()`. In GEN Framework it is not so. The real main is encapsulated within the framework infrastructure. This allows the boot to be homogeneous on all supported platforms, that the differences between Windows, Linux, Android or embedded systems are resolved internally and that the application only defines the class that represents its behavior.

The usual mechanism for integrated applications in the framework is to declare an application instance using a macro of creation. In `inibase` the pattern appears:

```
APPLICATIONCREATEINSTANCE(INIBASE, inibase)
```

The user does not implement main, but an application class. The framework creates the instance, calls its life-cycle methods, and coordinates the main process. This way of working is one of the differences between using GEN Framework as a framework and simply using it as a set of libraries.

This philosophy is also observed in the base use examples. Even when the full console or graphics AppFlow modality is not used, the code is structured around initialization, update and closure functions managed by the GEN Framework infrastructure. The application defines what should happen, but does not need to rewrite the native input of each platform.

4. The life cycle of an application with GEN Framework

An application with GEN Framework is organized around a clear life cycle. This cycle appears in the examples and becomes a design discipline: initialize resources, run periodic updates and correctly release everything that has been created.

Phase	Usual method	Responsibility
Initialisation	<code>AppProc_Ini()</code>	Create routes, prepare settings, initialize traces, register states, create timers, prepare services, and leave the app ready to start.
First update	<code>AppProc_FirstUpdate()</code>	Run tasks that must be done once when the environment is already started: get devices, prepare input, show headers, load system dependent items.
Update	<code>AppProc_Update()</code>	Run the application's regular work: main logic, current status, input, render, communications, timers and state machine.
Last update	<code>AppProc_LastUpdate()</code>	Available in some modalities for tasks immediately prior to closing or final

		transition.
Termination	AppProc_End()	Close services, release objects created with factories, save status, stop extended modules and destroy configuration singletons.

Not all applications have to use all phases with the same intensity. A console application can concentrate almost all its logic on AppProc_Update. A graphical application can divide the work between screen initialization, input, canvas, user interface and drawing. A minimal application can use a simpler pattern. The important thing is to keep the criterion: everything that is created must have a clear place to initialize and a clear place to finish.

5. Base use without console or graphics AppFlow mode

The examples in the Base directory are useful to understand the most direct use of GEN Framework. In this mode we are not building a console application with APPFLOWCONSOLE or a graphical application with APPFLOWGRAPHICS. The basic infrastructure of the framework and the necessary modules is used, but the application cycle is lighter.

This mode is useful when you want to test a class, validate a functionality, create a very small utility or integrate GEN Framework into an environment where you don't want to load the entire AppFlow infrastructure. It is also common for microprocessors or embedded systems, where memory costs, number of dependencies and main loop control can be more important than the convenience of a complete application.

In these scenarios there may be output per console, or even graphic output to a small display connected by SPI or I2C, but that does not necessarily imply using AppFlowGraphics. You can work with specific modules of the framework and with the minimum necessary for the platform.

This mode should be used with criteria. It is more flexible, but also leaves more decisions to the programmer. When you want a complete application, with main configuration, logs, traces, resource paths, events and homogeneous cycle, it is normal to use AppFlow.

6. Console application with AppFlow

The console mode is supported by APPFLOWCONSOLE. It is the right option for tools without graphical interface, services, background processes, communication tests, network utilities, examples of databases or applications that interact via keyboard and text output.

The example inibase belongs to this family. Its main class inherits from APPFLOWCONSOLE and XFSMACHINE. This indicates two things: on the one hand, it integrates into the GEN Frameworkconsole application cycle; on the other, it organizes its internal logic with a finite state machine.

```
class INIBASE : public APPFLOWCONSOLE, public XOBSERVER, public XFSMACHINE
{
public:
    bool AppProc_Ini();
    bool AppProc_FirstUpdate();
    bool AppProc_Update();
    bool AppProc_End();

    bool InitFSMachine();
};
```

The application does not define main. It defines a class, an application instance and the methods that GEN Framework calls at the right time. This allows the main flow to be homogeneous and the platform differences to remain within the framework.

7. Finite Status Machine with XFSMACHINE

One of the most important patterns of the inibase example is the use of XFSMACHINE. The finite state machine allows you to clearly separate in what state the application is in, what events can cause state changes and what logic should be run continuously while remaining in a specific state.

To use XFSMACHINE it is not enough to create the class. First you have to define two enumerations: one for events and one for states. Events represent what can happen. States represent the stable situations in which the application can be found.

```
enum INIBASEXFSMEVENTS
{
    INIBASE_XFSMEVENT_NONE = 0,
    INIBASE_XFSMEVENT_INI,
    INIBASE_XFSMEVENT_UPDATE,
    INIBASE_XFSMEVENT_END,

    INIBASE_LASTEVENT
};

enum INIBASEXFSMSTATES
{
    INIBASE_XFSMSTATE_NONE = 0,
    INIBASE_XFSMSTATE_INI,
    INIBASE_XFSMSTATE_UPDATE,
    INIBASE_XFSMSTATE_END,

    INIBASE_LASTSTATE
};
```

Then, at the beginning of the application, usually during AppProc_Ini, states and events are related through InitFSMachine. Each call to AddState states which events a state accepts and which new state each event leads to.

```
bool INIBASE::InitFSMachine()
{
    AddState(INIBASE_XFSMSTATE_NONE,
        INIBASE_XFSMEVENT_INI, INIBASE_XFSMSTATE_INI,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND);

    AddState(INIBASE_XFSMSTATE_INI,
        INIBASE_XFSMEVENT_UPDATE, INIBASE_XFSMSTATE_UPDATE,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND);

    AddState(INIBASE_XFSMSTATE_UPDATE,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND);

    return true;
}
```

The most interesting point appears in `AppProc_Update`. The pattern used by GEN Framework divides the logic into two parts. The first part is executed when there is no new event. In that case, the application remains in the current state and executes the continuous logic corresponding to that state. The second part is executed when a new event exists. In that case the transition is checked, the state is changed if applicable and the logic associated with the change of status is executed.

```
bool INIBASE::AppProc_Update()
{
    if(GetEvent() == INIBASE_XFSMEVENT_NONE)
    {
        switch(GetCurrentState())
        {
            case INIBASE_XFSMSTATE_UPDATE:
                // Logic continues while the application is in UPDATE.
                break;

            default:
                break;
        }
    }
    else
    {
        if(GetEvent() < INIBASE_LASTEVENT)
        {
            CheckTransition();

            switch(GetCurrentState())
            {
                case INIBASE_XFSMSTATE_INI:
                    SetEvent(INIBASE_XFSMEVENT_UPDATE);
                    break;

                default:
                    break;
            }
        }
    }

    return true;
}
```

This separation gives the state machine a special grace. It does not mix the permanent work of a state with the point logic that should only be executed when a transition occurs. In a state we can read keyboard, update communications, refresh status or check timers. In a transition we can initialize something once, prepare the next state or decide whether the flow should end.

In `inibase`, for example, the `INI` event is set to start. The machine moves to the initialization state and then the `UPDATE` event is launched. From there, the `UPDATE` state contains the periodic work of the application. When the user requests to leave, the output type is set and the application ends in an orderly manner.

8. Typical structure of a console application

An AppFlow-based console application is usually organized with a simple structure: a font folder, a CMake folder, and an asset folder for all resources that are not the executable.

```
MiAplicacion/
```

```

|
├── assets/
│   ├── miaplicacion.ini
│   ├── miaplicacion.log
│   └── ...
├── CMake/
│   ├── CMakeLists.txt
│   └── CMakePresets.json
└── Sources/
    ├── MiAplicacion.cpp
    ├── MiAplicacion.h
    ├── MiAplicacion_CFG.cpp
    ├── MiAplicacion_CFG.h
    └── APP_GEN_Defines.h

```

All resource files in the application, except the generated executable, must be placed under assets. This includes configuration, logs, languages, graphics, fonts, layouts, local databases, Android packages or other resources that the application needs in running time. This rule makes it easy for XPATHMANAGER to sort and locate resources in a homogeneous manner.

9. XPATHMANAGER and folder assets

XPATHMANAGER aims to keep all resource files in the application sorted and accessible. Instead of dispersing absolute paths by code, the application works with path sections. Each section represents a type of resource: root, graphics, fonts, interface layouts, databases or other groups that the application needs.

The asset directory acts as the root of the resources. Within assets, the default directories defined by XPATHMANAGER and the active modules of the application are used. For example, in graphical applications it is common to find subdirectories such as graphics, fonts or ui_layouts. In database applications there may be databases. In Android applications there may be a package. In addition, the main INI file and logs are usually located directly within assets.

```

assets/
|
├── miaplicacion.ini
├── miaplicacion.log
├── miaplicacion.lng
|
├── graphics/
├── fonts/
├── ui_layouts/
├── databases/
└── package/

```

Not all applications need all these directories. A simple console can only have the INI and the log. A graphical application with UserInterface needs layouts, fonts and graphical resources. A database application can add databases. The important thing is that they all follow the same criteria: resources are grouped into assets and accessed through XPATHMANAGER.

In the initialization the root path is adjusted and the necessary sections are created. In examples as inibase you can see the use of the default root:

```

GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();

```

In graphic applications, in addition, specific sections for visual resources are added:

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_FONTS, APPFLOW_DEFAULT_DIRECTORY_FONTS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, APPFLOW_DEFAULT_DIRECTORY_UI_LAYOUTS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

The default names can be changed if the application needs it. New path sections for own resources can also be created. However, it is advisable to start by respecting the default organization because it makes all applications with GEN Framework behave similarly and easier to maintain.

10. Main application configuration

A real application needs configuration. GEN Framework solves this need by means of a main file in INI format and a specific configuration module per application. Instead of reading the INI directly from any part of the code, a class of its own is created, usually called [APLICACION]_CFG, which inherits APPFLOWCFG.

In inibase, for example, there are IniBase_CFG.cpp and IniBase_CFG.h files. In an application called MyApp, it would normally be to create MiAplicacion_CFG.cpp and MiAplicacion_CFG.h. This module represents the main configuration of the application and centralizes the entries in the INI file.

```
Sources/
├─ MiAplicacion.cpp
├─ MiAplicacion.h
├─ MiAplicacion_CFG.cpp
└─ MiAplicacion_CFG.h

assets/
└─ miaplicacion.ini
```

APPFLOWCFG relies on XFILECFG to manage the configuration file. XFILECFG provides INI file support, sections, keys, value types, default values and file maintenance. APPFLOWCFG adds the common sections of the application flow: general configuration, logs, traces, Internet services, hardware resources or other areas activated by CMake.

The configuration class of the application usually functions as singleton. To do this, a macro is defined in the CMakeLists.txt indicating which is the main instance of configuration that will be used by APPFLOW. A pattern like this appears in inibase:

```
set(APP_CFG_INIBASE "-DAPPFLOW_CFG=INIBASE_CFG::GetInstance()")
add_definitions(${APP_CFG_INIBASE})
```

This definition makes the common code used by APPFLOW_CFG point to the application's specific instance of configuration. Thus, the generic modules of AppFlow can work with a common configuration, but each program maintains its own configuration class.

11. Configuration variables mapping

The main configuration should not be treated as a scatter string dictionary. The recommended way is to map each INI input against class member variables [APLICACION]_CFG. Then, the rest of the application accesses those values using getters and setters. This gives typing, centralization and control.

The configuration class usually implements two main methods: DoVariableMapping and DoDefault. DoVariableMapping declares which entries exist in the configuration file and which internal variables are associated with. DoDefault assigns the initial or default values.

```
class MIAPLICACION_CFG : public APPFLOWCFG
{
```

```

public:
    static MIAPLICACION_CFG& GetInstance(bool ini = true);
    static bool DelInstance();

    bool DoVariableMapping();
    bool DoDefault();

    bool GetServerActive();
    void SetServerActive(bool value);

private:
    bool server_active;
};

```

The mapping is done within `DoVariableMapping`. First, the mapping of `APPFLOWCFG` is called to preserve the common inputs in the framework. Then the application's own entries are added.

```

bool MIAPLICACION_CFG::DoVariableMapping()
{
    if(!APPFLOWCFG::DoVariableMapping())
    {
        return false;
    }

    AddValue(XFILECFG_VALUETYPE_BOOLEAN,
        __L("General"),
        __L("server_active"),
        &server_active,
        __L("Enables server mode"),
        APPFLOW_CFG_DEFAULT_REMARK_COLUMN);

    return true;
}

```

The `DoDefault` method loads the default values. You must also call `APPFLOWCFG::DoDefault` first to avoid losing the common values in the framework.

```

bool MIAPLICACION_CFG::DoDefault()
{
    if(!APPFLOWCFG::DoDefault())
    {
        return false;
    }

    server_active = false;
    return true;
}

```

This system has a very important advantage: the configuration file is automatically updated. If an entry exists in the INI but is no longer mapped in `DoVariableMapping`, it is considered obsolete and can be deleted from the file due to non-use. In this way, the configuration does not accumulate old, renamed or abandoned keys. The main file is kept clean and synchronized with the real configuration class.

In other words: the INI does not command over the code. The valid contract is in the mapping of the configuration class. What is mapped is preserved, updated and written with its values. What is no longer mapped ceases to be part of the active configuration.

12. Relationship between AppFlow, Configuration and Traces

During AppProc_Ini the application sets name, version, paths and traces. The automatic trace targets defined by the configuration are then loaded or prepared and the extended module of AppFlow is started. A conceptual sequence of this type is observed in inibase:

```
Application_GetName()->AddFormat(__L("%s"), APPLICATION_NAMEAPP);

GEN_SET_VERSION(Application_GetName()->Get(),
    APPLICATION_NAMEFILE,
    APPLICATION_VERSION,
    APPLICATION_SUBVERSION,
    APPLICATION_SUBVERSIONERR,
    APPLICATION_OWNER,
    APPLICATION_YEAROFCREATION)

XTRACE_SETAPPLICATIONNAME((*Application_GetName()));
XTRACE_SETAPPLICATIONVERSION(APPLICATION_VERSION, APPLICATION_SUBVERSION, APPLICATION_SUBVERSIONERR);

APPFLOW_CFG_SETAUTOMATICTRACETARGETS
APPFLOW_EXTENDED.APPStart(&APPFLOW_CFG, this);
```

The objective is that the application does not have to manually reconstruct all common services. The main configuration decides which traces are activated, which sections of log are used, which services are enabled and how part of the environment behaves. The application code consumes that configuration through the class [APLICACION]_CFG.

13. CMakeLists.txt minimum for a console application

An application with GEN Framework requires a CMakeLists.txt that activates the necessary features and adds the application sources. The example inibase activates AppFlow, console, traces, memory, DIO and other features. For a minimal application, the CMakeLists.txt would have the same philosophy: select options, define the configuration macro, include GEN_Main.cmake, add directories and fonts, and end up with GEN_End.cmake.

```
cmake_minimum_required(VERSION 3.16.0)
project(miaplicacion)

set(GEN_DIRECTORY "../.../GEN_FrameWork")

option(DEBUG_FEATURE           "Debug mode"           ON)
option(XMEMORY_CONTROL_FEATURE "XMemory Control"      ON)
option(XTRACE_FEATURE          "XTrace"               ON)
option(APPFLOW_FEATURE         "Application Flow"      ON)
option(APPFLOW_CONSOLE_FEATURE "Application Flow Console" ON)
option(APPFLOW_CFG_GENERAL_FEATURE "Application CFG General" ON)
option(APPFLOW_LOG_FEATURE     "Application Flow Log"  ON)

set(APP_CFG_MIAPLICACION "-DAPPFLOW_CFG=MIAPLICACION_CFG::GetInstance()")
add_definitions(${APP_CFG_MIAPLICACION})

include("${GEN_DIRECTORY}/Common/CMake/GEN_Main.cmake")

set(APP_DIRECTORY_APPLICATION "../Sources")
list(APPEND GEN_INCLUDES_DIR_LIST ${APP_DIRECTORY_APPLICATION})
```

```
list(APPEND GEN_SOURCES_MODULES_LIST "${APP_DIRECTORY_APPLICATION}/MiAplicacion.cpp")
list(APPEND GEN_SOURCES_MODULES_LIST "${APP_DIRECTORY_APPLICATION}/MiAplicacion_CFG.cpp")

include("${GEN_DIRECTORY}/Common/CMake/GEN_End.cmake")
```

The real CMakeLists.txt can activate more modules as needed: DIO, Cipher, Compression, Databases, Scripts, Graphics or UserInterface. The important thing in this chapter is to understand the pattern: the application is not built in isolation, but integrated into the GEN Framework build system.

14. Graphic application with AppFlowGraphics

The graphical mode is supported by APPFLOWGRAPHICS. It is the basis for applications with screen, input, canvas, viewports, render and, optionally, UserInterface. Examples from the Graphics directory show this approach.

A typical graphic class inherits from APPFLOWGRAPHICS and you can add XFSMACHINE if you want to organize your logic through states. The example UI_Message follows this model.

```
class UI_MESSAGE : public APPFLOWGRAPHICS, public XFSMACHINE
{
public:
    bool AppProc_Ini();
    bool AppProc_FirstUpdate();
    bool AppProc_Update();
    bool AppProc_End();

private:
    bool UpdateInput();
    bool Ini_Graphics(GRPSCREEN* screen);
    bool Ini_Canvas(GRPSCREEN* screen);
    bool Ini_UserInterface(bool on);
    bool DrawFrame();
};
```

In AppProc_Ini, initial graphical flow options are usually set. For example, create main screen, display it, and enable input:

```
SetInitOptions(APPFLOWGRAPHICS_INIOPTION_CREATEMAINSCREEN |
               APPFLOWGRAPHICS_INIOPTION_SHOWMAINSCREEN  |
               APPFLOWGRAPHICS_INIOPTION_INPUT);
```

Graphic resource paths are also prepared using XPATHMANAGER. The root is still in assets, but sections for graphics, fonts and interface layouts are added.

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_FONTS, APPFLOW_DEFAULT_DIRECTORY_FONTS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, APPFLOW_DEFAULT_DIRECTORY_UI_LAYOUTS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

The graphical application divides the update work between input, drawing and screen update. In the main run state devices are read, the frame is drawn and views are updated. The status machine is used in the same way as in console: a continuous part when there is no event and a point part when a transition occurs.

15. Add UserInterface to a graphical application

UserInterface is a top layer over graphical modules. It allows you to load layouts, manage visual elements, receive input events and respond to user actions. Not every graphical application needs UserInterface. An application can

draw directly on a canvas. But when you want buttons, messages, options, panels or more structured views, `UserInterface` provides a more comfortable organization.

`UserInterface` layouts are normally placed within `assets/ui_layouts`. In `UI_Message`, for example, the path of that section is obtained and a packaged layout file is loaded:

```
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, xpath);
xpath.Slash_Add();
xpath.Add(__L("message.zip"));

GEN_USERINTERFACE.Load(xpath, screen, 0);
GEN_USERINTERFACE.SubscribeInputEvents(true);
GEN_USERINTERFACE.SubscribeOutputEvents(true, this, &GEN_USERINTERFACE.GetInstance());
```

The initialization of `UserInterface` must also have its corresponding closure. At the end of the application events are unsubscribed, the instance is destroyed and associated resources are released. This follows the general rule of the framework: any resource created in the initialization must have a clear closure in `AppProc_End`.

```
GEN_USERINTERFACE.SubscribeOutputEvents(false, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.SubscribeInputEvents(false);
GEN_USERINTERFACE.DeleteInstance();
```

16. Typical structure of a graphical application

A graphical application has a structure similar to a console application, but its `assets` folder usually contains more subdirectories. In addition to the `INI` and the `log`, `graphics`, `fonts`, `interface layouts` and, if compiled for `Android`, `resource package` appear.

```
MiAplicacionGrafica/
|
├── assets/
|   ├── miaplicaciongrafica.ini
|   ├── miaplicaciongrafica.log
|   ├── miaplicaciongrafica.lng
|   ├── graphics/
|   ├── fonts/
|   ├── ui_layouts/
|   └── package/
|
├── CMake/
|   ├── CMakeLists.txt
|   └── CMakePresets.json
|
└── Sources/
    ├── MiAplicacionGrafica.cpp
    ├── MiAplicacionGrafica.h
    ├── MiAplicacionGrafica_CFG.cpp
    ├── MiAplicacionGrafica_CFG.h
    └── APP_GEN_Defines.h
```

The `CMakeLists.txt` of a graphical application activates additional modules: `graphics`, `input`, `AppFlowGraphics` and, if applicable, `UserInterface`. The idea does not change: the application declares what you need and `GEN_Main.cmake` / `GEN_End.cmake` integrates those needs into the general construction system.

17. Recommended Skeleton of a First Application

If you want to create a first application with GEN Framework, the recommended path is to start from an existing example. For console, `inibase` is a good starting point. For graphics, `UI_Message`, `UI_Options` or `Canvas2D` allow you to see different levels of complexity. It is not advisable to start by writing all files from scratch without looking at the examples, because the examples already contain the right integration decisions: instance macro, configuration, routes, CMake and life cycle.

Element	Console application	Graphic application
Base class	APPFLOWCONSOLE	APPFLOWGRAPHICS
Internal State	XFSMACHINE if state flow is required	XFSMACHINE recommended to separate initialization, execution and closure
Resources	assets with INI, log and resource requirements	assets with INI, log, graphics, fonts, ui_layouts and package if applicable
Settings	[APLICACION]_CFG inheriting from APPFLOWCFG	[APLICACION]_CFG inheriting from APPFLOWCFG
Entry	Console Keyboard using APPFLOWCONSOLE	INPFACTORY / GEN_INPMANAGER, keyboard, mouse, touch or other devices
Output	Text, logs and traces	Display, canvas, viewports and optionally UserInterface
Reference example	Examples/Console/IniBase	Examples/Graphics/UI_Message, UI_Options or Canvas2D

The recommended work order would be: copy the structure of a nearby example, rename the application, adjust `APPLICATION_NAMEAPP` and `APPLICATION_NAMEFILE`, create the class `[APLICACION]_CFG`, define the macro `APPFLOW_CFG` in `CMakeLists.txt`, review assets, compile with Common scripts and then start modifying the logic of `AppProc_Ini`, `AppProc_Update` and `AppProc_End`.

18. Why this skeleton matters

The skeleton of an application with GEN Framework is not a formality. It defines the way in which the application is integrated with the framework. When we respect this structure, we get uniform initialization, centralized configuration, orderly resources, homogeneous traces, platform isolation and a clear way to maintain the code.

The alternative of writing one's own main and calling one's own classes may seem more direct at first, but it breaks some of the framework philosophy. Integration with the life cycle is lost, configuration is dispersed, platform decisions are duplicated and the programmer is forced to manually solve problems that GEN Framework already has organized.

Therefore the first application should not be seen as a simple test program. It should be seen as a mental template. An application with GEN Framework is defined by its main class, its life cycle, its state machine if applicable, its main configuration, its assets resources and its integration with CMake. From that pattern you can grow into more complex applications without losing order.

The following chapters delve into each part: configuration, traces, routes, AppFlow, graphics, input, UserInterface and service modules. The important thing at this point is to be clear about the skeleton: GEN Framework provides infrastructure and the application provides logic.

Chapter V: XUtils, the extended utilities of the framework

1. The Origin of XUtils

XUtils means eXtended Utilities. The name sums up very well its role within GEN Framework: it is not an accidental collection of auxiliary functions, but the layer of extended utilities on which the framework began to form. Before GEN Framework had the current form, before there were graphic, input, sound, communications or AppFlow factories, there was already a need to have a common basis to work with strings, buffers, files, paths, dates, traces, timers, containers, configuration and small system abstractions.

That practical origin is important. Many C/C++ programmers end up creating a utility folder that travels from project to project. At first they are loose functions. Then there are classes to not repeat code. Later those classes need a coherent way to initialize, release resources, work with Unicode, register errors, access the file system or compile on several platforms. XUtils is born exactly in that field: as the part of the framework that turns those recurring utilities into a stable, common and reusable base.

That's why XUtils should be understood as the GEN Frameworkgerm. It's not the most spectacular module in the framework, but it's one of the most important. Any application will need strings, routes, buffers, time, files, lists, maps, traces or configuration. If each application solves those problems in its own way, the framework will lose homogeneity. XUtils avoids that dispersion and provides a common way of working.

In practice, when an application is developed with GEN Framework, XUtils appears continuously. A path is expressed using XPATH. A text is manipulated with XSTRING. A binary block is saved in XBUFFER. A date is obtained with XDATETIME. A timer is created with XTIMER. A list of objects can be arranged with XVECTOR, XLIST, XMAP or XTREE. A configuration is read through XFILECFG or XFILEINI. A trace is emitted by XTRACE. This constant presence makes XUtils the basic vocabulary of the framework.

2. XUses as common framework basis

The main function of XUtils is to create a homogeneous working layer for the rest of modules. DataIO should not invent its own type of string. Graphic should not use a different class for routes. AppFlow should not have a different way of measuring times. Cipher should not define another type of binary buffer. If each group did that, GEN Framework would end up being a sum of independent libraries. XUtils allows all modules to speak the same language.

This common language is based on several principles. The first is portability: classes must be able to compile on different systems with as few changes as possible. The second is API stability: public modules use GEN Framework types such as XCHAR, XSTRING, XBUFFER, XDWORD, XPATH or XVECTOR. The third is control: C/C++ style of the framework is maintained, without completely concealing resource management. The fourth is integration: XUtils classes are designed to be used among themselves and by the rest of groups.

XUtils does not replace the standard C++ library. GEN Framework can be supported internally by language or system mechanisms, but the visible interface of the framework uses its own types and patterns. This decision may seem conservative, but it brings a clear advantage: an application with GEN Framework maintains a coherent aspect, regardless of whether an internal platform uses a API of Windows, a call POSIX, a function of the C language, an external library or an implementation of its own.

3. XFACTORY and platform insulation

Within XUtils, XFACTORY is a fundamental piece. Its goal is to create and destroy those instances of XUtils whose implementation may depend on the platform. The programmer should not manually decide if you need a

Windows timer, Linux, Android or a microcontroller. The application asks for a XTIMER to GEN_XFACTORY and the framework returns the proper implementation for the compiled platform.

This pattern follows the general philosophy of GEN Framework. The application works with a common interface. The specific code is confined to the platform modules. When a class needs a different implementation in Windows, Linux, Android or an embedded environment, the factory acts as a boundary. The result is that the application code is not filled with `#ifdef` or native calls.

In XFACTORY we find the creation of objects such as XTIMER, XDATETIME, XRAND, XFILE, XDIR, XCONSOLE, XMUTEX and XTHREAD. Some of them seem simple, but they are not when you want to maintain homogeneous behavior between platforms. Sleeping a thread, measuring time, creating a mutex, opening a file, or listing a directory is not implemented the same on all systems. XFACTORY prevents that difference from spreading upwards.

```
XTIMER* xtimer = GEN_XFACTORY.CreateTimer();
if(!xtimer) return false;

xtimer->Reset();

// ... application work ...

if(xtimer->GetMeasureSeconds() >= 1)
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("one second"));
}

GEN_XFACTORY.DeleteTimer(xtimer);
xtimer = NULL;
```

The practical rule is simple: when a class of XUtils has a possible platform dependency and there is a creation method in XFACTORY, the factory must be used. Creating a specific class directly can work in a local case, but it breaks the architecture. Factory is not a convenience: it is an isolation barrier.

4. General Map of XUtils

Before reviewing the classes, you should look at XUtils as a set of functional groups. Not all classes have the same weight or type of use. Some are basic types used constantly. Others are collection templates. Others are responsible for files and formats. Others provide events, traces, synchronization, serialization or platform support. The following table summarizes this conceptual organization.

Element	Description, relationship and example
Basis and fundamental types	XBase, XString, XFString, XBuffer, XBufferBits, XPath, XVariant, XRect, XUUID. These are the pieces that appear in almost any application with GEN Framework.
Collections and structures	XVector, XVectorSTL, XMap, XPair, XList, XStack, XTree, XAVLTree, XBinaryMap, XDictionary, XOrderedList, XInheritEnum. They allow to organize data in the style of the framework.
System, time and platform	XFactory, XTimer, XDateTime, XRand, XDir, XFile, XConsole, XSystem, XProcessManager, XSharedMemoryManager, XSleep, XThread and XMUTEX using the factory.
Files and formats	XFile, XFileTXT, XFileINI, XFileCFG, XFileCSV, XFileJSON, XFileXML, XFileRIFF, XFileZIP, XFileHEX, XFileDFU, XFileDBF, XFileCobol, XFileHash and other auxiliary formats.
Events and publishing/observer model	XEvent, XSubject, XObserver, XPublisher and specific events such as XBER_XEVENT, XFILEZIP_XEVENT, XLICENSE_XEVENT, XSCHEDULER_XEVENT or XDRIVEIMAGEMANAGER_XEVENT.
Diagnosis and control	XTrace, XLog, XMemory_Control, XFeedback_Control, XErrorMsg. They help diagnose, record, control memory, and communicate states.

Concurrency and planning	XThread, XThreadCollected, XThreadListNonPreemptive, XSchedular, XSchedular_XEvent and XSleep. They provide a base for tasks, threads and timing.
Serialization and codification	XSerializable, XSerializationMethod, XSerializationMethodBinary, XSerializationMethodJSON, XASN1, XBER, XMPInteger. They serve to convert objects and data to transportable formats.
Language, translation and licences	XTranslation, XTranslation_GEN, XLanguage_ISO_639_3, XLicense, XLicense_XEvent. They cover transducible texts, language codes and license control.
Assistant modules	XSTL, XFactoryMacros, XFileID, XFileHashID, XEEPROMMemoryManager and XDriveImageManager. They are support pieces to integrate specific functionalities.

5. Basic types, chains, buffers and routes

The first group of XUtils contains classes that are used transversally. They are so common that they are often missed as individual modules. XSTRING appears in configuration, logs, paths, protocols and messages. XBUFFER appears in binary files, encryption, communications and serialization. XPATH appears in any disk access. XVARIANT allows transporting variable-type values. This layer is the equivalent of GEN Framework's "alphabet".

```
XSTRING name;
XBUFFER buffer;
XPATH xpath;

name.Format(__L("%s_%02d"), __L("test"), 1);
name.ConvertToASCII(buffer, XSTRINGASCII_CODEPAGE_437);

xpath.Set(__L("assets"));
xpath.Slash_Add();
xpath.Add(__L("config.ini"));
```

Element	Description, relationship and example
XBASE	Defines shared basic functions and elements. Relationship: Almost all XUtils depend directly or indirectly on XBase. Example: Use the basic types and macros of the framework before building higher modules.
XSTRING	GEN Framework Main Unicode String. Relationship: XBUFFER, XPATH, XFILETXT, XFILEINI, XLOG, XTRACE. Example: string.Format(__L("value %d"), value);
XFSTRING	Quick string oriented to scenarios where it is important to reduce text manipulation costs. Relation: XBUFFER and text operations. Example: use it to mount intensive temporary text without going through heavier routes.
XBUFFER	Buffer binary mutable. Relationship: XSTRING, XFILE, Cipher, Compress, DataIO and serialization. Example: buffer.Add(((XBYTE*)data, size);
XBUFFERBITS	Bit stream reading on a XBUFFER. Relationship: parsers binary, protocols and compact formats. Example: read fields that are not aligned to byte.
XPATH	It represents file paths with homogeneous operations. Relationship: XFILE, XDIR, XPATHSMANAGER, APPFLOWCFG. Example: xpath.Slash_Add(); xpath.Add(__L("file.txt"));
XPATHSMANAGER	Singleton ordering the application's resource paths. Relationship: AppFlow, assets, XPATH and XFILE. Example: GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
XVARIANT	Variable-type value container. Relationship: XFileJSON, serialization, configuration and heterogeneous data. Example:

	variant.Set(((XDWORD)10);
XRECT	Generic rectangle. Relationship: Graphic, UserInterface and geometric calculations. Example: XRECT rect(0, 0, width, height);
XUUID	Generation and handling of unique identifiers. Relationship: XRand and identification systems. Example: uuid.Generate(); uuid.GetToString(string);

6. Data collections and structures

GEN Framework is not limited to using standard containers visible in the public API. XUtilities offers its own structures to maintain consistency with the rest of the framework, control object ownership, use the existing iteration style and maintain compatibility with platforms where direct use of STL may not be desirable or not fit the historical style of the code.

The most common class is XVECTOR. In many modules of the framework it appears as pointer container to GEN Framework objects. When those objects are owned by the vector, it is common to use DeleteContents() followed by DeleteAll(). This form clearly expresses two different operations: removing the objects and cleaning the list.

```
XVECTOR<XSTRING*> names;

XSTRING* name = GEN_NEW XSTRING();
name->Set(__L("main"));
names.Add(name);

for(int c=0; c<names.GetSize(); c++)
{
    XSTRING* item = names.Get(c);
    if(item) XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, item->Get());
}

names.DeleteContents();
names.DeleteAll();
```

Element	Description, relationship and example
XVECTOR	Main generic vector of the framework. Relation: virtually all modules that keep ready. Example: vector.Add(item); item = vector.Get(index);
XVECTORSTL	STL-based adaptation when the module needs that implementation. Relation: XSTL and specific scenarios. Example: use it if the local subsystem already works with that variant.
XMAP	GEN Framework style key/element map. Relationship: configurations, relationship tables and search by key. Example: map.Add(key, element);
XPAIR	Pair of values. Relationship: XMAP and auxiliary structures. Example: XPAIR<XDWORD, XSTRING*> pair;
XLIST	Doubly linked list. Relationship: queues, paths and structures where inserting/eliminating in intermediate positions matters. Example: list.Add(data);
XSTACK	Generic battery. Relationship: parsers, LIFO paths and algorithms. Example: stack.Push(item); item = stack.Pop();
XTREE	Generic tree with nodes. Relationship: XML, UI hierarchies, project structures and nested data. Example: node->AddChild(child);
XAVLTREE	Balanced AVL tree. Relationship: structures that need efficient orderly search. Example: insert keys while maintaining automatic balance.
XBINARYMAP	A binary search-oriented map of sorted keys. Relation: compact tables and quick access. Example: search for an item per key without going through the entire collection.

XDICTIONARY	Dictionary of entries. Relationship: translation, catalogues, tables of texts and equivalencies. Example: dictionary.Add(__L("id"), value);
XORDEREDLIST	List sorted by comparison. Relationship: planning, ranking, object sorting. Example: Insert objects keeping the order defined by XCOMPARATOR.
XINHERITENUM	Utility for working with hereditary or extended enum erations. Relationship: APIs that need to keep base values and expand them. Example: define a range of enum own over an existing range.

7. Time, system and platform-dependent objects

There are utilities that seem neutral until you try to make them portable. Measure time accurately, sleep a thread, get system information, create processes or use shared memory changes according to the platform. XUtilities encapsulates those differences and, where appropriate, channels them through XFACTORY.

```

XDATETIME* datetime = GEN_XFACTORY.CreateDateTime();
if(datetime)
{
    XSTRING text;

    datetime->Read();
    datetime->GetDateTimeToString(XDATETIME_FORMAT_STANDARD, text);

    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, text.Get());

    GEN_XFACTORY.DeleteDateTime(datetime);
    datetime = NULL;
}

```

Element	Description, relationship and example
XFACTORY	XUtils Factory. Create platform-dependent instances such as XTIMER, XDATETIME, XRAND, XFILE, XDIR, XCONSOLE, XMUTEX and XTHREAD. Example: GEN_XFACTORY.CreateTimer();
XFACTORYMACROS	Auxiliary macros to create and destroy objects from factories. Relation: common pattern of GEN Frameworkfactories. Example: use creation macros when the module defines a standard flow.
XTIMER	Time measurement and timing. Relationship: XFactory, XDateTime, AppFlow and Scheduler. Example: timer->GetMeasureSeconds();
XDATETIME	Date and time. Relationship: XFactory, XLog, XFileZIP, Scheduler. Example: datetime->Read();
XRAND	Random Number Generation. Relationship: XFactory, UUID, testing and data generation. Example: rand->Max(100);
XSLEEP	Controlled pauses or waits. Relationship: threads, timers and platform. Example: GEN_XSLEEP.MilliSeconds(100);
XSYSTEM	Information and system operations. Relationship: platforms, AppFlow, hardware resources. Example: consult operating system or hardware features.
XPROCESSMANAGER	External process management. Relationship: tools, scripts, utilities and platform. Example: launch an auxiliary tool and check its completion.
XSHAREDMEMORYMANAGER	Shared memory between processes. Relationship: IPC, processes and synchronization. Example: create a common block for data exchange.
XCONSOLE	Consol Abstraction. Relationship: AppFlowConsole, DevTestsConsole and text input/output. Example: console->KBHit(); console->GetChar();

8. Files, directories and XPATHSMANAGER

File management is one of the first needs that solves any utility base. GEN Framework separates several levels. XPATH represents paths. XDIR works with directories. XFILE abstracts file operations. About XFILE specialized classes appear for specific formats. XPATHSMANAGER, for its part, orders where the application resources should be.

The idea is that an application does not have paths scattered by the code. Resources are grouped under assets and, within that folder, known sections are used: logs, databases, web, scraperweb, fonts, graphics, ui_layouts, sounds, scripts, firmware, certificates and generic sections. The name of those folders can be changed and new paths can be added, but the goal is always the same: that the application has a homogeneous way of finding its resources.

```
XPATH xpath;

GEN_XPATHSMANAGER.AdjustRootPathDefault( __L("assets"));
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, __L("graphics"));
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
xpath.Slash_Add();
xpath.Add(__L("logo.png"));
```

Element	Description, relationship and example
XFILE	Base file access class. Relation: all derived formats, XFactory, XPath, XBuffer. Example: file->Open(xpath); file->Read(buffer);
XDIR	Listing and managing directories. Relationship: XFactory, XPath and resource tools. Example: xdir->FirstSearch(xpath, __L("*.txt"));
XFILEID	Defines file type identifiers. Relationship: modules that recognize formats by header or extension. Example: use ID file to sort content.
XFILEHASHID	Define identifiers associated with files with hash. Relationship: XFileHash and integrity validation. Example: select the expected hash type.
XFILEHASH	File with hash included. Relationship: integrity, XHash32 and data validation. Example: check that the content has not been modified.
XFILESENTENCES	File of sentences or hash texts. Relationship: translation, texts and resources. Example: load sentences identified by key.
XFILEXDB	Format dialog or phrases. Relationship: text resources and applications with structured messages. Example: load a sentence bank or dialogs.

9. File formats supported in XUtils

An important part of XUtils is in FormatFiles. These classes solve the reading and writing of usual or necessary formats for specific projects. Again appears the practical philosophy of GEN Framework: formats are not added by catalog, but because some application or tool needed to work with them.

```
XFILEINI ini;
XPATH xpath;
XSTRING value;

xpath.Set(__L("assets"));
xpath.Slash_Add();
xpath.Add(__L("application.ini"));

if(ini.Open(xpath, true))
{
    ini.ReadValue(__L("General"), __L("Name"), value);
    ini.Close();
}
```

Element	Description, relationship and example
XFILETXT	Read and write text. Relationship: XSTRING, XBUFFER and encodings. Example: <code>filetxt.AddLine(__L("hello"));</code>
XFILEINI	INI files with sections, keys and comments. Relationship: XFILECFG and APPFLOWCFG. Example: <code>read [General] Name=...</code>
XFILECFG	Configuration layer over INI. Relationship: APPFLOWCFG, variable mapping and main application configuration. Example: Map an entry to keep it updated.
XFILECSV	CSV Files. Relation: import/export of tables. Example: browse records and fields of a file separated by commas.
XFILEJSON	Reading/writing JSON using objects, arrays and values. Relation: XVARIANT and JSON serialization. Example: <code>json.GetRoot()->GetValue(__L("name"));</code>
XFILEXML	Reading/writing XML using elements and attributes. Relationship: XTREE, XSTRING and structured configuration. Example: tour nodes children of a root element.
XFILERIFF	Format RIFF. Relationship: audio, video and binary containers. Example: locate chunks within a WAV or AVI file.
XFILEZIP	Compression/decompression ZIP. Relationship: XFILEZIP_XEVENT, packaged resources and distribution. Example: compress a folder assets into a ZIP.
XFILEZIP_XEVENT	Progress or status event for ZIP operations. Relationship: XPublisher and XObserver. Example: report compression percentage.
XFILEHEX	Intel HEX format or other ASCII hexadecimal data. Relationship: firmware and microcontrollers. Example: read firmware logs.
XFILEDFU	DFU firmware update format. Relationship: devices and embedded update. Example: load firmware images for a device.
XFILEDBF	Format DBF. Relationship: old databases, import and compatibility. Example: read records from a DBF table.
XFILECOBOL	COBOL file structures. Relationship: integration with legacy systems. Example: interpret fields defined by PIC.
XFILEHASH	File format with integrated hash. Relationship: validation and integrity. Example: read content and verify your footprint.
XFILESENTENCES	Format of phrases or sentences. Relationship: application texts and translation. Example: obtain a phrase per identifier.
XFILEXDB	Format of dialog phrases. Relationship: structured text resources. Example: load messages organized by key.

10. Configuration with XFILECFG

XFILECFG deserves a specific mention because it connects XUtilities to AppFlow. The main configuration of an application with GEN Framework is supported by INI files, but it is not reduced to reading text values. There is usually an application-specific configuration class, derived from APPFLOWCFG, which declares internal variables, defines an input mapping and provides getters and setters. XFILECFG is the basis for keeping that file neat, synchronized and clean.

When a configuration entry is mapped, the application knows what it means, what type it is and where it should be stored. When an entry is no longer mapped, the system can remove it because it is no longer part of the current configuration. This prevents INI s from accumulating obsolete keys for years. In long projects this

automatic cleaning has a lot of value: the configuration file reflects the actual configuration of the application, not an accumulated history of previous versions.

```
// Concept example within a APLICACION_CFG class

bool APLICACION_CFG::DoVariableMapping()
{
    AddValue(_L("General"), _L("Name"),          XFILECFG_VALUETYPE_STRING, &name);
    AddValue(_L("Trace"), _L("IsActive"),         XFILECFG_VALUETYPE_BOOLEAN, &istraceactive);
    AddValue(_L("Server"), _L("Port"),             XFILECFG_VALUETYPE_INTEGER, &port);

    return true;
}
```

The exact syntax of mapping depends on the concrete implementation of the configuration class, but the idea that should be clear is this: the INI file is not treated as a free text, but as a persistent projection of variables known to the application.

11. Events, observers and publication

XUtilities includes the framework's event system base. XEVENT represents the event. XSUBJECT represents the object that can be observed. XOBSERVER represents the receiver. XPUBLISHER organizes the publication. This structure is important because GEN Framework uses events in various modules: sound, communications, scheduler, licenses, ZIP, BER and other subsystems.

The model avoids directly docking producer and consumer. A module can issue an event without knowing who receives it. An application can subscribe to certain events without changing the class that produces them. This separation fits very well with a modular framework.

```
class MYOBSERVER : public XOBSERVER
{
public:

    bool HandleEvent(XEVENT* xevent)
    {
        if(!xevent) return false;

        switch(xevent->GetEventType())
        {
            case XEVENT_TYPE_APPLICATION : // processing event
                break;
        }

        return true;
    }
};
```

Element	Description, relationship and example
XEVENT	Event base class. Relationship: XSubject, XObserver and specific events. Example: transport an event type and associated data.
XSUBJECT	Observable subject. Relationship: XObserver and XPublisher. Example: a class that allows subscriptions.
XOBSERVER	Observer receiving events. Relationship: modules that react to changes. Example: an application subscribed to sound events.
XPUBLISHER	Publisher and manager of subject/observer relationships. Relationship: XSubject and XEvent. Example: publish an event to all subscribed observers.
XBER_XEVENT	Event associated with BER processing. Relationship: XBER and XPUBLISHER. Example: reporting progress or data in a BER encoding.
XSCHEDULER_XEVENT	Planner event. Relationship: XScheduler and scheduled tasks. Example: warn of a task to be executed.

XLICENSE_XEVENT	License Event. Relationship: XLicense. Example: Notify validation, error or change of license status.
XDRIVEIMAGEMANAGER_XEVENT	Disk Image Management Event. Relationship: XDriveImageManager. Example: report copy progress or image creation.
XFILEZIP_XEVENT	ZIP Operations Event. Relationship: XFile ZIP. Example: report compression or decompression progress.

12. Finite State Machine

XFSMACHINE already appeared in the chapter dedicated to the first application, but it belongs to XUtils and should therefore be understood as a general utility of the framework. Its function is to control states and events in an explicit way. Instead of having scattered Boolean variables, an application defines states, defines events and records which transitions are valid.

The grace of XFSMACHINE is not only in changing from one state to another. Its practical utility is in separating the continuous logic from a state of logic that should only be executed when a transition occurs. In the examples of GEN Framework, the Update usually has two parts: when there is no new event, the logic proper to the current state is executed; when there is an event, the transition is checked and the input or change of state logic is executed.

```
enum MYAPP_XFSMEVENTS
{
    MYAPP_XFSMEVENT_NONE = 0 ,
    MYAPP_XFSMEVENT_INI    ,
    MYAPP_XFSMEVENT_UPDATE ,
    MYAPP_XFSMEVENT_END    ,
};

enum MYAPP_XFSMSTATES
{
    MYAPP_XFSMSTATE_NONE = 0 ,
    MYAPP_XFSMSTATE_INI    ,
    MYAPP_XFSMSTATE_UPDATE ,
    MYAPP_XFSMSTATE_END    ,
};

bool MYAPP::IniFSMachine()
{
    if(!AddState(MYAPP_XFSMSTATE_NONE,
        MYAPP_XFSMEVENT_INI, MYAPP_XFSMSTATE_INI,
        MYAPP_XFSMEVENT_END, MYAPP_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND)) return false;

    return true;
}
```

The correct use of XFSMACHINE helps make an application easier to debug. We can see what status is active, what event has caused the change and what transitions are allowed. This is especially useful in applications with long life cycle, communications, graphical displays, protocols or operating modes.

13. Traces, logs, memory and feedback

A framework used in real applications needs diagnosis. Traces let you know what is going on. logs allows you to keep information. Memory control helps you find faulty leaks or life cycles. Feedback allows you to communicate states or progress. XUtils provides these pieces so that the diagnosis is not improvised in each application.

```
XTRACE_SETAPPLICATIONNAME((*Application_GetName()));
XTRACE_SETAPPLICATIONVERSION(APPLICATION_VERSION, APPLICATION_SUBVERSION,
APPLICATION_SUBVERSIONERR);

XTRACE_PRINTCOLOR(XTRACE_COLOR_GREEN, __L("Application initialized"));
```

Element	Description, relationship and example
XTRACE	Trace system. Relationship: AppFlow, DataIO and diagnostic. Example: XTRACE_PRINTCOLOR(...);
XLOG	Persistent message log. Relationship: XDateTime, XFileTXT, XMutex and configuration. Example: Type an error level message into a log file.
XMEMORY_CONTROL	Framework memory control. Relationship: GEN_NEW, leak detection and DEBUG compilations. Example: activate MEMCTRL during development.
XFEEDBACK_CONTROL	Feedback control and status codes. Relationship: long processes, UI and traceability. Example: report progress of an operation.
XERRORMSG	Error message management. Relationship: classes that return codes or states. Example: translate a code to readable error text.

14. Concurrency, threads and planning

The competition is another terrain where GEN Framework prefers a layer of its own. XTHREAD and XMUTEX are not simple aesthetic wrappers: they allow the application to use a common way to create threads, group them, destroy them and synchronize resources even if the implementation changes on each platform. Access is done using XFACTORY when applicable.

```
static void ThreadFunction(void* param)
{
    MYAPP* app = (MYAPP*)param;
    if(!app) return;

    // thread work
}

XTHREAD* thread = GEN_XFACTORY.CreateThread(XTHREADGROUPID_APPOWNER, __L("worker"),
ThreadFunction, this);
if(!thread) return false;
```

Element	Description, relationship and example
XTHREAD	Core yarn class. Relationship: XFactory, XMutex, AppFlow and tasks. Example: GEN_XFACTORY.CreateThread(...);
XTHREADCOLLECTED	Thread managed within a collection. Relationship: XThreadCollected manager and controlled life cycles. Example: threads grouped by type of work.
XTHREADLISTNONPREEMPTIVE	List of non-preemptive threads. Relationship: environments where execution is manually controlled. Example: microcontrollers or cooperative loops.
XSCHEDULER	Task Planner. Relationship: XTimer, XDateTime, XThreadCollected and Events. Example: execute a task every certain interval.
XSCHEDULER_XEVENT	Event of the planner. Relationship: XScheduler and XPublisher. Example: reporting execution of a scheduled task.
XSLEEP	Timed waits. Relationship: threads, Polling and Timers. Example: XSLEEP::MilliSeconds(50).

15. Serialization, coding and binary structures

When an application needs to save objects, network them, or convert them to a stable format, serialization appears. XUtilities separates the serializable object from the serialization methods. XSerializable defines the idea of object that can export or import its status. XSerializationMethod and its variants define how that state is represented: binary, JSON or other methods.

```

class MYDATA : public XSERIALIZABLE
{
public:

    bool Serialize()
    {
        // Add fields to active serialization method
        return true;
    }
};

```

Element	Description, relationship and example
XSERIALIZABLE	Base for serialization objects. Relationship: XSerializationMethod and data classes. Example: a class that saves its fields in JSON or binary.
XSERIALIZATIONMETHOD	Abstract method of serialization. Relationship: XBuffer, XVariant and concrete formats. Example: Write fields using a common interface.
XSERIALIZATIONMETHODBINARY	binary serialization. Relationship: XBuffer and compact data. Example: save structures in a buffer for file or network.
XSERIALIZATIONMETHODJSON	Serialization in JSON. Relationship: XFile JSON, XVariant and configuration. Example: Export an object to a readable structure.
XASN1	Reading and writing ASN.1. Relationship: BER, certificates and cryptographic structures. Example: interpret a block ASN.1.
XBER	Basic Encoding Rules. Relationship: ASN.1, certificates and binary structures. Example: decode tags and lengths.
XBER_XEVENT	BER processing events. Relationship: XBER and observers. Example: report during complex decoding.
XMPIINTEGER	Multi-precision integers. Relationship: cryptography and large mathematical operations. Example: handle numbers larger than native types.

16. Internationalization, translation and texts

GEN Framework includes classes to work with languages and translations. This support should not be seen as an aesthetic addition, but as part of the construction of real applications. An application that deploys in various environments may need translationable texts, common framework messages, language selection and standardized codes.

Element	Description, relationship and example
XLANGUAGE_ISO_639_3	ISO 639-3 language codes. Relationship: XTranslation and language selection. Example: GEN_XTRANSLATION.SetActual(XLANGUAGE_ISO_639_3_CODE_SPA);
XTRANSLATION	Application Translation Manager. Relationship: phrase files, XPATH and XSTRING. Example: load a language file and get text by identifier.
XTRANSLATION_GEN	Internal translations of the framework. Relationship: common messages from GEN Framework. Example: obtain a standard phrase from the framework itself.
XFILESENTENCES	Judgment files. Relationship: XTranslation. Example: store texts translatable by ID.
XFILEXDB	Dialogue files or phrases. Relationship: structured texts of application. Example: Load interface dialogs or messages.

17. Licenses, firmware, storage and special resources

Some classes of XUtils respond to very specific needs of real projects: license control, firmware update, EEPROM memory management, disk images or persistent resources. They are good examples of GEN

Framework philosophy: when a functionality appears several times in different projects, it becomes common utility.

Element	Description, relationship and example
XLICENSE	License generation and control. Relationship: XLicense_XEvent, XFile, Cipher and identification data. Example: Validate a license before activating modules.
XLICENSE_XEVENT	Event associated with the license status. Relationship: XLicense and Observers. Example: Warning of invalid license or next to expire.
KEEPROMMEMORYMANAGER	EEPROM Memory Management. Relationship: microcontrollers and small persistent storage. Example: Reserve zones for parameters.
XDRIVEIMAGEMANAGER	Disk or drive image management. Relationship: XDriveImageManager_XEvent, files and progress. Example: create or restore an image.
XDRIVEIMAGEMANAGER_XEVENT	Image Manager Progress Event. Relationship: XDriveImageManager. Example: Notify copy percentage.
XFILEDFU	DFU firmware update. Relationship: firmware, devices and assets/firmware. Example: Load an update package.
XFILEHEX	HEX firmware files. Relationship: microcontrollers. Example: read firmware logs in hexadecimal text format.

18. Auxiliary and integration modules

Not all XUtils files represent a great kind of direct use. Some are integration, compatibility or definition pieces.

Their presence is important because they make the framework consistent and avoid repeating small decisions in several modules.

Element	Description, relationship and example
XSTL	Definitions and includes STL when integration is needed. Relationship: XVector STL and code that must coexist with STL. Example: encapsulate standard dependencies at a common point.
XFACTORYMACROS	Factory support macros. Relationship: XFactory, DIOFactory, GRPFactory and other creation patterns. Example: simplify module creation and registration.
XFILEID	File type identifiers. Relationship: FormatFiles modules. Example: recognize a resource type.
XFILEHASHID	Identifiers for hash formats. Relationship: XFileHash. Example: select algorithm or type of validation.
XPAIR	Pair of values used by data structures. Relationship: XMap and auxiliary algorithms. Example: group key and value.
XINHERITENUM	Support to extend enumerations. Relationship: events and types ranges. Example: Reserve an event range for an application.
XINTERPOLATOR	Interpolation of values. Relationship: Graphic, animations, UI and timed processes. Example: interpolator between two values depending on a time.
XBINARYMAP	Optimized binary map. Relationship: compact searches and tables. Example: consult an element per binary key.

19. How to study XUtils from DevTestsConsole

DevTestsConsole is a very useful tool for understanding XUtils because it does not limit itself to compiling classes: it contains tests for actual use. Its sources contain tests for XString, XBuffer, XRand, XTrace, XLog, XTimer, XVector, XTree, XDir, XVariant, DateTime, XSystem, SharedMemory, XFileJSON, XFileXML, XFileRIFF, XFileDFU, XLicense, XSerializable, XScheduler, Compress and XPath, among others. This makes DevTestsConsole a kind of framework behavior laboratory.

When you want to study a class of XUtils, the recommended order is to first read the header to understand your public API, then review the Doxygen documentation to view the list of methods and finally search DevTestsConsole for a test that uses it. This combination is more useful than just reading the implementation, because it shows the intention of use.

For example, to learn XSTRING you should check Test_XString. To understand XBUFFER, Test_XBuffer. To see how trees are managed, Test_XTree. To see routes and directories, Test_XPath and Test_XDir. For formats, Test_XFileJSON, Test_XFileXML, Test_XFileRIFF or Test_XFileDFU. For timing, Test_XTimer. For planning, Test_Scheduler. This approach helps to move from theory to practice.

20. Criteria for use in an application with GEN Framework

The important question is not only what classes exist, but when they should be used. In an application with GEN Framework it is appropriate to adopt a simple rule: if there is an XUtils class to solve a common need, that class should be preferred rather than introducing an isolated solution. This does not mean that it is forbidden to use standard C++ or an external library, but that the public boundary of the application should be kept consistent with the framework.

If we need text, we use XSTRING. If we need binary data, XBUFFER. If we need a path, XPATH. If we need a framework object list, XVECTOR. If we need a INI configuration file, XFILECFG or XFILEINI. If we need a timer, XFACTORY and XTIMER. If we need traces, XTRACE. If we need to publish events, XEVENT, XSUBJECT, XOBSERVER and XPUBLISHER. This discipline reduces application differences and accelerates maintenance.

You must also respect the property of objects. If an object is created by a factory, it must be destroyed by the corresponding factory. If a container has the objects it stores, it must release its contents. If a class has Ini(), End() and Clean(), that life cycle must be used. XUtilities is not just a list of classes: it is a way to write code within GEN Framework.

21. Practical summary

XUtils is the practical basis of GEN Framework. Its name, eXtended Utilities, reflects its origin as a set of extended utilities that over time became a common infrastructure. It is the point where the pieces that any application needs are concentrated: strings, buffers, paths, files, dates, timers, collections, events, traces, configuration, threads, serialization and system support.

Its importance is not that each class is complex separately. Its importance is in the sum. When all applications use the same classes to solve common problems, the framework gains coherence. When platform dependent instances are created using XFACTORY, the application code becomes cleaner. When resources are located using XPATHSMANAGER, the project maintains order. When the configuration is managed with XFILECFG, the INI ceases to be loose text and becomes part of the application model.

This chapter should therefore be read as a vocabulary guide. Before entering more specific modules such as DataIO, Graphic, Input, Sound, Cipher, Databases or UserInterface, it is best to master XUtilities. Whoever understands XUtils understands much of GEN Framework's way of thinking.

Chapter VI: DataIO, data input and output in GEN Framework

1. The origin of DataIO

DataIO means Data Input Output. The name accurately describes its function within GEN Framework: grouping classes that allow an application to communicate with the outside, receive data, send data, talk to devices, access networks, work with protocols, consult web services, handle hardware buses and resolve input and output operations that in a real application appear continuously.

If XUtils was the initial germ of GEN Framework, DataIO was the second big block that began to shape it into a framework. Extended utilities solved common needs: strings, buffers, routes, dates, files, traces, events or data structures. But an application does not live in isolation. Before or after it has to open a serial port, listen to a socket, connect to a server, consult a URL, send an email, read a sensor, write on a I2C bus, list USB devices, receive a UDP plot, sync the time by NTP or dialogue with an industrial protocol.

The importance of DataIO within GEN Framework is not only in the number of classes it contains. It is in the idea it represents: an application should not be full of scattered calls to sockets system, Bluetooth APIs, specific functions of Linux, details of WinSock, serial port handlers or particular implementations of each bus. All of this must be organized behind a common layer, with a coherent way to configure, open, close, read, write and diagnose communications.

As in the rest of GEN Framework, DataIO is not born as a theoretical abstraction. Its classes exist because there was a specific need before. Many of them respond to real projects: TCP/IP communications, industrial instrumentation, sensors, I2C and SPI devices, remote updates, alerts, network protocols, embedded web servers, service scraping, DNS queries or hardware integration. That practical background explains why DataIO mixes very low-level elements with high-level protocols: both extremes are needed in real applications.

2. DataIO as second block of GEN Framework

DataIO is constantly supported by XUtilities. It does not invent a new type of string, nor a new buffer class, nor a parallel event system. It uses XSTRING for text, XBUFFER for binary data, XPATH for routes, XEVENT for events, XTHREAD for attendance, XTIMER for waiting times and XUtils for platform-dependent objects. This is important because it maintains the homogeneity of the framework: DataIO is not an isolated library attached to the project, but a natural extension of the GEN Framework vocabulary.

The programmer who already understands XUtilities finds in DataIO a logical continuation. A DIOSTREAM reads and writes XBUFFER. A DIOWEBCLIENT works with DIOURL and XBUFFER. A protocol can issue GEN Framework events. A stream configuration uses own DataIO classes, but maintains the Ini(), End(), Open(), Close(), Read() and Write() life cycle style. This continuity reduces the mental cost of moving from one part of the framework to another.

3. DIOFACTORY and platform insulation

The piece that marks the DataIO architecture is DIOFACTORY. Just like XFACTORY hides the creation of certain XUtils platform dependent objects, DIOFACTORY centralizes the creation of input and output objects whose actual implementation can change between Windows, Linux, Android, Raspberry Pi or an embedded environment.

A stream TCP/IP is not implemented the same on all platforms. A Bluetooth device enumerator also does not. Access to a serial port may depend on different APIs. A PCap capture object or a Wi-Fi manager may need system-specific code. The application should not directly decide which specific kind of instance platform. That decision belongs to the factory.

The common version of DIOFACTORY defines the interface. Platform versions specialize in creating specific objects when the platform supports them. If a type of stream or device is not available, the factory can return NULL. This behavior is deliberate: it forces the application to check availability and avoid assuming that all platforms have all the capabilities.

```
DIOSTREAMTCIPCONFIG diostreamcfg;
DIOSTREAM*          diostream = NULL;

diostreamcfg.GetRemoteURL()->Set(__L("endorasoft.com"));
diostreamcfg.SetMode(DIOSTREAMMODE_CLIENT);
diostreamcfg.SetRemotePort(25);

diostream = GEN_DIOFACTORY.CreateStreamIO(&diostreamcfg);
if(!diostream) return false;

if(diostream->Open())
{
    bool status = diostream->WaitToConnected(5);

    if(status)
    {
        XBUFFER buffer;

        diostream->WaitToFilledReadingBuffer(30, 5);
        diostream->Read(buffer);
    }

    diostream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(diostream);
```

This example summarizes the philosophy of DataIO: the application configures what it needs, asks the factory for an object capable of doing so and works with a common interface. The concrete platform is outside the application logic.

4. General Map of DataIO

DataIO is a large block. To study it, it is advisable to group the classes by responsibility. It is not useful to read the files in alphabetical order, because that mixes very different concepts. It is better to distinguish between network utilities, streams, protocols, web, scraping, nodes, drivers, cameras, notifications and auxiliary modules.

Group	Principal responsibility	Representative classes
Core and utilities	Addresses IP, MAC, URL, email, DNS, connectivity, Wake on LAN, PCap, alerts, update and general objects.	DIOFACTORY, DIOIP, DIOMAC, DIOURL, DIODNSRESOLVER, DIOWAKEONLAN
Streams	Common communication layer: open, close, read, write, wait for connection, list devices and configure transport.	DIOSTREAM, DIOSTREAMCONFIG, DIOSTREAMTCIP, DIOSTREAMUDP, DIOSTREAMUART, DIOSTREAMUSB, DIOSTREAMI2C, DIOSTREAMSPI
TLS	Safe transport layer and internal TLS message structures.	DIOSTREAMTLS, DIOSTREAMTLSCONFIG, DIOSTREAMTSMESSAGES
Protocols	Protocols built on streams or web: NTP, SMTP, SNMP, ModBus, DNS, Ping, AT, IEC 60870-5, Dyn DNS.	DIONTP, DIOSMTP, DIOSNMP, DIOMODBUS_CLIENT, DIOPING, DIOATCMDS
Web	HTTP client, web server, headers, API REST, plugins and page generation.	DIOWEBCLIENT, DIOWEBSERVER, DIOWEBHEADER, DIOWEBSERVER_APIREST
ScraperWeb	Structured collection of data from specific web services or external pages.	DIOSCRAPERWEB, DIOSCRAPERWEBPUBLICIP, DIOSCRAPERWEBWEATHER
Nodes	Node model, items, values, units, handlers and events for control systems.	DIONODE, DIONODEITEM, DIONODEITEMVALUE, DIONODEMANAGER
Drivers	Concrete hardware devices and buses I2C/SPI.	DIOI2CGPIOMCP2317, DIOSPIFTDISPLAYST7789,

5. Addresses, identifiers and network utilities

The first group of DataIO contains classes that look small, but prevent the application from working with free text for network concepts. DIOIP represents a IP address. DIOMAC represents a MAC address. DIOURL represents a URL with its parts. DIOEMAILADDRESS encapsulates an email address. DIODNSRESOLVER resolves names. DIOPUBLICINTERNETIP gets the public IP through external mechanisms. DIOWAKEONLAN builds the magic package to wake up a team.

```
DIOURL url;
DIOIP ip;

url.Set(__L("www.genframework.com"));

if(GEN_DIODNSRESOLVER.ResolveURL(url.Get(), ip))
{
    XSTRING iptext;

    ip.GetXString(iptext);
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, iptext.Get());
}
```

These classes are directly related to streams and protocols. A DIOSTREAMTCPIPConfig can use a URL or a remote IP. A NTP protocol uses DIOURL to indicate the time server. Wake on LAN needs DIOMAC and DIOIP to build the shipment. The advantage is that network concepts become framework objects, not loose strings scattered by the code.

6. Streams: the main abstraction of communication

DIOSTREAM is one of the most important classes in DataIO. It represents the idea of a communication channel. A stream can be TCP/IP, UDP, UART, USB, Bluetooth, I2C, SPI, ICMP or TLS, but the application always tries to work with the same idea: set up, open, wait for status, read, write, empty buffers and close.

The configuration is separated into specific classes. DIOSTREAMConfig contains the common base. DIOSTREAMTCPIPConfig adds URL, ports and client/server mode. GODSTREAMUDPConfig, DIOSTREAMUARTConfig, GODSTREAMUSBConfig, GODSTREAMI2CConfig and GODSTREAMSPIConfig adapt that idea to concrete transport. This separation is important because it prevents huge builders full of parameters and allows the configuration to move to DIOFACTORY to create the right stream.

```
DIOSTREAMUDPConfig udpcfg;
DIOSTREAM*          udpstream = NULL;

udpcfg.SetMode(DIOSTREAMMODE_CLIENT);
udpcfg.GetRemoteURL()->Set(__L("192.168.1.50"));
udpcfg.SetRemotePort(5000);

udpstream = GEN_DIOFACTORY.CreateStreamIO(&udpcfg);
if(udpstream && udpstream->Open())
{
    udpstream->WriteStr(__L("GEN UDP test"));
    udpstream->WaitToFlushOutXBuffer();
    udpstream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(udpstream);
```

The use of DIOSTREAM also allows higher protocols not to depend directly on transportation. A protocol may receive a DIOSTREAM* and speak for it without knowing if there is TCP/IP, UART or USB below, provided that the necessary operations are supported.

7. Listing of devices and descriptors

In real communications it is not enough to open a known port. Many times you have to discover which devices exist. DataIO includes enumeration classes and descriptors. DIOSTREAMENUMDEVICES is the basis of enumerations. From it appear enumerations of devices USB, UART, Bluetooth, Bluetooth LE, Wi-Fi or UDP servers. DIOSTREAMDEVICE objects and their derivatives represent the information found: name, identifier, address, type of connection or specific data of each technology.

```
DIOSTREAMENUMDEVICES* enumdevices = NULL;

enumdevices = GEN_DIOFACTORY.CreateStreamEnumDevices(DIOSTREAMENUMTYPE_BLUETOOTH_REMOTE);
if(enumdevices)
{
    enumdevices->Search();

    for(int c=0; c<enumdevices->GetDevices()->GetSize(); c++)
    {
        DIOSTREAMDEVICE* device = enumdevices->GetDeviceByIndex(c);
        if(device) XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, device->GetName()->Get());
    }

    GEN_DIOFACTORY.DeleteStreamEnumDevices(enumdevices);
}
```

Again the factory appears: the way to list Bluetooth devices in Windows does not have to match Linux. The application asks for an enumerator by type; the platform decides if it can offer it and how it implements it.

8. TLS and streams secure

The TLS folder adds an important layer: safe transport. DIOSTREAMTLS and DIOSTREAMTLSCONFIG allow you to work with encrypted communications on the streams infrastructure. DIOSTREAMTLSMESSAGES, DIOSTREAMTLSMESSAGESEXTENSION, DIOSTREAMTLSMESSAGESHANDSHAKECLIENTHELLO and DIOSTREAMTLSMESSAGESHANDSHAKESERVERHELLO classes represent internal parts of the TLS protocol and help analyze or build negotiation messages.

This group is especially interesting because it shows how far DataIO goes: it does not just call an external library, but structure the protocol concepts within the GEN Framework style. This allows you to debug, inspect messages, work with extensions and keep a controlled view of what happens during the handshake.

```
DIOSTREAMTLSCONFIG tlsconfig;
DIOSTREAM*         tlsstream = NULL;

tlsconfig.SetMode(DIOSTREAMMODE_CLIENT);
tlsconfig.GetRemoteURL()->Set(__L("www.genframework.com"));
tlsconfig.SetRemotePort(443);

tlsstream = GEN_DIOFACTORY.CreateStreamIO(&tlsconfig);
if(tlsstream && tlsstream->Open())
{
    if(tlsstream->WaitToConnected(10))
    {
        tlsstream->WriteStr(__L("GET / HTTP/1.1\r\nHost: www.genframework.com\r\n\r\n"));
    }

    tlsstream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(tlsstream);
```

9. High-level protocols

On top of the streams protocols appear. DataIO includes general network protocols, industrial protocols and integration protocols. Some are simple, such as DIONTP to get the time UTC or DIOPING to check range. Others are more specific, such as DIOMODBUS_CLIENT, DIOMODBUSELECTRICMETER or DIOIEC60870_5. Others solve daily communication needs, such as DIOSMTP for mail, DIOSNMP for device management, DIODYNDNS for dynamic DNS or DIOATCMDS and DIOATCMDGSM for modems GSM.

```
DIONTP*    ntp = GEN_NEW DIONTP();
DIOURL     url;
XDATETIME* datetime = GEN_XFACTORY.CreateDateTime();

url = __L("1.es.pool.ntp.org");

if(ntp && datetime)
{
    ntp->GetTime(url, DIONTP_DEFAULTTIMEOUT, GEN_XSYSTEM.HardwareUseLittleEndian(), (*datetime));
}

GEN_DELETE ntp;
GEN_XFACTORY.DeleteDateTime(datetime);
```

The protocols are a good example of the practical philosophy of GEN Framework. Not everyone has the same level of abstraction or all solve generic problems. Some exist because a project needed to talk with an electrical meter, send a AT command to a modem, or synchronize a device without relying on external system services. That is the value of DataIO: turn specific needs into reusable parts.

10. DIOCoreProtocol and DIOProtocol

Within Protocol there are two families that deserve a separate explanation. DIOCoreProtocol represents a kernel protocol of its own, with header, messages, configuration, data log, connections and events. DIOProtocol, within Protocol/Protocol, represents a generic binary protocol database, with application data, connection manager and associated events.

These classes are useful when an application not only consumes existing protocols, but needs to define its own communication protocol. Instead of scattering headers, states, connections and messages across different parts of the program, the framework provides a basis for organizing those pieces in the DataIO style.

```
// Conceptual outline of a protocol on a stream

DIOSTREAM* stream = GEN_DIOFACTORY.CreateStreamIO(&cfg);
if(stream && stream->Open())
{
    DIOPROTOCOL* protocol = GEN_NEW DIOPROTOCOL();

    // The protocol organizes plots, application data and events.
    // The application works with higher-level messages, not loose bytes.

    GEN_DELETE protocol;
    stream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(stream);
```

11. Web, HTTP client and embedded server

The Web folder groups classes to work with HTTP from two points of view. DIOWEBCLIENT allows to perform operations GET, PUT and POST on a URL and store the answer in a XBUFFER or in a file. DIOWEBHEADER represents HTTP headers. DIOWEBSERVER implements a web server. Around it appear API REST, plugins, CGI, PHP, endpoint resolution, query strings and events.

```

DIOWEBCLIENT webclient;
DIOURL        url;
XBUFFER       response;

url.Set(__L("https://www.genframework.com"));

if(webclient.Get(url, response))
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("Respuesta HTTP recibida"));
}

```

The web server is especially useful when an application needs to display a control, diagnostic or configuration interface without relying on an external server. In industrial, embedded or automation applications, a small integrated HTTP server can be a very practical way to query status, launch commands, or integrate the application with other systems.

12. ScraperWeb and obtaining external information

ScraperWeb is a DataIO block aimed at obtaining information from specific web resources. DIOSCRAPERWEB acts as a base. DIOSCRAPERWEBCACHE adds cache. Specializations solve specific needs: geolocation by IP, manufacturer of a MAC, public IP, sex detection by name, translation, user agent ID or weather information.

This block should be understood with caution. It does not replace a stable API when an official API exists, but it can be very useful when a project needs to obtain an external data in an automated and controlled way. The cache is important because it avoids repeating unnecessary queries and reduces network dependency.

```

DIOSCRAPERWEBPUBLICIP publicip;
XSTRING               ip;

if(publicip.Get(ip))
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, ip.Get());
}

```

13. Nodes: nodes, items, values and handlers

The Nodes group organizes a common idea in control systems: represent nodes containing items, values, units and handlers. A node can represent a device or a logical entity. An item can be an input, output, sensor or actuator. DIONODEITEMVALUE stores the value; DIONODEITEMVALUEUNITFORMAT helps represent units; DIONODEITEMHANDLER and its derivatives connect the item to a real source, such as a digital GPIO or a AM2315 sensor.

This model is useful when an application should treat in a homogeneous way many entry and exit points. Instead of writing specific logic for each sensor or actuator, you can organize a tree of nodes, items and values, and let handlers adapt each specific case.

```

DIONODE      node;
DIONODEITEM  item;
DIONODEITEMVALUE value;

item.GetName()->Set(__L("temperature"));
value.Set(23.5f);

// The node groups items; each item can have value, format and handler associated.

```

14. Drivers, I2C, SPI and physical devices

DataIO includes drivers for specific hardware. Some are generic or supportive, such as DIOBUSPIRATE, DIOMPSSE, DIOGPIO, DIOLEDNEOPIXELWS2812B, DIOPIFACE or DIOTEMHUMSENSORAM2301. Others are grouped under I2C and SPI and represent integrated or specific devices: GPIO expanders, OLED screens, TFT screens, light sensors,

temperature and humidity sensors, EEPROM, PWM controllers, battery chargers, AD/DA converters or touch controllers.

These drivers show one of the most important facets of GEN Framework: the framework is not intended only for desktop applications. It also serves applications close to hardware. A program can use the same general architecture of GEN Framework and, at the same time, speak with a display by SPI, a sensor by I2C or a WS2812B LED chain.

```
DIOI2CGPIOMCP2317* mcp2317 = GEN_NEW DIOI2CGPIOMCP2317();

if(mcp2317 && mcp2317->Ini(1, 0x23, 10))
{
    XBYTE value = 0;

    mcp2317->Configure();
    mcp2317->ReadInputs(DIOI2CGPIOMCP2317PORT_A, value);
    mcp2317->WriteOutputs(DIOI2CGPIOMCP2317PORT_B, 0xFF);

    mcp2317->End();
}

GEN_DELETE mcp2317;
```

The above example, based on DevTestsConsole tests, illustrates the usual way of working: to create the driver, initialize it with bus, address and timeout, configure the device, read or write records, finish and release. The exact detail changes for each integrated, but the life cycle pattern is consistent.

15. Cameras, notifications, alerts and updates

Other DataIO modules solve application needs. DIOCAMERA abstracts camera access and emit events using DIOCAMERA_XEVENT. DIONOTIFICATIONSMANAGER groups system notifications. DIOALERTS allows sending alerts across different channels, and DIOALERTSUDPSERVER can receive alerts by UDP. DIOAPPLICATIONUPDATE targets application update and uses events to report its progress. DIORemoteFileCFG allows you to work with remote settings.

These modules connect DataIO to AppFlow. A real application not only opens communications; it also needs to alert, update, post incidents, receive external signals or display status. DataIO provides input and output parts; AppFlow integrates them into the application life cycle.

```
DIONOTIFICATIONSMANAGER* notifications = GEN_DIOFACTORY.CreateNotificationsManager();

if(notifications)
{
    DIONOTIFICATION notification;

    notification.GetTitle()->Set(__L("GEN"));
    notification.GetMessage()->Set(__L("Proceso terminado"));

    notifications->Send(notification);

    GEN_DIOFACTORY.DeleteNotificationsManager(notifications);
}
```

16. Diagnosis, connectivity and operational security

DataIO includes auxiliary classes to diagnose the state of communication. DIOCHECKINTERNETCONNECTION checks general connectivity. DIOCHECKTCPIPCONNECTIONS checks TCP/IP connections. DIOPCAP and DIOPCAP_FILTERS allow you to work with packet capture when the platform and dependencies support it. DIOXTRACEINTERSTREAMS connects the trace system with DataIO's streams. DIOOSPIPELINE allows you to channel process input and output from the operating system.

There are also more specialized integration classes, such as DIOSSHREVERSE, DIOBACKDOOR or DIONATIVEMSGBROWSEREXTENSION. These pieces are not of everyday use in all applications, but show the amplitude of DataIO: communication, diagnosis, browser integration, external execution and reverse connection are part of the same input and output ecosystem.

17. Relationship with XUtils, AppFlow and other factories

DataIO should not be seen as an isolated block. XUtils provides you with types, buffers, routes, events, threads, timers and tracers. AppFlow uses it to integrate communications into console or graphics applications. UserInterface can use DataIO to display status received from sensors, protocols or web services. Graphic can render data obtained by DataIO. Cipher can encrypt information before sending it or integrating it with streams secure. GEN Framework modular architecture is based precisely on these controlled relationships.

The key is that DataIO provides the border with the outside. Everything that comes in or out of the application usually ends up going through this block or a class related to it. That's why it's good to understand its patterns: separate configuration, creation using DIOFACTORY when there is a platform, reading and writing with XBUFFER, events to report changes, and symmetric release using the same factory or life cycle that created the object.

18. Catalogue of DataIO classes and modules

The following reference summarizes the classes found in the DataIO block. It does not replace the Doxygen documentation or the reading of the .h files, but it serves as a practical map. The relationship column indicates with which other pieces the class tends to live together or which pattern of use is natural.

Core, utilities and general services

Class / module	What does it bring?	Relationship and typical use
DIOALERTS	DataIO alert system.	It integrates with XUtils and the rest of DataIO modules.
DIOALERTSUDPSERVER	UDP server to receive or distribute alerts.	It integrates with XUtils and the rest of DataIO modules.
DIOALERTS_CONDITIONS	Definition of conditions used by the alert system.	It integrates with XUtils and the rest of DataIO modules.
DIOAPPLICATIONUPDATE	Application update management.	It integrates with XUtils and the rest of DataIO modules.
DIOAPPLICATIONUPDATE_XEVENT	Extended event associated with application update.	Transports events to GEN Framework observers.
DIOBACKDOOR	Support for reverse SSH connections in Linux.	It integrates with XUtils and the rest of DataIO modules.
DIOCHECKINTERNETCONNECTION	General Internet connectivity check.	It integrates with XUtils and the rest of DataIO modules.
DIOCHECKTCPCONNECTIONS	Check for TCP/IP connections open or available.	It integrates with XUtils and the rest of DataIO modules.
DIODNSRESOLVER	DNS name resolution to IP addresses.	It integrates with XUtils and the rest of DataIO modules.
DIODEVICE	Generic DataIO device.	It integrates with XUtils and the rest of DataIO modules.
DIODEVICEI2C	Generic device connected by I2C.	It integrates with XUtils and the rest of DataIO modules.
DIODEVICESPI	Generic device connected by SPI.	It integrates with XUtils and the rest of DataIO modules.
DIODISPLAYDEVICE	Generic display device.	It integrates with XUtils and the rest of DataIO modules.
DIODYDNS_MANAGER	DynDNS Service Manager.	It integrates with XUtils and the rest of DataIO modules.
DIOEMAILADDRESS	Email address.	Support type used by streams, protocols and configuration.
DIOFACTORY	Platform-dependent DataIO object creation factory.	Main input to create platform dependent objects.

DIOGPIO	General purpose GPIO access.	It integrates with XUtils and the rest of DataIO modules.
DIOIP	Representation of a IP address.	Support type used by streams, protocols and configuration.
DIOMAC	Representation of a MAC address.	Support type used by streams, protocols and configuration.
DIOMACMANUFACTURED	Manufacturer information associated with a MAC.	It integrates with XUtils and the rest of DataIO modules.
DIONATIVEMSGBROWSEREXTENSION	Native messaging channel with browser extensions.	It integrates with XUtils and the rest of DataIO modules.
DIONOTIFICATIONSMANAGER	System or Application Notification Manager.	It integrates with XUtils and the rest of DataIO modules.
DIOPIPELINE	Input/output channeling with operating system processes.	It integrates with XUtils and the rest of DataIO modules.
DIOPCAP	Integration with PCap packet capture.	It integrates with XUtils and the rest of DataIO modules.
DIOPCAP_FILTERS	Definition and management of filters for PCap.	It integrates with XUtils and the rest of DataIO modules.
DIOPUBLICINTERNETIP	Getting the public IP output to the Internet.	It integrates with XUtils and the rest of DataIO modules.
DIOREMOTEFILECFG	Remote file-based configuration.	It integrates with XUtils and the rest of DataIO modules.
DIOSSHREVERSE	Tunnel management or reverse SSH connection.	It integrates with XUtils and the rest of DataIO modules.
DIOURL	Representation and manipulation of a URL.	Support type used by streams, protocols and configuration.
DIOWAKEONLAN	Construction and delivery of Wake on LAN packages.	It integrates with XUtils and the rest of DataIO modules.
DIOWIFIMANAGERMODE	Wi-Fi mode management when the platform supports it.	It integrates with XUtils and the rest of DataIO modules.
DIOTRACEINTERSTREAMS	Interconnection of the trace system using streams.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.

Streams, device configuration and enumeration

Class / module	What does it bring?	Relationship and typical use
DIOSTREAM	Stream DataIO base.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTH	stream BLUETOOTH implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTHCONFIG	stream BLUETOOTH configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTHLEREMOTEENUMDEVICES	Remote device enumerator BLUETOOTHLE.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTHLEREMOTEENUMDEVICES_X_EVENT	Extended event associated with the corresponding module.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTHLOCALENUMDEVICES	Local device enumerator BLUETOOTH.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTHREMOTEENUMDEVICES	Remote device enumerator BLUETOOTH.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMBLUETOOTH_XEVENTS	Extended event associated with the corresponding module.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMCIPHER	Stream with encryption layer.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMCONFIG	streams base configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMCONFIGSTRING	Configuration conversion from stream to string and vice versa.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMDEVICE	stream device base descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMDEVICEBLUETOOTH	stream BLUETOOTH device descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.

DIOSTREAMDEVICEBLUETOOTHLE	stream BLUETOOTHLE device descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMDEVICEIP	stream IP device descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMDEVICEUSB	stream USB device descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMDEVICEWIFI	stream WIFI device descriptor.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMENUMBLUETOOTHDEVICES	Lister of devices or servers associated with streams.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMENUMDEVICES	Lister of devices or servers associated with streams.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMENUMSERVERS	Lister of devices or servers associated with streams.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMI2C	stream I2C implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMI2CCONFIG	stream I2C configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMICMP	stream ICMP implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMICMPCONFIG	stream ICMP configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMIPLOCALENUMDEVICES	Local device enumerator IP.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMSPI	stream SPI implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMSPICONFIG	stream SPI configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTCPIP	stream TCP/IP implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTCPIPCONFIG	stream TCP/IP configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTCPIPSEVER	TCP/IP multi-socket server.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUART	stream UART implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUARTCONFIG	stream UART configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUARTLOCALENUMDEVICES	Local device enumerator UART.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUDP	stream UDP implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUDPACKNOWLEDGE	Stream UDP with confirmation of reception.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUDPCONFIG	stream UDP configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUDPLOCALENUMSERVERS	Lister of devices or servers associated with streams.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUSB	stream USB implementation.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUSBCONFIG	stream USB configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMUSBLOCALENUMDEVICES	Local device enumerator USB.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMWIFIREMOTEENUMDEVICES	Remote device enumerator WIFI.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAM_XEVENT	Extended event associated with the corresponding module.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.

TLS and Safe Transport Messages

Class / module	What does it bring?	Relationship and typical use
----------------	---------------------	------------------------------

DIOSTREAMTLS	Stream TLS for safe transport.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTLSCONFIG	stream TLS specific configuration.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTSLMESSAGES	Support structure for messages and extensions TLS.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTSLMESSAGE	Support structure for messages and extensions TLS.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTSLMESSAGESHANDSHAKECLIENTHELLO	Message TLS ClientHello used during handshake.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.
DIOSTREAMTSLMESSAGESHANDSHAKESERVERHELLO	Message TLS ServerHello used during handshake.	It relies on DIOSTREAMCONFIG, DIOSTREAMDEVICE and DIOFACTORY.

High-level and specific protocols

Class / module	What does it bring?	Relationship and typical use
DIOCLIPROTOCOL	Command line-oriented protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCLIPROTOCOLBUS	CLI protocol bus.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL	DatalO own core protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_CFG	Core protocol configuration.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTION	Individual connection of the core protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTIONSMANAGER	Core protocol connection manager.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTIONSMANAGER_XEVENT	Extended event associated with the corresponding module.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_HEADER	Core protocol header.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_MESSAGES	Core protocol message management.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_REGISTERDATA	Core protocol data log.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOCOREPROTOCOL_XEVENT	Extended event associated with the corresponding module.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOATCMDGSM	AT command specialization for GSM modules.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOATCMDMS	Base for AT command-based protocols.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIODNSPROTOCOL	Structures of DNS protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIODNSPROTOCOL_CLIENT	DNS protocol client.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIODNSPROTOCOL_MITM_SERVER	DNS MitM server for interception or filtering.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIODNSPROTOCOL_MITM_SERVER_XEVENT	Extended event associated with the corresponding module.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIODYNDNS	DynDNS update protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOIEC60870_5	IEC 60870-5 protocol for electrical and industrial environments.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOMODBUS_ELECTRICMETER	ModBus specialized access for electrical counter.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOMODBUS_CLIENT	ModBus protocol client.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIONTP	NTP client for UTC time.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOOBEXPUSH	OBEX Push Protocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.

DIOPING	Ping range check.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPING_XEVENT	Extended event associated with the corresponding module.	Transports events to GEN Frameworkobservers.
DIOSMTP	SMTP client for sending mail.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOSNMP	SNMP support for device management.	It usually works above DIOSTREAM or DIOWEBCLIENT.

Generic binary protocol base

Class / module	What does it bring?	Relationship and typical use
DIOPROTOCOL	Generic binary protocol base.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPROTOCOLCONNECTIONSINUSEDDEFINITIONS	Definitions of connections in use for DIOProtocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPROTOCOL_APPLICATIONDATA	Application data associated with DIOProtocol.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPROTOCOL_CONNECTIONSMANAGER	DIOProtocol Connection Manager.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPROTOCOL_CONNECTIONSMANAGER_XEVENT	Extended event associated with the corresponding module.	It usually works above DIOSTREAM or DIOWEBCLIENT.
DIOPROTOCOL_XEVENT	Extended event associated with the corresponding module.	It usually works above DIOSTREAM or DIOWEBCLIENT.

Client, Web server and components HTTP

Class / module	What does it bring?	Relationship and typical use
DIOWEBCLIENT	HTTP client for GET, PUT and POST operations.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBCLIENT_XEVENT	Extended event associated with the corresponding module.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBHEADER	Header HTTP common.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBPAGEHTMLCREATOR	HTML Page Generator.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBPAGEHTMLCREATOR_XEVENT	Extended event associated with the corresponding module.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER	Integrated web server.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_APIREST	API REST support on DIOWebServer.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_PLUGIN	Plugin base for the web server.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_PLUGIN_CGI	Plugin CGI for the web server.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_PLUGIN_PHP	Plugin PHP for the web server.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_PLUGINSMANAGER	Web server plugin manager.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_QUERYSTRINGS	Query strings management HTTP.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_RESOLVEENDPOINT	Web server endpoint resolution.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.
DIOWEBSERVER_XEVENT	Extended event associated with the corresponding module.	He works with DIOURL, DIOWEBHEADER and streams TCP/IP.

ScraperWeb and external services

Class / module	What does it bring?	Relationship and typical use
DIOSCRAPERWEB	Base for extracting information from web resources.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBCACHE	Cache for ScraperWeb results.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBGEOLOCATIONIP	Scraper web specialized in geolocationip.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBMACMANUFACTURER	Scraper web specialized in macmanufacturer.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBPUBLICIP	Scraper web specialized in publicip.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBSEXNAME	Scraper web specialized in sexname.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBTRANSLATION	Scraper web specialized in translation.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBUSERAGENTID	Scraper web specialized in useragentid.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.
DIOSCRAPERWEBWEATHER	Scraper web specialized in weather.	Specialization on DIOSCRAPERWEB and DIOWEBCLIENT.

Node and value model

Class / module	What does it bring?	Relationship and typical use
DIONODE	Base node for control systems.	It is part of the node model, items, values and events.
DIONODEITEM	Item associated with a node.	It is part of the node model, items, values and events.
DIONODEITEMHANDLER	Handler base to link an item to a real source.	It is part of the node model, items, values and events.
DIONODEITEMHANDLER_GPIODIGITAL	Item handler for GPIO digital.	It is part of the node model, items, values and events.
DIONODEITEMHANDLER_SENSORAM2315	Item handler for AM2315 sensor.	It is part of the node model, items, values and events.
DIONODEITEMVALUE	Value associated with a node item.	It is part of the node model, items, values and events.
DIONODEITEMVALUEUNITFORMAT	Format and unit of a node value.	It is part of the node model, items, values and events.
DIONODEMANAGER	Node manager.	It is part of the node model, items, values and events.
DIONODE_XEVENT	Extended event associated with the corresponding module.	It is part of the node model, items, values and events.

General hardware Drivers

Class / module	What does it bring?	Relationship and typical use
DIOAIOBIMAINBOARD	AIOWI Main board.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOBUSPIRATE	interface Bus Pirate 4.0.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOLEDNEOPIXELWS2812B	Led Neopixel WS2812B.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOLIGHTSENSORLDRANALOG	IO Sensor Light LDR (Light Dependent Resistor) Analog in port digital.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOMPSSE	Multi-Protocol Synchronous Serial Engine (MPSSE).	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOPIFACE	Raspberry Pi Face.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOTEMHUMSENSORAM2301	AOSONG AM2301 Sensor (Temperature and	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an

	humidity module).	equivalent physical driver.
--	-------------------	-----------------------------

Drivers I2C

Class / module	What does it bring?	Relationship and typical use
DIOI2C6AXISTRACKINGBMI270	Driver I2C for 6AXISTRACKINGBMI270.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2C6AXISTRACKINGLSM303DLHC	Driver I2C for 6AXISTRACKINGLSM303DLHC.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2C9AXISTRACKINGMPU9150	Driver I2C for 9AXISTRACKINGMPU9150.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CADDACONVERTERPCF8591	Driver I2C for ADDACONVERTERPCF8591.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CAIRQUALITYCCS811	Driver I2C for AIRQUALITYCCS811.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CBATTERYCHARGERBQ24295	Driver I2C for BATTERYCHARGERBQ24295.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CEEPROM24XXX	Driver I2C for EEPROM24XXX.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CGPIOMCP2317	Driver I2C for GPIOMCP2317.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CGPIOPCF8574	Driver I2C for GPIOPCF8574.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CLIGHTSENSORSL2561	Driver I2C for LIGHTSENSORSL2561.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CMONITORGAUGELTC2942	Driver I2C for MONITORGAUGELTC2942.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2COLEDDISPLAYSSD1306	Driver I2C for OLEDDISPLAYSSD1306.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CPWMCONTROLLERPCA9685	Driver I2C for PWMCONTROLLERPCA9685.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CTEMHUMSENSORAM2315	Driver I2C for TEMHUMSENSORAM2315.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CTEMHUMSENSORSHT20	Driver I2C for TEMHUMSENSORSHT20.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2CTOUCHSENSORAT42QT1060	Driver I2C for TOUCHSENSORAT42QT1060.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOI2C_DEVICES	Driver I2C for _DEVICES.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.

Drivers SPI

Class / module	What does it bring?	Relationship and typical use
DIOSPIGPIOMCP23S17	Driver SPI for GPIOMCP23S17.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPILCDDISPLAYPCF8833	Driver SPI for LCDDISPLAYPCF8833.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPIOLEDDISPLAYSSD1306	Driver SPI for OLEDDISPLAYSSD1306.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPIOLEDDISPLAYSSD1331	Driver SPI for OLEDDISPLAYSSD1331.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPITFTDISPLAYILI9341	Driver SPI for TFTDISPLAYILI9341.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPITFTDISPLAYST7789	Driver SPI for TFTDISPLAYST7789.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPITOUCHSCREENSTMPE610	Driver SPI for TOUCHSCREENSTMPE610.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.
DIOSPI_DEVICES	Driver SPI for _DEVICES.	It is used on DIOSTREAMI2C, DIOSTREAMSPI or an equivalent physical driver.

Cameras and camera events

Class / module	What does it bring?	Relationship and typical use
DIOCAMERA	Camera abstraction.	It integrates with XUtils and the rest of DataIO modules.
DIOCAMERA_XEVENT	Extended event emitted by a camera.	Transports events to GEN Frameworkobservers.

Location and addresses

Class / module	What does it bring?	Relationship and typical use
DIOLOCATIONADDRESS	Address container and location data.	Support type used by streams, protocols and configuration.

19. How to study DataIO from DevTestsConsole

DevTestsConsole is the best add-on to study DataIO because it contains actual usage tests. Its sources include examples of DIOSTREAMTCPIP, DIOSTREAMTLS, GODOTreamUSB, Bluetooth enumeration, DIONTP, DIODNSResolver, DIOWakeOnLAN, DIOWifiManagerModel, DIONotificationsManager, DIOATCMDGSM, DIOSNMP, drivers I2C and drivers SPI. Not all tests will be enabled on all platforms, but the code shows the pattern to be learned.

The recommended order to study a DataIO class is the same as in other GEN Framework modules. First, the header is checked to see what API offers. Then, the Doxygen documentation is consulted to understand inheritance, methods, and relationships. Finally, a test is sought in DevTestsConsole or official examples to see how the object is created, initialized, and released in a real application.

In DataIO it is especially important to review also the build macros. Many modules are conditioned by flags such as DIO_STREAMTCPIP_ACTIVE, DIO_STREAMUDP_ACTIVE, DIO_STREAMI2C_ACTIVE, DIO_STREAMSPI_ACTIVE, DIO_PCAP_ACTIVE or equivalent. That a class exists in the source code does not mean that it is active in all compilations or that the platform can create it.

20. Criteria for use in an application with GEN Framework

When an application needs to communicate with the outside, the general rule is not to skip DataIO. If you have to open a TCP/IP connection, you must use DIOSTREAMTCPIPCONFIG and DIOFACTORY. If you have to send UDP, use DIOSTREAMUDP. If you have to speak with a serial port, use DIOSTREAMUART. If you have to consult a URL, use DIOWEBCLIENT. If you have to get network time, use DIONTP. If you have to treat a IP, a MAC or a URL, use DIOIP, DIOMAC and DIOURL.

This discipline prevents each application from inventing its own way of solving the same problems. In addition, it allows the code to move from one platform to another with fewer changes. If an application calls a native API directly, it docks to that platform. If it uses DataIO, the dependency is confined to the platform layer and the corresponding factory.

What is created with GEN_DIOFACTORY must be destroyed with GEN_DIOFACTORY. What is created with GEN_NEW must be released with GEN_DELETE or with the corresponding class pattern. A stream must be closed before being destroyed. A driver must run End() if it was initialized. This order is not just style: in communications and hardware it avoids leaving open resources, sockets hanged, busy buses or unfinished threads.

21. Practical summary

DataIO is the second big block that shaped GEN Framework. Its name, Data Input Output, summarizes its responsibility: to organize the input and output of an application in a portable, coherent and reusable way.

Its architectural center is DIOFACTORY, which allows to isolate platform differences in the creation of streams, enumerations, managers, PCap, Wi-Fi, notifications and other system dependent objects. On top of that base appear streams, protocols, web, scraping, nodes, drivers and auxiliary services.

The application that uses DataIO correctly gains order. Instead of mixing native calls, sockets, buffers, timeouts and protocols anywhere in the code, it concentrates communication on framework classes. This makes the code easier to read, easier to carry, and easier to diagnose.

After XUtils, DataIO is the block that most clearly demonstrates the practical vocation of GEN Framework: solving real communication, network, hardware and external services problems without breaking the common philosophy of framework.

Chapter VII: Graphic, the graphical system of GEN Framework

1. The third block that shaped GEN Framework

Graphic is the third big block that finished converting GEN Framework into something larger than a collection of utilities and input/output classes. XUtilities solved the basic pieces that any C/C++ programmer just needed. DataIO added communication with the outside, streams, protocols, devices and network services. Graphic incorporated the ability to build visual applications, draw on screen, work with images, manage graphic formats, create windows, use canvas 2D and generate vector or statistical representations.

In a real application, the graphic part is not just aesthetic. A screen is a frontier of communication with the user. A canvas can represent a machine, a device, a production line, an industrial panel, a diagnostic interface, a data viewer, an embedded screen or a configuration tool. Therefore, the Graphic module should not be seen only as a set of functions for painting lines and rectangles. It is a complete layer for converting data and internal states into visual representation.

As in the rest of GEN Framework, Graphic is not born as an academic abstraction. Its classes respond to specific problems: creating screens on different platforms, having a common canvas 2D, loading bitmaps, rendering SVG, interpreting DXF, generating graphics, controlling views, measuring frames, managing image sequences, reading video or presenting a buffer using OpenGL ES. Each functionality exists because in some projects it was necessary to solve that need in a portable and reusable way.

The result is a practical graphic block, closely linked to the rest of the framework. It is not intended to replace all existing graphics engines or compete with tools of very different purpose. Its aim is to give applications with GEN Framework a coherent, portable and sufficient graphic base to develop interfaces, viewfinders, examples, control tools, dashboards, embedded screens and multiplatform graphical applications without filling the native call code of each operating system.

2. GRPFACTORY and platform insulation

The centerpiece of the graphic architecture is GRPFACTORY. Just as XFACTORY and DIOFACTORY isolate platform-dependent objects in their respective blocks, GRPFACTORY centralizes the creation of graphical components whose implementation may depend on the operating system, the graphical backend or pixel mode.

The most obvious case is the screen. Creating a window in Windows, a surface in Linux, a graphical activity in Android or an output on an embedded display is not resolved with the same APIs. The application should not decide which specific kind of display to instance. That responsibility belongs to the factory and platform classes confined in Platform.

The common code of Graphic defines the interface and part of the shared logic. Concrete platform implementations specialize that factory to create screens, desktop managers or native integrations. The same pattern applies to other objects when the platform intervenes. Instead, there are objects that can be created commonly, such as certain bitmaps or canvas based on AGG, because their operation depends more on the pixel format than the native window.

This distinction is important: GRPFACTORY is not just a window factory. It is the point where the application ceases to know the detail of the platform. If a graphical functionality depends on the system, it must live behind the factory or inside the Platform tree. If a functionality can be common, it is kept inside the Graphic block and reused on all platforms.

```
GRPPROPERTIES properties;
GRP2DCANVAS* canvas = NULL;
```

```

properties.SetWidth(800);
properties.SetHeight(480);
properties.SetMode(GRPPROPERTYMODE_32_RGBA_8888);

canvas = GEN_GRPFACTORY.CreateCanvas(&properties);
if(canvas)
{
    GRP2DCOLOR_RGBA8 color(255, 255, 255);

    canvas->Buffer_Create();
    canvas->SetFillColor(&color);
    canvas->Rect(0.0f, 0.0f, 800.0f, 480.0f, true);

    GEN_GRPFACTORY.DeleteCanvas(canvas);
}

```

The example shows the typical pattern of the factory: a configuration or properties is prepared, the object is requested, it is checked if it exists, it is worked with and it is released by the same factory. In a complete graphic application, AppFlowGraphics is usually responsible for creating the main screen and issuing the corresponding events, but the philosophy of ownership and creation is the same.

3. Graphic within an AppFlowGraphics application

The examples in the Graphics directory show the practical use of this block. Canvas2D focuses on drawing directly on a canvas. StatisticsCharts shows the generation of statistical graphics. UI_Message and UI_Options combine Graphic with UserInterface. WindowChromes works with custom window frame configuration. They are all supported by AppFlowGraphics, which encapsulates program input, life cycle, and reception of graphic events.

An important detail, already discussed in the chapter of the first application, is that the programmer does not write a main function of his own. In an application with GEN Framework, the input is integrated into the framework. The application receives extension points and events: initialization, update, completion, screen creation, creation of canvas, user input and other messages. This allows the graphic part to be portable and the platform to be confined to the framework.

The usual pattern is to receive a GRPXEVENT screen creation, configure the GRPSCREEN object, create one or more views and load resources from assets using XPATHSMANAGER. The Canvas2D example follows precisely that model.

```

void CANVAS2D::HandleEvent_Graphics(GRPXEVENT* event)
{
    switch(event->GetEventType())
    {
        case GRPXEVENT_TYPE_SCREEN_CREATING : { GRPSCREEN* screen = event->GetScreen();
                                                if(!screen) break;

                                                Ini_Graphics(screen);
                                                }
                                                break;
    }
}

```

The event does not contain complex business logic. It serves to connect the exact moment when the framework is creating the screen with the graphical configuration code of the application. That separation prevents the application from relying on the main function or native calls of the platform.

4. Displays, views and graphic properties

GRPSCREEN represents the main screen or window from the framework point of view. It allows setting width, height, position, title, styles, selected desktop screen, rotation and views. Internally, the actual implementation can change depending on the platform, but the application works with the same conceptual interface.

GRPVIEWPORT defines a rendering zone within the screen. This class is essential when you want to separate the physical size of the window from the logical drawing coordinates. A screen can be a specific size in pixels and, at the same time, a viewport representing a different logical space. This is useful for scaling backgrounds, adapting interfaces, drawing maps, rendering scenes or isolating work regions.

GRPPROPERTIES groups common properties such as dimensions and pixel mode. Many graphic classes inherit or copy these properties because a bitmap, a canvas or a screen need to share information about formatting and size. GRPRECT allows to express rectangular areas and is used in cuts, refresh regions and geometric operations.

```
bool CANVAS2D::Ini_Graphics(GRPSCREEN* screen)
{
    if(!screen) return false;

    screen->SetPosition(GRPPROPERTYMODE_SCREEN_CENTER, GRPPROPERTYMODE_SCREEN_CENTER);
    screen->SetWidth(1024);
    screen->SetHeight(768);
    screen->SetTitle()->Set(__L("Canvas 2D"));
    screen->SetDesktopScreenSelected(GRPSCREENTYPE_DESKTOP_SCREEN1);

    return screen->CreateViewport(APPLICATION_GUI_MAIN_VIEWPORT_ID,
                                0.0f, 0.0f,
                                (float)screen->GetWidth(), (float)screen->GetHeight(),
                                0, 0,
                                screen->GetWidth(), screen->GetHeight());
}
```

This skeleton reflects the central idea: the application configures the screen using GEN Framework properties, not native APIs. The main viewport is associated with an identifier and can be recovered later to obtain the canvas and draw.

5. Canvas 2D, colors, paths and reconstruction areas

GRP2DCANVAS is the 2D drawing surface of the framework. On it are drawn primitives, pixels, lines, rectangles, ellipses, texts, bitmaps and paths. The application should not worry about how the buffer is stored internally or why backend is used. It works with a common drawing interface.

GRP2DCANVASAGG is AGG based implementation. AGG provides high quality 2D rasterization and allows you to work with different pixel formats. The factory can create canvas specific for RGB565, RGB888, BGRA8888 or other modes. This makes a lot of sense in a framework that aims to reach both desktop and embedded systems: not all screens have the same color format and memory cost.

GRP2DCOLOR and its variants allow to express colors. GRP2DPATH represents vector paths. GRP2DVERTICES groups vertices. GRP2DREBUILDAREAS helps to manage areas that need to be redrawn. In graphic interfaces or small displays, it is not always interesting to repaint the entire screen; being able to identify modified areas helps to optimize the refresh.

```
GRP2DCOLOR_RGBA8 colorwhite(255, 255, 255);
GRP2DCOLOR_RGBA8 colorgray ( 10,  10,  10, 150);

GRPVIEWPORT* viewport = GetMainScreen()->GetViewport(APPLICATION_GUI_MAIN_VIEWPORT_ID);
if(viewport)
{
    GRP2DCANVAS* canvas = viewport->GetCanvas();
    if(canvas)
    {
        canvas->SetFillColor(&colorgray);
        canvas->Ellipse(75.0f, 130.0f, 40.0f, 8.0f, true);

        canvas->SetFillColor(&colorwhite);
        canvas->Rect(10.0f, 10.0f, 200.0f, 80.0f, true);
    }
}
```

The call to the canvas expresses graphic intent, not platform detail. The same logic can be run on a desktop window, a Android surface or a backend adapted to a particular display, provided that the platform provides the necessary implementation.

6. Bitmaps, sequences and raster formats

GRPBMAP represents a raster image in memory. Unlike a simple buffer, it knows its dimensions, its pixel mode and its own operations of an image: obtain or write pixels, mix with alpha, clone, copy, convert mode, flip, rotate, scale, crop, compare or search for a sub-bitmap. This makes it a central piece for 2D interfaces, sprites, backgrounds, captures and image manipulation.

GRPBMAPSEQUENCE allows you to group multiple bitmaps as an animated sequence. In the example Canvas2D is used to load a frame animation to frame and play it. This pattern is useful when you want to animate a character, an icon, an indicator or a transition without using a more complex animation system.

GRPBMAPFILE is the base class for loading and saving bitmaps. GRPBMAPFILEBMP, GRPBMAPFILEPNG, GRPBMAPFILEJPG and GRPBMAPFILETGA specialisations encapsulate specific formats. Thus, the application does not need to know each library or image parser. It asks to load a file and receives a GRPBMAP adapted to screen mode.

```

XPATH          xpath;
GRPBMAPFILE*   bitmapfile = GEN_NEW GRPBMAPFILE();
GRPBMAP*       background = NULL;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
xpath.Slash_Add();
xpath.Add(__L("background.jpg"));

background = bitmapfile->Load(xpath, GetMainScreen()->GetMode());

if(background)
{
    // The bitmap can already be drawn on a canvas or used as a background.
}

delete bitmapfile;

```

The example also shows the relationship with XPATHSMANAGER. The graphical resources of the application should be under assets, usually in the Graphics section. This keeps files sorted and makes all applications use a homogeneous way to locate images, fonts, SVG, DXF or other resources.

7. Vector files: common base, SVG and DXF

The FormatFiles/Vector block is one of the most interesting parts of Graphic. GRPVECTORFILE defines the common basis for vector documents. From it, specializations appear for SVG and DXF. This separation allows working with the general idea of vector file without the application having to know all the details of the specific format from the first moment.

SVG is a very useful format for interfaces, icons, independent resolution scaling and graphics generation. parser SVG from GEN Framework organizes the document as an object tree: rectangles, circles, ellipses, lines, paths, polygons, polylines, texts, gradients, styles, transformations and references use. Each object has a specific class, but they are all part of the same document model.

DXF, for its part, connects Graphic to the CAD world. DXF classes manage sections, values, entities and geometric objects such as lines, circles, arcs, ellipses, polylines, texts, MText, points and 3DFace. This allows loading planes, schemas, technical geometry or test files for vector rendering.

```

XPATH          pathvf;
GRPVECTORFILE* vectorfile = NULL;

```

```

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, pathvf);
pathvf.Slash_Add();
pathvf.Add(__L("logo.svg"));

vectorfile = GRPVECTORFILE::CreateInstance(pathvf);
if(vectorfile)
{
    if(vectorfile->Load(pathvf) == GRPVECTORFILERESULT_OK)
    {
        // The document can be inspected or rendered on a canvas.
    }

    delete vectorfile;
}

```

The creation by path allows the framework to select the appropriate type from the format. It can also be created by explicit type, as in StatisticsCharts when a SVG is generated in memory to contain a graph.

8. Vector render over canvas

The GRP2DVECTORFILERENDERAGG, GRP2DVECTORFILESVGRENDERAGG and GRP2DVECTORFILEDXFRENDERAGG classes connect the world of vector documents with the canvas 2D. A SVG or a DXF are not directly pixels; they are geometric structures, styles, transformations and entities. The renderer runs through that structure and translates it into drawing operations on a GRP2DCANVAS.

This separation is important from an architectural point of view. parser from SVG or DXF should not depend on a specific screen. canvas should not know how a DXF is organized internally. The renderer is the intermediate piece: it understands the vector document and knows how to convert it into primitive 2D.

In recent versions of the framework, work has been done precisely on this area to centralize the vector rendering cache and avoid duplicating logic between SVG and DXF. This fits the philosophy of GEN Framework: when a common part appears, it must go up to a common class; when a format difference appears, it must be confined to the corresponding specialization.

```

GRP2DVECTORFILERENDERAGG renderer;

if(vectorfile && canvas)
{
    renderer.SetCanvas(canvas);
    renderer.Render(vectorfile);
}

```

The above fragment expresses the conceptual pattern. The exact name of the functions may depend on the concrete version, but the idea is the same: the vector document is kept separate from the canvas and the renderer performs the translation between the two worlds.

9. OpenGL ES, EGL and accelerated submission

Graphic also includes a OpenGL layer. GRPEGLCONTEXT manages the creation and administration of a EGL context. GRPBLITGLES acts as OpenGL ES 3.0: takes a canvas or buffer 2D and presents it using the GPU. This architecture is very useful when the 2D drawing is generated in memory, but the final presentation must be efficient.

The idea is not to force every graphical application to program directly in OpenGL. On the contrary: many applications can draw with GRP2DCANVAS and let the platform or backend present the result. OpenGL ES appears as an output layer, not necessarily as the API that must use the entire graphical logic of the application.

This separation allows the same drawing code to coexist with different presentation mechanisms. On desktop there may be a native window. In Android there may be a graphic surface. In a embedded system there may be a

framebuffer or display with particular restrictions. The application should not have to rewrite its logic for each destination.

10. Video and extraction of frames

GRPVIDEOFILE and GRPVIDEOFILEAVI extend the graphic block to video reading. They are not the core of most applications, but they allow working with frames sequences and video properties when the project needs it. Examples show uses of GRPVIDEOFILEAVI to open a AVI file, query properties and extract frames that can subsequently be converted or drawn as bitmaps.

```
GRPVIDEOFILEAVI* avifile = GEN_NEW GRPVIDEOFILEAVI();

if(avifile)
{
    if(avifile->Open(xpath))
    {
        GRPVIDEOFILE_PROPERTYYS propertyys;

        avifile->GetPropertyys(propertyys);
        // From here you can get frames and treat them as images.

        avifile->Close();
    }

    delete avifile;
}
```

The pattern is the same again: specialized object, opening, consulting or reading, closing and release. Although the AVI format is concrete, the style of use remains that of GEN Framework.

11. Statistical Charts

The StatisticsChars directory contains a small graphic generation system. The historical name of the directory should not distract from its function: creating statistical representations from data, visual configuration and an output builder. The base class GRPSTATISTICSCHART defines common logic. GRPSTATISTICSCHARTDATA contains categories and series. GRPSTATISTICSCHARTCONFIG defines title, colors, grid, legend, font sizes and margins. GRPSTATISTICSCHARTBUILDER abstracts output, and GRPSTATISTICSCHARTBUILDERSVG allows generating a SVG.

Above that base appear specific types: columns, lines, areas, bars, stacked columns and circular sectors, with 2D and 3D versions. This allows generating dashboards, visual reports or graphic elements without relying on a high-level external library.

```
GRPSTATISTICSCHART* chart = GEN_NEW GRPSTATISTICSCHARTCOLUMNS();

if(chart)
{
    chart->GetConfig()->SetTitle(__L("Sales by quarter"));
    chart->GetConfig()->SetShowValues(true);
    chart->GetConfig()->SetShowLegend(true);

    GRPSTATISTICSCHARTDATA* data = chart->GetData();
    data->AddCategory(__L("Q1"));
    data->AddCategory(__L("Q2"));
    data->AddCategory(__L("Q3"));
    data->AddCategory(__L("Q4"));

    GRPSTATISTICSCHARTSERIE* serie = data->AddSerie(__L("2026"));
    serie->AddValue(10.0);
    serie->AddValue(18.0);
    serie->AddValue(15.0);
    serie->AddValue(22.0);

    GRPSTATISTICSCHARTBUILDERSVG builder;
    if(chart->Generate(builder, 420.0, 420.0) == GRPVECTORFILERESULT_OK)
```

```

    {
        XSTRING svg;
        builder.GetResult(svg);
    }

    delete chart;
}

```

The example is representative of the design: the graph does not write directly on the screen. It generates an output through a builder. In this case the output is SVG, which can be stored, rendered, loaded as GRPVECTORFILE or integrated into an interface.

12. Graphic and UserInterface

UserInterface is naturally supported in Graphic. An interface needs display, canvas, colors, bitmaps, fonts, graphic events and, in many cases, SVG or images. That is why the examples UI_Message and UI_Options combine both blocks. Graphic provides the drawing and presentation infrastructure; UserInterface provides widgets, layouts, interaction and composition of interface elements.

This separation allows you to use Graphic without UserInterface when you want to draw directly on a canvas, as in Canvas2D, or use UserInterface when you want to work with higher level controls. In small applications or embedded screens you can simply draw directly. In applications with more interaction, forms, buttons, messages or menus, UserInterface avoids reinventing controls in each project.

The programmer must understand that UserInterface does not replace Graphic. Similarly, Graphic does not replace AppFlowGraphics. It integrates with it to receive the life cycle and events. This layered composition is one of the keys to GEN Framework.

13. General Map of Graphic

The Graphic block is wide. To study it, it is advisable to group it by responsibility, not alphabetical order. The following table summarizes its large areas.

Group	Principal responsibility	Representative classes
Graphical core	Displays, properties, views, events, frame rate and graphical factory.	GRPFACORY, GRPSCREEN, GRPVIEWPORT, GRPPROPERTIES, GRPXEVENT
Canvas 2D	2D raster drawing, colors, paths, vertices and repainting areas.	GRP2DCANVAS, GRP2DCANVASAGG, GRP2DPATH, GRP2DCOLOR, GRP2DREBUILDAREAS
Bitmaps	Images in memory, animated sequences and raster format loading/saved.	GRPBITMAP, GRPBITMAPSEQUENCE, GRPBITMAPFILE, GRPBITMAPFILEPNG, GRPBITMAPFILEJPG
Vector	Load, analyze and render SVG and DXF documents.	GRPVECTORFILE, GRPVECTORFILESVG, GRPVECTORFILEDXF, GRP2DVECTORFILERENDERAGG
OpenGL	Context EGL and accelerated presentation using OpenGL ES.	GRPEGLCONTEXT, GRPBLITGLS
Video	Reading video formats and extracting information or frames.	GRPVIDEOFILE, GRPVIDEOFILEAVI
Statistical Charts	Data model, configuration and generation of 2D/3D graphics, especially towards SVG.	GRPSTATISTICSCHART, GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTBUILDERSVG

14. Catalogue of Graphic classes and modules

The following reference summarizes the classes found in the Graphic block. It should be read as a guidance map. The Doxygen documentation and the examples in the Graphics directory complete the detail of exact methods, parameters and dependencies.

Core

Class / module	What it does	Relationship and typical use
GRPBITMAP	Raster image in memory. Provides buffer, pixel access, copy, scale, rotation, trimming and comparison.	It is created by GRPFACTORY or bitmap readers.
GRPBITMAPSEQUENCE	Bitmap sequence used for frame animations to frame.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRPDESKTOPMANAGER	Desktop and monitor manager when the platform supports it.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.
GRPDISPLAYDEVICE	Generic description of a physical or logical display device.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.
GRPFACTORY	Graphical Factory. Create screens, canvas, bitmaps and platform dependent managers when applicable.	It is the entry point for graphical objects created by platform.
GRPFramerate	Measurement and control of frame rhythm.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRPPROPERTIES	Common properties of graphic objects: width, height, pixel mode and other shared attributes.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRPRECT	Template for rectangles and areas. It is used in cuts, refresh zones and geometric operations.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRPSCREEN	Abstraction of screen or graphic window. Defines size, position, styles, rotation, views and creation events.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.
GRPSCREENCFGCHROMES	Custom window chromes configuration: title, buttons, edges and frame appearance.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.
GRPVIEWPORT	Logical rendering zone within a screen. Allows mapping of screen coordinates and content coordinates.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.
GRPXEVENT	Display-issued graphic event, canvas and graphic subsystems.	It is used from AppFlowGraphics and emits XEVENT_TYPE_GRAPHICS events.

2D and canvas

Class / module	What it does	Relationship and typical use
GRP2DCANVAS	2D drawing surface. It offers primitive drawing, text, bitmaps, strokes, fillers and operations on the buffer.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.
GRP2DCANVASAGG	canvas 2D implementation based on AGG for specific pixel modes.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.
GRP2DCOLOR	2D color representation in RGBA or other supported formats.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.
GRP2DPATH	2D vector path. Group movements, lines, curves and reusable geometry.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.
GRP2DREBUILDAREAS	Management of areas that need to be redrawn or rebuilt.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.
GRP2DVECTORFILE DXFRENDERAGG	DXF renderer on canvas AGG.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRP2DVECTORFILE RENDERAGG	Common vector file renderer over canvas AGG.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRP2DVECTORFILE SVGRENDERAGG	SVG renderer on canvas AGG.	It integrates with XUtilities, AppFlowGraphics and the rest of the Graphic block.
GRP2DVERTXS	2D vertex container.	He works with GRPSCREEN, GRPVIEWPORT and GRPBITMAP.

Bitmap formats

Class / module	What it does	Relationship and typical use
GRPBITMAPFILE	Base class for loading and saving bitmap files.	Load or save GRPBITMAP from assets/Graphics.
GRPBITMAPFILE BMP	Support for BMP files.	Load or save GRPBITMAP from assets/Graphics.
GRPBITMAPFILE JPG	Support for JPEG files.	Load or save GRPBITMAP from assets/Graphics.
GRPBITMAPFILE PNG	Support for PNG files.	Load or save GRPBITMAP from assets/Graphics.
GRPBITMAPFILE TGA	Support for TGA/TARGA files.	Load or save GRPBITMAP from assets/Graphics.

Common Vector

Class / module	What it does	Relationship and typical use
GRPVECTORFILE	Base class of vector files. Allows to create instances by type or path.	Common base for SVG, DXF and vector file events.
GRPVECTORFILECONFIG	Common configuration of vector files.	Common base for SVG, DXF and vector file events.
GRPVECTORFILE_XEVENT	Event associated with loading, processing or rendering vector files.	Common base for SVG, DXF and vector file events.

SVG

Class / module	What it does	Relationship and typical use
GRPVECTORFILESVG	Document SVG: Load, analyze and maintain the SVG object tree.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG CSSSTYLESHEET	CSS style sheet applied to a SVG.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG CONFIG	SVG specific configuration.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ	Base object of the SVG tree and SVG node factory.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ CIRCLE	Circle SVG node.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ ELLIPSE	Ellipse SVG node.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ GRADIENT	Paint server for linear or radial gradients.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ LINE	SVG line node.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ PATH	SVG node of path. It interprets the attribute d.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ POLYLINE	SVG polyline node.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ POLYGON	SVG node of polygon.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ RECT	SVG node of rectangle.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ TEXT	SVG node of text and tspan.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ USE	Node SVG use, reference another item per identifier.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG STYLE	SVG style: filling, stroke, opacity, width and rules of painting.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG TRANSFORM	Related transformation SVG into matrix 2x3.	It is loaded with GRPVECTORFILE and can be rendered with GRP2DVECTORFILESVGRENDERAGG.

DXF

Class / module	What it does	Relationship and typical use
GRPVECTORFILEDXF	Document DXF: Load, analyze sections and entities of a CAD file.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF CONFIG	DXF reader-specific configuration.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF ENTITY	Entity DXF base.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF POINT	auxiliary definition DXF: POINT.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF STRINGVALUE	auxiliary definition DXF: STRINGVALUE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF VERTEX	auxiliary definition DXF: VERTEX.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF ENTITYOBJ	Geometric entity DXF base.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ 3DFACE	Geometric entity DXF specialized: 3DFACE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ ARC	Geometric entity DXF specialized: ARC.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ CIRCLE	Geometric entity DXF specialized: CIRCLE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ ELLIPSE	Geometric entity DXF specialized: ELLIPSE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ LWPOLYLINE	Geometric entity DXF specialized: LWPOLYLINE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.

LWPOLYLINE		rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ LINE	Geometric entity DXF specialized: LINE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ MTEXT	Geometric entity DXF specialized: MTEXT.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ POINT	Geometric entity DXF specialized: POINT.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ POLYLINE	Geometric entity DXF specialized: POLYLINE.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ TEXT	Geometric entity DXF specialized: TEXT.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTBLOCK	Internal text structure of the parser DXF.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTPART	Internal text structure of the parser DXF.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTSECTION	Textual section DXF base.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION BLOCKS	DXF text section: BLOCKS.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION ENTITIES	DXF text section: ENTITIES.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION HEADER	DXF text section: HEADER.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF VALUE	DXF value associated with group codes.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF XDATACTRL	XDATA Extended Data Control from DXF.	It is part of the parser DXF and can be rendered with GRP2DVECTORFILEDXFRENDERAGG.

Video

Class / module	What it does	Relationship and typical use
GRPVIDEOFILE	Base class for video formats.	Video reading and extraction of frames as bitmaps.
GRPVIDEOFILEAVI	AVI read support.	Video reading and extraction of frames as bitmaps.

OpenGL

Class / module	What it does	Relationship and typical use
GRPBUTGLES	Blitter OpenGL ES to bring a canvas or buffer 2D screen using GPU.	Acceleration and presentation layer on OpenGL ES/EGL.
GRPEGLCONTEXT	EGL Context Manager for OpenGL ES.	Acceleration and presentation layer on OpenGL ES/EGL.

Statistical Charts

Class / module	What it does	Relationship and typical use
GRPSTATISTICSCHART	Abstract database of statistical graphs.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART AREA	Area chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART AREA3D	3D area chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART BARS	Horizontal bar chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART BARS3D	3D Bar Chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART BUILDER	Builder abstract output for graphics.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART BUILDERSVG	Builder which converts the graph to SVG.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART COLUMNS	Chart of grouped columns.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART	3D Column Chart.	Use GRPSTATISTICSCHARTDATA,

COLUMNS3D		GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART CONFIG	Visual configuration of the graphics: title, palette, legend, grid and fonts.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART DATA	Chart data model: categories, series and values.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART LINES	Line chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART LINES3D	3D Line Chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART PIE	Circular graph or doughnut.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART PIE3D	Circular 3D chart.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART STACKEDCOLUMNS	Chart of stacked columns.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART STACKEDCOLUMNS3D	Chart of stacked 3D columns.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.
GRPSTATISTICSCHART COLOR	Style types independent of format: color, fill, stroke and text.	Use GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG and builders such as SVG.

15. How to study Graphic from examples

To study Graphic it is not advisable to start with the inner classes. It is better to follow the official examples. Canvas2D shows the relationship between AppFlowGraphics, GRPSCREEN, GRPVIEWPORT, GRP2DCANVAS, GRPBITMAPFILE, XPATHSMANAGER and graphic events. It is the right example to understand direct drawing.

StatisticsCharts is the right example to study statistical graphics classes and generation flow to SVG. It allows you to see how a chart is created, how categories and series are fed, how the legend is configured and how a builder produces a vector result.

UI_Message and UI_Options show the relationship between Graphic and UserInterface. They show that a user interface does not live outside the graphical system: it needs display, canvas, events, resources and window settings. WindowChromes shows window customization and use of GRPSCREENCFGCHROMES.

The recommended order of study is: first Canvas2D, then WindowChromes, then UI_Message or UI_Options, and finally StatisticsCharts if you want to delve deeper into generation of vector graphics and statistics. While studying those examples, you should consult Doxygen to see inheritances, enumerations and available methods.

16. Criteria for use in an application with GEN Framework

When an application needs to display graphic information, the general rule is not to skip Graphic. If you have to create a screen, you have to work using AppFlowGraphics and GRPSCREEN. If you have to draw, you use GRP2DCANVAS. If you have to load an image, you use GRPBITMAPFILE. If you have to treat an image in memory, you use GRPBITMAP. If you have to load a SVG or DXF, you use GRPVECTORFILE and its specialisations. If you have to generate a statistical graph, you use the GRPSTATISTICSCHART block.

This discipline prevents each application from inventing its own way of drawing or uploading resources. It also allows the graphical logic of the application to survive platform changes better. A direct call to a native window API links the program to that platform. A call to Graphic maintains dependency within the framework.

The property order must also be respected. Objects created by GRPFACTORY must be destroyed by GRPFACTORY. Objects created with GEN_NEW must be released with GEN_DELETE or by the pattern marking the class. Graphical resources must be loaded from assets and located using XPATHSMANAGER. Displays and canvas controlled by

AppFlowGraphics must not be treated as native objects owned by the application if the framework is managing them.

17. Practical summary

Graphic is the third block that consolidated GEN Framework. It provides the visual base: screens, canvas, bitmaps, graphical formats, SVG, DXF, video, OpenGL ES and statistical graphics. Its goal is not to hide the existence of the platform, but to confine it to specific factories and implementations, offering the programmer a homogeneous way of working.

The key to its architecture is GRPFACTORY, GRPSCREEN, GRPVIEWPORT and GRP2DCANVAS. On that basis the other services are built: image loading, vector rendering, statistics generation, accelerated presentation and relationship with UserInterface.

An application with GEN Framework that uses Graphic correctly keeps the application logic separate from the native window details or the graphical backend. This separation allows you to compile on multiple platforms, reuse resources, maintain framework style and develop interfaces or visualizations without losing control of C/C++ itself.

Chapter VIII: Input, input devices and interaction abstraction

1. The Input block inside GEN Framework

The Input module appears as a natural consequence of the graphical block. As soon as an application ceases to be just a console tool and starts to have a screen, canvas, visual elements, windows or direct interaction with the user, an immediate need arises: receive actions from the outside evenly. It is not enough to draw; you need to know what the user is doing and turn that information into decisions from the application.

The problem is that the user input is not homogeneous between platforms. Windows, Linux, Android, a Raspberry Pi with touch screen, a system embedded with a small SPI display, a physical keyboard, a mouse, a touch surface or an external controller do not offer exactly the same model of events. Each system has its codes, messages, event queues, coordinates, conventions and limitations.

Input is born to prevent each application from having to solve that problem again. Its objective is that the application code does not depend directly on whether the action comes from a mouse, a touch screen, a keyboard, a joystick, a controller or a particular implementation of a platform. The application must work with stable concepts of the framework and let the platform layer translate the specific details.

That is why Input should not be seen as a simple collection of keyboard and mouse classes. It is an interaction normalization layer. Just as Graphic tries to offer a common way of drawing, Input tries to offer a common way of interpreting what the user does about the application.

2. A block born after the graphical system

Historically, this block appears when performing the graphical block. When GEN Framework starts to handle screens, canvas, bitmaps, interfaces and graphical applications, it becomes clear that the graph alone is not enough. A graphical application needs to know when a key is pressed, when a cursor moves, when it is dragged over a zone, when a selection is made, or when a touchstroke occurs.

The design of Input is closely linked to the graphic examples of the framework. In applications like Canvas2D, UI_Options, UI_Message, WindowChromes or StatisticsCharts you can see a repeated pattern: the application gets the devices from the input manager, saves pointers to the buttons and cursors that interest you, and in each update cycle consults its status to transform that input into application actions or interface events.

This relationship with Graphic does not mean that Input only serves large graphic applications. An application without AppFlow, running on a microcontroller, could use a similar idea to handle physical buttons, a crossbreed, an encoder, a resistive touchscreen or a small display by SPI or I2C. The bottom idea remains the same: to isolate the logic of the application from the concrete physical origin of the input.

3. The key idea: break down devices in cursors and buttons

One of the most important decisions of this collection of classes is to break down all input devices into two fundamental elements: cursors and buttons. This idea is simple, but it has an enormous practical force.

A button represents a discrete action: pulsed, released, maintained, released with flank detection, physical key, mouse button, touch click or button of a controller. A cursor represents a position or a movement: X/Y/Z coordinates, mouse scrolling, contact point on a touch screen, drag or path.

With this decomposition, the application code stops asking if the user has a mouse or touch screen. The application asks for a cursor and a selection button. On one platform that cursor can come from the mouse; on

another, from a touch screen; on another, from a specific device. The application's behavior can be the same without touching the main logic.

This is especially important in GEN Framework because the framework seeks to make the application portable. Scheduling a configuration screen, menu or interactive canvas directly against Windows mouse events requires rewriting parts of the application for Android or for a touch screen plate. If the application works with INPCURSOR and INPBUTTON, the platform change is confined to the Input layer and the corresponding factory.

4. INPFACTORY and platform insulation

Platform-dependent instances are managed by INPFACTORY. This factory fulfills the same architectural role as other GEN Framework factories: separating the conceptual request of an object from its concrete implementation. The application should not directly build a Windows, Linux or Android device; it should ask for a device to enter the framework and let the actual platform factory create the right class.

In the common code, INPFACTORY defines operations such as CreateDevice(), DeleteDevice(), CreateSimulator() and CreateCapture(). The base implementation does not create anything specific: it returns NULL or false when there is no concrete implementation available. The utility appears when the platform installs its own specialized factory, such as a Windows, Linux or Android factory capable of creating keyboards, mice, touch screens, captures or simulators adapted to the real system.

This design prevents the application code from having to include headers such as INPWINDOWSDEVICEMOUSE, INPLINUXDEVICETOUCHSCREEN or INPANDROIDDEVICEKEYBOARD. These classes may exist in the platform area, but should not contaminate the common application code. The application works with INPDEVICE, INPBUTTON, INPCURSOR and INPMANAGER.

```
INPDEVICE* device;

device = GEN_INPFACTORY.CreateDevice(INPDEVICE_TYPE_MOUSE);
if(device)
{
    GEN_INPMANAGER.AddDevice(device);
}
```

The above example shows the general idea. In a normal AppFlow application, much of this creation can be integrated into the graphical flow boot when the Input option is activated. The application code usually consumes the devices already registered in INPMANAGER, but the creation responsibility still belongs to the factory.

5. INPMANAGER: the device access point

INPMANAGER is the central input device manager. It is your responsibility to maintain an orderly collection of INPDEVICE, allow you to locate devices by index or type, update them and release the collection when no longer required. In the graphic examples it appears as the usual entry point for the application to find keyboard, mouse or touchscreen.

The manager uses an internal map protected by mutex. This fits the GEN Framework philosophy: it does not expose a messy list of random objects, but a structure that allows registering devices, consulting them and updating their status in a homogeneous way. The application does not need to know how native events are read; it just needs to know which devices are available.

```
INPDEVICE* inpdevice;

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD);
if(inpdevice)
{
```

```

    button[MYAPP_BUTTON_ESC] = inpdevice->GetButton(INPBUTTON_ID_ESCAPE);
    button[MYAPP_BUTTON_SPACE] = inpdevice->GetButton(INPBUTTON_ID_SPACE);
}

```

This pattern appears repeatedly in the graphic examples. During the first update of the application the devices are located and pointers are saved to the necessary buttons. Then, in the Update, the code checks those pointers and checks their status. This avoids searching for the device and the button in each cycle when the references that interest the application are already known.

6. INPDEVICE: a device reduced to common elements

INPDEVICE represents a generic input device. Its type enumeration allows to distinguish between keyboard, mouse, joystick, touch screen or Wiimote, but the class is not limited to one of those families. Its value is that all devices can be described by two collections: buttons and cursors.

A keyboard has many buttons and usually no cursor. A mouse has buttons and a cursor. A touch screen has at least one selection conceptual button and one or more touch cursors. A joystick can display buttons and, depending on the implementation, cursors or axes translated into an equivalent form. Application logic can treat all those devices with the same basic operations.

The GetButton(), GetButtonByCode(), GetButtonState(), IsPressButton(), ReleaseAllButtons(), GetCursor() and IsChangeCursor() functions make INPDEVICE the standard entry status query point. The Update() function is virtual because the actual reading depends on the platform and the specific type of device.

```

INPDEVICE* mouse = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
if(mouse)
{
    INPBUTTON* select = mouse->GetButton(INPBUTTON_ID_MOUSE_LEFT);
    INPCURSOR* cursor = mouse->GetCursor(0);

    if(select && select->IsPressedWithRelease())
    {
        // Selection in the current cursor position.
    }
}

```

7. INPBUTTON: pulse, status and transition

INPBUTTON is the abstraction of a discrete input. It does not represent only a keyboard key. It can also represent a mouse button, a control button, a smartphone button or the conceptual click of a touch screen. That is why the enumeration INPBUTTON_ID includes groups of identifiers for keyboard, mouse, smartphone, Wiimote and touchscreen.

The class retains the button's logical identifier, native code or key code, associated symbol, state, pressure, and press time. The presence of an internal timer allows you to measure how long it takes to press a button, which is useful to distinguish a short press from a sustained press or to apply repetitions.

The difference between IsPressed() and IsPressedWithRelease() is important. IsPressed() lets you know if the button is in a pressed state. IsPressedWithRelease() is used to detect the complete click-and-drop gesture, which is usually suitable for interface actions: accept, open, select, change option or execute a single command.

```

if(button[MYAPP_BUTTON_ESC])
{
    if(button[MYAPP_BUTTON_ESC]->IsPressedWithRelease())
    {
        SetEvent(MYAPP_XFSMEVENT_END);
    }
}

```

```
    }
}
```

This way of working avoids involuntary repeated actions. If only `IsPressed()` is used, a held key could execute the same action dozens of times while the pulsation lasts. When using `IsPressedWithRelease()`, the code responds to the complete transition.

8. INPCURSOR: position, change and selection

`INPCURSOR` represents an input position. Hereda from `INPCURSORMOTIONPOINT`, so that a cursor is also a point with X, Y and Z coordinates. This choice allows the cursor to work directly with positions and, at the same time, can be integrated with the movement class.

The class allows you to query and modify the coordinates, know if the cursor has changed, check if its position falls into a rectangle and access the associated `INPCURSORMOTION` object. In graphic applications, this allows you to transform the physical movement of the mouse or finger into interface events or actions on a canvas.

```
if(cursor)
{
    int cursor_x = (int)cursor->GetX();
    int cursor_y = (int)cursor->GetY();

    if(cursor->IsPositionInRect(0, 0, width, height))
    {
        // The cursor is within the active area.
    }
}
```

The Y coordinate may require adaptation according to the graphical system used. In some examples the Y axis is reversed before sending events to `UserInterface`, because the origin of the input coordinates and the source of coordinates of the canvas or interface may not match.

9. INPCURSORMOTIONPOINT and INPCURSORMOTION

`INPCURSORMOTIONPOINT` is the minimum unit of a trajectory: a point with coordinates X, Y and Z.

`INPCURSORMOTION` is the collection and analyzer of those points. Together they allow to study the movement of the cursor, not only its current position.

`INPCURSORMOTION` allows you to add points, delete the path, build synthetic paths from a line, circle or ellipse, calculate angle, module, direction and time elapsed. It also allows you to reduce points by modes such as `BYSTEPS`, `NMAX` or `NMAXPERCENT`, which is useful when a path contains too many samples and only requires a simplified representation.

This class is the basis for recognizing simple movements. It does not pretend to be a complex engine of gestures, but it provides what is necessary to interpret a drag direction, a significant displacement or a sufficiently stable trajectory.

```
if(cursor && cursor->GetMotion()->IsReadyToTest(50))
{
    INPCURSORMOTION_DIR dir;

    dir = cursor->GetMotion()->GetDirectionByAngle(true);
    switch(dir)
    {
        case INPCURSORMOTION_DIR_UP : // Action up
            break;
```

```

        case INPCURSORMOTION_DIR_DOWN : // Action Down
            break;

        default : break;

    }

    cursor->GetMotion()->Reset();
}

```

In Canvas2D this type of logic is used to translate a cursor movement into actions such as walking towards a direction. The code does not need to know if the movement comes from the mouse or a touch screen. It only receives a standardized path.

10. The pattern of use in graphic applications

The graphic examples show a very clear pattern. First Input is enabled in the graphical application settings. Then, in `AppProc_FirstUpdate()`, the devices are ordered from the manager, the buttons and cursors of interest are saved, and finally in each update the status of those objects is checked.

```

option(INP_FEATURE                                     "Inputs"
ON )

```

The `APPFLOWGRAPHICS_INIOPTION_INPUT` option usually appears in the boot of a graphical application with `AppFlow`. That option indicates the graphical flow that the input must prepare. From that point on, the application can use `GEN_INPMANAGER` to locate devices.

```

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
if(inpdevice)
{
    button[MYAPP_BUTTON_MOUSE] = inpdevice->GetButton(INPBUTTON_ID_MOUSE_LEFT);
    cursor = inpdevice->GetCursor(0);
}

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_TOUCHSCREEN);
if(inpdevice)
{
    button[MYAPP_BUTTON_TOUCHSCREEN] = inpdevice->GetButton(INPBUTTON_ID_TOUCHSCREEN);
    cursor = inpdevice->GetCursor(0);
}

```

This fragment perfectly illustrates the philosophy of the block. The same cursor member can end up pointing to the mouse cursor or touch screen cursor. From that moment on, the rest of the code can behave the same. The application is no longer organized around “mouse” or “touch” but around “cursor” and “selection button”.

11. Input and UserInterface

When an application uses `UserInterface`, Input does not usually act directly on the visual elements. It is usual to convert the status of buttons and cursors into interface events. Thus, `UserInterface` receives events such as cursor movement, selection, pulsation or gesture, without knowing the physical device that produced them.

This pattern is especially powerful because it separates three responsibilities: the platform reads native events, Input normalizes them as buttons and cursors, and `UserInterface` interprets them as actions on visual elements. The application acts as a bridge when it needs to adapt coordinates, filter actions, or translate specific buttons to interface events.

```

if(changed)
{
    GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_CURSOR_MOVE, cursor_x, cursor_y);
}

```

```

    }

    if(button[MYAPP_BUTTON_MOUSE]->IsPressedWithRelease())
    {
        GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECTION, cursor_x, cursor_y);
        cursor->GetMotion()->Reset();
    }

```

The same logic can work with touchscreen. If the INPBUTTON_ID_TOUCHSCREEN conceptual button is pressed, the application can send a selection event using the coordinates of the touch cursor. UserInterface code does not need a special branch for each type of device.

12. INPCAPTURE and INPCAPTURE_XEVENT

INPCAPTURE is aimed at capturing input, especially keyboard sequences, and publishing them using the GEN Frameworkevent system. Hereda from XSUBJECT, indicating that it can act as an event source for interested observers. It maintains a key counter, a maximum limit, a XBUFFER and a XSTRING with captured content.

INPCAPTURE_XEVENT carries the specific information of a capture event: virtual code, scan code, flags, lock state, key number, limit, buffer and string. This design separates the object that captures from the notification sent to the rest of the system.

The input capture does not have the same role as the query of buttons in a graphical application. It is more appropriate when you are interested in receiving text, sequences or global keyboard events. Being also managed by the factory, the actual implementation may depend on the platform.

```

#ifdef INP_CAPTURE_ACTIVE
INPCAPTURE* capture = GEN_INPFACTORY.CreateCapture();
if(capture)
{
    capture->SetLimit(256);
    capture->Application_SetHandle(applicationhandle);

    // The object can publish INPCAPTURE_XEVENT when keys are produced.

    GEN_INPFACTORY.DeleteCapture(capture);
}
#endif

```

13. INPSIMULATE: Software-generated input

INPSIMULATE represents the opposite side of the capture. Instead of reading physical input, it allows you to simulate it from software. Its operations allow you to press, drop or click on a key, send text, move your mouse and translate literals to keyboard codes taking into account alternative keys such as ALTGR or SHIFT.

This functionality can be useful for testing, automation, internal tools or script integration. As with capture, actual implementation depends on the platform. Simulating a key in Windows, Linux or Android is not done the same way, so the factory is again the point of isolation.

```

#ifdef INP_SIMULATE_ACTIVE
INPSIMULATE* simulate = GEN_INPFACTORY.CreateSimulator();
if(simulate)
{
    simulate->Key_ClickByLiteral(__L("A"));
    simulate->Key_ClickByText(__L("GEN Framework"), 50);
    simulate->Mouse_SetPos(100, 100);
}

```

```

        GEN_INPFACTORY.DeleteSimulator(simulate);
    }
#endif

```

14. Platform classes

The common Input block defines stable concepts. Platform classes specialize those concepts. The documentation includes implementations for Windows, Linux and Android, such as keyboard, mouse, touchscreen, specific factories, and system-adapted capture or simulation classes.

This division is fundamental. INPDEVICE defines that a device has buttons, cursors and a virtual Update(). A specific class of Windows or Linux implements how the keyboard or mouse is actually read. INPFACTORY defines that a device can be ordered; the specific factory of the platform decides which class to create. INPMANAGER keeps the devices already created and allows the application to access them without knowing their real class.

Thanks to this confinement, the application code is kept clean. There should be no blocks #ifdef WINDOWS or #ifdef LINUX to read the keyboard in a normal application. Those details must live in the platform area and reach the program already converted into buttons and cursors.

Group	Main classes	Function within Input
Factory and management	INPFACTORY, INPMANAGER	Platform dependent creation, device registration and global update.
Device	INPDEVICE	Common base for keyboard, mouse, joystick, touch screen or controls.
Buttons	INPBUTTON	It represents keys, physical buttons, tactile pulsations and pressure states.
Cursors	INPCURSOR, INPCURSORMOTIONPOINT	Represents normal position and point of entry.
Movement	INPCURSORMOTION	Saves trajectory, calculates direction, angle, module and simple gestures.
Catch	INPCAPTURE, INPCAPTURE_XEVENT	Capture sequences or keyboard events and post them as GEN Framework events.
Simulation	INPSIMULATE	Generates software input for testing, automation or integration.

15. Relationship between classes

The ratio of Input classes can be summarized hierarchically. INPFACTORY creates specific objects when they depend on the platform. INPMANAGER retains the available devices. Each INPDEVICE contains a collection of INPBUTTON and a collection of INPCURSOR. Each INPCURSOR contains a INPCURSORMOTION, which in turn contains a list of INPCURSORMOTIONPOINT. INPCAPTURE and INPSIMULATE are ancillary services that can also depend on the platform.

```

INPFACTORY
+-- CreateDevice() -> INPDEVICE specialized platform
+-- CreateCapture() -> INPCAPTURE specialized platform
+-- CreateSimulator() -> INPSIMULATE specialized platform

INPMANAGER
+-- INPDEVICE keyboard
|   +-- INPBUTTON escape, space, arrows, ...
+-- INPDEVICE mouse
|   +-- INPBUTTON left, right, middle
|   +-- INPCURSOR mouse cursor
+-- INPDEVICE touchscreen
    +-- INPBUTTON touchscreen
    +-- INPCURSOR touch cursor

```

This structure explains why the application code is so compact. Once the device exists and is registered, the program only needs to ask for the button or cursor that interests you. The rest of the complexity lies below.

16. Example of input skeleton in a graphical application

A typical skeleton of a graphical application of GEN Framework declares pointers to the buttons of interest and a common cursor. In the first update these pointers are assigned from the available devices. In each Update states are reviewed and translated into actions.

```
enum MYAPP_BUTTON
{
    MYAPP_BUTTON_UP = 0,
    MYAPP_BUTTON_DOWN,
    MYAPP_BUTTON_ESC,
    MYAPP_BUTTON_MOUSE,
    MYAPP_BUTTON_TOUCHSCREEN,
    MYAPP_BUTTON_MAX,
};

INPBUTTON* button[MYAPP_BUTTON_MAX];
INPCURSOR* cursor;
bool MYAPP::AppProc_FirstUpdate()
{
    INPDEVICE* inpdevice;

    inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD);
    if(inpdevice)
    {
        button[MYAPP_BUTTON_UP] = inpdevice->GetButton(INPBUTTON_ID_UP);
        button[MYAPP_BUTTON_DOWN] = inpdevice->GetButton(INPBUTTON_ID_DOWN);
        button[MYAPP_BUTTON_ESC] = inpdevice->GetButton(INPBUTTON_ID_ESCAPE);
    }

    inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
    if(inpdevice)
    {
        button[MYAPP_BUTTON_MOUSE] = inpdevice->GetButton(INPBUTTON_ID_MOUSE_LEFT);
        cursor = inpdevice->GetCursor(0);
    }

    inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_TOUCHSCREEN);
    if(inpdevice)
    {
        button[MYAPP_BUTTON_TOUCHSCREEN] = inpdevice->GetButton(INPBUTTON_ID_TOUCHSCREEN);
        cursor = inpdevice->GetCursor(0);
    }

    return true;
}

bool MYAPP::UpdateInput()
{
    if(button[MYAPP_BUTTON_ESC])
    {
        if(button[MYAPP_BUTTON_ESC]->IsPressedWithRelease())
        {
            SetEvent(MYAPP_XFSMEVENT_END);
        }
    }
}
```

```

        }
    }

    if (cursor && button[MYAPP_BUTTON_MOUSE])
    {
        if (button[MYAPP_BUTTON_MOUSE]->IsPressedWithRelease())
        {
            int x = (int)cursor->GetX();
            int y = (int)cursor->GetY();

            GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECTION, x, y);
        }
    }

    return true;
}

```

This example is not intended to replace the actual code of the framework examples, but it shows the conceptual skeleton. The application does not process native Windows messages, X11 events or callbacks Android. It works with high-level buttons, cursors and events.

17. Input in applications without AppFlow

Although Input is closely linked to graphical applications with AppFlow, its model can also be applied in applications without AppFlow. In an embedded system there may not be a classic window or a complete graphic loop, but there may be physical buttons, touch sensors, a small display or a controller connected by GPIO, I2C or SPI.

In that case, the application could create or receive input devices more directly and update them in its own loop. The advantage of maintaining the same abstraction is that logic keeps talking about buttons and cursors, not pins, interruptions or specific logs. The platform or driver part translates the physical into the common form of Input.

This fits the philosophy of GEN Framework: you can use the complete framework when there is AppFlow, or you can use specific modules when the environment is more limited. The important thing is not to lose the separation between application logic and platform infrastructure.

18. Common errors when using Input

A common error is to write mouse-specific logic and then add a different logic for touchscreen. That duplication breaks the idea of the block. It is advisable to locate the available cursor and selection button and write a single behavioral path.

Another error is to check devices in each Update cycle instead of saving the buttons and cursor references during the first update. Although not always serious, it dirtys the code and repeats unnecessary searches.

It is also important to distinguish between continuous status and transition. To move an object while a key is pressed it can make sense. To accept a menu option it is usually better `IsPressedWithRelease()`. To choose this difference badly causes repeated actions, double selections or feeling of lack of control in the interface.

Finally, remember that the coordinate axis can change according to the graphical origin. If the input delivers Y from below and the canvas or the interface expects Y from above, the application must adapt the coordinate before sending the event. In the graphic examples this transformation is seen when calculating the position that is finally sent to `UserInterface`.

19. Summary class catalogue of the Input block

Class	Description	Example of use
INPFACTORY	Factory of creation of devices, simulators and platform dependent captures.	Order a INPDEVICE_TYPE_MOUSE device without knowing its real class.
INPMANAGER	singleton Manager of registered devices.	GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD).
INPDEVICE	Common base of a device with buttons and cursors.	Get INPBUTTON_ID_ESCAPE or the main cursor.
INPBUTTON	Normalized button with status, symbol, pressure and timing.	Detect IsPressedWithRelease() to accept an action.
INPCURSOR	Normalized cursor with associated position and movement.	Use GetX(), GetY() and IsPositionInRect().
INPCURSORMOTIONPOINT	Path point with X/Y/Z. coordinates	Individual element within a trajectory.
INPCURSORMOTION	Collection and analysis of movement points.	Detect address with GetDirectionByAngle().
INPCAPTURE	Capture posted entry as events.	Capture text or keyboard sequences.
INPCAPTURE_XEVENT	Event with capture data.	Transport vkcode, scancode, flags, buffer and string.
INPSIMULATE	Keyboard and mouse simulation.	Send Key_ClickByText() or Mouse_SetPos().

20. Practical summary

Input is the block that allows an application with GEN Framework to interpret user interaction portably. It appears as a natural complement to the graphical system, but its design is also useful in smaller or embedded environments.

Its central idea is to reduce all devices to buttons and cursors. This reduction allows a mouse, touchscreen or a different device to power the same application logic. INPFACTORY isolates platform-dependent creation, INPMANAGER organizes devices, INPDEVICE groups buttons and cursors, INPBUTTON represents discrete actions and INPCURSOR together with INPCURSORMOTION represents position and movement.

By combining Input with UserInterface, the application transforms buttons and cursors into interface events. The result is a more portable, clean and maintainable code.

Chapter IX: Sound, sound and playback in GEN Framework

1. The last factory block of GEN Framework

After studying XUtilities, DataIO, Graphic and Input, the next natural block within the GEN Framework architecture is Sound. This module closes, at least for the moment, the main factory set in the framework. If XFACTORY provides the common base, DIOFACTORY allows communicating the application with the outside, GRPFACTORY offers graphical output and INPFACTORY abstracts the user input, SNDFACTORY completes the circle by providing a sound output abstraction.

Sound does not always appear in early applications. Many console utilities do not need audio and many industrial applications can work only with traces, files or screen. However, when an application needs sound warnings, background music, effects, event response, animation synchronization or acoustic feedback for the user, having an integrated layer avoids solving from scratch a problem that also depends on the platform.

The Sound block maintains the same philosophy as the rest of GEN Framework: the application should not know directly the native API that plays the sound. It should create sound elements, configure how they are played and ask the factory to execute them. The concrete implementation is below. In the version studied, the main backend is supported by OpenAL, but the architectural idea is broader: the application works against SNDFACTORY and the module's common classes, not against direct calls from OpenAL distributed by the application code.

2. SNDFACTORY and platform insulation

SNDFACTORY is the factory that centralizes the creation and control of sound elements. As with the other factories in the framework, its role is not limited to constructing objects. SNDFACTORY defines a boundary. Above that boundary is the portable code of the application. Below are the concrete implementations, devices, buffers, audio contexts, playback threads and APIs used to emit sound on each platform.

The application uses high-level operations: creating a sound from a file, creating a synthetic note, playing, pausing, stopping, modifying volume, modifying pitch, or waiting for it to end. The factory is responsible for converting those operations into real work. In an implementation based on OpenAL, for example, that conversion involves creating sources, buffers, assigning PCM data, controlling playback states, and maintaining an update thread.

This separation is important for three reasons. The first is portability: the application code is not tied to a API of concrete sound. The second is coherence: all sounds are managed using the same types of GEN Framework, such as XPATH, XBUFFER, XVECTOR, XEVENT or XFSMACHINE. The third is control: the framework can emit sound events, maintain states and centralize resource destruction.

```
// Typical Activation of Sound Block in CMake.
option(SND_FEATURE "Sound"
ON )
option(SND_FILE_WAV_FEATURE "Sound WAV file"
ON )
option(SND_FILE_OGG_FEATURE "Sound OGG file"
ON )
```

These options show the same idea we saw in previous chapters: the build system decides which parts of the framework are incorporated. An application that does not need sound does not have to load this functionality. A graphical application like Canvas2D can actually activate it to play music and effects synchronized with animation.

3. General Map of Sound Block

The Sound block is smaller than XUtilities, DataIO or Graphic, but it is quite well structured. Its classes can be grouped into five areas: factory and events, sound elements, file formats, backend OpenAL and playback settings.

- Factory and Events: SNDFACTORY and SNDFACTORY_XEVENT define the common access point and events that report state changes in playback.
- Sound elements: SNDITEM represents a reproducible sound, whether it comes from a file or from a generated note.
- Play Settings: SNDPLAYCFG encapsulates volume and pitch to apply specific parameters when playing.
- File formats: SNDFILE, SNDFILEOGG and SNDFILEWAV load and decode sound data from disk.
- Backend OpenAL: SNDOPENALFACTORY, SNDOPENALPLAYITEM, SNDOPENALBUFFER and SNDOPENALSOURCE execute the actual playback using OpenAL.

This organization allows the normal user of the framework to stay at the high level. In most applications it is enough to prepare routes using XPATHMANAGER, create elements with GEN_SNDFACTORY.CreateItem, adjust a playback settings and call Sound_Play. OpenAL classes are important to understand how it works internally, but they should not become the application's usual API.

4. SNDITEM: the reproducible unit

SNDITEM is the class that represents a sound drive. It is the object that the application retains when it wants to play, pause or stop a sound. A SNDITEM can be either a file type or a note type. This distinction appears in the enum SNDITEM_TYPE, where the element may be in an unknown state, associated with a note or associated with a sound file.

In addition to the type, SNDITEM maintains a state of play. The state lets you know if the sound is unstated, initialized, playing, paused, stopped or finished. This is essential to avoid playing effects out of control. In Canvas2D, for example, before launching a step sound you check whether it is already playing to avoid overlapping multiple copies of the same effect at the same instant.

The class also retains repeat information: number of times played, number of times to play, playback counter, accumulated time, current timer and internal timer. In this way the system can manage both single-time sounds and loop sounds.

```
// Creating a sound item from a path.
XPATH xpath;
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SOUNDS, xpath);
xpath.Slash_Add();
xpath.Add(__L("efecto.ogg"));

SNDITEM* item = GEN_SNDFACTORY.CreateItem(xpath);
if(item)
{
    GEN_SNDFACTORY.Sound_Play(item, NULL, 1);
}
```

The important detail is that the application does not directly create a SNDFILEOGG or a buffer OpenAL. The application creates a SNDITEM through the factory. Internally, the system deduces the file type by extension, loads the data and prepares the item so that the backend can play it.

5. SNDPLAYCFG: volume and pitch per playback

SNDPLAYCFG encapsulates playback parameters. In the analyzed version it contains volume and pitch. The volume allows to adjust the intensity of a particular sound. The pitch allows to alter the playback speed or tone, which can be useful for variations of effects, warnings or interface resources.

It is appropriate to distinguish between the global configuration and the configuration of a particular sound. SNDFACTORY has master volume as general volume, while SNDPLAYCFG allows to apply parameters to a specific item. This separation allows, for example, to download background music without changing the volume of important effects, or to play the same resource with different settings depending on the context.

```
SNDPLAYCFG playCFG;
playCFG.SetVolume(25);
playCFG.SetPitch(1.0f);

SNDITEM* background = GEN_SNDFACTORY.CreateItem(xpath);
if(background)
{
    GEN_SNDFACTORY.Sound_Play(background, &playCFG, SNDFACTORY_INLOOP);
}
```

The SNDFACTORY_INLOOP value indicates that the sound should be played in a loop. This constant is very practical for background music or ambient sounds. For point effects, such as a tread, a single repetition is normally used.

6. Sounds from file: SNDFILE, OGG and WAV

The sound file system is grouped into FormatFiles. The base class is SNDFILE. Its responsibility is to represent an already loaded and converted sound file to data that the backend can deliver to the audio device. It maintains the path, number of channels, number of samples, sampling frequency, duration and buffer with decoded data.

SNDFILE acts as the point of creation. From the file extension decides which specific class to instance. The revised code includes OGG and WAV files. OGG are managed by SNDFILEOGG and WAV by SNDFILEWAV. This separation prevents the application from having to know the specific decoder.

SNDFILEOGG loads the file, reads its contents into a XBUFFER and decodes Vorbis using stb_vorbis. At the end, it obtains channels, sampling frequency, number of samples, duration and PCM data interspersed. SNDFILEWAV, for its part, relies on the RIFF reader of the framework to locate fmt and data chunks, read the format and obtain audio data.

```
// The specific type is decided by extension.
SNDFILE* soundfile = SNDFILE::Create(__L("assets/sounds/greensleeves.ogg"));
if(soundfile)
{
    XDWORD duration    = soundfile->GetDuration();
    XDWORD samplerate  = soundfile->GetSampleRate();
    XWORD  channels     = soundfile->GetChannels();
}
```

Although a normal application does not need to create SNDFILE directly, understanding this layer helps to understand the full flow: file in assets/sounds, load using XPATH, decoding to XBUFFER, mapping to buffer OpenAL and playback using audio source.

7. SNDNOTE: sounds generated by frequency and duration

Not all sounds have to come from a file. GEN Framework also allows you to create an item from a frequency and duration. In that case SNDNOTE is used. This class is minimal: it contains frequency and duration. Its utility appears when you want to generate tones, simple warnings or acoustic signals without depending on an external resource.

Internally, the backend OpenAL can generate a buffer from that note. SNDOPENALBUFFER includes GenerateNote methods that fill a XBUFFER with calculated samples, and then Assign delivers that data to OpenAL. It is a small but interesting example: the framework not only reproduces resources, it can also synthesize a basic signal.

```
// Create and play a simple tone of 440 Hz for 250 ms.
SNDITEM* beep = GEN_SNDFACTORY.CreateItem(440, 250);
if(beep)
{
    SNDPLAYCFG cfg;
    cfg.SetVolume(50);
    GEN_SNDFACTORY.Sound_Play(beep, &cfg, 1);
}
```

This type of functionality is especially useful in diagnostic tools, simple interfaces, small devices or applications where you want to emit a sound signal without including audio files.

8. Sound events

SNDFACTORY inherits from XSUBJECT. This means that the sound system can publish events within the framework observer model. The SNDFACTORY_XEVENT class encapsulates those events and allows them to be associated with the SNDITEM involved. The main events are initialization, playback, pause, stop and end.

This integration with XEVENT is very important because it allows an application to observe the audio behavior without blocking the main flow. An application may know when a sound started, when it ended or when it stopped, and react accordingly. A visual indicator could be activated in a user interface; an animation could be coordinated in a game or demo; an audio alarm could be recorded in an industrial tool.

```
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_INI    , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PLAY  , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PAUSE , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_STOP  , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_END   , &GEN_SNDFACTORY.GetInstance());
```

The pattern is the same as in other GEN Framework modules: the event producer is the factory, and the application or observer class subscribes to the types of event that interest you.

9. SNDOPENALFACTORY: playback implementation

SNDOPENALFACTORY is the concrete implementation based on OpenAL. Hereda from SNDFACTORY and provides the real functionality that the base class only defines as a contract. In Ini opens the audio device, creates the context OpenAL, makes it current, creates a mutex, prepares a playback thread and marks the system as active.

The existence of a thread and mutex is not accidental. Sound playback should not depend on the application being blocked waiting for an effect to end. The system should be able to update the sound status, check times, detect endings, restart loops and publish events while the application continues its normal cycle. That is why the backend maintains a list of SNDOPENALPLAYITEM and runs Update periodically.

SNDOPENALFACTORY also limits the number of simultaneous channels using maxchannels. In the code studied it initializes to 16. This limit avoids creating unlimited fonts and allows better control of the playback cost. When

Sound_Play is launched, a OpenAL source is created, a play item, volume and pitch is applied, it is set if it goes in loop and the state machine is launched.

```
bool SNDOPENALFACTORY::Ini()
{
    device = alcOpenDevice(NULL);
    context = alcCreateContext(device, NULL);

    alcMakeContextCurrent(context);

    playmutex = GEN_XFACTORY.Create_Mutex();
    playthread = GEN_XFACTORY.CreateThread(XTHREADGROUPID_SOUND,
        __L("SNDOPENALFACTORY::SNDOPENALFACTORY"),
        SNDOPENALFACTORY::ThreadPlay,
        this);

    soundactive = true;
    return true;
}
```

The above fragment summarizes the concept, does not intend to replace the real code. What matters to the reader is to understand that SNDFACTORY offers the interface and SNDOPENALFACTORY deals with the device, context, buffers, fonts, threads and internal states.

10. Buffer, source and play item

In OpenAL there are three internal concepts that appear reflected in the GEN Framework classes.

SNDOPENALBUFFER represents the audio buffer, that is, the sample block that is delivered to the device.

SNDOPENALSOURCE represents the source that plays that buffer. SNDOPENALPLAYITEM unites a SNDITEM with a source and adds a state machine to control its life.

SNDOPENALBUFFER can be created, destroyed, received data and assigned to OpenAL. If the sound comes from a file, the buffer is filled with SNDFILE decoded data. If it comes from a note, a wave is generated from the frequency and duration. SNDOPENALSOURCE controls operations such as Play, Stop, Pause, UnPause, volume, pitch, loop and actual source status.

SNDOPENALPLAYITEM is especially interesting because it is not limited to being a binding structure. XFSTATEMACHINE Hereda. This allows modeling the playback process as states: NONE, INI, PLAY, PAUSE, STOP and END. This idea fits what was seen in the IV chapter: a state machine allows to separate continuous logic and point transitions.

```
// Internal conceptual relationship.
SNDITEM
-> describes which sound you want to play.
SNDOPENALPLAYITEM
-> associates the item with a source and controls its state.
SNDOPENALSOURCE
-> runs Play, Pause, Stop and controls volume/pitch.
SNDOPENALBUFFER
-> contains samples delivered to OpenAL.
```

11. The reproduction state machine

The sound also uses a finite state machine. This is consistent with the rest of GEN Framework and allows playback to be controllable. When requesting Sound_Play, the item is not played in any way. A play item is created, its

source is prepared, parameters are assigned and the initialization event is set. From there, Update processes the transition.

During the PLAY status, the system measures the time elapsed, updates the playback time and decides whether the sound is over. If the item is in a loop, check if the source stopped ringing to re-launch it or maintain the playback. If it is a sound with a specific number of repetitions, decrease the counter and restart the source when appropriate. When there are no repetitions left, move to STOP or END and publish the corresponding event.

This behavior is important because it allows you to play effects without blocking the application. The application launches the sound and continues to draw, read input or process communications. The sound system maintains the state internally.

12. Sound within a graphical application

The Canvas2D example shows well why the Sound module appears after Graphic and Input. A graphical application can draw a scene, move a character, read user input and, in addition, synchronize sounds with animation. That synchronization should not force writing low-level audio code within the render.

Canvas2D activates the sound module in CMake and places the sound resources in assets/sounds. During initialization you get the path of the sound section using XPATHMANAGER, load a background music and play it in a loop. Then load two tread sounds, which are played at specific moments of character animation.

```

XPATH xpathsounds;
XPATH xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SOUNDS, xpathsounds);
xpathsounds.Slash_Add();

xpath = xpathsounds;
xpath.Add(__L("greensleeves.ogg"));

backgroundsound = GEN_SNDFACTORY.CreateItem(xpath);
if(backgroundsound)
{
    GEN_SNDFACTORY.Sound_Play(backgroundsound, &playCFGsound, SNDFACTORY_INLOOP);
    GEN_SNDFACTORY.Sound_SetVolume(backgroundsound, 25);
}

```

This fragment shows the recommended pattern. First you get the correct path by XPATHMANAGER. Then you build the path to the resource. Finally, the factory creates the item and reproduces it. There are no absolute routes or direct dependence of OpenAL on the logic of the application.

13. Synchronization of Footsteps in Canvas2D

The Canvas2D demo is especially didactic because it synchronizes the sounds of the footprints with specific frames of the character. The character has an animation sequence. When the current frame reaches certain indices, the application plays an armored sound walking. This creates the feeling that the audio is part of the animation, not that it sounds independently.

The code checks the frame of the sequence and only reproduces the effect if the corresponding SNDITEM is no longer in PLAY status. This detail avoids saturating the sound output with multiple overlapping reproductions of the same effect. It is a simple technique, but it reflects very well how Sound should be integrated into the graphic cycle.

```

if(charactersequence[indexcharactersequence]->GetActualFrameIndex() == 4)

```

```

{
    if(armorwalkingsounds[0]->GetStatus() != SNDITEM_STATUS_PLAY)
    {
        playCFGsound.SetVolume(10);
        GEN_SNDFACTORY.Sound_Play(armorwalkingsounds[0], &playCFGsound, 1);
    }
}

if(charactersequence[indexcharactersequence]->GetActualFrameIndex() == 17)
{
    if(armorwalkingsounds[1]->GetStatus() != SNDITEM_STATUS_PLAY)
    {
        playCFGsound.SetVolume(10);
        GEN_SNDFACTORY.Sound_Play(armorwalkingsounds[1], &playCFGsound, 1);
    }
}

```

The logic is clear: the animation decides the moment, but the playback is managed by SNDFACTORY. This allows to keep the visual behavior separate from the technical audio detail. The programmer thinks in terms of “when the foot touches the ground, I throw a step”, not in terms of buffers, sources or states of OpenAL.

14. assets/sounds and organization of resources

As we saw in chapter IV, the resources of an application with GEN Framework are placed under assets. Sound follows the same rule. Audio files must be inside the sound section, usually assets/sounds. Thus XPATHMANAGER can provide a homogeneous path and the application does not depend on an absolute location.

```

Canvas2D/
|
├── assets/
|   ├── sounds/
|   |   ├── greensleeves.ogg
|   |   ├── armour-walking1.ogg
|   |   └── armour-walking2.ogg
|   ├── graphics/
|   └── fonts/
└── Sources/

```

This structure facilitates packaging and portability. A desktop application, a Linux compilation, a Android application or an embedded version can maintain the same logical criterion even if the final deployment changes. The code should not assume fixed routes of the operating system, but request the section corresponding to XPATHMANAGER.

15. Closing and cleaning pattern

Sound closing is as important as initialization. If an application subscribes to events, it must be unsubscribed. If it created sounds, it must stop them and release items. If the factory created threads, fonts, buffers or contexts, they must end in an orderly manner. GEN Framework insists a lot on this discipline: Ini creates, End destroys and Clean leaves members in a known state.

```

UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_INI    , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PLAY   , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PAUSE  , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_STOP   , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_END    , &GEN_SNDFACTORY.GetInstance());

```

```
GEN_SNDFACTORY.Sound_StopAll();  
GEN_SNDFACTORY.DeleteAllItems();
```

In a real application, leaving active sounds or unreleased sources can lead to errors that are difficult to detect: closing locks, busy audio resources, live threads or references to already destroyed items. Therefore, the closing pattern should not be considered optional.

16. Relationship with other modules of the framework

Sound does not work in isolation. It relies on many pieces already studied. It uses XUtilities for routes, chains, buffers, vectors, timers, threads, mutexes, events and status machines. It relates to AppFlow because initialization and closure are integrated into the application life cycle. It relates to Graphic when the sound accompanies an animation. It relates to Input when a user action triggers an effect. And it relates to XPATHMANAGER to locate resources in a homogeneous way.

It also has a natural relationship with DIO, although in the high-level code it is not always seen. Sound files are data that are read from disk, and that reading ends up relying on file classes, buffers and formats. The WAV module uses RIFF, and OGG uses Vorbis decoding on a buffer. Therefore, Sound benefits from the common infrastructure without forcing the application programmer to handle all those details.

17. Summary class catalog of the Sound block

SNDFACTORY is the main class of the module. Defines the singleton, records events, maintains the list of SNDITEM, manages the master volume, creates items from file or note, and offers playback, pause, stop, wait, volume and pitch operations. The application should access sound through this factory.

SNDFACTORY_XEVENT represents the events of the sound system. It transports the event type and the associated SNDITEM. It allows to integrate the sound into the observation system of the framework.

SNDITEM represents a reproducible sound. It can be associated with a SNDFILE or a SNDNOTE. It saves ID, type, status, play counters, times, timer and playback settings.

SNDPLAYCFG encapsulates the playback parameters. In the revised version it contains volume and pitch, and allows copying settings between instances.

SNDNOTE represents a sound generated by frequency and duration. It is useful for simple warnings or acoustic signals that do not depend on external files.

SNDFILE is the common base for sound files. It maintains path, channels, samples, frequency, duration and buffer decoded data. Its creation method selects the specific class from the extension.

SNDFILEOGG loads and decodes OGG/Vorbis files. Use stb_vorbis to convert the contents of the file to PCM data that you can then use the backend audio.

SNDFILEWAV loads WAV files. It supports the RIFF structure to read formatting and data, calculate size, frequency, channels and duration.

SNDOPENALFACTORY implements playback over OpenAL. Opens device, creates context, manages threads, mutex, lists active reproductions and publishes status events.

SNDOPENALPLAYITEM unites a SNDITEM with a SNDOPENALSOURCE and controls its cycle using XFSMACHINE. It is the piece that allows each reproduction to be treated as an independent state machine.

SNDOPENALSOURCE represents a OpenAL font. It controls Play, Stop, Pause, volume, pitch, loop, offsets, queue and actual playback status.

SNDOPENALBUFFER represents the buffer OpenAL. Creates and destroys buffers, retains an auxiliary XBUFFER, generates notes and assigns PCM mono or stereo data to OpenAL.

18. Criteria for use in an application with GEN Framework

The first criterion is to always use the factory. An application with GEN Framework should not directly create OpenAL sources or OpenAL buffers. That detail belongs to backend. The application code must work with SNDITEM, SNDPLAYCFG and GEN_SNDFACTORY.

The second criterion is to organize resources in assets/sounds and access through XPATHMANAGER. This maintains the same discipline as graphics, sources, layouts or databases.

The third criterion is not to reproduce effects without control. Before launching very frequent sounds it is advisable to check the state of the item, as Canvas2D does with the footprints. This avoids unwanted overlaps and reduces load.

The fourth criterion is to subscribe to events when the sound state is relevant. If the app doesn't care when a sound ends, it doesn't need events. If it needs to react at the end of an alarm, music, or effect, SNDFACTORY_XEVENT is the natural path.

The fifth criterion is to close in an orderly manner. Sound_StopAll, DeleteAllItems and Event Unsubscription must be part of the application closing when the sound module has been used.

19. Practical summary

The Sound block completes the main GEN Framework factory set. Its aim is to abstract the sound output in the same way that Graphic abstracts the visual output and Input abstracts the user input. The application creates sound items, configures their playback and requests actions to SNDFACTORY. The specific backend takes care of technical details.

The module allows you to work with sounds from file, especially OGG and WAV in the revised version, as well as with notes generated by frequency and duration. SNDPLAYCFG allows you to adjust volume and pitch. SNDFACTORY_XEVENT integrates sound into the framework's event system.

The OpenAL implementation shows how GEN Framework encapsulates a specific API under a common interface. SNDOPENALFACTORY manages device, context, threads, mutex, buffers, fonts and playback status machines. For the application programmer, however, the pattern remains simple: locate the resource with XPATHMANAGER, create the SNDITEM, play it with GEN_SNDFACTORY and close it properly.

Canvas2D is a good example of real integration. Background music plays in loop and the character's footprints are synchronized with specific frames animation. That's the philosophy of GEN Framework applied to sound: integrate practical functionality into the application without breaking the architecture or polluting the code with platform details.

Chapter X: Cipher, encryption and hashes in GEN Framework

1. The objective of the chapter

After reviewing XUtilities, DataIO, Graphic, Input and Sound, the next block to be studied is Cipher. In GEN Framework, Cipher groups the classes related to practical cryptography: hashes calculation, checksums, symmetric encryption, asymmetric encryption, key exchange, key handling, X.509 certificates and key container reading in its own or standard formats.

This block should not be seen as an isolated collection of algorithms. In a real application, security appears mixed with many other parts of the framework. A hash can be used to check the integrity of a file read with XFile, a symmetric encryption can protect a XBUFFER before sending it by a GODOTream, a RSA key can be loaded from assets/certificates using XPATHSMANAGER and an X.509 certificate can be part of a secure connection or validation process.

The name of the block, Cipher, is used in a broad sense. It includes encryption, deciphering, signature generation, abstracting, checksums and key management. Not everything within Cipher encrypts data. A hash, for example, does not encrypt. It produces a footprint or summary that allows identifying data, checking integrity or feeding other cryptographic processes. This difference is important and should be made clear from the beginning.

As in other GEN Framework modules, the design is oriented to be practical. Classes work with the usual types of framework: XBUFFER, XSTRING, XPATH, XFILE, XMPINTEGER, XVECTOR and the memory and life cycle mechanisms of GEN Framework. This allows the Cipher block to fit the rest of the framework without introducing an external programming form.

2. Hash, checksum, encryption and signature are not the same

An application may need security or integrity in many different ways. Sometimes it is enough to know if a data block has changed. In that case it can serve a checksum such as CRC16 or CRC32. Other times a cryptographic fingerprint is needed to identify a content much more robustly; there appear MD5, SHA1, SHA2 or Whirlpool. In other scenarios you have to hide the content by symmetric encryption. And when identity, key exchange or signature intervenes, asymmetric algorithms such as RSA or elliptical curve mechanisms such as X25519 enter into play.

GEN Framework separates these needs into different classes. HASH and its derivatives calculate abstracts. CIPHER and its derivatives transform data with a key to encrypt or decrypt. CIPHERKEY classes represent keys. CIPHERKEYSFILEGKF and CIPHERKEYSFILEPEM classes are responsible for reading or maintaining keys and certificates in files. X.509 certificate classes store and process certificate information.

This separation helps to avoid confusion. A hash should not be used as if it were encrypted, because it does not allow to recover the original data. A simple checksum should not be used as if it were a strong cryptographic test, because its main objective is to detect errors or accidental changes. Nor should a security protocol be invented by combining random parts. The Cipher block provides tools, but the application must use them with criteria.

3. General map of the Cipher block

Cipher classes can be grouped by responsibility. This grouping makes it easy to understand the block before entering class by class.

Group	Main classes	Responsibility
Hash base	HASH	Common interface to calculate summaries about memory, XBUFFER, XFILE or XPATH

		routes.
Hashes and checksums	HASHCKS16, HASHCRC16, HASHCRC32, HASHMD5, HASHSHA1, HASHSHA2, HASHWHIRLPOOL, HASHCOMP128V1	Concrete implementations of checksum, cryptographic hash or specialized algorithm.
Encryption base	CIPHER	Common interface to encrypt, decrypt, define chain mode, padding, initialization vector, keys and result.
Symmetrical Encryption	CIPHERDES, CIPHER3DES, CIPHERAES, CIPHERBLOWFISH	cipher algorithms by shared key over blocks or buffers.
Keys	CIPHERKEY, CIPHERKEYSYMMETRICAL, CIPHERKEYPUBLICRSA, CIPHERKEYPRIVATERSA, CIPHERKEYECDSA	Common representation of symmetrical keys, RSA and ECDSA.
Key Files	CIPHERKEYSFILEGKF, CIPHERKEYSFILEPEM	Reading, maintenance, export or decoding of keys and certificates.
Asymmetric Encryption	CIPHERRSA, CIPHERRSA_CONTEXT	Encryption, decryption, signature and generation of RSA keys using multi-precision integers.
Key exchange	CIPHERECDSEX25519	Private key generation, public key and shared key using Curve25519/X25519.
Certificates	CIPHERCERTIFICATEX509, CIPHERCERTIFICATEX509_ID, CIPHERTRUSTEDROOTCERTIFICATESX509	Representation of X.509 certificates, identity of the certificate and roots of trust.

4. The HASH class

HASH is the common interface for summary classes. Its main value is that it allows you to treat very different algorithms homogeneously. The application can call Do over a memory block, over a XBUFFER, over a XPATH path or over an open XFILE. Then you can recover the result as XBUFFER, as a byte pointer or as a hexadecimal string using GetResultString.

This class is supported by XBuffer to store the result and in XFile/XPath to operate directly on files. That integration with XUtils is important: hashes calculation is not limited to byte arrays, but can be incorporated into the usual flow of an application with GEN Framework.

```

XBUFFER input;
XSTRING text;
XSTRING result;

text = __L("The quick brown fox jumps over the lazy dog");
input.Add((XBYTE*)text.Get(), text.GetSize());

HASH* hash = GEN_NEW HASHSHA2(HASHSHA2TYPE_256);
if(hash)
{
    hash->Do(input);
    hash->GetResultString(result);

    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("SHA2-256: %s"), result.Get());

    GEN_DELETE hash;
}

```

The same pattern can be applied to a complete file. The application gets the path using XPATHMANAGER and lets HASH read the contents by internal blocks.

```

XPATH  xpath;
XSTRING result;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpath);
xpath.Slash_Add();
xpath.Add(__L("data.bin"));

HASHCRC32 crc32;
crc32.Do(xpath);
crc32.GetResultString(result);

```

5. Checksums and hashes fast

HASHCKS16, HASHCRC16 and HASHCRC32 cover quick checking needs. They should not be confused with strong cryptographic hashes. Their typical utility is to detect accidental errors, validate blocks in simple protocols, check basic file integrity or compare changes in data where there is no attacker trying to generate collisions.

HASHCKS16 represents a 16-bit checksum. HASHCRC16 and HASHCRC32 implement 16-bit and 32-bit CRC. These classes inherit or support the HASH interface, but also display specific methods such as GetResultCRC 16, GetResultCRC 32 or Update for incremental calculation.

Class	Typical use	Example of result
HASHCKS16	Simple 16-bit checksum for light validation.	XWORD using GetResultCKS16().
HASHCRC16	16-bit CRC in protocols, frames or devices.	XWORD by GetResultCRC16().
HASHCRC32	32-bit CRC for files, buffers and quick checks.	XDWORD by GetResultCRC32().

```

XBUFFER  frame;
HASHCRC16 crc16;

frame.Add(data, size);
crc16.Do(frame);

XWORD value = crc16.GetResultCRC16();

```

6. General cryptographic Hashes

HASHMD5, HASHSHA1, HASHSHA2 and HASHWHIRLPOOL represent general purpose cryptographic hashes. The common class HASH allows you to use them with the same flow: create the object, call Do, get the result and release the instance if created dynamically.

MD5 and SHA1 continue to appear in many code bases for compatibility, historical file identification or integration with old systems, but they should not be the main choice for new security functions. For new developments, the SHA2 block is usually the most reasonable option within these classes, especially SHA-256 or SHA-512, depending on the need for output size and compatibility.

HASHSHA2 allows you to select the size using HASHSHA2TYPE. In DevTestsConsole SHA2-224, SHA2-256, SHA2-384 and SHA2-512 are tested using the same usage pattern.

Class	Type	Recommended use within an application with GEN Framework
HASHMD5	128-bit Hash	Compatibility, historical footprints, not as a new security base.
HASHSHA1	160-bit hash	Compatibility with legacy systems, certificates or protocols.
HASHSHA2	SHA-224, SHA-256, SHA-384, SHA-512	Main option for new cryptographic fingerprints within the block.
HASHWHIRLPOOL	512 bit Hash	Robust footprint of large size, useful when you want a wide output.

```

XBUFFER input;
XSTRING result;

input.Add((XBYTE*)buffer, size);

HASHSHA2 sha512(HASHSHA2TYPE_512);
sha512.Do(input);
sha512.GetResultString(result);

```

7. HASHCOMP128V1

HASHCOMP128V1 is a more specialized class. It does not respond to the typical file hash pattern or generic buffer, but to an algorithm associated with classic GSM authentication. It works with RAND and Ki, and generates two differentiated results: SRES and Kc. For that reason it exposes specific methods such as SetInput, Get SRES and GetKc.

In an application with normal GEN Framework it is not usually the first kind of hash to be used. Its presence shows that the Cipher block contains not only generic algorithms, but also specific pieces that appeared due to real integration or communication needs.

```

XBYTE rand[HASHCOMP128V1_RAND_SIZE];
XBYTE ki[HASHCOMP128V1_KI_SIZE];
XBUFFER input;
XBUFFER sres;
XBUFFER kc;

HASHCOMP128V1 comp128;
comp128.SetInput(rand, ki, input);
comp128.Do(input);
comp128.GetSRES(sres);
comp128.GetKc(kc);

```

8. The CIPHER class

CIPHER is the common basis for classes that transform data using a key. Its responsibility is greater than that of a simple encryption function. It maintains the encryption type, the chain mode, the padding type, the padding setting, the keys, the initialization vector and the result buffer.

The public interface offers Cipher and Uncipher over byte pointers or over XBUFFER. After running the operation, the result is obtained with GetResult. This design fits the way the framework works: the data enters a XBUFFER, the algorithm produces another XBUFFER, and the result can be passed to DIO, Compression, Databases, a file or a network layer.

CIPHER also defines the chain modes CIPHERCHAININGMODE_ECB, CBC, CFB and CTR. Not all algorithms implement all modes in the same way, but the interface allows you to configure behavior from a common point. tests de DevTestsConsole is used CBC with zero padding and an initialization vector.

Concept	Related methods	Function
Encryption type	GetType(), SetType()	Identify the specific algorithm.
Chain mode	GetChainingMode(), SetChainingMode()	Select ECB, CBC, CFB or CTR.
Padding	GetPaddingType(), SetPaddingType()	Adjust data to block size when necessary.
Key	GetKey(), SetKey()	Associates the key to the algorithm.
Initialisation Vector	GetInitVector(), SetInitVector()	It provides the IV for modes that require it.
Result	GetResult(), GetResultString()	Returns the buffer generated by Cipher or Uncipher.

9. Symmetrical Encryption

Symmetric encryption uses the same key to encrypt and decrypt. It is the usual mechanism to protect large volumes of data because it is usually much faster than asymmetric encryption. In GEN Framework, the main classes of this family are CIPHERDES, CIPHER3DES, CIPHERAES and CIPHERBLOWFISH, in addition to the CIPHER base implementation that acts as a single XOR.

The CIPHERKEYSYMMETRICAL class stores the key as a XBUFFER. The same key can be associated with a specific CIPHER using SetKey. The algorithm is configured with chain mode, padding and initialization vector. The usage pattern is repeated between DES, 3DES, AES and Blowfish.

Class	Description	Practical comment
CIPHER	Base implementation type XOR.	Useful as a basis or test, not as strong encryption.
CIPHERDES	64-bit block DES.	Inherited algorithm; useful for compatibility.
CIPHER3DES	Triple DES based on DES.	Compatibility with old systems that still demand it.
CIPHERAES	AES with 128 bit blocks.	Main symmetrical option in new developments.
CIPHERBLOWFISH	Blowfish with variable key.	Symmetrical alternative present in the block.

```
XBYTE keydata[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe,
                    0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
                    0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61, 0x08, 0xd7,
                    0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
```

```
XBYTE inivector[] = { 0x00, 0x01, 0x02, 0x03,
                     0x04, 0x05, 0x06, 0x07,
                     0x08, 0x09, 0x0A, 0x0B,
                     0x0C, 0x0D, 0x0E, 0x0F };
```

```

XBUFFER input;
XBUFFER ciphered;
CIPHERKEYSYMMETRICAL key;
CIPHERAES cipher;

key.Set(keydata, sizeof(keydata));
input.Add(data, size);

cipher.SetChainingMode(CIPHERCHAININGMODE_CBC);
cipher.SetPaddingType(XBUFFER_PADDINGTYPE_ZEROS);
cipher.SetInitVector(inivector, sizeof(inivector));
cipher.SetKey(&key);

if(cipher.Cipher(input))
{
    ciphered.CopyFrom((*cipher.GetResult()));
}

```

To decrypt, an instance configured with the same key, the same mode, the same padding and the same initialization vector is created or reused.

```

XBUFFER plain;

if(cipher.Uncipher(ciphered))
{
    plain.CopyFrom((*cipher.GetResult()));
}

```

10. Keys and internal representation

CIPHERKEY is the common basis of the keys. It maintains the key type, allows you to query its size and offers CopyFrom to copy a compatible key. Derivative classes adapt that base to specific families:

CIPHERKEYSYMMETRICAL for symmetrical keys, CIPHERKEYPUBLICRSA for public keys RSA, CIPHERKEYPRIVATERSA for private keys RSA and CIPHERKEYECDSA for ECDSA keys.

The decision to represent keys as own classes has a clear advantage: the algorithm does not work with loose data without context. A key knows its type, size and structure. This makes it easier for CIPHER, CIPHERRSA or key files to better validate what they receive.

Class	Represents	Relationship
CIPHERKEY	Generic key.	Base for all block keys.
CIPHERKEYSYMMETRICAL	Shared binary key.	Used by DES, 3DES, AES, Blowfish and other symmetrical encryptions.
CIPHERKEYPUBLICRSA	Module and public exponent RSA.	Used by CIPHERRSA for operations with public key.
CIPHERKEYPRIVATERSA	Prime factors and private exponent RSA.	Used by CIPHERRSA for private operations and optimization CRT.
CIPHERKEYECDSA	ECDSA key.	Derives from the symmetric key as specialized binary storage.

11. GKF and PEM key files

An application cannot rely solely on keys created in memory. In practice, keys are saved, loaded, exported or distributed. GEN Framework includes two important classes for that work: CIPHERKEYSFILEGKF and CIPHERKEYSFILEPEM.

CIPHERKEYSFILEGKF works with its own .gkf format, GEN Framework key file (GKF). This format is based on XML and allows to store creator metadata, creation date, last modified date and different keys with attributes such as type, size, module, exponent or prime factors. DevTestsConsole uses a keys.gkf file in assets/certificates to load public and private RSA keys.

CIPHERKEYSFILEPEM works with PEM inputs, keys and certificates. It supports XASN1, XBER and the observer pattern to decode encoded structures. It also collaborates with CIPHERCERTIFICATEX509 when content is a certificate.

```

XPATH xpath;
XPATH xpathgeneric;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathgeneric);
xpath.Create(3, xpathgeneric.Get(), __L("keys"), CIPHERKEYSFILEGKF_EXT);

CIPHERKEYSFILEGKF filekeys(xpath);

CIPHERKEYPUBLICRSA* publickey = (CIPHERKEYPUBLICRSA*)filekeys.GetKey(CIPHERKEYTYPE_RSA_PUBLIC);
CIPHERKEYPRIVATERSA* privatekey = (CIPHERKEYPRIVATERSA*)filekeys.GetKey(CIPHERKEYTYPE_RSA_PRIVATE);

```

12. RSA and CIPHERRSA

CIPHERRSA implements RSA asymmetric encryption within the GEN Frameworkstyle. Hereda from CIPHER, but adds own methods because RSA needs to work with public and private keys, block sizes, padding PKCS#1, multi-precision integers and, in some cases, signature.

The class is supported by XMPINTEGER for large integer operations. CIPHERRSA_CONTEXT maintains the public module, exponents, prime factors, CRT values and auxiliary data. The class CIPHERRSA can load keys using SetKey, encrypt with one key, decrypt with the other and generate new keys through GenerateKeys.

In DevTestsConsole a key pair is loaded from keys.gkf, a long text is encrypted using the private key and decrypted using the public key, checking after the result matches the original. That test shows two ideas: first, that the algorithm works with XBUFFER; second, that CIPHERRSA internally divides the data into blocks appropriate to the size of key and padding.

```

CIPHERRSA cipher;
XBUFFER input;
XBUFFER output;

input.Add((XBYTE*)text, size);

if(cipher.SetKey(publickey))
{
    if(cipher.SetKey(privatekey))
    {
        if(cipher.Cipher(input, CIPHERKEYTYPE_RSA_PRIVATE))
        {
            output.CopyFrom((*cipher.GetResult()));
        }
    }
}

```

```

        cipher.Uncipher(output, CIPHERKEYTYPE_RSA_PUBLIC);
    }
}

bool same = input.Compare(cipher.GetResult());

```

RSA also offers Sign, which combines the data with a HASH class. This point is important: to sign you must not sign a string improvisedally, but build a clear flow where the chosen hash is part of the operation.

```

CIPHERRSA rsa;
HASHSHA2 hash(HASHSHA2TYPE_256);

rsa.SetKey(privatekey);
rsa.Sign(input, CIPHERKEYTYPE_RSA_PRIVATE, &hash);

```

13. Curve25519 / X25519

CIPHERECDSAX25519 provides a key exchange-oriented piece using Curve25519/X25519. Although the class's historical name includes ECDSA, DevTestsConsole uses private key generation, public key creation, and key calculation shared between two extremes.

This type of mechanism is not used to directly encrypt a large file. Its utility is to allow two participants to generate a shared key from their private keys and the public keys at the other end. That shared key can then power a symmetric encryption, for example AES, within a properly designed protocol.

DevTestsConsole's test creates two objects, generates two private keys, creates corresponding public keys, exchanges public keys and generates two shared keys. The final test compares byte to byte that both shared keys match.

```

CIPHERECDSAX25519 peer1;
CIPHERECDSAX25519 peer2;

peer1.GenerateRandomPrivateKey();
peer2.GenerateRandomPrivateKey();

peer1.CreatePublicKey();
peer2.CreatePublicKey();

peer1.CreateSharedKey(peer2.GetKey(CIPHERECDSAX25519_TYPEKEY_PUBLIC));
peer2.CreateSharedKey(peer1.GetKey(CIPHERECDSAX25519_TYPEKEY_PUBLIC));

XBYTE* shared1 = peer1.GetKey(CIPHERECDSAX25519_TYPEKEY_SHARED);
XBYTE* shared2 = peer2.GetKey(CIPHERECDSAX25519_TYPEKEY_SHARED);

```

14. X.509 certificates and trusted roots

Cipher also includes classes to work with certificates. CIPHERCERTIFICATEX509 represents an X.509 certificate: version, serial number, algorithm, validity dates, issuer or subject identity, associated public key, hash and certificate data. CIPHERCERTIFICATEX509_ID groups identity fields such as country, organization, organizational unit and common name.

CIPHERTRUSTEDROOTCERTIFICATESX509 allows you to read a root certificate file and keep your lines in memory. You can also generate an embedded header from a source file. In DevTestsConsole you see the use of assets/certificates/cacert.pem as source to decode certificates using CIPHERKEYSFILEPEM.

These classes are related to DIO and TLS, but belong to Cipher because the certificate is a cryptographic piece: it contains identity, validity periods, algorithm, public key and data that must be validated.

```
CIPHERTRUSTEDROOTCERTIFICATESX509 trustedrootcertificates;
CIPHERKEYSFILEPEM                filekeys;
XPATH                             xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_CERTIFICATES, xpath);
xpath.Slash_Add();
xpath.Add(__L("cacert.pem"));

if(trustedrootcertificates.ReadFromFile(&xpath))
{
    filekeys.DecodeCertificates(trustedrootcertificates.GetLines());
}
```

15. Relationship with assets and XPATHMANAGER

The Cipher block must not work with absolute paths. The keys, certificates and test files must be placed inside assets, usually in a certified section or equivalent. This criterion maintains the consistency with what is explained in the chapter of the first application: all the application resources, except the executable, live under assets and are accessed by XPATHMANAGER.

DevTestsConsole follows this criterion with assets/certificates. Cacert.pem, cacert.h, certificate.crt, certificate.key, id_rsa, id_rsa.pub, root.pem and keys.gkf. A proprietary application can use a similar structure when you need to load keys or certificates.

```
assets/
|
+-- certificates/
|   +-- cacert.pem
|   +-- keys.gkf
|   +-- certificate.crt
|   +-- certificate.key
|
+-- miaplicacion.ini
```

16. Block activation in CMake

As with other blocks in the framework, Cipher is enabled by functionalities. This allows you to compile only what the application needs. DevTestsConsole activates hashes, symmetric encryption, key files, RSA and X25519 using specific CMake options.

An application that only needs SHA2 does not have to activate the entire asymmetric block. An application that uses TLS or certified usually requires more parts. This granularity helps control dependencies, build times, and final size.

```
option(HASH_FEATURE                "Hash"                ON )
option(HASH_CRC32_FEATURE          "Hash CRC32"        ON )
```

option(HASH_MD5_FEATURE	"Hash MD5"	ON)
option(HASH_SHA1_FEATURE	"Hash SHA1"	ON)
option(HASH_SHA2_FEATURE	"Hash SHA2"	ON)
option(HASH_WHIRLPOOL_FEATURE	"Hash Whirlpool"	ON)
option(CIPHER_SYMMETRIC_FEATURE	"Cipher Symetric"	ON)
option(CIPHER_SYMMETRIC_DES_FEATURE	"Cipher Symetric DES"	ON)
option(CIPHER_SYMMETRIC_AES_FEATURE	"Cipher Symetric AES"	ON)
option(CIPHER_SYMMETRIC_BLOWFISH_FEATURE	"Cipher Symetric BlowFish"	ON)
option(CIPHER_ASYMMETRIC_FEATURE	"Cipher Asymetric"	ON)
option(CIPHER_ASYMMETRIC_FILEKEY_GKF	"Cipher Asymetric File Key (.Gen Key)"	ON)
option(CIPHER_ASYMMETRIC_FILEKEY_PEM	"Cipher Asymetric File Key (.PEM)"	ON)
option(CIPHER_ASYMMETRIC_RSA_FEATURE	"Cipher Asymetric RSA"	ON)
option(CIPHER_ASYMMETRIC_X25519_FEATURE	"Cipher Asimetric ECDSA X25519"	ON)

17. Class catalogue of the Cipher block

Class	What it does	Related classes / example of use
HASH	Common hash interface over memory, XBUFFER, XPATH and XFILE.	Base HASHCRC32, HASHMD5, HASHSHA1, HASHSHA2 and others.
HASHCKS16	16-bit checksum.	Light validation of data or plots.
HASHCRC16	16-bit CRC.	Protocols and devices waiting for CRC16.
HASHCRC32	32-bit CRC.	Quick track of files and buffers.
HASHMD5	Hash MD5.	Compatibility with old systems.
HASHSHA1	Hash SHA1.	Compatibility and legacy protocols.
HASHSHA2	SHA2 family.	SHA-224, SHA-256, SHA-384 and SHA-512.
HASHWHIRLPOOL	512 bit Hash Whirlpool.	Big trace of data.
HASHCOMP128V1	Algorithm COMP128v1.	RAND, Ki, SRES and Kc.
CIPHER	Encryption base.	Manage key, mode, padding, IV and result.
CIPHERDES	DES.	Symmetrical encryption inherited.
CIPHER3DES	Triple DES.	Compatibility with old environments.
CIPHERAES	AES.	Main symmetric encryption for new uses.
CIPHERBLOWFISH	Blowfish.	Alternative symmetric encryption.
CIPHERKEY	Keybase.	Type, size and key copy.
CIPHERKEYSYMMETRICAL	Symmetrical key.	Used by AES, DES, 3DES and Blowfish.
CIPHERKEYPUBLICRSA	RSA public key.	Module and exponent.
CIPHERKEYPRIVATERSA	Private key RSA.	Factors primes and exponent.
CIPHERKEYECDSA	ECDSA key.	ECDSA specific key representation.
CIPHERKEYSFILEGKF	GEN Framework key file (GKF).	Load and update .gkf keys.
CIPHERKEYSFILEPEM	PEM reading and certificates.	Decodes keys/certificates with XASN1/XBER.
CIPHERRSA_CONTEXT	Internal context RSA.	N, E, D, P, Q, CRT values and auxiliary data.
CIPHERRSA	RSA.	Encrypt, decipher, sign and generate keys.
CIPHERECD SAX25519	Curve25519/X25519.	Private, public and shared keys.

CIPHERCERTIFICATEX509_ID	Identity X.509.	Country, organisation, organizational unit and common name.
CIPHERCERTIFICATEX509	Certificate X.509.	Version, series, algorithm, dates, public key and hash.
CIPHERTRUSTEDROOTCERTIFICATEX509	Trust roots.	Read root certificates and can generate embedded header.

18. How to study Cipher from DevTestsConsole

DevTestsConsole is the best entry point to see the practical use of the Cipher block. Its tests show five very useful blocks: Test_Hash, Test_Cipher_Simetric, Test_CipherFileKeys, Test_CipherRSA and Test_CipherECDSAX25519.

Test_Hash shows how to create different HASH classes and get their results. Test_Cipher_Simetric shows the common encryption and decryption pattern with symmetric key, CBC mode, padding and IV. Test_CipherFileKeys shows certificate reading and use of CIPHERKEYSFILEPEM. Test_CipherRSA shows key loading from keys.gkf and RSA encryption/decryption test. Test_CipherECDSAX25519 shows key exchange between two participants.

To learn the module, you should run a test each time and follow the flow with the debugger. The goal is not to memorize the internal algorithm, but to understand what class each concept represents, what object has the result and where each resource is released.

19. Criteria for use in an application with GEN Framework

The first criterion is to choose the right tool. To check if a file accidentally changed, CRC32 may be enough. For a cryptographic fingerprint, SHA2 is a more appropriate option. To protect large data, it is normal to use symmetric encryption. For key exchange or identity exchange, enter RSA, X25519 and certificates.

The second criterion is not to disperse keys or routes. Keys and certificates must live under assets, in a section controlled by XPATHMANAGER, and the code must access them by routes managed by the framework. This avoids absolute routes and facilitates moving the application between platforms.

The third criterion is to close the life cycle. Objects created with GEN_NEW must be released with GEN_DELETE or by the intended mechanism. Buffers must be emptied when they are no longer needed. Keys should not be copied without need or left in logs. Traces must be useful, but they must not print secrets in production.

The fourth criterion is not to invent cryptographic protocols. Cipher offers low-level parts. Real security depends on how those parts are combined. Using AES, RSA or SHA2 does not automatically convert an application into secure if the key design, IVs, padding, authentication or storage is incorrect.

20. Practical summary

Cipher is the GEN Framework block dedicated to hashes, checksums, encryption, keys and certificates. Its main value is to integrate these operations into the framework style: XBUFFER for data, XPATH and XFILE for files, XSTRING for readable results, XMPINTEGER for asymmetric operations and assets/XPATHMANAGER for cryptographic resources.

HASH provides a common interface for abstracts. CIPHER provides a common interface for encryption. Key classes separate the representation of each key type. GKF and PEM files allow to persist or import cryptographic material. RSA and X25519 cover asymmetrical scenarios. X.509 classes allow you to handle certificates and trusted roots.

In real applications, Cipher usually works together with DataIO, Compression, AppFlow and XUtilities. Data is read, compressed, encrypted, signed, transmitted or stored. The programmer must understand the role of each class

and use it within a clear architecture, without converting cryptography into improvised code dispersed by the application.

Chapter XI: Compression of data and files in GEN Framework

1. The objective of the chapter

After studying the main blocks of the framework, the input and output systems, graphics, user input, sound and encryption, a very common need appears in real applications: reducing the size of the data. Compression is not a decorative element. In an application with resources, traces, configurations, update packages, communications, backups or file exchange, compressing correctly can make the difference between a comfortable system to deploy and another one heavy, slow or difficult to maintain.

The Compress GEN Framework block groups the classes dedicated to data compression and decompression. Its aim is not to replace all existing formats or become a universal packaging tool, but to offer a coherent layer, with GEN Framework style, to work with compression algorithms from the framework itself. As with other blocks, the idea is that the application does not have to directly use external APIs or third-party library structures when it can do so through its own classes, GEN Framework types and common buffers.

This chapter explains how the Compress block is organized, which paper fulfills its classes, when each algorithm should be used, and how it relates to XUtilities, DataIO, XPATHMANAGER and XFile ZIP. It also includes examples of use inspired by DevTestsConsole, where a buffer is compressed and decompressed and a file is added to a ZIP to validate the entire flow.

2. The Compress block inside GEN Framework

Compress is a cross-sectional block. It does not belong directly to the file system, even if it can compress files. It does not belong to DataIO, although it can be used on data entering or leaving a stream. It does not belong to Cipher, although many times compression and encryption appear together in the same stream. Its natural place is that of an auxiliary service that transforms data: it receives a sequence of bytes and produces another sequence of bytes, usually smaller or packaged.

This separation is important. The application should not mix compression logic with business logic. Nor should it repeat zlib code, LZW or internal algorithms in every module you need to compress. It is reasonable to encapsulate that responsibility in specific classes and use a common interface. That common interface is represented by COMPRESSBASE and by derivatives that implement specific algorithms.

In an application with GEN Framework, Compress usually works with XBUFFER. This fits the framework philosophy: binary data is manipulated by a common class, instead of passing pointers, sizes and manually reserved memory scatteredly. Using XBUFFER allows the output of a compression to be then written to a file, sent by network, stored in a database, encrypted or integrated into a resource package.

3. Data compression versus file compression

A practical distinction that should be clear from the beginning is the difference between compressing data and packing files. Compressing data means taking a block of bytes and generating another compressed block. The source can be a UTF-8 string, a buffer memory, an already loaded image, a trace, a configuration or any other binary content. The result is also a buffer. COMPRESS_ZIP, COMPRESS_GZ, COMPRESS_LZW and COMPRESS_LZRW1KH classes work mainly at this level.

Packing files is a top-level operation. In that case, it is not only interesting to compress a block, but to keep file names, internal paths, entries inside a file, metadata and package structure. XFILEZIP appears within the file system, which relates to Compress but has a different responsibility. DevTestsConsole displays both ideas: on the

one hand, it uses XFILEZIP to add a file to a ZIP; on the other hand, it uses COMPRESSBASE to compress and decompress a buffer.

This difference helps to choose the right tool. If the application needs to reduce the size of a XBUFFER before sending it over the network, Compress is enough. If you need to create a package with multiple files inside, it is natural to use XFILEZIP or an equivalent file layer. If you need to compress a loose file, you can read it to XBUFFER, compress it and write the result; but if you need a container with internal inputs, then a file format class is needed.

4. General map of classes

The Compress block is small when compared to XUtilities, DataIO or Graphic, but it is very clearly structured. There is a base class, a creation manager and several algorithm implementations. This simplicity is precisely one of its virtues: the programmer can quickly understand what class to use.

Class / module	Responsibility	Typical use
COMPRESSBASE	Common base class for compression and decompression operations.	Common interface, null class or direct copy when there is no specific algorithm.
COMPRESSMANAGER	Compress Block Light Manager/factory.	Create an algorithm instance from COMPRESSBASE_TYPE.
COMPRESS_ZIP	Compression based on zlib/deflate.	Buffers compressed in zlib format, general use.
COMPRESS_GZ	Zlib based GZip compression with GZ header.	Data supported by GZip streams or files.
COMPRESS_LZW	Algorithm LZW, with internal support for use type GIF.	Resources, repetitive data and compatibility with LZW flows.
COMPRESS_LZRW1KH	Algorithm LZRW1/KH oriented at speed and low cost.	Data where quick compression and light logic are of interest.
XFILEZIP	File format ZIP within XUtilities/FormatFiles.	Create ZIP packages, add files and work with entries.

5. COMPRESSBASE: the common interface

COMPRESSBASE is the base class of the block. It defines the contract that compression classes must fulfill: a Compress method to transform original data into compressed data and a Decompress method to recover original data. The class works with XBYTE as data drive, XDWORD as size and XBUFFER as output container. In this way it maintains the style of types of the framework and avoids exposing standard types or external structures.

The enum COMPRESSBASE_TYPE identifies the available algorithms. Sources include COMPRESSBASE_TYPE_NONE, COMPRESSBASE_TYPE_LZW, COMPRESSBASE_TYPE_LZRW1KH, COMPRESSBASE_TYPE_ZIP and COMPRESSBASE_TYPE_GZ. This enum allows the application to request an algorithm without directly creating the derived class. The selection can be centralized in COMPRESSMANAGER.

The base implementation acts as a neutral class. If you call Compress or Decompress on COMPRESSBASE without a specific algorithm, the behavior is to copy the source content to the target buffer. This may seem simple, but it makes sense as a base case: it allows a code path to work even when the actual compression is not enabled or when you want to maintain the same flow without modifying the external logic.

```
XSTRING source;
XBUFFER origin;
XBUFFER packed;
```

```

source = __L("Test text for compression");
source.ConvertToUTF8(origin, false);

COMPRESSBASE compress;
compress.Compress(origin.Get(), origin.GetSize(), &packed);

// In the base class the result is a copy of the original buffer.

```

In real applications, COMPRESSBASE is usually not instanced directly except as fallback. It is usual to use COMPRESSMANAGER or create a specific derivative class when you know exactly what algorithm is needed.

6. COMPRESSMANAGER: creating algorithms

COMPRESSMANAGER acts as the creation manager for the Compress block. It is not a main factory of the framework such as XFACTORY, DIOFACTORY, GRPFACTORY, INPFACTORY or SNDFACTORY, but it follows the same conceptual idea: centralize object creation. The Create method receives a COMPRESSBASE_TYPE and returns a pointer to COMPRESSBASE. Internally, according to the active macros, it creates COMPRESS_LZW, COMPRESS_LZRW1KH, COMPRESS_ZIP, COMPRESS_GZ or a base instance.

This class also has a static instance pattern using GetInstance and DeleteInstance. In simple code you can create a local COMPRESSMANAGER with GEN_NEW, as DevTestsConsole does. In more global code you can use the shared instance. The important thing is that the application doesn't need to know all the included and specific classes if it works through the compression type.

```

COMPRESSMANAGER* manager    = GEN_NEW COMPRESSMANAGER();
COMPRESSBASE*   compressor = NULL;

if(manager)
{
    compressor = manager->Create(COMPRESSBASE_TYPE_ZIP);
}

// ... use compressor ...

if(compressor) delete compressor;
if(manager)    delete manager;

```

This pattern reinforces the idea of modularity. The application code can choose the type of compression in configuration or context, and the manager is responsible for returning the appropriate class. If an algorithm is not compiled, the manager does not offer it as actual implementation and the case NULL or fallback must be treated.

7. COMPRESS_ZIP: deflate compression over buffers

COMPRESS_ZIP implements compression using zlib with classic deflate flow. Although the name may suggest a complete .zip file, in this class the main job is to compress and decompress buffers. Creating a ZIP file with internal inputs belongs to XFILEZIP. This difference is important to not confuse compression algorithm with container format.

The class displays Compress and Decompress with the same signature as COMPRESSBASE. In Compress the target buffer is cleaned, initial memory is reserved, ZCompress runs and the final size of the XBUFFER is adjusted to the compressed size. In Decompress, the target buffer size must be prepared to contain the decompressed result; therefore, in the DevTestsConsole example it resizes decmp buffer to the original size before calling Decompress.

```

XSTRING sentence;
XBUFFER origin;
XBUFFER compressed;
XBUFFER decompressed;

sentence = __L("Repeated data, repetitive data, repetitive data.");
sentence.ConvertToUTF8(origin, false);

decompressed.Resize(origin.GetSize());

```

```

COMPRESS_ZIP zip;
if(zip.Compress(origin.Get(), origin.GetSize(), &compressed))
{
    zip.Decompress(compressed.Get(), compressed.GetSize(), &decompressed);
}

```

COMPRESS_ZIP is a good choice when you want a general data compression in memory and zlib is available. It is suitable for configuration buffers, network responses, temporary data or any block that is then stored or transmitted inside another container.

8. COMPRESS_GZ: GZip compatible compression

COMPRESS_GZ is also supported by zlib, but configures the flow to generate or read data with GZ ip header. Sources show the use of deflateInit2 with the combination of GZ window and encoding, and inflateInit2 with 16 + MAX_WBITS to recognize the GZ ip format. This differentiates COMPRESS_GZ from COMPRESS_ZIP: both use zlib, but do not generate exactly the same data wrapper.

The use of GZip is common when working with data that must be supported by external tools, web streams, compressed HTTP responses or .gz files. If the application only needs to compress internally a buffer, ZIP/deflate may be sufficient. If you need to interoperate with a GZip format, then COMPRESS_GZ is more appropriate.

```

COMPRESS_GZ gz;
XBUFFER      origin;
XBUFFER      packed;
XBUFFER      unpacked;

// origin already contains binary data.
unpacked.Resize(origin.GetSize());

if(gz.Compress(origin.Get(), origin.GetSize(), &packed))
{
    gz.Decompress(packed.Get(), packed.GetSize(), &unpacked);
}

```

As in the other algorithms, the application must know or retain the expected size of the decompressed result if you want to correctly reserve the output buffer. This can be done by saving the original size in a header, in a metadata file, in a package structure or in the protocol that transports the data.

9. COMPRESS_LZW: LZW compression and GIF type of support

COMPRESS_LZW implements the LZW algorithm. LZW is a classic dictionary-based algorithm, especially known for its historical presence in image formats such as GIF. In GEN Framework, the class not only exhibits Compress and Decompress, but also internal methods oriented to encoding and decoding data type GIF: LZW_GIF_Encode and LZW_GIF_Decode.

The code maintains a table of strings, clear/end codes, output bits, hash table and parameters related to width, height, interlace and pixel reading. This shows that the class is not only intended as generic text compressor, but also as support for graphical formats that need LZW in their internal process.

In a normal application not all internal methods are usually called. Usual use consists of applying Compress and Decompress over XBUFFER. LZW_GIF_Encode and LZW_GIF_Decode methods belong to a more specialized level and make sense when implementing or maintaining support for a particular graphic format.

```

COMPRESS_LZW lzw;
XBUFFER      origin;
XBUFFER      packed;
XBUFFER      unpacked;

// The buffer decompressed output must be reserved to the desired size.
unpacked.Resize(origin.GetSize());

if(lzw.Compress(origin.Get(), origin.GetSize(), &packed))
{

```

```

    lzw.Decompress(packed.Get(), packed.GetSize(), &unpacked);
}

```

LZW can be useful in repeat data and modules that need compatibility with historical formats. As a practical criterion, it should not be automatically chosen for everything. In GEN Framework it is advisable to choose the algorithm according to the format, the necessary compatibility and the cost of CPU/acceptable memory.

10. COMPRESS_LZRW1KH: quick and light compression

COMPRESS_LZRW1KH implements a variant of LZRW1/KH. This type of algorithm is fast compression oriented, with relatively small hash structures and a simple matching logic. A hash table of 4096 entries, matching search and writing of copied or compressed data markers is observed in the sources. Its natural orientation is to reduce data at low processing cost.

An interesting feature of this class is that it offers two forms of work. The first uses XBUFFER as a destination, as well as the rest of compressors. The second one accepts a void* param1 and uses CompressResult to emit the result byte to byte. This design allows adapting the algorithm to special outputs without always forcing the construction of an intermediate XBUFFER.

```

COMPRESS_LZRW1KH lzw;
XBUFFER          origin;
XBUFFER          packed;
XBUFFER          unpacked;

packed.Resize(origin.GetSize() + 1024);
unpacked.Resize(origin.GetSize());

lzw.Compress(origin.Get(), origin.GetSize(), &packed);
lzw.Decompress(packed.Get(), packed.GetSize(), &unpacked);

```

The mode with callback using CompressResult can be useful in advanced scenarios: write directly to a stream, feed a queue, send data by blocks or adapt the result to a proprietary format. In such cases, the derivative class or integration code can redefine CompressResult and use param1 as context.

11. XFILEZIP: ZIP packages and files

Although XFILEZIP does not strictly belong to the Compress block as an algorithm, it appears naturally when it comes to file compression. The reason is simple: a ZIP file is not only a compressed buffer; it is a container with inputs, internal names, structure and metadata. That is why GEN Framework separates the data compressor from the file format.

DevTestsConsole shows a very clear example. First create a path to the file you want to add, another internal path inside the ZIP and a path to the resulting .zip file. Then create XFILEZIP, open the ZIP, add the file and close the container. That example serves to understand that file packaging is resolved from the file layer, although internally it can be supported by compression.

```

#define NAMEFILECMPINZIP __L("testcmp.txt")

XPATH xpathcmpfile;
XPATH xpathcmpinzip;
XPATH xpathzipfile;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathcmpfile);
xpathcmpfile.Add(NAMEFILECMPINZIP);

xpathcmpinzip.Add(NAMEFILECMPINZIP);

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathzipfile);
xpathzipfile.Slash_Add();
xpathzipfile.Add(__L("test.zip"));

XFILEZIP* zipfile = GEN_NEW XFILEZIP();
if(zipfile)

```

```

{
    if (zipfile->Open(xpathzipfile))
    {
        zipfile->AddFile(xpathcmpfile, xpathcmpinzip);
        zipfile->Close();
    }

    delete zipfile;
}

```

This class is used when the goal is to generate a file package, not just compress a memory block. It is useful for backup, data export, resource packages, multiple file sending or creating temporary files with internal structure.

12. Relationship with assets and XPATHMANAGER

In an application with GEN Framework, resource files are placed under assets and accessed by XPATHMANAGER. This also affects compression. If you are going to create a ZIP configuration, a logs package, a GZ file or a compressed copy of a local database, it is reasonable to get the path from XPATHMANAGER instead of building an absolute path manually.

The general pattern consists of ordering the path of the appropriate section, adding the filename and working with XFILE, XFILEZIP or XBUFFER. This keeps the portable application and prevents rigid routes. In Windows, Linux or Android the physical location can change, but the application continues to use the logical sections defined by the framework.

```

XPATH xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpath);
xpath.Slash_Add();
xpath.Add(__L("backup.zip"));

// xpath points to the file within the resource tree of the application.

```

If the application needs a specific section, such as backups, exports or packages, you can create a new section in XPATHMANAGER or reuse an existing section. The criterion is the same as we saw in previous chapters: all resources must be sorted and traceable in a homogeneous way.

13. Complete example: compress and decompress a text

The following example summarizes the typical use of Compress on a buffer. Part of a XSTRING, converts to UTF-8 into a XBUFFER, creates a ZIP compressor using COMPRESSMANAGER, compresses the buffer and then decompresses to reconstruct the original text. It is the same conceptual pattern that appears in DevTestsConsole.

```

COMPRESSMANAGER* manager    = GEN_NEW COMPRESSMANAGER();
COMPRESSBASE*   compressor = NULL;

XSTRING sentence;
XSTRING finalsentence;
XBUFFER buffer;
XBUFFER cmpbuffer;
XBUFFER decmpbuffer;

sentence = __L("Compress test from GEN Framework.");
sentence.ConvertToUTF8(buffer, false);

decmpbuffer.Resize(buffer.GetSize());

if(manager)
{
    compressor = manager->Create(COMPRESSBASE_TYPE_ZIP);
}

if(compressor)
{
    if(compressor->Compress(buffer.Get(), buffer.GetSize(), &cmpbuffer))
    {

```

```

        compressor->Decompress(cmpbuffer.Get(), cmpbuffer.GetSize(), &decmpbuffer);
        finalsentence.ConvertFromUTF8(decmpbuffer);
    }

    if(compressor) delete compressor;
    if(manager) delete manager;

```

There are two important details. The first is that the decompressed buffer is reserved with the original size. The second is that the final verification should not be done by comparing compressed sizes, but by reconstructing the original data and comparing it with the initial content. In integrity testing, this can be reinforced by calculating a hash before and after, using the Cipher/Hash block studied in the previous chapter.

14. Compression, encryption and order of operations

Compress is naturally related to Cipher. Many applications need to store data in a compact and also protected way. In that case you have to decide the order of operations. The usual practical rule is to compress first and encrypt later. The reason is that encrypted data must appear random; when trying to compress after encrypting, there are usually no usable patterns and the size is not significantly reduced.

A typical flow would be: serialize data to XBUFFER, compress with Compress, encrypt with Cipher and write the result into a file or send it by DataIO. To recover the information, the order is reversed: read, decrypt, decompress and de-serialize. Separate each step in its corresponding module makes the code more maintainable and allows to change the compression or encryption algorithm without altering the entire application.

```

// Flujo conceptual:
// 1. original data -> XBUFFER
// 2. COMPRESS_ZIP -> buffer comprimido
// 3. CIPHERAES -> buffer cifrado
// 4. XFILE/DIOSTREAM -> storage or shipping

// To read:
// 1. XFILE/DIOSTREAM -> buffer cifrado
// 2. CIPHERAES -> buffer comprimido
// 3. COMPRESS_ZIP -> original data

```

This stream also shows why GEN Framework organizes blocks by responsibility. Compress does not encrypt. Cipher does not compress. DataIO does not decide the algorithm. Each module provides a piece and the application combines them in a controlled way.

15. Activation using macros and compilation

Compress sources are prepared to compile specific algorithms using activation macros. Conditions such as COMPRESS_LZW_ACTIVE, COMPRESS_LZRW1KH_ACTIVE, COMPRESS_ZIP_ACTIVE and COMPRESS_GZ_ACTIVE appear in CompressManager.cpp. This allows to include or exclude algorithms according to the needs of the project, platform or dependencies available.

This technique fits the GEN Frameworkbuild system. A small or embedded application may not need all algorithms. A package export desktop application can activate ZIP. An application that works with GZip-compatible formats can activate GZ. A graphical module with historical formats may need LZW. The goal is not to drag unnecessary dependencies when they do not add value.

```

// Conceptual example of selection by compilation:
#define COMPRESS_ZIP_ACTIVE
#define COMPRESS_GZ_ACTIVE

// CompressManager can create COMPRESS_ZIP and COMPRESS_GZ
// when those options are enabled in the project.

```

The exact way to activate these macros depends on the CMake Lists.txt and common framework options. As a general criterion, applications with GEN Framework should not define macros in a scattered way in each source, but should centralize the selection of functionalities in CMake and build configuration files.

16. Usual errors when using Compress

The most common error when using Compress is not to properly reserve the buffer decompression output. The class can compress a buffer and adjust the compressed size, but to uncompress it needs enough space to rebuild the original content. If the application does not retain the original size, it has to store it somewhere: a header, a protocol field, a file metadata or a package structure.

Another common error is to confuse COMPRESS_ZIP with XFILEZIP. The first compresses data using deflate; the second works with a ZIP file as a file container. If you want to add multiple files to a package, you need XFILEZIP. If you want to compress a memory block, you need a derivative COMPRESSBASE.

It should also be remembered that not all data is compressed well. A JPEG, a PNG, a MP3, a OGG or an already compressed file may not be reduced just. They can even grow slightly by headers or internal structure.

Compression is especially useful in text, configurations, logs, repetitive structures, uncompressed serialized data or packages with many small resources.

Finally, it is not advisable to mix manual memory without control. GEN Framework offers XBUFFER precisely to sort this work. If the application starts to save memory with new, malloc or large local buffers without a clear policy, it increases the probability of errors. The recommended style is to use XBUFFER and let the classes in the framework adjust sizes and contents.

17. Compress block summary catalogue

Group	Classes	Practical comment
Common base	COMPRESSBASE	Defines the common contract and offers neutral copy behavior.
Management	COMPRESSMANAGER	Create algorithms by type and centralize the compressor selection.
Zlib/deflate	COMPRESS_ZIP	General buffer compression using deflate. It is not a complete ZIP container.
GZip	COMPRESS_GZ	Generates/reads GZip data, useful for external compatibility.
Classic Dictionary	COMPRESS_LZW	Algorithm LZW, with specialized support for GIF flows.
Quick compression	COMPRESS_LZRW1KH	Lightweight algorithm with output to buffer or emission using CompressResult.
File Format	XFILEZIP	ZIP Container for adding and organizing files within a package.

18. Criteria for use in an application with GEN Framework

To choose a class of Compress you should start with the correct question: do you want to compress a data block or create a container file? If you want to compress a XBUFFER, you choose a Compress algorithm. If you want to

create a ZIP with files, you use XFILEZIP. Then you decide the algorithm according to compatibility, speed, final size and dependencies.

COMPRESS_ZIP is a good general option when you want standard compression based on zlib. COMPRESS_GZ is useful when you need support with GZip. COMPRESS_LZW makes sense in formats or data where LZW is needed or convenient. COMPRESS_LZRW1KH may interest you when the speed and simplicity of the algorithm are more important than getting the maximum compression rate.

In all cases, the result should be integrated with the rest of the framework: XPATHMANAGER for routes, XBUFFER for binary memory, XFILE or DataIO for storage and transmission, Cipher for protection and AppFlow to place the process within the application life cycle. Compress should not be seen as an isolated utility, but as one more part of the GEN Framework data stream.

19. Practical summary

Compress provides GEN Framework with a coherent layer for compressing and decompressing data. Its design is simple: a common base class, a creation manager and several algorithm implementations. This simplicity allows to use it without introducing direct dependencies or specific compression logic in each application.

COMPRESSBASE defines the contract. COMPRESSMANAGER creates the right algorithm. COMPRESS_ZIP and COMPRESS_GZ support zlib for general cases and GZip compatibility. COMPRESS_LZW covers dictionary-based scenarios and GIF formats. COMPRESS_LZRW1KH offers a quick and lightweight alternative. XFILEZIP complements the block when the goal is not to compress a buffer, but to create a file package.

The central idea is the same as in the rest of the framework: to organize a regular functionality of C/C++ within a homogeneous architecture. Instead of repeating compression code in each program, GEN Framework offers a common way to work with buffers, routes, files and algorithms, maintaining the style of the framework and facilitating the evolution of applications.

Chapter XII: Databases, SQL databases in GEN Framework

1. The role of Databases within the framework

The Databases block groups the classes that GEN Framework uses to access relational databases using SQL. Its objective is not to convert the application into a specific engine dependent, but to provide a homogeneous access layer to work with different engines from the same idea of connection, query, result, row, data variant and transaction.

In this phase of the framework, Databases is mainly oriented to SQL databases. This includes SQLite for local or embedded scenarios, MySQL/MariaDB for widely used servers and PostgreSQL for environments where an advanced relational engine is required. The architecture allows for the incorporation of document connectors, such as MongoDB or other NoSQL models, although the current block is organized around relational access and SQL language.

The philosophy coincides with the rest of GEN Framework: if an application can express its intention in a common way, the specific detail must be encapsulated in the corresponding module. The application should not have a different branch for each database engine except at those points where the SQL syntax really differs. The Databases block tries to reduce those differences and group them into connectors that share a user interface.

2. Why a common connector is important

In C/C++ it is common for each database engine to have its own client library, its own types, its own way of opening connections, preparing queries, linking parameters, going through results and managing errors. If an application directly uses those APIs, it is tied to the chosen engine. Switching from SQLite to PostgreSQL, or from MySQL to SQLite for local testing, can become a partial rewrite of data access.

GEN Framework tries to avoid that direct dependency. The programmer works with DB_SQL block classes and selects the database type using an enumeration value. From there, the block factory creates the right connector. The application continues to see a database, a connection, a query and a result. Internally, the framework uses the specific classes of SQLite, MySQL/MariaDB or PostgreSQL.

This common layer does not completely eliminate differences between engines. SQL is not identical in all of them. Data types, parameter placeholders, dates, transactions, savepoints or table creation may have nuances. What GEN Framework does is provide a common working point and leave inevitable differences localized.

Engine	Model	Usual use
SQLite	Local base in file	Tests, embedded applications, advanced configuration, desktop tools and serverless deployments.
MySQL/MariaDB	SQL client/server base	Applications connected to a server, web solutions, internal tools or systems already based on MySQL/MariaDB.
PostgreSQL	SQL customer/advanced server base	Applications that need a robust relational engine, complete transactions, advanced types or server deployments.

3. DB_SQL_FACTORY and engine selection

The creation of a SQL database is done using DB_SQL_FACTORY. This class concentrates the selection of the connector and prevents the application from having to directly build a class MYSQL_DATABASE, SQLITE_DATABASE or POSTGRES_SQL_DATABASE. The type is expressed by DB_SQL_DATABASE_TYPE.

```
DB_SQL_DATABASE* database = NULL;

database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);
if(!database) return false;
```

The main values of DB_SQL_DATABASE_TYPE are DB_SQL_DATABASE_TYPE_SQLITE, DB_SQL_DATABASE_TYPE_MYSQL and DB_SQL_DATABASE_TYPE_POSTGRES_SQL. Each value is compiled only if the corresponding support is enabled using the project options. This allows generating applications with a subset of connectors, avoiding unnecessary dependencies.

In the Databases example, the keystroke allows you to dynamically change the database type: M selects MySQL, L selects SQLite and P selects PostgreSQL. Then, the T key runs a common test on the selected engine. This example clearly shows the intent of the block: the test flow is practically the same even if the engine changes.

```
case 'M' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_MYSQL);      break;
case 'L' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_SQLITE);     break;
case 'P' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_POSTGRES_SQL); break;
case 'T' : Database_DoTest();                                     break;
```

4. SQL Block Organization

The SQL block is divided into a common layer and multiple implementation layers. The common layer defines the vocabulary: database, connection, query, result, row, variant and error. Concrete implementations inherit from those classes and adapt each operation to the native SQLite, MySQL/MariaDB or PostgreSQL client.

Group	Main classes	Responsibility
Common layer	DB_SQL_DATABASE, DB_SQL_CONNECTION, DB_SQL_QUERY, DB_SQL_RESULT, DB_SQL_VARIANT, DB_SQL_ROW, DB_SQL_ERROR, DB_SQL_FACTORY	Defines the common working interface.
SQLite	SQLITE_DATABASE, SQLITE_CONNECTION, SQLITE_QUERY, SQLITE_RESULT	Adapts the common interface to a SQLite database in file.
MySQL/MariaDB	MYSQL_DATABASE, MYSQL_CONNECTION, MYSQL_QUERY, MYSQL_RESULT, MYSQL_VARIANT	Adapts the common interface to the compatible MySQL client.
PostgreSQL	POSTGRES_SQL_DATABASE, POSTGRES_SQL_CONNECTION, POSTGRES_SQL_QUERY, POSTGRES_SQL_RESULT	Adapts the common interface to the PostgreSQL client.

5. DB_SQL_DATABASE: database abstraction

DB_SQL_DATABASE is the base class that represents a SQL database from the point of view of the application. It is not a connection or a specific query, but the object that groups the engine, the opening state, the associated connection, the table list, the error stack, the encoding and the common execution logic.

Among their responsibilities are to open and close the database, create connections, create queries, run already built queries, manage transactions, do commit or rollback, create and release savepoints, list tables, create or delete tables in an auxiliary way and consult the number of records in a table.

The class also contains an important idea: the database can be thread safe if you have an internal mutex. The `IsThreadSafe` method allows you to consult that condition, and `Lock/Unlock` allows you to protect operations when implementation or application use requires it.

```
if(database->Open())
{
    if(database->IsThreadSafe())
    {
        database->Lock();
    }

    // Operations on the database.

    if(database->IsThreadSafe())
    {
        database->Unlock();
    }

    database->Close();
}
```

`DB_SQL_DATABASE` should not be seen as a minor auxiliary class. It is the main contract of the Databases block. `MYSQL_DATABASE`, `SQLITE_DATABASE` and `POSTGRESQL_DATABASE` classes specialize that contract for each specific engine.

6. DB_SQL_CONNECTION: connection options

`DB_SQL_CONNECTION` represents the data needed to connect to a database. In client/server engines, the usual options are URL, PORT, DATABASE, USER, PASSWORD and TIMEOUT. In SQLite, in addition, PATH appears because the database is normally in a file within the databases section of assets.

The connection is not configured by a different rigid structure for each engine, but by key/value pairs. This design allows sharing a common way to configure different connectors, maintaining sufficient flexibility for specific options.

```
DB_SQL_CONNECTION* connection = database->CreateConnection();
if(!connection) return false;

connection->SetOption(__L("URL")      , __L("127.0.0.1"));
connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("USER")     , __L("sirgraham"));
connection->SetOption(__L("PASSWORD"), __L("sirgraham"));
connection->SetOption(__L("TIMEOUT") , __L("30"));
```

In the case of SQLite, the example obtains the database directory through `XPATHMANAGER` and passes it as option `PATH`. This integration is consistent with the rest of the book: the application resources live under assets and are accessed by route sections, not by absolute paths scattered by the code.

```
XPATH path;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_DATABASES, path);
connection->SetOption(__L("PATH"), path.Get());
```

7. Database configuration in the application

The Databases example maintains the main configuration in Databases_CFG.cpp and Databases_CFG.h. As we saw in the first application chapter, the GEN Framework configuration should not be distributed by the program. It is centralized in a class derived from APPFLOWCFG, mapped against INI file entries and then accessed by specific getters.

The database section of the INI contains the fields url, port, databasename, user, password and timeoutconnection. This allows you to change the server or database without recompiling. In SQLite, the name of the database can be combined with the path of assets/databases. In MySQL/MariaDB and PostgreSQL, those same values allow you to open a server connection.

```
[database]
url=127.0.0.1
port=0
databasename=test
user=sirgraham
password=sirgraham
timeoutconnection=30
```

The variable mapping keeps the file updated and avoids obsolete keys. The actual variables remain inside DATABASES_CFG and the rest of the application does not need to directly manipulate the INI file.

```
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_URL, &db_URL);
AddValue(XFILECFG_VALUETYPE_INT, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_PORT, &db_port);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_DATABASENAME, &db_databasename);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_USER, &db_user);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_PASSWORD, &db_password);
AddValue(XFILECFG_VALUETYPE_INT, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_TIMEOUTCONNECTION, &db_timeoutconnection);
```

8. Integration with assets and XPATHSMANAGER

The Databases block uses the XPATHSMANAGERSECTIONTYPE_DATABASES section to keep the database files sorted. In an application with GEN Framework, the executable should not be the place where resources, configurations or data files are placed. Those resources must live within assets.

The example activates the database section and creates the required folders on disk. This pattern allows a local application with SQLite to have its database file in a homogeneous location, and for a client/server application to keep its configurations, logs and auxiliary resources under assets.

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_DATABASES, APPFLOW_DEFAULT_DIRECTORY_DATABASES);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

9. DB_SQL_QUERY: construction, binding and execution

DB_SQL_QUERY represents a SQL query. The class allows you to assign the query text via Set, link parameters via Bind, run the query and get the associated result. This separation is important because a query should not be built by concatenating user values within a string if it can be avoided. binding parameters helps to separate the SQL text from the data.

GEN Framework supports binding integers, XCHAR chains, char chains, floating and XDATETIME dates. Each specific implementation is responsible for adapting those values to the engine parameter mechanism. SQLite and MySQL use the "?" position marker. PostgreSQL uses parameters numbered as \$1, \$2, \$3 and \$4.

```
DB_SQL_QUERY* query = database->CreateQuery();
if(!query) return false;

query->Set(__L("INSERT INTO test (id,name,value,date) VALUES (?, ?, ?, ?);"));

query->Bind(0, 1);
query->Bind(1, __L("Sir Graham"));
query->Bind(2, 11.5f);
query->Bind(3, (*datetime));

bool status = database->Execute(query);

query->UnbindAll();
delete query;
```

The difference of placeholders forces you to have some prudence when writing common SQL. The example Databases shows this explicit adaptation: for MySQL and SQLite a sentence with ? is generated, while for PostgreSQL a sentence with \$1, \$2, \$3 and \$4 is generated. The rest of the flow remains common.

```
switch(database->GetType())
{
    case DB_SQL_DATABASE_TYPE_MYSQL :
    case DB_SQL_DATABASE_TYPE_SQLITE : querystring.Format(__L("INSERT INTO %s (id,name,value,date)
VALUES (?, ?, ?, ?);"), DATABASES_TABLE_TEST_NAME);
        break;

    case DB_SQL_DATABASE_TYPE_POSTGRESQL : querystring.Format(__L("INSERT INTO %s (id,name,value,date)
VALUES ($1,$2,$3,$4);"), DATABASES_TABLE_TEST_NAME);
        break;
}
```

10. Transactions, commit, rollback and savepoints

Access to a database should not be understood only as executing isolated sentences. In real applications, it is often necessary for several operations to behave as a logical unit. If all are executed correctly, the transaction is confirmed. If a failure, changes are undone.

DB_SQL_DATABASE defines Transaction, Commit, Rollback, Savepoint and Release Savepoint. Each engine implements these methods according to its own API and syntax. The application code can work with the common idea of transaction and let the connector specify the detail.

```
database->Transaction();

bool status = database->Execute(queryinsert1);
status      &= database->Execute(queryinsert2);

if(status)
    database->Commit();
else database->Rollback();
```

savepoints allows you to create intermediate points within a transaction. Not all applications need them, but they are useful when a large operation can be divided into blocks and you want to back up only part of the job.

11. Creation and management of tables

Although Databases allows you to run SQL directly, DB_SQL_DATABASE includes auxiliary methods for standard table operations: Table_IsThere, Table_Create, Table_Delete and Table_GetNRecords. These methods do not replace the full design of a database scheme, but simplify examples, tools and simple cases.

The Databases example deletes the test table, checks if it exists and creates it again with a set of fields adapted to the engine. This shows an important point: the common interface does not prevent differences in SQL types. SQLite, MySQL/MariaDB and PostgreSQL do not define exactly the same fields, keys and dates.

```
XCHAR* fields[] = { __L("id int PRIMARY KEY"),
                    __L("name varchar(50)"),
                    __L("value float"),
                    __L("date DATETIME") };

bool isexist = false;

database->Table_IsThere(__L("test"), __L("id"), isexist);
if(!isexist)
{
    database->Table_Create(__L("test"), fields, 4);
}
```

For real applications it is best to be more careful: to version the schema, to control migrations, to separate the initial creation of tables from the normal logic of execution and to record errors clearly. The Databases block provides the base, but the data design remains the responsibility of the application.

12. DB_SQL_RESULT, DB_SQL_ROW and DB_SQL_ROWHEADER

When a query returns data, the result is represented by DB_SQL_RESULT. This class provides a common interface to cross rows, query the number of columns, move to the first or last record, move forward with Next, abort the path, and process the current row.

Each row is represented by DB_SQL_ROW, which contains a collection of DB_SQL_VARIANT. The header is represented as DB_SQL_ROWHEADER, a collection of column names. Thus, the code can run columns without depending on the native types of each database client.

```
DB_SQL_RESULT* result = query->GetResult();
if(result)
{
    result->ProcessRow();

    while(result->HasNext())
    {
        DB_SQL_ROW* row = result->GetRow();
        DB_SQL_ROWHEADER* header = result->GetHeader();

        for(XDWORD c=0; c<result->GetNumColumns(); c++)
        {
            console->Printf(__L("%s = "), header->Get(c)->Get());
            // Interpret row->Get(c) according to its type.
        }

        result->Next();
    }
}
```

The travel pattern is deliberately similar between engines. The concrete result is implemented by `MYSQL_RESULT`, `SQLITE_RESULT` or `POSTGRESQL_RESULT`, but the application works with `DB_SQL_RESULT`.

13. DB_SQL_VARIANT: SQL data types in a common form

`DB_SQL_VARIANT` derives from `XVARIANT` and adapts the generic value idea to the database context. A field read from SQL can be null, integer, double integer, floating, string, date, time or date-time. The variant allows transporting that value without the application code depending on a native type of MySQL, SQLite or PostgreSQL.

In addition to the type, `DB_SQL_VARIANT` maintains flags associated with the field: not null, primary key, unique, unsigned, zerofill, binary, auto increment, blob, timestamp and other useful indicators to describe column metadata.

```
DB_SQL_VARIANT value = row->Get(c);

switch(value.GetType())
{
    case DB_SQL_VARIANT_TYPE_STRING : console->Printf(__L("%s"), (XCHAR*)value); break;
    case DB_SQL_VARIANT_TYPE_INTEGER : console->Printf(__L("%d"), (int)value); break;
    case DB_SQL_VARIANT_TYPE_FLOAT : console->Printf(__L("%f"), (float)value); break;

    case DB_SQL_VARIANT_TYPE_DATETIME : { DB_SQL_DATETIME datetime = value;
        XSTRING string;
        datetime.GetDateTimeToString(XDATETIME_FORMAT_STANDARD, string);
        console->Printf(__L("%s"), string.Get());
    }
    break;
}
```

`MYSQL_VARIANT` specializes in some behaviors for MySQL, especially in the conversion of dates and data sizes. In other engines, the result is adapted within the corresponding query and result classes.

14. DB_SQL_ERROR and error management

`DB_SQL_ERROR` represents an error produced during database operations. The `DB_SQL_DATABASE` base class maintains a list of errors and offers `GetLastError`, `Error` and `ClearPreviousErrors`. This architecture allows operations to fail by returning false, but there is also a description of what happened.

The types of error include memory error, connection error, judgment error and transaction error. In a real application it is important not to limit itself to knowing that an operation failed. It is necessary to record the context: which query was executed, which engine was being used, if the base was open, which connection had been configured and what was the last description of error.

```
if(!database->Execute(query))
{
    XCHAR* error = database->GetLastError();
    if(error)
    {
        APPFLOW_LOG_ENTRY(XLOGLEVEL_ERROR, APPFLOW_CFG_LOG_SECTIONID_GENERIC, false, __L("Database error: %s"), error);
    }
}
```

15. SQLite: local base in file

SQLite is the easiest connector to deploy because it does not require an external server. The database is stored in a file, usually within assets/databases. This makes it a very convenient option for examples, local tools, tests, embedded applications or programs that need persistence without installing a service.

The `SQLITE_DATABASE`, `SQLITE_CONNECTION`, `SQLITE_QUERY` and `SQLITE_RESULT` classes implement the common interface. The connection uses `PATH` to locate the directory and `DATABASE` to indicate the name of the base. The query adapts the binding to the API of SQLite and the result allows you to cross rows using common methods.

```
database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);

DB_SQL_CONNECTION* connection = database->CreateConnection();
connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("PATH"), path.Get());

database->Open();
```

SQLite is especially interesting in GEN Framework because it fits very well with the idea of assets. The application can carry its local base as a data resource, create it if it does not exist and keep it in a homogeneous location on all supported platforms.

16. MySQL and MariaDB: client/server connection

The MySQL connector allows you to work with a MySQL server or compatible server, such as MariaDB when using a supported client library and protocol. Unlike SQLite, here the database is not in a local file managed by the application, but on a server accessed by URL, port, user, password and base name.

`MYSQL_DATABASE` defines the type, name, transactions, query creation, connection creation and encoding. `MYSQL_CONNECTION` opens and closes the real connection. `MYSQL_QUERY` prepares and executes sentences, links parameters and builds results. `MYSQL_RESULT` allows you to run rows and columns with the same interface as all other engines.

```
database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_MYSQL);

DB_SQL_CONNECTION* connection = database->CreateConnection();
connection->SetOption(__L("URL"), APPFLOW_CFG.Database_GetURL()->Get());
connection->SetOption(__L("PORT"), __L("3306"));
connection->SetOption(__L("DATABASE"), APPFLOW_CFG.Database_DatabaseName()->Get());
connection->SetOption(__L("USER"), APPFLOW_CFG.Database_GetUser()->Get());
connection->SetOption(__L("PASSWORD"), APPFLOW_CFG.Database_GetPassword()->Get());

database->Open();
```

In a professional environment, the use of MySQL/MariaDB requires taking care of network configuration, credentials, user permissions, encoding, reconnection and error treatment. GEN Framework provides the access layer; server security and management remain the responsibility for deployment.

17. PostgreSQL: Advanced Relational Connector

PostgreSQL is integrated using `POSTGRESQL_DATABASE`, `POSTGRESQL_CONNECTION`, `POSTGRESQL_QUERY` and `POSTGRESQL_RESULT`. Conceptually it is used just like the rest: a database is created using `DB_SQL_FACTORY`, a connection is set up, queries are opened, created and results are traversed.

A visible difference in the example is how to parameterize queries. PostgreSQL uses parameters numbered as \$1, \$2, \$3, while SQLite and MySQL use the '?' marker. This detail is located at the point where the query is built, not spread across the application.

```
query->Set(__L("INSERT INTO test (id,name,value,date) VALUES ($1,$2,$3,$4);"));

query->Bind(0, 1);
query->Bind(1, __L("Sir Graham"));
query->Bind(2, 11.5f);
query->Bind(3, (*datetime));

database->Execute(query);
```

PostgreSQL is usually a good option when the application needs a solid relational engine, robust transactions, advanced data types, server functions or deployments with higher consistency requirements. The GEN Framework layer allows to maintain a code style similar to that of other engines.

18. Complete example of basic flow

The following skeleton summarizes the usage pattern that appears in the Databases example: creating the engine, creating the connection, setting options, opening, creating query, running, going through results and closing. It is not intended to be a complete application, but a reading guide to understand the pieces.

```
DB_SQL_DATABASE* database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);
if(!database) return false;

DB_SQL_CONNECTION* connection = database->CreateConnection();
if(!connection)
{
    delete database;
    return false;
}

connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("PATH"), path.Get());

if(database->Open())
{
    DB_SQL_QUERY* query = database->CreateQuery();
    if(query)
    {
        query->Set(__L("SELECT id,name,value,date FROM test;"));

        if(database->Execute(query))
        {
            DB_SQL_RESULT* result = query->GetResult();
            if(result)
            {
                result->ProcessRow();
                while(result->HasNext())
                {
                    DB_SQL_ROW* row = result->GetRow();
                    // Read row values.
                    result->Next();
                }
            }
        }
    }
}
```

```

    }

    delete query;
}

database->Close();
}

delete connection;
database->SetConnection(NULL);
delete database;

```

Cleaning is as important as opening. A query created with `CreateQuery` must be destroyed when it is no longer used. The connection must be released and the base must be closed before it is destroyed. This discipline follows the general pattern of GEN Framework: Ini/End, opening/closing, creation/destruction and pointers to NULL when applicable.

19. Activation by CMake

The Databases block is activated using CMake options. The Databases example activates the general SQL support and, in addition, the specific connectors for SQLite, MySQL and PostgreSQL. In a real application, only the necessary connectors can be activated.

```

option(DATABASES_SQL_FEATURE          "DataBases SQL"          ON)
option(DATABASES_SQL_SQLITE_FEATURE    "DataBases SQLite"      ON)
option(DATABASES_SQL_MYSQL_FEATURE     "DataBases MySQL"       ON)
option(DATABASES_SQL_POSTGRESQL_FEATURE "DataBases PostgreSQL" ON)

```

Activating a connector means that the corresponding libraries and headers must be available for the build platform. SQLite is usually easier because it can be integrated more locally. MySQL/MariaDB and PostgreSQL may require specific client libraries, including paths and correctly linked libraries.

The example `CMakeLists.txt` also activates AppFlow, console, log, general configuration and log configuration. This fits the idea of the example: not only try SQL, but do it within an application with full GEN Framework, with configuration, traces, status and resources.

20. Relationship with AppFlow, DIO, XUtilities and Cipher

Databases do not live in isolation. It is supported by XUtilities for strings, routes, dates, variants, vectors, maps, mutexes and traces. It is supported by AppFlow for configuration, logs, application cycle and asset paths. It can be related to DIO when the application obtains network data before persisting, or when a database operation is part of a communication service.

Cipher can also intervene if the application needs to protect credentials, encrypt sensitive fields before storing them or calculating integrity hashes. It is important to distinguish between encrypting communication with the server, which can depend on the engine and its client, and encrypting application data before writing them into tables. GEN Framework provides blocks for both needs, but the architecture decision must be taken on a case-by-case basis.

21. Summary catalogue of the Databases block

Class/group	What does it bring?	Relationship
DB_SQL_Base	Defines base types, aliases and	It is included in the rest of the SQL block.

	enumeration DB_SQL_DATABASE_TYPE.	
DB_SQL_FACTORY	Creates the concrete base according to DB_SQL_DATABASE_TYPE.	Entry point to select SQLite, MySQL/MariaDB or PostgreSQL.
DB_SQL_DATABASE	Main SQL database contract.	Open, Close, Execute, transactions, tables, errors and mutex.
DB_SQL_CONNECTION	Key/value connection options.	URL, PORT, DATABASE, PATH, USER, PASSWORD, TIMEOUT.
DB_SQL_QUERY	See SQL and binding for parameters.	Set, Bind, Execute, GetResult and UnbindAll.
DB_SQL_RESULT	Result of a consultation.	Route by rows, columns and cursor.
DB_SQL_VARIANT	Generic SQL value based on XVARIANT.	Field types and metadata flags.
DB_SQL_ROW / DB_SQL_ROWHEADER	Row and column names.	Homogenous reading of results.
DB_SQL_ERROR	Error representation SQL.	Connection, statement, memory and transaction.
SQLITE_*	SQLite implementation.	Local base in file.
MYSQL_*	MySQL/MariaDB implementation compatible.	SQL client/server.
POSTGRES_*	PostgreSQL implementation.	SQL server with numbered parameters.

22. Criteria for use in an application with GEN Framework

To choose an engine you should first think about the deployment type. If the application needs a local database without a server, SQLite is the natural option. If you need to share data between multiple clients or integrate with existing infrastructure, MySQL/MariaDB or PostgreSQL make more sense. If the goal is to develop and test quickly, SQLite can serve even when the final deployment is a server, provided that the application does not abuse specific syntax.

It is also appropriate to separate the business logic from the specific SQL. The code that decides what an application operation means should not be mixed with placeholders details, table creation or variant conversion. A good application with GEN Framework can encapsulate your access to data in your own classes and use Databases as a connection and execution layer.

We must avoid building SQL by concatenating external data without control. Although the example uses strings to build some structure queries, data values must be linked to Bind whenever possible. This improves clarity, reduces conversion errors and avoids many typical problems of manually constructed SQL.

23. Usual errors when using Databases

- Enable a connector in CMake without having installed or available the required client library.
- Forget to configure DATABASE, PATH, URL, PORT, USER, PASSWORD or TIMEOUT before calling Open.
- Use absolute routes for SQLite instead of the XPATHMANAGER database section.
- Write the same SQL statement for all engines when the syntax of parameters or types is not identical.
- Do not call UnbindAll after reusing a query with parameters.
- Do not close the database before destroying the connection or DB_SQL_DATABASE object.
- Do not consult GetLastError or the error stack when Execute returns false.
- Confused local persistence with server: SQLite and PostgreSQL/MySQL solve different problems even if they share SQL interface.

24. Practical summary

Databases provides GEN Framework with a common layer to work with relational databases. Its design revolves around a simple idea: the application must be able to talk about databases, connections, queries, results, rows and variants without being tied directly to the API native to a particular engine.

DB_SQL_FACTORY creates the engine. DB_SQL_DATABASE defines the general contract. DB_SQL_CONNECTION sets up the connection. DB_SQL_QUERY contains the sentence and its parameters. DB_SQL_RESULT allows data to be traversed. DB_SQL_VARIANT adapts the SQL types to the world of GEN Framework. Implementation SQLITE, MYSQL and POSTGRESQL specify that contract for each engine.

The block is oriented to SQL and relational databases. Its architecture leaves open the incorporation of documental connectors or other models, but its current value is already clear: allowing an application with GEN Framework to access SQLite, MySQL/MariaDB or PostgreSQL through the same architecture, integrated with assets, XPATHMANAGER, AppFlow, XUtils and the general style of the framework.

Chapter XIII: Scripts, Dynamic Algorithms and Real-Time Plugins

1. The role of Scripts within GEN Framework

The Scripts block allows an application developed with GEN Framework to run external algorithms written in script languages. Its aim is not to replace the C/C++ code of the application, but to complement it where it is interested in gaining flexibility: auxiliary tasks, modifiable rules, automations, testing, maintenance actions, small business algorithms or extensions loaded at runtime.

In previous chapters we have seen that GEN Framework tries to separate the application logic from the platform dependencies. Scripts adds an additional separation: it allows separating certain behavioral decisions from the compiled binary. When a part of the algorithm is written into a script file, it can be modified, copied, replaced or expanded without recompiling the application. This is especially useful in projects where certain rules change frequently or where the advanced user is given a controlled way to extend the behavior.

The idea must be understood in practical terms. An industrial application may need to vary a calculation formula. A test tool may require new sequences of actions. A diagnostic application can load small verification procedures. A graphical utility can automate repetitive operations. In all these cases, recompiling the entire application for each small change can be uncomfortable. The Scripts block allows placing that variable part in assets/scripts and running it from GEN Framework using a common interface.

The module is designed to work with three languages: JavaScript, Lua and .GEN, represented by files with .g extension. The latter can be understood as a language of its own, a simplified variant of the C language to run external logic within the GEN Framework environment. The application does not have to treat each language completely independently. The SCRIPT base class provides a common layer and each specific language implements its way of loading, registering functions and running the code.

2. From compiled algorithms to loadable algorithms

In C/C++, the natural way to add functionality is to write code, compile and link. That strategy is excellent for the kernel of an application: it offers performance, control, strong types and full access to the framework. However, it is not always the best answer for everything. There are parts of an application that are more changing than critical: maintenance scripts, validation rules, test sequences, integration with external tools, parametric calculations or interface automations.

When those parts live within the executable, each modification forces to recompile, test and redistribute. When they live in scripts, the application can load the running file and apply the new logic directly. This converts the Scripts block into a kind of lightweight plugin system. Binary libraries are not loaded or the main executable modified: text is loaded, interpreted or evaluated using the corresponding engine and the script is exposed to the functions that the application decides to allow.

This flexibility should not be confused with lack of control. The application continues to decide which scripts are loaded, from which directory, with which libraries, with which functions available and at which time they are executed. The script should not have indiscriminate access to the entire framework; only to what is exposed through script libraries. That frontier is important because it allows the power level to be adapted to each project.

3. Three supported languages

The Scripts block recognizes the script type by its extension. The SCRIPT class includes SCRIPT_TYPE and the GetTypeByExtension method, which associates the .g, .lua and .js files with their specific implementations when the corresponding build options are active.

Extension	Internal type	Main class	Typical use
.g	SCRIPT_TYPE_G	SCRIPT_LNG_G	GEN Framework language, close to C language, useful for simple and controlled scripts.
.lua	SCRIPT_TYPE_LUA	SCRIPT_LNG_LUA	Integration with Lua for lightweight, embedded and easy to modify scripts.
.js	SCRIPT_TYPE_JAVASCRIPT	SCRIPT_LNG_JAVASCRIPT	JavaScript integration using Duktape for dynamic algorithms and automations.

The existence of three languages does not mean that an application should use them all. Each project can activate only those that you need through CMake. A small application can only work with Lua. A tool that already has user scripts in JavaScript can activate JavaScript. A highly controlled environment can only use .g. The selection is made in compilation and allows for adjustment of dependencies, size and functionality.

4. The SCRIPT class as a common point

SCRIPT is the base class of the system. It represents a loaded script, its type, its path, its name, its text, its available libraries, its exported functions, its timer, its possible execution thread and its error code. By inheriting from XSUBJECT you can also post events, which allows you to alert the application when an error or interruption occurs during execution.

Among his most important methods are Create, Load, LoadAdd, Save, Run, RunWithThread, IsRunThread, CancelExecution, AddLibrary, AddLibraryFunction and LoadScriptAndRun. Create builds the proper implementation according to the type or extension. Load reads a script file. LoadAdd allows you to concatenate multiple files in the same instance. Run executes the loaded content. RunWithThread allows you to launch the execution into a thread. LoadScriptAndRun is especially convenient when the application has a list of scripts configured and wants to load them and run them sequentially.

```
SCRIPT* script = SCRIPT::Create(__L("test.lua"));
if(script)
{
    XPATH xpath;

    GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
    xpath.Slash_Add();
    xpath += __L("test.lua");

    if(script->Load(xpath))
    {
        int returnvalue = 0;
        int error       = script->Run(&returnvalue);
    }

    GEN_DELETE script;
}
```

The above example shows manual use. In a real app based on AppFlow it is usually more convenient to use the list of configuration scripts and delegate to LoadScriptAndRun. That function creates each script, adjusts libraries using an optional callback, loads the content from assets/scripts, uses the cache if it is active and executes the script.

```
SCRIPT::LoadScriptAndRun(APPFLOW_CFG.Scripts_GetAll(), SCRIPTSEXAMPLE::AdjustLibraries);
```

5. Load, routes and assets/scripts

As with other GEN Framework blocks, scripts should not be located using absolute paths scattered by the code. The application must use XPATHSMANAGER and the XPATHSMANAGERSECTIONTYPE_SCRIPTS section. In the ScriptsExample example, during initialization the root of assets is set and the script section is added so that the framework knows where to search for external files.

```
GEN_XPATHSMANAGER.AdjustRootPathDefault (APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection (XPATHSMANAGERSECTIONTYPE_SCRIPTS ,
APPFLOW_DEFAULT_DIRECTORY_SCRIPTS);
GEN_XPATHSMANAGER.AddPathSection (XPATHSMANAGERSECTIONTYPE_GRAPHICS,
APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

The typical structure is as follows:

```
assets/
├─ scriptsexample.ini
├─ scriptsexample.log
├─ scripts/
│   ├─ test.g
│   ├─ test.js
│   ├─ test.lua
│   ├─ test1.lua
│   ├─ test2.lua
│   └─ inputsimul.js
└─ graphics/
    └─ inputsimul.png
```

This organization allows the executable to remain separate from external algorithms. The program is compiled once, but scripts can be changed within assets/scripts. If the project needs to organize scripts by families, it can also create subdirectories or additional sections, but the starting point must always be the same: use XPATHSMANAGER to maintain a homogeneous way of locating resources.

6. Configuration of the script list

The ScriptsExample example shows a direct integration between Scripts and AppFlowCFG. The main configuration contains a scriptslist section where the scripts that the application must load are declared. The application does not need to have the names written rigidly in the code; it gets them through APPFLOW_CFG.Scripts_GetAll().

```
[scriptslist]
scripts001=test1.lua
scripts002=test2.lua
scripts003=
```

This mechanism fits the general philosophy of the GEN Framework configuration file. The application can change the list of scripts by modifying the INI. If SCRIPT_CACHE is enabled, those scripts can be preloaded in memory during AppProc_FirstUpdate to speed up subsequent runs.

```
#ifdef SCRIPT_CACHE_ACTIVE

GEN_SCRIPT_CACHE.Cache_DelAll();
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
GEN_SCRIPT_CACHE.Cache_AllDirectory(xpath);
GEN_SCRIPT_CACHE.Cache_AllList (APPFLOW_CFG.Scripts_GetAll());

#endif
```

7. SCRIPT_CACHE: script cache

SCRIPT_CACHE is a singleton that keeps script texts in memory. Its utility appears when an application repeatedly executes the same files or when it wants to preload all the scripts in a directory. Instead of reading the file every time from disk, an identifier is generated and associated content is stored.

The class provides GenerateID, Cache_Add, Cache_Get, Cache_Set, Cache_Del, Cache_DelAll, Cache_AllDirectory and Cache_AllList. The identifier is calculated from a standardized string or path. Load and LoadScriptAndRun can check the cache if SCRIPT_CACHE_ACTIVE is enabled.

```
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
GEN_SCRIPT_CACHE.Cache_AllDirectory(xpath);

XDWORD ID = GEN_SCRIPT_CACHE.GenerateID(xpath);
XSTRING* scripttext = GEN_SCRIPT_CACHE.Cache_Get(ID);
```

The cache does not change the semantics of the script. It only avoids repeating accesses to the file system. In testing tools, automation or frequent execution of light plugins can be an appreciable improvement. In development, it should be remembered that an active cache can make changes in the file not seen until it is reloaded or cleaned.

8. Events and Script Errors

Scripts may fail for many reasons: non-existent file, unsupported type, syntax error, insufficient parameters, internal engine problem or failure in a function exposed from C++. GEN Framework represents this information using SCRIPT_ERRORCODE and SCRIPT_XEVENT.

SCRIPT_ERRORCODE defines common errors as SCRIPT_ERRORCODE_NONE, SCRIPT_ERRORCODE_INTERNALERROR and SCRIPT_ERRORCODE_INSUF_PARAMS, leaving SCRIPT_ERRORCODE_OWN as an extension point for errors specific to each language or implementation. SCRIPT_XEVENT allows transporting information to observers: affected script, name, error code, error text, line and current token.

In ScriptsExample, the event handler translates those events to traces. When SCRIPT_XEVENT_TYPE_ERROR arrives, the error code, text, line and current token are displayed. When SCRIPT_XEVENT_TYPE_BREAK arrives, the line and token where the interruption occurred are reported.

```
void SCRIPTSEXAMPLE::HandleEvent_Script(SCRIPT_XEVENT* event)
{
    switch(event->GetEventType())
    {
        case SCRIPT_XEVENT_TYPE_ERROR : XTRACE_PRINTCOLOR(4,
            __L("Script ERROR [%d]: %s line %d -> \"%s\""),
            event->GetError(),
            event->GetErrorText()->Get(),
            event->GetNLine(),
            event->GetCurrentToken()->Get());
            break;
    }
}
```

9. Function libraries: SCRIPT_LIB and SCRIPT_LIB_FUNCTION

The real power of the Scripts block is not just running JavaScript, Lua or .g. It is possible to expose functions of GEN Framework or a specific application to those scripts. SCRIPT_LIB and SCRIPT_LIB_FUNCTION are used for this purpose.

SCRIPT_LIB represents a library of functions available for a script. Each library has an identifier and an AddLibraryFunctions method. SCRIPT_LIB_FUNCTION represents a specific function: it keeps the pointer to the library, the name visible from the script language, and the pointer to the C/C++ function that is called when the script invokes that name.

The common signature of a library function is SCRFUNCTIONLIBRARY. It receives the library, the script that makes the call, a list of parameters in XVECTOR<XVARIANT*> and a XVARIANT return. This allows you to convert the values of JavaScript, Lua or .g to the world of GEN Framework using a common representation.

```
typedef void (*SCRFUNCTIONLIBRARY) (SCRIPT_LIB* library,
                                    SCRIPT* script,
                                    XVECTOR<XVARIANT*>* params,
                                    XVARIANT* returnvalue);
```

A library of its own always follows the same pattern: it inherits SCRIPT_LIB, registers its functions in AddLibraryFunctions and defines Call_... functions that convert parameters, execute C/C++ code and write the result.

```
bool SCRIPT_LIB_MIAPP::AddLibraryFunctions(SCRIPT* script)
{
    if(!script) return false;

    script->AddLibraryFunction(this, __L("MiFuncion"), Call_MiFuncion);

    return true;
}

void Call_MiFuncion(SCRIPT_LIB* library, SCRIPT* script,
                   XVECTOR<XVARIANT*>* params, XVARIANT* returnvalue)
{
    if(!returnvalue) return;

    (*returnvalue) = true;
}
```

With this idea, a project can create its own bridge between scripts and C/C++. For example, an industrial application can display ReadingSensor, WriteOut, RunSequence, or CalculateConsign. A test tool can expose ConnectTeam, SendCommand, or CheckAnswer. The script is clean and actual access to hardware or framework remains in C++.

10. Standard internal libraries

When building a SCRIPT instance, the system adds internal libraries. Some are standard and others depend on compilation options. The standards provide basic functions for math, random numbers, timing, strings, routes, directories and traces.

Library	Class	Main functions	Use
Math	SCRIPT_LIB_MATH	Abs	Basic mathematical operations available from scripts.
Random	SCRIPT_LIB_RAND	RandMax, RandBetween	Generation of random numbers for testing, simulation or

			external logic.
Timer	SCRIPT_LIB_TIMER	Sleep	Controlled pauses during automations or test sequences.
String	SCRIPT_LIB_STRING	AddString, FindString, CompareString, ReplaceString, SPrintf	Text manipulation without repeating logic in each language.
Path	SCRIPT_LIB_PATH	GetPathScript, GetNameScript	Get the path and name of the script running.
Dir	SCRIPT_LIB_DIR	IsItExists, ChangeDir, RemoveDir, MakeDir	Basic operations on directories.
Trace	SCRIPT_LIB_TRACE	TraceClearScreen, TraceClearMsgsStatus, TracePrintColor	Integrated diagnostic output with GEN Framework trace system.

These libraries make a script not an isolated text. It runs within the GEN Framework ecosystem and can use common framework services. In the example scripts it is clearly seen: test.g, test.js and test.lua calculate a maximum, print per console, generate random numbers and write color traces.

```
function main()
{
    Console_Printf("JavaScript version %s\n", "0.1.0");
    Console_Printf("Random numbers from 1 to 100: \n");

    for(i=0; i<10; i++)
    {
        Console_Printf("%d ", RandBetween(1,100));
    }

    TracePrintColor(1, "Proof of shipment %.2f con cadena %s", 10.45, "hola radiola");
}
```

11. Optional libraries and integration with GEN Framework

In addition to standard libraries, the Scripts block has optional libraries that are enabled by CMake settings or options. These libraries allow scripts to access console, configuration, logs, system, processes, windows, input simulation or specific development functions.

Library	Class	Exposed functions	Relationship with GEN Framework
Console	SCRIPT_LIB_CONSOLE	Console_GetChar, Console_PutChar, Console_Printf	Integrates scripts with the APPFLOWCONSOLE console.
CFG	SCRIPT_LIB_CFG	GetFileCFGValue	It allows you to read configuration file values using XFILECFG.
Log	SCRIPT_LIB_LOG	Log_Ini, Log_CFG_SetLimit, Log_CFG_SetFilters, Log_CFG_SetBackup, Log_AddEntry	It integrates scripts with the XLOG/APPFLOW_LOG system.
System	SCRIPT_LIB_SYSTEM	System_GetType, System_Reboot, System_PowerOff, System_Logout, System_GetEnvironmentVar	Access information or actions of the system through XUtilities.
Process	SCRIPT_LIB_PROCESS	OpenURL, ExecApplication, MakeCommand, TerminateApplication, TerminateApplicationWithWindows	It allows you to open URLs, launch and close processes.
Window	SCRIPT_LIB_WINDOW	Window_SetPosition, Window_Resize,	Integrates scripts with a graphical application and search

		Window_SetFocus, Window_GetPosX, Window_GetPosY, Window_SetBmpFindCFG	for on-screen items.
InpSimulate	SCRIPT_LIB_INPUTSIMULATE	InpSim_Key_Press, InpSim_Key_ClickByText, InpSim_Mouse_SetPos, InpSim_Mouse_Click	It allows simulating keyboard and mouse from scripts.
Developed Test	SCRIPT_LIB_DEVTEST	DevTest_Func1	Test library to validate parameter step and object return.

The example `inputsimul.js` demonstrates a very powerful combination: opening an external application, moving your window, searching for a position using a bitmap mask, simulating mouse clicks, typing text and finishing the application. That kind of test would be much more rigid if it were compiled inside the executable. As a script, it can be quickly adjusted.

12. SCRIPT_LNG_JAVASCRIPT

SCRIPT_LNG_JAVASCRIPT implements JavaScript execution. Internally it uses a Duktape context, records C/C++ functions as global engine functions and converts parameters to X VARIANT. The script defines a main function, which acts as a conventional input point for the file.

When a function is recorded using `AddLibraryFunction`, the class pushes a function of the C language in the Duktape context, associates the actual pointer of the GEN Framework function and publishes it with the visible name for the script. When JavaScript calls that function, `LibraryCallback` retrieves the pointer, extracts the arguments, transforms them into X VARIANT and delegates them to the library function.

This design allows JavaScript to be just the expression layer of the algorithm. The access to console, traces, routes, directories, processes or functionality of the application is still implemented in C/C++. The boundary between script and framework is clear: the script calls by name; the registered library executes native code.

```
SCRIPT* script = SCRIPT::Create(__L("test.js"));
SCRIPT_SET_LIB_CFG(script, (XFILECFG*) &APPFLOW_CFG);
script->Load(xpath);
script->Run();
```

13. SCRIPT_LNG_LUA

SCRIPT_LNG_LUA implements Lua script execution. When created, it initializes a Lua state and opens its standard libraries. As in JavaScript, a callback function is recorded that allows you to link visible names to the script with C/C++ functions of GEN Framework. The script can return a value at the end of the execution.

Lua is especially suitable for embedded environments and for light logic. Its syntax is simple, the engine is small and comfortable for rules that do not justify recompiling the application. In the example `test.lua` a `max` function is defined, console messages are printed, random numbers are generated and a value is returned.

```
function main()

    Console_Printf("Lua version %s\n", _VERSION);
    Console_Printf("Random numbers from 1 to 100: \n");

    for i=0,10,1 do
        Console_Printf("%d ", RandBetween(1,100));
    end
```

```

    return 25

end

return main()

```

14. SCRIPT_LNG_G: the language .GEN

SCRIPT_LNG_G implements GEN Framework's own language for files with .g extension. The source code shows that this implementation is not limited to delegating to an external engine. Define tokens, double operators, internal types, commands, variables, functions, parameters and error codes. It is therefore a specific piece of the framework.

The SCRIPT_LNG_G_COMMAND, SCRIPT_LNG_G_FUNCTIONTYPE and SCRIPT_LNG_G_VAR auxiliary classes represent internal parts of the language. SCRIPT_LNG_G_COMMAND associates command text with tokens. SCRIPT_LNG_G_FUNCTIONTYPE maintains the name, return type and location of a function. SCRIPT_LNG_G_VAR represents variables with name, type, value and marks as argument or return value.

Class	Responsibility
SCRIPT_LNG_G	Language engine .g: tokenization, variables, functions, command call and execution.
SCRIPT_LNG_G_COMMAND	Relationship between language commands and internal tokens.
SCRIPT_LNG_G_FUNCTIONTYPE	Description of functions defined in the script: name, return and location.
SCRIPT_LNG_G_VAR	Variable language .g with name, type, value and execution attributes.

The example test.g shows a syntax close to the C language. Defines a max function, declares integer variables, uses if, for, return, calls to Console_Printf, RandBetween, TracePrintColor and Console_GetChar. For a C/C++ programmer, this form is familiar and makes it easy to write simple logic without jumping mentally into a very different language.

```

int max(int num1, int num2)
{
    int result;

    if(num1 > num2)
    {
        result = num1;
    }
    else
    {
        result = num2;
    }

    return result;
}

```

15. Integration into ScriptsExample

ScriptsExample is a console application based on APPFLOWCONSOLE and XFSMACHINE. Its aim is to demonstrate how to integrate the Scripts block into a real framework application. As in other examples of GEN Framework, no C++ main appears; the application defines a class, an instance with APPLICATIONCREATEINSTANCE, initialization, first update, periodic update and closure.

During AppProc_Ini name, version, thread groups, traces, script and graphics paths, state machine, mutex, automatic trace targets and AppFlow Extended are configured. During AppProc_FirstUpdate the cache is preloaded if it is enabled and timers are created. In AppProc_Update the console is served and the key is reacted to. The R key launches the execution of configured scripts.

```
case 'R' : { if(xtimerscriptrun) xtimerscriptrun->Reset();

                SCRIPT::LoadScriptAndRun(APPFLOW_CFG.Scripts_GetAll(),
                                          SCRIPTSEXAMPLE::AdjustLibraries);

                XQWORD timereleased = xtimerscriptrun->GetMeasureMilliseconds();
                XTRACE_PRINTCOLOR(1, __L("Run exec in %d milliseconds."),
                                (XDWORD)timereleased);
            }
        break;
```

The callback AdaptLibraries allows you to adjust libraries before running the script. In the example it is used to pass the main configuration to SCRIPT_LIB_CFG, so that a function exposed to the script can consult XFILECFG values.

```
void SCRIPTSEXAMPLE::AdjustLibraries(SCRIPT* script)
{
    #ifdef SCRIPT_LIB_CFG_ACTIVE

        SCRIPT_SET_LIB_CFG(script, (XFILECFG*)&APPFLOW_CFG);

    #endif
}
```

16. Activation using CMake

The Scripts block is enabled using CMake options. ScriptsExample displays a wide configuration, with all three languages, cache and several optional libraries. In a real application only the necessary pieces should be activated.

```
option(SCRIPT_FEATURE           "Activate Scripts"           ON)
option(SCRIPT_G_FEATURE         "Activate Script G Language" ON)
option(SCRIPT_LUA_FEATURE       "Activate Script LUA Language" ON)
option(SCRIPT_JAVASCRIPT_FEATURE "Activate Script Javascript" ON)
option(SCRIPT_CACHE_FEATURE     "Activate Script Cache"      ON)

option(SCRIPT_LIB_CONSOLE_FEATURE "Script Lib Console"      ON)
option(SCRIPT_LIB_CFG_FEATURE     "Script Lib Config File"   ON)
option(SCRIPT_LIB_LOG_FEATURE     "Script Lib Log"           ON)
option(SCRIPT_LIB_PROCESS_FEATURE "Script Lib Process"       ON)
option(SCRIPT_LIB_WINDOW_FEATURE  "Script Lib Window"        ON)
option(SCRIPT_LIB_INPUTSIMULATE_FEATURE "Script Lib Input Simulated" ON)
```

This granularity is important. Activating JavaScript involves including your engine. Activating Lua involves linking to Lua. Activating process libraries, window or input simulation increases the power of scripts, but also expands what a script can do. In applications where scripts can be modified by external users, it is appropriate to limit the available libraries to those strictly necessary.

17. Expand the system with its own functions

One of the most interesting ideas of the Scripts block is that it is not closed to the libraries included in the framework. A project can create new specific libraries and expose functions related to your domain. This is the

correct way to convert a script into plugin of an application with GEN Framework: not giving uncontrolled access to the internal code, but by publishing a small, stable and designed API for the script.

The recommended pattern is to create a `SCRIPT_LIB` class... for the project, register its functions in `AddLibraryFunctions` and add the library to the script before executing it. The function receives parameters like `XVARIANT`, validates types, calls the C/C++ part and returns another `XVARIANT`.

```
class SCRIPT_LIB_CONTROL : public SCRIPT_LIB
{
public:
    bool AddLibraryFunctions(SCRIPT* script)
    {
        script->AddLibraryFunction(this, __L("Control_SetOutput"), Call_SetOutput);
        script->AddLibraryFunction(this, __L("Control_GetInput") , Call_GetInput);
        return true;
    }
};
```

From there, a script could write something as direct as `Control_SetOutput(3, true)` or `value = Control_GetInput(1)`. The algorithm remains in the script, but actual device access continues in C/C++, using the GEN Framework classes and respecting platform insulation. That separation allows changing the rule without recompiling, keeping the technical control in the framework.

18. Examples of use

The examples included in `ScriptsExample` cover several scenarios. `test.g`, `test.js` and `test.lua` show the same idea implemented in three languages: a max function, random number generation, console output, trace and key reading. This shows that the Scripts block tries to offer a homogeneous surface even if the underlying language changes.

`test1.js` demonstrates the most complex data exchange. Defines a JavaScript object and calls `DevTest_Func1`, an exposed function from C++ using `SCRIPT_LIB_DEVTEST`. The goal is to validate that not only integers or simple strings can be passed, but also data structures that the JavaScript engine represents as objects.

`inputsimul.js` is the most advanced example. Use `Log_Ini` and `Log_AddEntry` to log activity, `ExecApplication` to launch WordPad, `Window_SetPosition` and `Window_Resize` to manipulate the window, `Window_GetPosX` and `Window_GetPosY` to locate a reference image, `InpSim_Mouse_Click` to simulate the mouse, `InpSim_Key_ClickByLiteral` or `InpSim_Key_ClickByText` for keyboard and `TerateAplicationWithWindow` to close the application. It is a very clear example of external automation controlled from a script.

19. Relationship to other modules

Scripts is a cross-sectional block. It is supported by `XUtilities` for strings, routes, variants, maps, vectors, events, threads, timers and tracers. It is supported by `AppFlow` for configuration, script list, life cycle and logs. It can relate to `DataIO` if a library itself exposes communications or file access. It can relate to `Graphic`, `Input` and `UserInterface` if scripts are created to automate screens, simulate input or adapt visual behaviors. It can also work with `Cipher`, `Compress` or `Databases` if the application decides to expose secure and well-defined functions.

This transversality is precisely its value. The script becomes an external decision layer, while GEN Framework maintains native services. The application decides which part of the framework appears visible for the script. This prevents each script from having to reinvent paths, logs, traces, settings or timers.

20. Criteria for use

- Use scripts for changing logic, external rules, automations, tests, auxiliary tasks or lightweight plugins.
- Keep in C/C++ the critical core of performance, memory management, direct access to hardware and logic that requires maximum control.
- Show the script only the necessary functions. A small, clear library is better than a huge surface that is difficult to control.
- Use assets/scripts and XPATHMANAGER to locate files. Do not enter absolute paths in the application code.
- Use AppFlowCFG and the scriptslist section when the application has to run a configurable list of scripts.
- Enable using CMake only the languages and libraries required for the project.
- Manage errors using SCRIPT_XEVENT and traces, especially when scripts can be modified without recompile.

21. Usual errors when using Scripts

- Treat scripts as substitutes for the design of the application. They must complement the C/C++ code, not turn the project into a messy mix.
- Activate all languages and libraries for convenience, even if the project only needs one or two elements.
- Do not control what functions are displayed. A script with access to processes, windows or system can perform high impact actions.
- Save scripts out of assets/scripts and lose XPATHMANAGER homogeneity.
- Forget that the cache can keep an older version of the script during a session.
- Do not validate parameters in C/C++ functions called from script. All callback must check the number, type and validity of its arguments.
- Do not collect or display SCRIPT_XEVENT_TYPE_ERROR, leaving the user without information when a script fails.

22. Summary catalogue of the Scripts block

Group	Classes / modules	Responsibility
Core	SCRIPT, SCRIPT_ERRORCODE, SCRIPT_XEVENT	Load, execute, events, and common script system errors.
Cache	SCRIPT_CACHE	Preload and reuse script text from memory.
Language .g	SCRIPT_LNG_G, SCRIPT_LNG_G_COMMAND, SCRIPT_LNG_G_FUNCTIONTYPE, SCRIPT_LNG_G_VAR	Own engine for .g files, with tokens, commands, functions and variables.
Lua	SCRIPT_LNG_LUA	Lua run and callbacks log to GEN Frameworkfunctions.
JavaScript	SCRIPT_LNG_JAVASCRIPT	Running using Duktape and converting arguments/returns to XVARIANT.
Libraries	SCRIPT_LIB, SCRIPT_LIB_FUNCTION	Basis for exposing C/C++ functions to any script language.
Standard libraries	Math, Random, Timer, String, Path, Dir, Trace	Common functions available for scripts.
Optional libraries	Console, CFG, Log, System, Process, Window, InputSimulate, DevTest	Integration with console, configuration, logs, system, processes, windows and input simulation.

23. Practical summary

The Script block makes GEN Framework a more flexible platform. It allows you to load external algorithms into JavaScript, Lua or .g, run them from a C/C++ application, register your own functions and modify uncompiled

behaviors. This fits very well with GEN Framework philosophy: maintain native code performance and control, but reduce repetitive work and facilitate real application adaptation.

The SCRIPT class provides the common base. SCRIPT_LNG_G, SCRIPT_LNG_LUA and SCRIPT_LNG_JAVASCRIPT implement specific languages. SCRIPT_LIB and SCRIPT_LIB_FUNCTION allow you to expose C/C++ functions to scripts. SCRIPT_CACHE improves repeated load. SCRIPT_XEVENT allows you to diagnose errors. ScriptsExample demonstrates full integration with AppFlow, XPATHMANAGER, configuration, cache, console, traces and optional libraries.

The most important idea is that the script should not be seen as an improvised addition, but as a controlled extension of the application. When designed well, it allows you to change algorithms, add tasks and create plugins in real time, keeping the main application compiled, robust and aligned with the architecture of GEN Framework.

Chapter XIV: UserInterface, widgets and multiplatform interface design

UserInterface is the GEN Framework block that converts the graphical and input capabilities of the framework into an organized user interface. Its aim is not just to draw controls, but to maintain an element tree, solve positions and sizes, apply a common appearance, receive mouse input, keyboard or touch screen, publish events and redraw only the areas that have changed.

The result is an own widget layer, supported by Graphic and Input, which retains the same appearance and behavior on different platforms. Menus, options, forms, text boxes, buttons, virtual keyboards and progress bars are described with the same concepts even if they change the operating system, resolution, or input device.

This chapter reviews all the public classes included in the UserInterface block provided. Classes are grouped by responsibility and not alphabetically, because the most useful way to understand the module is to follow the complete path: configuration, loading, composition, interaction, drawing and updating.

1. UserInterface location within GEN Framework

UserInterface is located on Graphic and Input. Graphic provides the screen, viewport, canvas 2D, bitmaps, fonts and reconstruction areas. Input abstracts mouse, keyboard, touch screen and cursor movement. UserInterface unites both worlds: it transforms input into operations on elements and transforms the state of those elements into drawing.

The application still retains control of the main cycle. In UI_Options, AppFlowGraphics creates the screen and canvas; then the XML layout is loaded, the user moves or selections are sent to the manager and, at the end of each frame, the areas are reconstructed and Update is called. This separation prevents business logic from relying on a native API of windows.

- Graphic provides GRPSCREEN, GRPVIEWPORT and GRP2DCANVAS, as well as images, fonts and drawing operations.
- Input provides INPCURSOR, INPCURSORMOTION and INPBUTTON, regardless of the physical device.
- UserInterface provides layouts, elements, properties, skins, events, resources and widgets.
- AppFlowGraphics coordinates the initialization, updating cycle and closing of the application.

2. General architecture and execution flow

The center of the block is UI_MANAGER, usually displayed using GEN_USERINTERFACE. The manager loads one or more layouts and retains the elements they contain. Each UI_LAYOUT has a UI_SKIN, a background and a UI_ELEMENT tree. Specialized elements add text, images, selection, editing, scroll or progress. The skin calculates geometry and draws; UI_XEVENT communicates input and output actions.

The normal flow of an application can be summarized as follows:

- Record graphics, fonts and layouts paths with XPATHMANAGER.
- Get the screen and viewport created by AppFlowGraphics.
- Load the XML or ZIP package with UI_MANAGER::Load.
- Subscribe to incoming and outgoing events.
- Send movement, selection or gesture using UI_MANAGER::SendEvent.
- Consult or modify widgets by name and type.
- Mark the modified elements with Elements_SetToRedraw.
- Rebuild the required areas and run Update on each frame.

3. Load an interface and minimum cycle

UI_Options contains a complete example of initialization. The layout is located under the UI_LAYOUTS section administered by XPATHMANAGER. Load receives the path, screen and viewport index. Subscriptions are then activated, the virtual keyboard is created, the background is placed and the first redrawn is requested.

```
GRPSCREEN* screen = GetMainScreen();
if(!screen) return false;

XPATH xpath;
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, xpath);
xpath.Slash_Add();
xpath.Add(__L("example/example.xml"));

if(!GEN_USERINTERFACE.Load(xpath, screen, 0)) return false;

GEN_USERINTERFACE.SubscribeInputEvents(true);
GEN_USERINTERFACE.SubscribeOutputEvents(true, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.CreaterVirtualKeyboard(GEN_USERINTERFACE.Layouts_Get(__L("example")), screen);
GEN_USERINTERFACE.Layout_PutBackground();
GEN_USERINTERFACE.Elements_SetToRedraw();
```

The reverse operation is performed at the closing: subscriptions are cancelled and the manager instance is deleted. It is appropriate to do so before destroying the screen or application objects that receive events.

```
GEN_USERINTERFACE.SubscribeOutputEvents(false, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.SubscribeInputEvents(false);
GEN_USERINTERFACE.DelInstance();
```

4. UI_MANAGER and UI_LAYOUT: global management and composition

UI_MANAGER. It is the singleton that coordinates the entire subsystem. Load and LoadLayout read the configuration; it also supports packaged layouts, maintains the layouts collection, searches for elements, manages the modal element, publishes events, resolves automatic texts, manages the animation cache and controls the virtual keyboard. Its Elements_SetToRedraw and Elements_RebuildDrawAreas methods connect the status change with the incremental rendering.

UI_LAYOUT. It represents a logical screen or widget composition. It has identifier, skin, background and collection of root elements. Elements_Add incorporates an element; Elements_Get allows to recover the collection or search by name, type or role of chrome; Update draws the elements that need it. A file can contain several layouts, including the COMMON layout for shared resources.

Search by name and type prevents the application from keeping pointers throughout the session. It also allows the same code to work with different configuration files, provided they keep the identifiers agreed.

```
UI_LAYOUT* layout = GEN_USERINTERFACE.Layouts_Get(__L("example"));
if(layout)
{
    UI_ELEMENT* menu = layout->Elements_Get(__L("menu_horz"));
    if(menu)
    {
        menu->SetVisible(!menu->IsVisible());
        GEN_USERINTERFACE.Elements_SetToRedraw(menu, true);
    }
}
```

5. UI_ELEMENT: common basis for all widgets

UI_ELEMENT. It is the root of the visual hierarchy and inherited from XSUBJECT. It retains name, type, role of chrome, parent, color, background, position, level Z, direction, margins, rectangle, visibility, activity, preselection and selection states, blinking, emergence transition and composite elements. GetComposeElements forms the widget tree; SetToRedraw propagates the need to draw again.

An element is not just a rectangle. The layer/compose ratio allows you to build forms, buttons with text and image, menus with options or indicators with a percentage tag. ZLevel sets the order, while IsDetached allows you to separate an element from the normal flow of the parent when the composition requires it.

UI_ELEMENT_CHROMEROLE adds window roles like icon, minimize, maximize and close. UI_Options demonstrates that a selection can be solved by role before servicing the specific name of the widget.

```
switch(element->GetChromeRole())
{
    case UI_ELEMENT_CHROMEROLE_MINIMIZE : GetMainScreen()->Minimize(true); break;
    case UI_ELEMENT_CHROMEROLE_MAXIMIZE : GetMainScreen()->Maximize(true); break;
    case UI_ELEMENT_CHROMEROLE_CLOSE    : SetExitType(APPFLOWBASE_EXITTYPE_BY_USER); break;
    default                             : break;
}
```

6. Geometry, boundaries and alignment of coordinates

UI_BOUNDARYLINE. Defines the solved rectangle of an element. It can be copied, expanded around another rectangle, checked if a point remains inside and obtained the left, right, top and bottom edges. It is the piece used for both calculation and hit testing.

UI_CSSBOX and UI_CSSBox_Get/UI_CSSBox_Set. UI_CSSAdapter.h offers an additive bridge between the internal model of GEN Framework and the box model CSS. UI_CSSBOX expresses left, top, width and height with upper left origin. Helpers read or write that box without forcing to change the internal system, whose stored Y value represents the lower edge.

UI_ELEMENT_CONTROLTRANSITION. It is currently an extension point. The class included only implements construction, destruction and cleaning, no state or additional public operations. It should not be presented as a transition system already completed; it serves as a skeleton for future evolution.

The CSS adapter is especially useful if an external layout engine is integrated or you want to import web geometry. The rest of UserInterface continues to use its usual coordinates.

```
UI_CSSBOX box;
box.left   = 24.0;
box.top    = 16.0;
box.width  = 320.0;
box.height = 80.0;

UI_CSSBox_Set(element, box);
UI_CSSBOX resolved = UI_CSSBox_Get(element);
```

7. XML configuration and declarative styles

UI_STYLEPROPERTY. It is a key/value pair used by the style system. It maintains the property as text so that it can come directly from a XML node.

UI_STYLE. Group style properties, allow you to define them, check their existence, retrieve them as text or number and fill them out from XFILEXMLELEMENT. It is the lightweight layer that decomposes the declarative attributes of the particular widget construction.

The UI_Options configuration starts with skin, colors, texts and global animations, and then declares a layout. Each element specifies type, name, position, size and visual properties. The nested elements become composite elements.

```
<layouts>
  <skin drawmode="canvas" vectorfont="Nunito-SemiBold.ttf"
    backgroundimg="seamless_pattern2.jpg" backgroundseamlesspattern="yes" />

  <color name="colormenu">black,50</color>
  <text name="button_text">#[TEST_TEXT]</text>

  <layout name="example">
    <element name="menu-btn" type="button" xpos="5" ypos="704">
      <element name="animation00" type="animation">
        <image>menu_button.png</image>
      </element>
    </element>
  </layout>
</layouts>
```

Separate definition and execution makes it easy to change colors, fonts, positions and unrecompiled resources. The application retains the responsibility of validating the identifiers it uses and reacting to events.

8. Colors, texts, animations and shared backgrounds

UI_COLOR. It represents a RGBA color and its validity. It can be built from a string, modify channels, copy or analyze expressions such as network, black,50 or 160,195,25. Transparency is part of the same object.

UI_COLORS. It is the singleton color record with a name. Add associates an identifier and a color expression; Get recovers the expression. XML from UI_Options uses colormenuline and colormenu to reuse a palette.

UI_TEXT. Stores the name and content of a textual resource.

UI_TEXTS. It is the singleton record of named texts. It allows you to declare a string once and refer it from various elements. The #[...] masks are resolved later by translation or by application event.

UI_ANIMATION. Loads a bitmap sequence from a resource, including a vector resource adapted to a reference canvas. It retains name, resource pattern and frames; GetBitmap allows you to recover a frame.

UI_ANIMATIONS. It is the singleton animation cache. Add, Get and DeleteAll avoid repeatedly loading the same resources. UI_MANAGER::GetOrAddAnimationCache encapsulates the usual pattern.

UI_BACKGROUND. Maintains the color and background resources of a layout: bitmap name, normal bitmap and pattern bitmap. UI_MANAGER can paint color, climbing image or repeated pattern.

These records convert the XML into a reusable system. The logical name remains stable and the specific resource can change between themes, languages or screen densities.

9. Skins and rendering on canvas

UI_SKIN. Defines the appearance contract. Saves name, type, drawing mode and raster/vector fonts. Calculates the limits of each type of widget and dispatches its drawing using Draw_Text, Draw_Form, Draw_Menu, Draw_ProgressBar and other specialized methods.

UI_SKIN_TEXTBOX_PART. It is an internal layout unit of a TextBox. It retains line, position, color, dimensions and the text or image that is drawn in that part.

UI_SKINCANVAS_REBUILDAREAS. Extend GRP2DREBUILDAREAS to associate canvas zones with layouts or elements. You can rebuild everything, rebuild an element, capture/restore bitmaps and locate the area corresponding to a widget.

UI_SKINCANVAS. Implement UI_SKIN over GRP2DCANVAS and add the relative position calculation, parent size, fonts, clipping, scrollbars and drawing of all widgets. It also solves center, auto and max positions and applies scroll margins and scrolls.

UI_SKINCANVAS_FLAT. Specializes UI_SKINCANVAS with a flat appearance. In the supplied block mainly overwrites Draw_Form, so it reuses the rest of pipeline skin from canvas.

The separation between model and skin is the reason why the interface can maintain a common behavior and change appearance. widgets do not draw directly: expose status, and the skin interprets that state in the selected graphic backend.

10. Reusable properties: selection, editing and scroll

UI_PROPERTY_SELECTABLE. Adds a selection status to an item: active, preselected, selected or disabled, along with a timer and the time during which the selection is maintained. SetSelectableStateFromString links directly to XML attributes such as active, preselect, selected or deactivated.

UI_PROPERTY_EDITABLE. Adds cursor position and maximum size to editable content. UI_ELEMENT_EDITTEXT combines this property with text and selection.

UI_PROPERTY_SCROLLEABLE_STATUS. It retains the status of an axis: activity, visibility, auto/forced mode, limit, displacement, step, overflow and appearance of the bar. It is the internal data block for horizontal and vertical.

UI_PROPERTY_SCROLLEABLE. Displays the complete displacement policy. Allows you to activate axes, fix steps or fractions, choose visible overflow/hidden/scroll/auto, calculate limits, display or hide the bar and adjust width, colors and separations. Scroll_ResolvePolicy relates viewport and content extension.

Properties are implemented as reusable bases. Thus, Form, TextBox, Menu and MultiOption can share scroll without duplicating logic; Option, ListBox and EditText share selection; EditText adds editing.

```
UI_PROPERTY_SCROLLEABLE* scroll = dynamic_cast<UI_PROPERTY_SCROLLEABLE*>(element_menu);
if(scroll)
{
    UI_COLOR thumb(__L("white,75"));
    UI_COLOR track(__L("white,50"));

    scroll->Scroll_SetOverflow(UI_PROPERTY_SCROLLEABLE_TYPE_VERTICAL, UI_OVERFLOW_AUTO);
    scroll->Scroll_SetBarWidth(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, 16.0);
    scroll->Scroll_SetBarColors(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, thumb, track);
    scroll->Scroll_SetBarSeparation(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, 5.0, 0.0);
}
```

11. Content Widgets: text, images and animation

UI_ELEMENT_TEXT. Displays a string with font size, effective text, mask and length limit. It can be independent or be part of a button, an option or a progress indicator.

UI_ELEMENT_TEXTBOX. Represents multiline text and inherits scroll capability. Controls interline, image separation, alignment, word cutting, mask and actual content height. Skin divides content into UI_SKIN_TEXTBOX_PART to model it.

UI_ELEMENT_IMAGE. Displays a GRPBITMAP with alpha level. The resource can come from raster formats or an image created from a vector file.

UI_ELEMENT_ANIMATION. Play frames from a UI_ANIMATION. It offers Play, Pause and Stop, timing, image index and alpha. It is the visible widget; UI_ANIMATION is the shared resource containing bitmaps.

UI_ELEMENT_SCROLL. It is the basic visual element associated with displacement. Politics and state reside in UI_PROPERTY_SCROLLLEABLE, while skin is responsible for drawing and testing bars.

UI_Options combines these components: TextBox mixes text and an animation; buttons contain an animation and a text; the percentage of a bar is another UI_ELEMENT_TEXT whose mask is automatically resolved.

12. Selection Widgets: Option, Button and CheckBox

UI_ELEMENT_OPTION. It is the basis of selectable controls. It can contain text and animation, decide where the text is placed and limit which subelements are visible according to active, preselect, select or deactive status.

UI_ELEMENT_BUTTON. It is a semantic specialization of Option. It takes advantage of the composition of text and image and publishes a selection when the user activates it.

UI_ELEMENT_CHECKBOX. Adds a boolean value to Option using GetStatus and SetStatus. Its visual representation continues in the skin's hands.

The option is more important than it seems: it concentrates the visual state machine. A button can display different colors, text or animations for rest, preselection, selection and deactivation without the application having to manually redraw each variant.

```
UI_ELEMENT_BUTTON* button = (UI_ELEMENT_BUTTON*)
    GEN_USERINTERFACE.Element_Get(__L("menu-btn"), UI_ELEMENT_TYPE_BUTTON);

if(button)
{
    button->SetActive(true);
    button->SetVisible(true);
    GEN_USERINTERFACE.Elements_SetToRedraw(button, true);
}
```

13. MultiOption and State Representation

UI_ELEMENT_MULTIOPTION. It groups several options that represent the same control in different states. Hereda scroll and selection and retains each variant in its collection of composite elements.

The XML of UI_Options declares chromeall-btn as multioption and nests four buttons with the same logical name, one for each selectable status. Each variant changes the position or color of the text and animation. The skin chooses the representation corresponding to the current state.

```
<element name="chromeall-btn" type="multioption" xpos="80" ypos="20">
  <element name="chrome-btn" type="button" selectablestatus="active">...</element>
  <element name="chrome-btn" type="button" selectablestatus="preselect">...</element>
  <element name="chrome-btn" type="button" selectablestatus="selected">...</element>
  <element name="chrome-btn" type="button" selectablestatus="deactive">...</element>
</element>
```

This pattern allows for a very rich appearance without conditional drawing in the application. It is also the base that the virtual keyboard uses to build keys with different visual states.

14. Containers, forms, menus and lists

UI_ELEMENT_FORM. It is a container with line color, visible rectangle and scroll capacity. It groups other elements and can act as a panel, dialogue, form or navigation band.

UI_ELEMENT_MENU_OPTION_CFG. It is the configuration structure used to create menu options at runtime. It contains index, name, legend, color, font, margins and dimensions.

UI_ELEMENT_MENU. Specialize Form and add Option_AddText. It can be declared empty in XML and populated from C++, as does UI_Options with eight options.

UI_ELEMENT_LISTBOX. Combines text and selection with an associated UI_ELEMENT_MENU. The text shows the current choice and the menu offers alternatives.

A dynamic menu is useful when options come from configuration, detected devices or external data. After adding elements, the tree limits are recalculated and the visible area is adjusted.

```
UI_ELEMENT_MENU* menu = (UI_ELEMENT_MENU*)
    GEN_USERINTERFACE.Element_Get(__L("ListBoxMenuID"), UI_ELEMENT_TYPE_MENU);

if(menu)
{
    UI_ELEMENT_MENU_OPTION_CFG cfg;
    cfg.index          = 1;
    cfg.nameoption     = __L("option01");
    cfg.leyend         = __L("Option 01");
    cfg.colorstr       = __L("white,255");
    cfg.sizefont       = 18;
    cfg.marginwidth    = 10;
    cfg.marginheight   = 15;
    cfg.width          = 250;
    cfg.height         = 35;

    UI_ELEMENT_OPTION* option = menu->Option_AddText(cfg);
    if(option) option->SetVisibleLimitType(UI_ELEMENT_OPTION_VISIBLE_LIMIT_PRESELECT |
                                           UI_ELEMENT_OPTION_VISIBLE_LIMIT_SELECT);

    GEN_USERINTERFACE.Element_GetLayout(menu)->GetSkin()->
        CalculeBoundaryLine_AllElements(menu, true);
    GEN_USERINTERFACE.Elements_SetToRedraw(menu, true);
}
```

15. Text editing and virtual keyboard

UI_ELEMENT_EDITTEXT. Combines UI_ELEMENT_TEXT, UI_PROPERTY_SELECTABLE and UI_PROPERTY_EDITABLE. Adds visible cursor, blink timer and cursor status change. The maxsizetext attribute limits input.

UI_VIRTUALKEYBOARD_KEYINFO. Describes the geometry and appearance of a key: color, font size, margin, position, width and height. It is used internally during keyboard construction.

UI_VIRTUALKEYBOARD. Creates a virtual keyboard inside a layout. Ini links it to layout and screen; Show presents it for an editable element; SelectInput interprets the key; End releases the elements. Manages uppercase, lowercase and symbols and modifies the text respecting cursor position and maximum size.

UI_MANAGER::CreaterVirtualKeyboard builds the keyboard associated with layout. When an EditText is selected, the manager can display it with that item as a target. By closing, **DeleteVirtualKeyboard** eliminates infrastructure.

```
UI_LAYOUT* layout = GEN_USERINTERFACE.Layouts_Get(__L("example"));
GEN_USERINTERFACE.CreaterVirtualKeyboard(layout, GetMainScreen());

UI_ELEMENT_EDITTEXT* edit = (UI_ELEMENT_EDITTEXT*)
    GEN_USERINTERFACE.Element_Get(__L("ID Edit"), UI_ELEMENT_TYPE_EDITTEXT);

// The manager displays the keyboard when the EditText becomes the editable target.
if(edit) GEN_USERINTERFACE.Elements_SetToRedraw(edit, true);
```

In the XML of **UI_Options**, the EditText is inside a Form. This demonstrates the normal case: a form composes the field, while the keyboard is an additional layer that is shown only when needed.

16. Linear, radial and image indicators of progress

UI_ELEMENT_PROGRESSBAR. It is a horizontal or vertical linear bar. It maintains level, line colors /fondo/relleno, gradient, rounded ends and progress rectangle. **ContinuousCycle** activates an animated segment for tasks of unknown duration.

UI_ELEMENT_PROGRESS_RADIAL. Represents the level as arc. Controls initial angle, sweep, thickness, rounded ends, line, color and gradient. A 360 degree sweep produces a full ring.

UI_ELEMENT_PROGRESS_IMAGE. Reveals a full image over another empty image according to the level. **OffsetStart** and **OffsetEnd** delimit the useful part of the resource; it is suitable for thermometers, tanks, batteries or shape indicators.

The three inherit from **Option**, so they can incorporate text or animation. The **#[PROGRESSBAR_PERCENT]** mask is resolved from the parent level. **UI_Options** simultaneously increases or decreases multiple bars with **F1** and **F2**.

```
UI_ELEMENT_PROGRESSBAR* progress = (UI_ELEMENT_PROGRESSBAR*)
    GEN_USERINTERFACE.Element_Get(__L("progressbar0"), UI_ELEMENT_TYPE_PROGRESSBAR);

if(progress)
{
    float level = progress->GetLevel();
    progress->SetLevel(level + 1.0f);
    GEN_USERINTERFACE.Elements_SetToRedraw(progress);
}

<element type="progressradial" name="progressbar1"
    xpos="50" ypos="475" width="150" height="150"
    level="15" thickness="18" roundcap="yes"
    startangle="-90" sweepangle="360"
    bckgrdcolor="32,40,48" color="160,195,255"
    gradientcolor="20,80,240" gradientmode="track">
    <element type="text" xpos="center" ypos="center" sizefont="35">
        #[PROGRESSBAR_PERCENT]
    </element>
</element>
```

17. Entry/exit events

UI_XEVENT. Extends **XEVENT** to carry position, cursor movement, element, skin and text mask values. Input types are movement, path and selection; output types report selection, text change request, and content change.

The application translates the physical device to UserInterface events. UI_Options reverses the Y axis of the cursor when necessary and sends movement or selection to the manager. Thus, the UI does not need to know if the point comes from mouse or touchscreen.

```
GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_CURSOR_MOVE, cursor_x, cursor_y);

if(button_mouse->IsPressedWithRelease())
{
    GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECTION, cursor_x, cursor_y);
}
```

The output events return to the application using the observe/subject pattern. The selected event delivers the UI_ELEMENT; TEXTTOCHANGE allows you to solve your own masks such as VERSION or TEST_TEXT; CHANGECONTENTS reports edits.

```
void MYAPP::HandleEvent_UserInterface(UI_XEVENT* event)
{
    switch(event->GetEventType())
    {
        case UI_XEVENT_TYPE_OUTPUT_SELECTED:
            if(event->GetElement()) UserInterface_ElementSelected(event->GetElement());
            break;

        case UI_XEVENT_TYPE_OUTPUT_TEXTTOCHANGE:
            UserInterface_ChangeLiteralText((UI_ELEMENT_TEXT*)event->GetElement(),
                                             event->GetMaskTextValue(),
                                             event->GetMaskTextResolved());

            break;

        case UI_XEVENT_TYPE_OUTPUT_CHANGECONTENTS:
            break;
    }
}
```

18. Automatic texts, translation and dynamic data

The #[...] masks allow a layout to contain data that you do not know when loading. UI_MANAGER tries to resolve resources and translations and, when the application must provide the value, issues UI_XEVENT_TYPE_OUTPUT_TEXTTOCHANGE. The observer writes the result in GetMaskTextResolved.

```
if(!maskvalue->Compare(__L("VERSION"), true))
{
    maskresolved->Format(__L("%d.%d.%d"),
                        APPLICATION_VERSION,
                        APPLICATION_SUBVERSION,
                        APPLICATION_SUBVERSIONERR);
}

if(!maskvalue->Compare(__L("TEST_TEXT"), true))
{
    maskresolved->Set(__L("Tests"));
}
```

This mechanism avoids matching the XML with the application's internal variables. It is useful for network status, date, user, version, counters or any data that changes during execution.

19. Interfaces adaptable to different screens and displays

UserInterface does not require all coordinates to be absolute. UI_SKINCANVAS interprets relative positions as center, auto or max sizes, margins and parent dimensions. Nested elements are calculated within your container and visible rectangles control clipping and scrolling.

The responsible configuration should be based on three levels:

- Relative geometry: use center, auto, max, margins and parent elements instead of repeating global coordinates.
- Alternative layouts: load different files or layouts according to resolution, orientation, density or device family.
- Appropriate resources and sources: keep logical names stable and change images, fonts or sizes by configuration.
- Overflow and scroll: declare which content is cut, moved or visible when the viewport is smaller.
- CSS adaptor: integrate, if necessary, an engine that works with top left source boxes without changing the core.

The example already uses xpos="center" and ypos="center" in internal texts, shapes like containers and a visiblirect for a horizontal bar. For a real application it is advisable to avoid that the complete layout depends on a single resolution and prepare variants for the main formats.

```
<element name="dialog" type="form"
    xpos="center" ypos="center" width="max" height="auto"
    margin="24,24,24,24" overflow="auto">
  <element name="title" type="text"
    xpos="center" ypos="20" width="auto" height="auto">
    #[DIALOG_TITLE]
  </element>
</element>
```

The exact names of attributes must be contrasted with the version of UI_MANAGER that is compiled. The design principle is stable: express relative intent, resolve it in the skin and select the appropriate configuration before drawing.

20. Update, reconstruction and performance areas

Changing a widget does not imply redrawing the entire screen. SetLevel, SetText or SetVisible change the model; then Elements_SetToRedraw marks the element and, optionally, its children. Elements_RebuildDrawAreas prepares the affected areas and Update delegates the drawing to the skin.

```
GEN_USERINTERFACE.Elements_SetToRedraw(element, true);
GEN_USERINTERFACE.Elements_RebuildDrawAreas();
GEN_USERINTERFACE.Update();
```

UI_Options runs that pattern at the end of the frame. It also rebuilds the entire area of the canvas when the application needs it, but the incremental path must be the usual one. A frequent error is to change a property and forget to mark the item, which makes the internal state correct but the screen retains the image above.

21. Integrated example based on UI_Options

The delivered example combines almost all the concepts of the block. AppFlowGraphics creates the screen; XPATHMANAGER records graphics, fonts and ui_layouts; GEN_USERINTERFACE loads example.xml; the XML declares resources, background, text, TextBox, Menu, progress bars, Form, EditText, MultiOption, Button and Image.

During initialization options are added to the menu from C++, their geometry is recalculated and scroll bars are customized. During entry, mouse and touch screen generate the same events. F1 and F2 modify indicators; arrows move the menu; F5 recharges the interface. During drawing the main elements are marked, areas are reconstructed and Update is called.

The menu-btn selection shows or hides menu_horz. The selections from a list can copy the chosen text to the ListBox, hide the menu and remove the modal. Text masks are solved in the observer. This example is especially valuable because it shows the correct border: XML for composition and appearance; C++ for dynamic data, navigation and application logic.

22. UserInterface block complete catalog

The revised block is organized as follows:

- Core and coordination: UI_MANAGER, UI_LAYOUT, UI_ELEMENT and UI_XEVENT.
- Geometry and compatibility: UI_BOUNDARYLINE, UI_CSSBOX, UI_CSSBox_Get, UI_CSSBox_Set and UI_ELEMENT_CONTROLTRANSITION.
- Settings and resources: UI_STYLEPROPERTY, UI_STYLE, UI_COLOR, UI_COLORS, UI_TEXT, UI_TEXTS, UI_ANIMATION, UI_ANIMATIONS and UI_BACKGROUND.
- Appearance and drawing: UI_SKIN_TEXTBOX_PART, UI_SKIN, UI_SKINCANVAS_REBUILDAREAS, UI_SKINCANVAS and UI_SKINCANVAS_FLAT.
- Properties: UI_PROPERTY_SELECTABLE, UI_PROPERTY_EDITABLE, UI_PROPERTY_SCROLLEABLE_STATUS and UI_PROPERTY_SCROLLEABLE.
- Visual content: UI_ELEMENT_SCROLL, UI_ELEMENT_TEXT, UI_ELEMENT_TEXTBOX, UI_ELEMENT_IMAGE and UI_ELEMENT_ANIMATION.
- Selection and action: UI_ELEMENT_OPTION, UI_ELEMENT_MULTIOPTION, UI_ELEMENT_BUTTON and UI_ELEMENT_CHECKBOX.
- Containers and lists: UI_ELEMENT_FORM, UI_ELEMENT_MENU_OPTION_CFG, UI_ELEMENT_MENU and UI_ELEMENT_LISTBOX.
- Edition: UI_ELEMENT_EDITTEXT, UI_VIRTUALKEYBOARD_KEYINFO and UI_VIRTUALKEYBOARD.
- Progress: UI_ELEMENT_PROGRESSBAR, UI_ELEMENT_PROGRESS_RADIAL and UI_ELEMENT_PROGRESS_IMAGE.

Not all these classes should be used directly by an application. UI_SKIN_TEXTBOX_PART, UI_PROPERTY_SCROLLEABLE_STATUS and UI_VIRTUALKEYBOARD_KEYINFO are mainly auxiliary parts; UI_MANAGER, UI_LAYOUT and widgets are the usual surface. UI_ELEMENT_CONTROLTRANSITION should be treated as incomplete infrastructure in the revised version.

23. Usual errors when working with UserInterface

- Find an item with an incorrect type and convert the pointer without checking the result.
- Change the status of a widget without calling Elements_SetToRedraw.
- Send cursor coordinates with a Y-axis incompatible with the canvas.
- Store pointers to items after reloading the layout with F5 or other mechanism.
- Create hierarchies without establishing layer, level Z or composite elements in a coherent way.
- Confused UI_ANIMATION, which is the frames resource, with UI_ELEMENT_ANIMATION, which is the player widget.
- Use absolute positions for the entire screen and expect automatic adaptation to any resolution.
- Force scroll without calculating viewport, content extension and overflow policy.
- Subscribe to events and do not unsubscribe during closing.

- Suppose UI_ELEMENT_CONTROLTRANSITION already implements complete transition animations.

24. Design criteria for an application with GEN Framework

- Define stable element names and treat them as a contract between XML and C++.
- Reserve the XML for structure, style and resources; maintain business logic in the application.
- Build complex controls by composition before creating new widget classes.
- Reusing colors, texts and animations through your global records.
- Choose a common skin to ensure the same appearance on all platforms.
- Prepare alternative layouts for screen families and use relative alignment within each.
- Translate all input devices to UI_XEVENT into a single point.
- Update only the modified areas and measure the cost of funds, transparencys and animations.
- Always keep a keyboard or focus path, not just pointer interaction.

25. Practical summary

UserInterface converts Graphic and Input into a complete system of interfaces. UI_MANAGER loads and coordinates; UI_LAYOUT organizes; UI_ELEMENT forms the tree; properties add selection, editing and scroll; widgets represent content and controls; UI_SKIN calculates and draws; UI_XEVENT communicates interaction; virtual keyboard and indicators cover usual needs for touch, embed and desktop applications.

The main strength of the block is the independence between appearance, structure and logic. The same application can retain C++ behavior, change the layout or skin by configuration and run on different systems and displays. The adaptation does not arise from blindly scaling a fixed screen, but from combining hierarchy, relative positions, layout variants, adequate resources and overflow policies.

UI_Options demonstrates the recommended pattern: declarative load, secure access by name and type, unified events, model modification, redraw marking and incremental update. From that base menus, dialogues, forms, keyboards, progress bars and controls can be built while maintaining the visual identity of GEN Framework between platforms.

Chapter XV: ThirdPartyLibraries, grow on open foundations

Building large software requires building on work already tested. A framework that intended to implement typography, compression, codecs, databases, network capture, script engines, audio and toolchains complete would consume your resources in repeating solved problems. Using third-party libraries is not a lack: it is a way to concentrate your own effort where GEN Framework brings value, in common architecture, factories and multiplatform abstraction.

GEN Framework applies, however, a demanding criterion. Whenever it is feasible to integrate dependency as source code, it compiles it together with the project and keeps behind its own classes the boundary with that library. This makes it easier to change compiler or platform, avoids relying on global installations and allows adapting compilation options. When dependency is a driver, a manufacturer SDK or a dynamic library, integration is deliberately limited to the platform layer.

The selection is not only based on functionality. Portability, maturity, size, maintainability, ability to audit the code and license are also important. "Open source" does not mean "without obligations": GPL, LGPL, BSD, MIT, public domain and specific licenses allow different uses and require different notices or conditions.

1. Selection and integration criteria

- Proven functionality: the library solves a complex problem better than improvised implementation.
- Portability: projects in C/C++ are favoured that can be compiled in various architectures and systems.
- Controlled integration: GEN Framework exposes its own API and confines external calls to the appropriate module or platform.
- Source code when possible: facilitates reproducibility, debugging, compiler configuration and cross builds.
- Optional Dependency: An application should only load or link the library if it activates the corresponding function.
- Compatible license: the legal text is retained and redistribution, link, modifications and attributions are reviewed.
- Fixed version: the ThirdPartyLibraries tree acts as a reproducible photograph; updating requires testing and a new audit.

2. Scope of this review

The review has been done on the content of ThirdPartyLibraries available for this chapter. The audited versions come from macros, VERSION files, source.properties, headers or licenses of the included copies. The general README is used as guidance, but not as authority when it contradicts the library code itself.

The term "audited version" describes the copy included in ThirdPartyLibraries, not the latest version available on the Internet. Unlabeled snapshots retain that name. The inventory is independent of a specific version of GEN Framework: documents the purpose of each component within the framework and the conditions necessary to integrate it.

3. General inventory

The following table summarizes the dependencies present or documented. It then explains why each one exists and how it fits into the architecture.

Component	Audited version	License	Primary use in GEN Framework	Integration
-----------	-----------------	---------	------------------------------	-------------

Component	Audited version	License	Primary use in GEN Framework	Integration
AGG	2.5	GPL-2.0-or-later	Graphic vector rasterizing and canvas 2D.	Source
Duktape	2.7.0	MIT	JavaScript engine embedded from the Script module.	Source
FreeType	2.12.1	FTL or GPL-2.0	Font loading and rasterizing for Graphic/AGG.	Source
GLEW	1.13.0	BSD modified + notices	Load OpenGL extensions, especially in Windows.	Source
InpOut32/64	1.5.0.1	Permissive type MIT	Access from Windows to I/O ports by DLL and driver.	Binary/driver
IJG JPEG	9e	IJG	Reading and writing JPEG in Graphic.	Source
libpcap	1.10.1	BSD-3-Clause	Package capture in Linux and Android.	Source
Npcap SDK	1.15	Npcap License	Package capture in Windows.	SDK/binary
libpng	1.6.40	PNG Reference Library 2	Reading and writing PNG; depends on zlib.	Source
Lua	5.3.4	MIT	Lua engine embedded from the Script module.	Source
MySQL Connector/C	6.1.11	GPL-2.0 + FOSS Exception	MySQL connector of the Databases module.	Source/binary
nameof	0.10.2	MIT	Type names/enum in compilation; optional utility available.	Header-only
OpenAL Soft	1.23.1	LGPL-2.0-or-later	Backend for Sound playback and spatialization.	Source/binary
OpenGL Registry	Snapshot	MIT per header/warnings	Extension statements; GEN Framework uses glext.h.	Headers
PostgreSQL/libpq	10.6	PostgreSQL License	PostgreSQL connector of the Databases module.	Source/binary
rpi_ws281x	Snapshot (test 1.1.0)	BSD-3-Clause; RP1 GPL-2.0+ module	LED WS2812/SK6812 control in Raspberry Pi.	Source/platform
SQLite	3.23.1	Public domain	Base SQL embedded, without server, in Databases.	Amalgamation
StackWalker	v5	License not included	Calling and diagnostic batteries in Windows.	Source
stb_vorbis	1.14	MIT or public domain	Ogg Vorbis decoding in Sound.	Single source
WinToast	1.3.0	MIT	Toast notifications of Windows 8/10/11.	Source
zlib	1.2.12	zlib	Deflate, GZip, ZIP and libpng dependency.	Source
Android NDK	r26b (26.1.10909125)	Apache-2.0 + notices	Toolchain Android and android_native_app_glue.	SDK
GoogleTest	1.12.1 and 1.10.0	BSD-3-Clause	Unit tests activated with GOOGLETEST_ACTIVE.	Source
ESP-IDF	5.2.0	Apache-2.0 + component licenses	SDK and HAL for ESP32 platforms.	SDK

4. Graphics, images and typography

Graphic combines several specialized libraries. The advantage of this composition is that GEN Framework retains its own API and can replace internal implementation without spreading an external dependency throughout the application.

Anti-Grain Geometry (AGG)

Audited version: 2.5

Use and motive: Graphic uses its primitives, scanners, path converters, colors and source engines for the canvas 2D. It was chosen by its high quality C++ rasterizer, independent of a specific window and suitable for generating pixels in memory.

Integration and license: The copy includes GPL v2 and the README declares GPL v2 or later; when linking or distributing a derivative work, the copyleft effect should be reviewed.

FreeType

Audited version: 2.12.1

Use and motive: It renders TrueType/OpenType font glyphs and feeds the `agg_font_freetype` adapter. It prevents font management from relying on an exclusive API of Windows, Linux or Android.

Integration and license: Source code. Dual License: FreeType Project License (permissive with attribution) or GPL v2, at the dealer's choice.

GLEW

Audited version: 1.13.0

Use and motive: **Initializes and displays OpenGL extensions at runtime.** InGEN Framework it appears in the graphical context of Windows using `glewInit`, which avoids manually maintaining function pointers by extension.

Integration and license: Source code/heads. Your LICENSE brings together a modified BSD license and additional notices from Mesa and Khronos; all applicable notices must be kept.

OpenGL Registry

Audited version: Snapshot without semantic version

Use and motive: **Provides official OpenGL extension statements.** TheGEN Framework inventory indicates that `glxext.h` is used; it is a header dependency, not a rendering engine.

Integration and license: The ventorized folder contains only GL and does not incorporate a root license. Khronos' modern headers include MIT identifiers or other proprietary notices; the license must be verified file to file when updating.

libpng

Audited version: 1.6.40

Use and motive: It implements PNG in Graphic, with support of the reference format and compression using `zlib`. It is used because PNG requires more than reading a header: filters, chunks, color, transparency, CRC and historical compatibility.

Integration and license: Source code under PNG Reference Library License version 2, permissive. Warnings must be kept and altered versions marked.

IJG JPEG

Audited version: 9e (16-01-2022)

Use and motive: It provides JPEG encoding and decoding used by `GRPBitmapFileJPG`. It is a mature and portable implementation that prevents DCT rewriting, quantization, Huffman and JPEG variants handling.

Integration and license: Source code under the IJG license, permissive but with its own conditions and notices; it should not be simply called MIT.

5. Sound and multimedia

Sound separates the high-level API from the backend and the file decoders. Thus an application can work with fonts, buffers and playback without docking to the API native to each system.

OpenAL Soft

Audited version: 1.23.1

Use and motive: Act as backend OpenAL multiplatform for playback, streaming, capture and spatial positioning. GEN Framework includes AL/al.h and AL/alc.h in its Windows, Linux and Android factories.

Integration and license: The copy contains code/projects and may require dynamic libraries depending on the platform. It is under LGPL 2 or later; the link form and the possibility of replacing the library are relevant to the distribution.

stb_vorbis

Audited version: 1.14

Use and motive: Decodifies Ogg Vorbis audio from memory in SNDFileOGG. Its implementation in a single file reduces integration complexity and fits well with embedded or small compilations.

Integration and license: It is compiled directly from stb_vorbis.c. The file itself allows you to choose MIT or public domain. 2.31 value of README of ThirdPartyLibraries does not correspond to this copy.

FFmpeg

Audited version: 4.0 in the README; missing from the package

Use and motive: The historical inventory is presented as a multimedia framework for video, codecs, mux/demux and streaming. However, there is no ffmpeg folder in the revised ThirdPartyLibraries content.

Integration and license: It should not be treated as a Vendorized Dependency in this package. If a project activates it externally, the license depends on the configuration: normally LGPL, but it can become GPL by enabling GPL components.

6. C++ script languages and utilities

Embedded engines allow you to modify rules and algorithms without recompiling the entire application. GEN Framework maintains the life cycle and data conversion within Script and leaves the language execution to the library.

Duktape

Audited version: 2.7.0

Use and motive: Run JavaScript embedded with a contained print and a API of the C language.

Script_Language_Javascript includes **duktape.h**, so **Duktape** provides the interpreter while GEN Framework provides integration, objects and application flow.

Integration and license: Source code under MIT, suitable for static integration and cross-platform with copyright notice preservation.

Lua

Audited version: 5.3.4

Use and motive: Run Lua using lua.hpp. It was chosen as a small, mature language designed to embed in C/C++ applications.

Integration and license: Source code under MIT. It is one of the easiest dependencies to compile for different platforms.

nameof

Audited version: 0.10.2

Use and motive: header-only Library for readable names of types, enumerations and members at compilation time. You can improve traces, serialization or debugging tools without manual RTTI.

Integration and license: MIT. The copy is present in ThirdPartyLibraries and is considered an optional utility: it is only incorporated into the compilation of modules or projects that include its header.

7. Compression and container formats

Compression is a good example of why it is not appropriate to reinvent a widely standardized piece. A compatible implementation must respect formats, checksums, streaming, limits and corrupt cases accumulated over decades.

zlib

Audited version: 1.2.12

Use and motive: Implement Deflate and zlib/gzip containers. GEN Framework uses it in CompressGZ, CompressZIP and XFileZIP; it is also dependent on libpng.

Integration and license: It is integrated as a source under the permissive zlib license. Modified versions must be marked and authorship cannot be falsified.

8. SQL databases

Databases presents common interfaces, but each engine maintains its protocol, format and connection cycle. Official client libraries reduce incompatibilities and retain the specific knowledge of each server.

SQLite

Audited version: 3.23.1

Use and motive: SQL base embedded and self-contained. sqlite3.c amalgamation allows you to incorporate local persistence without deploying a separate daemon or server.

Integration and license: Public domain. It is portable and simple to compile, although the included version is old and should be evaluated before exposing it to unreliable entries.

MySQL Connector/C

Audited version: 6.1.11

Use and motive: libmysqlclient implements connection, protocol, results and MySQL types for GEN Framework MySQL_* classes.

Integration and license: The distribution includes code and binaries. GPL v2 with MySQL FOSS License Exception; the exception must be checked against the specific license of the project that links to the connector.

PostgreSQL/libpq

Audited version: 10.6

Use and motive: libpq provides the native client interface used by GEN Framework's PostgreSQL_* classes.

Integration and license: PostgreSQL License, permissive. The general README indicates 11.1, but pg_config.h of the audited copy declares PostgreSQL 10.6.

9. Network capture and specific functions of Windows

These dependencies encapsulate capabilities that depend on the operating system or even on a driver. GEN Framework keeps them behind platform classes so that the rest of the code does not know their details.

libpcap

Audited version: 1.10.1

Use and motive: Package capture and injection interface used by DIOPCap in Linux and Android. It also reads and writes pcap captures.

Integration and license: Source code under BSD of three clauses. Version is confirmed in VERSION file; 1.8.1 of detail of README is obsolete.

Npcap SDK

Audited version: 1.15

Use and motive: Catch equivalent for Windows, with headers, import libraries and driver/Npcap service installed.

Integration and License: Npcap License: the code is visible, but free editing is not open source software and cannot be redistributed with an unlicensed OEM product. It is a real exception to the declared GEN Framework policy and must be resolved before distributing.

InpOut32/64

Audited version: 1.5.0.1

Use and motive: It allows DIOWNDOWSGPIOPCParallel to open inpout32.dll or inpout64.dll and access hardware ports from user space.

Integration and license: DLL and driver only for Windows. The license.txt grants permissions equivalent to MIT, but the deployment of driver s also requires signature, architecture and compatibility with the version of Windows.

StackWalker

Audited version: v5

Use and motive: Gets DbgHelp call stacks in Windows and facilitates diagnostics when no debugger is connected.

Integration and license: The included folder does not contain LICENSE or license text. The source refers to CodeProject; therefore, the redistribution must be blocked until attaching and verifying the exact license of the sold copy.

WinToast

Audited version: 1.3.0

Use and motive: It integrates modern Windows 8, 10 and 11 toast notifications in DIOWNDOWSNotificationsManager.

Integration and license: Source code under MIT. The version is taken from wintoastlib.h and LICENSE.txt; the values 1.1 and 1.4.0 of the general README do not describe the current copy.

10. Embedded Hardware and SDK Platform

NDK and ESP-IDF are complete toolchains/SDK, not small libraries. Their presence allows to compile platform layers and access native APIs, but also multiplies the size, transitive licenses and maintenance cost.

Android NDK

Audited version: r26b (26.1.10909125)

Use and motive: Provide compiler, sysroot, libraries and android_native_app_glue, which GEN Framework includes in its native activity Android.

Integration and license: SDK with Apache 2.0 main license and numerous ad/component licenses. The actual copy is r26b, not r17 as indicated by the README.

ESP-IDF

Audited version: 5.2.0

Use and motive: Espressif's official environment for compiling GEN Framework over ESP32: HAL, FreeRTOS, drivers, network and tools.

Integration and license: Apache 2.0 at the top level, with components under additional licenses. When redistributing an image or SDK, notices of the components used must be kept.

rpi_ws281x

Audited version: Snapshot; 1.1.0 test tool

Use and motive: It controls WS2812/SK6812 strips from Raspberry Pi and is used by DIOLINUXLedNeoPixelWS2812BRPi.

Integration and license: The kernel contains BSD notices of three clauses. The variant for Raspberry Pi 5 adds a RP1 module marked GPL-2.0-or-later; that part must be treated separately.

11. Unitary tests

A test library is not part of the final product in many configurations, but it is part of the quality of the framework. Keeping it in ThirdPartyLibraries allows you to play the same test infrastructure on different equipment.

GoogleTest

Audited version: 1.12.1 and inherited copy 1.10.0

Use and motive: GEN Framework initializes GoogleTest and executes `RUN_ALL_TESTS` when `GOOGLETEST_ACTIVE` is defined, both in Windows and in Linux.

Integration and license: Source code under BSD of three clauses. Maintaining two copies facilitates old projects, but increases the risk that different tests compile against different versions.

12. Discrepancies identified in the inventory

The revision shows why the version should not be copied only from a table held by hand. These relevant differences appear in the revised content:

- FreeType: Summary indicates 2.12.1 and detail 2.10.1; header confirms 2.12.1.
- libpcap: Summary indicates 1.10.1 and detail 1.8.1; VERSION confirms 1.10.1.
- PostgreSQL/libpq: README indicates 11.1; pg_config.h confirms 10.6.
- WinToast: README mix 1.1 and 1.4.0; header and license confirm 1.3.0.
- zlib: Summary indicates 1.2.12 and detail 1.2.11; zlib.h confirms 1.2.12.
- Android NDK: the README indicates r17; source.properties confirms r26b.
- ESP-IDF: README indicates 5.1; version.cmake confirms 5.2.0.
- stb: README indicates 2.31; the file actually used is stb_vorbis 1.14.
- Npcap: the README mixes 1.13, 1.15 and 1.60. The folder corresponds to the SDK 1.15, but the version of the runtime is determined in the installation.
- FFmpeg 4.0 is documented, but does not exist in the revised package. nameof 0.10.2 does exist and does not appear in the general list.

13. Practical reading of licences

Permissive licenses — MIT, BSD, zlib, PostgreSQL, IJG, PNG Reference Library and FreeType License — often allow for commercial use, modification and static integration, but require the retention of notices and discharges. Public domain, such as SQLite, removes much of these conditions, although it is appropriate to maintain provenance by traceability.

Copyright licenses need more attention. AGG is distributed in this copy under GPL v2 or later; MySQL Connector/C uses GPL v2 with the exception of FOSS; OpenAL Soft uses LGPL. The result depends on whether it is distributed, how it is linked, whether the user can replace the library and license of the rest of the product. Source integration does not eliminate these obligations.

Npcap is the most important finding: its code can be consulted, but the free edition is not open source and its redistribution is restricted. A product that installs or distributes Npcap needs the corresponding OEM modality or a separate installation strategy compatible with its terms. StackWalker requires another documentary correction: the included copy must incorporate its exact license before being distributed.

This chapter describes the technical inventory; it does not replace a legal review of the final product, its linking method or the components that are actually distributed.

14. Recommended policy to maintain ThirdPartyLibraries

- Store by library a VERSION file or manifest with version or revision, URL of source, date and hash of the package.
- Keep LICENSE/NOTICE next to the code and generate a THIRD_PARTY_NOTICES for each distributed product.
- Register if the dependency is compiled statically or dynamically, it is integrated as header-only, as driver or as SDK of build.
- Distinguish present, used, optional, historical and only necessary for tests; they are not equivalent states.
- Automate the comparison between the manifest and the macros version of the headers.
- Run tests on all supported platforms before updating a dependency.
- Check CVE and upstream support, especially for SQLite 3.23.1, PostgreSQL 10.6, GLEW 1.13.0 and other old components.
- Block publication if a license is missing or if a commercial/non-open license contradicts project policy.

15. Practical summary

ThirdPartyLibraries is not an accidental collection of files: it is the specialized knowledge layer on which GEN Framework builds graphics, audio, scripts, compression, databases, network capture, testing and embedded platforms. Integrating many of these libraries into source helps keep builds reproducible and cross-platform, while GEN Framework factories and modules prevent dependency from invading the application.

The quality of this strategy depends on maintaining the inventory. Version, use, form of integration and license must travel together. If one of those data becomes obsolete, the compilation can continue to work and, even so, the project loses reproducibility, security or right of redistribution.

The conclusion is not to avoid libraries of third parties, but to select them with criteria, to isolate them architecturally, to fix their versions, to keep their licenses and to be able to replace them. That discipline allows to build something greater without giving up the technical control of the framework.

16. Sources of the review

- Official ThirdPartyLibraries repository: <https://gitlab.genframework.com/gen-framework/thirdpartylibraries>
- Official website of GEN Framework: <https://www.genframework.com/>
- Npcap License and Guide: <https://npcap.com/guide/>
- OpenGL Khronos Registry: <https://registry.khronos.org/OpenGL/>

Appendix I: C++ Coding Style Guide for GEN Framework

Purpose. This appendix summarizes and adapts the style guide used to write new modules in GEN Framework. Its aim is to make the new code appear written within the framework itself, keeping the same visible rules of structure, names, life cycles, comments, factories, memory and platform isolation.

Non-negotiable principle. When a modern standard C++ convention conflicts with the way you type code in GEN Framework, you must follow the style of GEN Framework. The goal is not to apply an external guide, but to respect the coherence of the existing repository.

Overview of the GEN Framework

A new module should maintain an appearance consistent with the rest of the framework. The way of writing is not a superficial detail: in GEN Framework, the visual structure of the code helps to quickly recognize the subsystem, the life cycle, the owner of the resources and the boundary between common code and platform-specific code.

- A .h header file and a .cpp implementation file by main class.
- GEN Framework Banner at the start of all source files.
- Use of #pragma eleven in headers.
- Visible section separators: INCLUDES, DEFINES & ENUMS, CLASS, CLASS MEMBERS, etc.
- Class names in capital letters, usually with module prefix: X, DIO, APPFLOW, GRP, UI, CIPHER, SCRIPT.
- Preferential use of GEN Frameworktypes: XBYTE, XWORD, XDWORD, XQWORD, XCHAR, XSTRING, XPATH, XBUFFER.
- Lifecycle Methods Ini(), End() and Clean().
- Indentation of two spaces and keys in line own.
- Statements aligned in columns.
- Doxygen GEN Framework blocks in files and functions.
- Use of GEN_NEW when the code must participate in the memory control of the framework.
- Specific platform code confined to Platform/Platforms directories and related factories.

File names and modules

GEN Framework does not require the file name to be fully capitalized. It is usual to use a Pascal type name with subsystem prefix. The important thing is to respect the style of the folder that is being expanded.

Module type	File Example	Expected Class
XUtils	XString.h / XString.cpp	XSTRING
DataIO	DIOProtocol.h / DIOProtocol.cpp	DIOPROTOCOL
AppFlow	APPFlowAlerts.h / APPFlowAlerts.cpp	APPFLOWALERTS
Graphic	GRP2DCanvas.h / GRP2DCanvas.cpp	GRP2DCANVAS
UserInterface	UI_Manager.h / UI_Manager.cpp	UI_MANAGER
Events	APPFlow_XEvent.h	APPFLOW_XEVENT
Windows Platform	XWINDOWS..., DIOWINDOWS..., GRPWINDOWS...	Dialcode WINDOWS
Linux Platform	XLINUX..., DIOLINUX..., GRPLINUX...	Dialcode LINUX

Events use the _XEvent form in the file and _XEVENT in the class. Variants such as Event, Events, Xevent or xevent should not be entered if the module belongs to the usual framework style.

```

APPFlow_XEvent.h      -> APPFLOW_XEVENT
DIOCamera_XEvent.h   -> DIOCAMERA_XEVENT
APPFlowUpdate_XEvent.h -> APPFLOWUPDATE_XEVENT

```

Header structure

The header defines the public interface of the module. It must be stable, readable and consistent with the rest of the subsystem. It is recommended to use advanced statements whenever possible to reduce unnecessary dependencies.

```

/**-----
-----
 *
 * @file      MYModule.h
 *
 * @class     MYMODULE
 * @brief     My Module class
 * @ingroup   MYGROUP
 *
 * @copyright EndoraSoft. All rights reserved.
 *
 * -----*/

#pragma once

/*---- INCLUDES -----
-----*/

#include "XBase.h"
#include "XString.h"

/*---- DEFINES & ENUMS -----
-----*/

#define MYMODULE_DEFAULT_VALUE      10

enum MYMODULE_STATUS
{
    MYMODULE_STATUS_UNKNOWN          = 0 ,
    MYMODULE_STATUS_ACTIVE           ,
    MYMODULE_STATUS_ERROR            ,
};

/*---- CLASS -----
-----*/

class XBUFFER;
class XTIMER;

class MYMODULE
{
public:
    virtual                MYMODULE                ();
    virtual                ~MYMODULE                ();

    bool                   Ini                        ();
    bool                   End                        ();

    XSTRING*               GetName                    ();
    void                   SetName                    (XCHAR* name);

private:
    void                   Clean                      ();

    XSTRING                name;
    bool                   isactive;
};

/*---- INLINE FUNCTIONS + PROTOTYPES -----
-----*/

```

Key points of the header:

- The initial banner is part of the framework style and should not be omitted.
- After the banner is placed #pragma eleven.
- Section separators are long and very visible.
- Classes are declared after DEFINES & ENUMS and advanced statements.
- Even if there are no inline functions, the INLINE FUNCTIONS + PROTOTYPES section is retained.

Structure of a .cpp file

The .cpp file contains the implementation and follows a very marked structure. In modules that participate in the framework GEN_Defines.h is included at the beginning and GEN_Control.h near the end of the included area.

```

/*---- PRECOMPILE INCLUDES -----
-----*/

#include "GEN_Defines.h"

/*---- INCLUDES -----
-----*/

#include "MYModule.h"

#include "XBuffer.h"
#include "XTrace.h"

/*---- PRECOMPILE INCLUDES -----
-----*/

#include "GEN_Control.h"

/*---- GENERAL VARIABLE -----
-----*/

MYMODULE* MYMODULE::instance = NULL;

/*---- CLASS MEMBERS -----
-----*/

```

- The .cpp includes GEN_Defines.h first when applicable.
- Then the header of the module itself is included.
- The implementation units are added below.
- GEN_Control.h is placed after normal inclusions.
- The static variables of singleton are placed in GENERAL VARIABLE.
- The functions are implemented in the same order as they are declared in the header.

Section Separators

The separators are one of the most recognizable visual brands of the GEN Framework style. They should not be replaced by generic comments or styles of other projects.

```

/*---- PRECOMPILE INCLUDES -----
-----*/

```

```

/*---- INCLUDES -----
-----*/
/*---- DEFINES & ENUMS -----
-----*/
/*---- CLASS -----
-----*/
/*---- GENERAL VARIABLE -----
-----
/*---- CLASS MEMBERS -----
-----*/
/*---- INLINE FUNCTIONS + PROTOTYPES -----
-----*/

```

Indentation, keys and spacing

GEN Framework uses an indentation of two spaces, not tabs. Function keys and multiline control blocks are placed in their own line.

```

bool MYMODULE::Ini()
{
    Clean();

    if(isactive)
    {
        value = 1;
    }

    return true;
}

```

Compact early outputs are common to validate parameters or simple states:

```

if(!cfg)    return false;
if(!stream) return false;

```

The alignment of statements is part of the visual style. It should not be removed with external automatic formatters.

```

class MYMODULE
{
public:

    bool                Ini                ();
    bool                End                ();

    XSTRING*            GetName            ();
    void                SetName            (XCHAR* name);

private:

    void                Clean              ();

    XSTRING              name;
    bool                isactive;
};

```

Conventions of names

Element	GEN Framework	Example
Classes	Shifts and module prefix	XSTRING, DIOSTREAM, APPFLOWCFG, GRP2DCANVAS
Methods	PascalCase and GEN Framework	Ini(), End(), Clean(), GetName(), SetName()
Variable members	Compact miniscules	isactive, description, applicationdata
Definitions	Shifts with prefix	DIOPROTOCOL_TIMEOUT, APPFLOW_ALERTS_WEBALERTCMD

Enums	Type and values in capital letters with prefix	MYMODULE_STATUS_UNKNOWN
Events	Suffix _XEVENT	APPFLOW_XEVENT

The names must carry the prefix of the subsystem. This allows you to quickly identify to which part of the framework a class or function belongs.

Area	Standard prefixes
XUtils	X, XFILE, XSTRING, XBUFFER, XTIMER
DataIO	DIO, DIOSTREAM, DIOPROTOCOL
AppFlow	APPFLOW, APPFLOWCFG, APPFLOWALERTS
Graphic	GRP, GRP2DCANVAS, GRPBITMAP
UserInterface	UI, UI_MANAGER, UI_ELEMENT
Cipher	CIPHER, HASH, CIPHERAES
Script	SCRIPT, SCRIPT_LIB
Input	INP, INPDEVICE
Sound	SND, SOUND
Platform Windows	XWINDOWS, DIOWINDOWS, GRPWINDOWS
Platform Linux	XLINUX, DIOLINUX, GRPLINUX
Microcontrollers	XSTM32, XESP32, DIOSTM32, DIOESP32

Life cycle of a class

GEN Framework classes usually follow a very recognizable cycle: builder, destroyer, Ini(), End() and Clean(). This pattern not only sorts the code; it also sets when resources are initialized, when they are released, and when internal members are reset.

```

MYMODULE::MYMODULE()
{
    Clean();
}

MYMODULE::~MYMODULE()
{
    End();

    Clean();
}

bool MYMODULE::Ini()
{
    if(!isinitialized) return true;

    isinitialized = true;

    return true;
}

bool MYMODULE::End()
{
    if(!isinitialized) return false;

```

```

    isinitialized = false;

    return true;
}

```

```

void MYMODULE::Clean()
{
    isinitialized = false;
    type          = MYMODULE_TYPE_UNKNOWN;
    stream        = NULL;
}

```

- Clean() should not reserve resources; only members should be left in known values.
- Ini() returns bool when initialization may fail.
- End() releases resources and leaves the class in a state of completion.
- The destroyer calls End() when there are resources to release.
- The released pointers are always placed at NULL.

singleton pattern in GEN Framework

When a class is used as singleton, GEN Framework maintains a classic pattern based on static pointer, GetInstance(), DelInstance() and private copy prevention.

```

#define MYMODULE_MANAGER          MYMODULEMANAGER::GetInstance()

class MYMODULEMANAGER
{
public:

    static bool          GetIsInstanced          ();
    static MYMODULEMANAGER& GetInstance          ();
    static bool          DelInstance             ();

    bool                Ini                     ();
    bool                End                     ();

private:

                                MYMODULEMANAGER          ();
                                MYMODULEMANAGER          (MYMODULEMANAGER const&); // Don't

implement
    virtual              ~MYMODULEMANAGER          ();

    void                  operator =              (MYMODULEMANAGER const&); // Don't
implement

    void                  Clean                    ();

    static MYMODULEMANAGER* instance;
};
MYMODULEMANAGER* MYMODULEMANAGER::instance = NULL;

MYMODULEMANAGER& MYMODULEMANAGER::GetInstance()
{
    if(!instance) instance = GEN_NEW MYMODULEMANAGER();

    return (*instance);
}

bool MYMODULEMANAGER::DelInstance()
{
    if(instance)
    {
        delete instance;
        instance = NULL;

        return true;
    }
}

```

```

    }
    return false;
}

```

Documentation Doxygen

GEN Framework documents files and functions with Doxygen blocks of its own format. In new modules, all functions implemented in the .cpp should have its block, including getters and setters if the module follows that pattern.

```

/**-----
 *
 * @fn      bool MYMODULE::Ini(XCHAR* name, XDWORD type)
 * @brief   Ini
 * @ingroup MYGROUP
 *
 * @param[in] name :
 * @param[in] type :
 *
 * @return   bool : true if successful.
 * -----*/
bool MYMODULE::Ini(XCHAR* name, XDWORD type)
{
    if(!name) return false;

    this->name.Set(name);
    this->type = type;

    return true;
}

```

Field	Use
@file	Name of the current file.
@class	Main class documented.
@brief	Short description with GEN Framework vocabulary.
@ingroup	Doxygen group of the subsystem.
@param[in/out]	Parameters with two points.
@return	Return value for functions not void.

Public Types and API

The public API of a GEN Framework module must use frame types. Standard C++ types can appear at borders with external libraries, but should not become the visible style of GEN Framework modules.

Use	Avoid in public API
XBYTE	uint8_t, unsigned char
XWORD	uint16_t, unsigned short
XDWORD	uint32_t, unsigned int
XQWORD	uint64_t, unsigned long long
XCHAR	Direct char/wchar_t
XSTRING	std::string
XBUFFER	std::vector<unsigned char>

XVECTOR	std::vector in public headers
---------	-------------------------------

Memory and ownership of objects

GEN Framework uses an explicit property style. When saving memory with framework objects, GEN_NEW is used if the module is involved in memory control. When releasing, the pointer must be placed at NULL.

```
object = GEN_NEW MYOBJECT();

if(object)
{
    delete object;
    object = NULL;
}
```

When an object has been created by a factory, it should be removed by the relevant factory:

```
if(timer)
{
    GEN_XFACTORY.DeleteTimer(timer);
    timer = NULL;
}
```

- Do not enter std::unique_ptr or std::make_unique into modules that follow the classic GEN Framework pattern.
- Do not use exceptions for normal error flow.
- Prefer bool or result enums to indicate failure.
- If the property of a pointer is optional, use members and access methods IsOwn... / SetIsOwn....

Including rules

Headers should include the minimum necessary. If a member is stored by value, its header is included. If used only as a pointer or reference, forward statement is preferred.

```
#include "XString.h"

class DIOSTREAM;

class MYMODULE
{
private:
    XSTRING          name;
    DIOSTREAM*       stream;
};
```

Control flow

Early exits are frequent and help reduce nesting. switch s usually align case s and use upper case enum values.

```
if(!buffer) return false;
if(!size)   return false;

switch(type)
{
    case MYMODULE_TYPE_A : result = DoA();
                          break;

    case MYMODULE_TYPE_B : result = DoB();
                          break;

    default :             break;
}
```

Platform and factory code

The platform-specific code must be confined to platform directories and hidden behind factories or abstraction classes. This rule is essential for an application with GEN Framework to be compiled on multiple systems without filling the application code with platform `#ifdef`.

- Use existing platform symbols: `WINDOWS`, `LINUX`, `ANDROID`, `MICROCONTROLLER`.
- Keep specific implementations under `Platforms/<PlatformName>/` when possible.
- Create platform-dependent objects using framework factories.
- Add `Create...` and `Delete...` methods symmetrically when a factory is expanded.
- Do not expose native details of Windows, Linux or Android in common APIs unless unavoidable.

```
#if defined(WINDOWS)
...
#endif

#if defined(LINUX) || defined(ANDROID)
...
#endif
```

Traces and diagnosis

Reusable modules should not enter `std::cout`, `printf` or improvised traces. Framework utilities should be used, respecting existing channels and macros.

```
GEN_XLOG
XTRACE_PRINTCOLOR(...)
```

Styles to Avoid

A GEN Framework module should not look like a modern generic C++ code if the neighbouring subsystem does not use that style. Local consistency is more important than an isolated modernization.

```
namespace gen { ... }           // Not the visible style of GEN
class MyModule { ... };         // Incorrect case for GEN classes
std::string name;               // Use XSTRING in public API
std::vector<Item> items;        // Use GEN containers in public API
nullptr // The classic GEN code uses NULL
auto value = ...;               // Reduces visible clarity of types
std::unique_ptr<T> ptr;         // Property not consistent with Clean/End
throw std::runtime_error(...);  // GEN uses bool/results enums
enum class Status // GEN uses class ic enum in these modules
if(condition) {                 // Key in position no GEN
    ... // Misidentification
}
```

Minimum module template

The following template summarizes the recommended way to start a new module not singleton.

```
class MYMODULE
{
public:
    MYMODULE()();
    virtual ~MYMODULE()();

    bool Ini()();
    bool End()();
    bool Reset()();

    bool IsInitialized()();
    void SetIsInitialized(bool isinitialized);
```

```

XSTRING*
bool
private:
    void
    bool
    XSTRING
};
MYMODULE::MYMODULE()
{
    Clean();
}

MYMODULE::~MYMODULE()
{
    End();
    Clean();
}

bool MYMODULE::Ini()
{
    if(isinitialized) return true;

    isinitialized = true;

    return true;
}

bool MYMODULE::End()
{
    if(!isinitialized) return false;

    isinitialized = false;

    return true;
}

void MYMODULE::Clean()
{
    isinitialized = false;
    name.Empty();
}

```

Checklist for new modules

Area	Check
Files	There are .h and .cpp with names consistent with the subsystem.
Banner	The GEN Framework banner appears at the beginning of the .h and the .cpp.
Header	Use #pragma eleven and GEN Frameworksection separators.
Includes	GEN_Defines.h goes first in .cpp and GEN_Control.h at the end of included.
Names	Classes, enums, definition and prefixes follow the subsystem.
Format	Two spaces, own online keys and column alignment.
Life cycle	Builder calls Clean(); destroyer calls End() and Clean().
Memory	GEN_NEW when applicable; released pointers placed at NULL.

API	Types of GEN Framework in public interfaces.
Doxygen	Documented implemented functions with GEN Frameworkblock.
Platform	Native code confined to platform/factories when possible.
Consistency	The module appears to be written next to the neighboring files.

Final style rule

When an existing GEN Frameworkfolder is extended, the most important rule is local consistency. Before creating a new module you should open two or three close .h/.cpp pairs and copy its visible structure: banner, separators, indentation, method order, Doxygen, group names, including, singleton pattern, events or factory if applied.

A correct GEN Framework module must appear written at the same time as the files surrounding it. C++ external guides are secondary to the style set by the repository.

Appendix II: Utilities — eXtended Trace Monitor

eXtended Trace Monitor is the utility of GEN Framework intended to receive, organize and visualize traces emitted by one or more applications. It works as a central observation point: applications act as producers and the monitor listens to configured destinations, identifies the origin and presents the information without forcing to stop or locally debug each process.

The utility is open source, included in GEN Framework and can be compiled along with the rest of the framework. It is also distributed compiled to facilitate immediate use during field development, testing, integration and diagnosis.

Its utility is especially appreciated when an application runs on another machine, on a service without interface, on a embedded system or on a device connected by network or serial port. Instead of consulting each computer separately, the developer can concentrate the traces in a single window.

What a trace is and what problem it solves

A trace is a diagnostic message issued during execution. It describes what the program is doing: initialization of modules, state changes, communications, notices, errors, relevant data or closure of services. Its main objective is to observe behavior in real time.

GEN Framework allows you to accompany each trace with information that helps classify it: application, version, identifier, network source, sequence, date and time, level and color. eXtended Trace Monitor uses that data to separate machines and applications and to apply filters without modifying the program you are sending.

The monitor is not just a text console. The XTRACE protocol also includes cleaning commands and status messages, so the same connection can maintain a panel of important variables in addition to the chronological flow of messages.

Traces and logs are not the same

In GEN Framework—a trace and a log are different mechanisms although they may contain similar messages. The trace is oriented to immediate observation and can be sent to several destinations. Log is oriented towards persistence: XLOGBASE and XLOG manage a file, levels INFO/WARNING/ERROR, sections, filters, limits, reduction, backup and emptying of pending entries.

The connection between the two systems is deliberate. XLOGBASE::Ini receives the isdumpintrace parameter, the default value of which is true. When enabled, the log entries are also issued as traces and appear in eXtended Trace Monitor with their level and section tag. The log file continues to exist; the monitor receives a copy for live observation.

```
XPATH xpathlog;
// Build xpathlog within the logs section of the application.

GEN_XLOG.Ini(xpathlog, __L("Mini Web Server"), true); // true: also send as trace
GEN_XLOG.SetFilters(__L("Ini,Status,End"), XLOGLEVEL_ALL);

GEN_XLOG.AddEntry(XLOGLEVEL_INFO, __L("Ini"), false,
    __L("Initializing Web Server ... : Ok."));
```

This difference avoids confusion: saving traces from the monitor creates a capture of the received; configuring XLOG creates the application's own persistent record, with its size, backup and filtering policies.

GEN Framework

The middle class is XTRACE. It maintains the destinations, size limit, application metadata, source addresses and message sequence. XTRACE_TARGET represents each destination and retains its type, target and number of shipments. In normal compilations several targets can be configured to send the same message on more than one track.

The general types defined by XTRACE are SPECIAL, FILE and NET. SPECIAL delegates to platform implementation; FILE writes in a file and NET packages the trace to be sent by network. XWINDOWSTRACE, XLINUXTRACE, XANDROIDTRACE, XESP32TRACE and XSTM32TRACE adapt the special output to each environment.

In Windows, the special destination searches for a window by its title and uses WM_COPYDATA to deliver the package to eXtended Trace Monitor. If the window is not available, the implementation can turn to OutputDebugString in Microsoft compilations. The network output uses DataIO streams and XTRACE package format; DIOXTRACEINTERSTREAMS allows processing traces that cross streams and optionally relate them to the log system.

- Producer: application with GEN Framework that configures XTRACE and issues messages.
- Target: specific destination, for example, a local window, a file or a host and a network port.
- Transport: UDP/network, special platform mechanism, pipeline or local channel, UART/series or other stream enabled by compilation.
- Receiver: eXtended Trace Monitor, which listens, decodes and groups packages.
- Presentation: tree of origins, status variables, list of traces and filtering/capture controls.

Basic configuration from an application with GEN Framework

The application must be identified before issuing. The name, version and ID allow different processes to be distinguished even if they share a machine or address. Then one or more targets is configured.

```
XTRACE_SETAPPLICATIONNAME(__L("Mini Web Server"));
XTRACE_SETAPPLICATIONVERSION(0, 1, 0);
XTRACE_SETAPPLICATIONID(__L("MINIWEB-LOCAL-01"));

// eXtended Trace Monitor local window in Windows.
XTRACE_SETTARGET(0, XTRACE_TYPE_SPECIAL, __L("eXtended Trace Monitor"));

// Remote monitor listening in UDP/red.
```

```

XTRACE_SETTARGET(1, XTRACE_TYPE_NET, __L("192.168.1.50:10001"));

// Additional copy in file.
XTRACE_SETTARGET(2, XTRACE_TYPE_FILE, __L("miniweb.trace"));

XTRACE_SETSIZELIMIT(64 * 1024);
XTRACE_RESOLVEALLRESOURCES;

```

Common_DefaultXTraceCFG includes predefined network destinations and a local listener on port 10001. In a network of its own it is preferable to explicitly configure the monitor address or the name DNS administered by the organization.

XTRACE_TARGET::GetNSendings lets you know how many messages have been sent by a target. The limit configured with XTRACE_SETSIZELIMIT controls the maximum size that supports a trace and prevents a diagnostic message from growing without control.

Emission of messages, colors and data blocks

The macros XTRACE_PRINT, XTRACE_PRINTLEVEL and XTRACE_PRINTCOLOR cover the usual cases. The level carries presentation and classification indicators. GEN Framework defines the colors black, blue, green, purple, red and grey; it also offers alias INFO, WARNING and ERROR.

```

XTRACE_PRINT(__L("Server started on %d port"), port);
XTRACE_PRINTCOLOR(XTRACE_COLOR_INFO, __L("Settings loaded correctly"));
XTRACE_PRINTCOLOR(XTRACE_COLOR_WARNING, __L("Retry %d connection"), retry);
XTRACE_PRINTCOLOR(XTRACE_COLOR_ERROR, __L("Could not open the socket"));

```

To inspect binary buffers, XTRACE_PRINTDATABLOCK and its variants are used by level or color. The receiver can display offset and text rendering, which is useful with protocols, files, serial plots and network packages.

```

XBUFFER packet;
// Fill in packet...

XTRACE_PRINTDATABLOCKCOLOR(XTRACE_COLOR_BLUE, packet, 0, 16, true, true);

```

The application should avoid excessive frequency. Traces within high-speed loops can alter times, saturate a transport or displace relevant information. It is best to emit in state changes, errors, operation milestones and regulated samples.

Main Window Reading

The screenshot below shows a real session with Mini Web Server and a STM32 device. The interface is organized to simultaneously observe recipient origin, status, messages and configuration.

eXtended Trace Monitor

All

- # Local (no internet)
 - Home [88.14.211.100]
 - UART STM32 F0

Name	Value
Version APP	Mini Web Server 0.1.0 (Beta)
Version S.O.	Microsoft Windows 11 Pro Bui

```

[30/07/2026 18:16:39.808] Mini Web Server 0.1.0
[30/07/2026 18:16:39.811] Mini Web Server 0.1.0
[30/07/2026 18:16:39.812] Mini Web Server 0.1.0
[30/07/2026 18:16:39.812] Mini Web Server 0.1.0
[30/07/2026 18:16:39.862] Mini Web Server 0.1.0
[30/07/2026 18:16:40.130] Mini Web Server 0.1.0
[30/07/2026 18:16:40.131] Mini Web Server 0.1.0
[30/07/2026 18:16:40.375] Mini Web Server 0.1.0
[30/07/2026 18:16:40.436] Mini Web Server 0.1.0
[30/07/2026 18:16:44.501] Mini Web Server 0.1.0
[30/07/2026 18:16:48.573] Mini Web Server 0.1.0
[30/07/2026 18:17:01.362] Mini Web Server 0.1.0
[30/07/2026 18:17:01.377] Mini Web Server 0.1.0
[30/07/2026 18:17:02.232] Mini Web Server 0.1.0
[30/07/2026 18:17:03.071] Mini Web Server 0.1.0
[30/07/2026 18:17:03.235] Mini Web Server 0.1.0
[30/07/2026 18:17:03.258] Mini Web Server 0.1.0
[30/07/2026 18:17:03.258] Mini Web Server 0.1.0
[30/07/2026 18:17:03.259] Mini Web Server 0.1.0

```

```

Mini Web Server 0.1.0 (Beta) [Debug]
Camino raíz de application: E:/Projects/GEN_Framework/Examples/Console/MiniWebServer/assets/

[Log] INFO Ini -----
[Log] INFO Ini Version S.O.: Microsoft Windows 11 Pro Build(26200) 64 bits
[Log] INFO Ini Memoria total 32580 Kb, libre 18329 Kb (56%).
[Log] INFO Ini Activando sistema de LOG ... : Correcto.
[Log] INFO Ini Activando estado aplicacion ... : Correcto.
[Log] INFO Ini Activando estado Internet ... : Correcto.
[Log] INFO Ini Inicializando Web Server ... : Ok.
[Log] INFO Ini Inicializando WebSocket ... : Ok.
[Log] WARNING Status Key pressed: 0x1B [?]
[Log] INFO General Aplicacion cerrada por el usuario
[Log] INFO End Desactivando servidor web ... : Ok.
[Log] INFO End Desactivando WebSocket ... : Ok.
[Log] INFO End Desactivando estado Internet ... : Correcto.
[Log] INFO End Desactivando estado aplicacion ... : Correcto.
[Log] INFO End Desactivando sistema de LOG ... : Correcto.
[Log] INFO End -----

```

IP Filter
☐ Enable text filter

Level Filter

- ☒ 0 - Black
- ☒ 1 - Blue
- ☒ 2 - Green
- ☒ 3 - Purple
- ☒ 4 - Red
- ☒ 5 - Gray

☒ Msg Status
☒ Add time mark
☒ Text tracing
☐ Show IPs

Clear
Stop
DynDNS
Del Stat.

Save
Minimize
Exit

xtracegen.dyndns.org:10001
GEN Framework
COM2,115200,8,N,1,NONE
UART STM32 F0

UDP
UART

Deactivated filter.
UART STM32 F0 COM2,115200,8,N,1,NONE: Close
Clear status from All.
Clear screen from All.

Figure B.1. eXtended Trace Monitor receiving traces of an application and a STM32 device.

Origin tree. The left panel groups the sources under All. A local source, a remote Home machine identified by IP and a UART STM32 F0 channel appear in the capture. This hierarchy allows you to select everything, a machine or a specific application/device.

State variables. The Name/Value table shows values published by the status messages. Version APP and Version S.O. appear in the example; the values are updated without adding a new line to the main stream.

Trace Panel. The central zone shows date and time, application, version and text. Colors differentiate normal, informative, warning or error messages. [Log] entries make the dump of XLOG visible within the trace channel.

Filters. The lower part allows you to filter by IP, text and levels/colors 0–5. It also controls the display of status messages, time mark, trace text and IPs.

Checks Clear clear view, Stop pause reception, DynDNS manages destination resolution, Del Stat removes states, Save saves capture, Minimize reduces window and Exit ends utility.

Destinations and internal status. Selectors display the host/network port, profile and serial configuration. UDP/UART indicators report enabled channels. The black console records internal operations such as port opening/closing and screen cleaning.

Machine organization and application

XTRACE packages include enough information for the monitor to separate messages. Public and local address allow you to build the network source; the name, version and ID identify the application. The sequence and date help order and diagnose losses or delays.

This organization allows you to have a single monitor open during a distributed test. For example, a graphical interface, a web service, a communications process and a microcontroller can send to the same server. The tree allows you to concentrate on one without stopping the others.

When working without Internet access, the monitor can use the local source. The example explicitly shows Local (not the Internet), which confirms that observation does not depend on an external service to function within the same machine or network.

Filters by IP, text, level, color and type

Filtering is a display operation. It allows to reduce noise without temporarily changing the producer program. The IP filter limits origins; the text filter looks for terms; Level Filter activates or disables colors/levels; Msg Status decides if the status panel is displayed.

- Black/INFO for normal flow and general messages.
- Blue or green for confirmations, correct steps or categories chosen by the application.
- Purple/WARNING for situations that require attention but allow to continue.
- Red/ERROR for failures and uncompleted operations.
- Grey for secondary information when the transmitter version and monitor allow it.

Colours are diagnostic categories, not a mandatory universal policy. Each project should document its use and maintain the same semantics in all its applications.

Filtering on the monitor does not reduce traffic already emitted. To limit cost of CPU, network or series you must reduce generation in the application, disable targets, use compilations without XTRACE_ACTIVE or enter a sampling policy.

Important variables window

XTRACE_PRINTMSGSTATUS publishes a name-identified value. The monitor does not add a chronological input each time: it updates the corresponding row. This makes it a lightweight telemetry panel for variables that is interested in continuously monitoring.

XTRACE_STATUS_MSG supports booleans, integers, chains, XDWORD, float and colors RGB.

XTRACE_STATUS_MSGS keeps the collection received and allows you to add, search or delete states. Each state also retains a timer since its last update, useful to detect values that have stopped arriving.

```
XTRACE_PRINTMSGSTATUS(__L("Version APP"), __L("Mini Web Server 0.1.0 (Beta)"));
XTRACE_PRINTMSGSTATUS(__L("Connected customers"), nclients);
XTRACE_PRINTMSGSTATUS(__L("Internet available"), isinternet);
XTRACE_PRINTMSGSTATUS(__L("Memory usage (KB)"), memory_kb);
XTRACE_PRINTMSGSTATUS(__L("Temperature"), temperature);
```

The panel is suitable for active connections, memory, queues, sensor status, version, operating mode, latency or latest error. It does not replace a historical platform of metrics: it represents the current state and is designed for interactive diagnosis.

```
// Clear the status of the current application or of all sources.
XTRACE_CLEARMSGSTATUS;
// XTRACE_CLEARALLMSGSTATUS;
```

Pause, clean up and save a catch

Stop allows to freeze the reception or presentation to analyze an interval without new lines displacing the information. Clear cleans the monitor screen, and XTRACE_CLEARSCREEN/XTRACE_CLEARALLSCREENS macros allow to request cleaning from the producer according to the desired range.

```
XTRACE_CLEARSCREEN;
XTRACE_PRINTHEADER(0, __L("Comm test start"));
```

Save retains the acquired traces to analyze them or attach them to an incident. Before saving it is advisable to select the correct range in the tree and check the filters: a filtered capture may not contain information that did reach the monitor.

Traces may contain paths, addresses, identifiers, protocol data or information entered by users. The saved file should be treated as potentially sensitive diagnostic material.

Remote transport, UDP, pipeline and UART

The monitor can receive via multiple channels because XTRACE separates the message from the target. In network, NET uses host and port and is supported by DataIO. In Windows, SPECIAL uses window messaging for a local delivery. Depending on the platform and compilation, pipelines or equivalent local channels can be enabled, and targets microcontroller can use UART or streams series.

The capture shows simultaneously network configuration and COM2 at 115200 bauds, 8 bits, one stop bit and no parity, plus UDP and UART indicators. This allows you to observe a PC application and a STM32 from the same tool.

UDP is suitable for diagnosis by its low coupling: the producer does not need to establish a long session. In return, it does not guarantee delivery or order. An isolated absence of trace does not show that the code will not be executed; for audit or persistence logs or other reliable channel should be used.

Control of volume and cost of traces

XTRACE records the number of shipments per target and applies a size limit. These functions help prevent disproportionate messages, but the main strategy must be in the instrumentation design.

- Do not emit on each iteration of a fast loop except during a narrowed test.
- Post status changes and errors, not the same value without variation.
- Use the state window for repetitive variables instead of generating continuous lines.
- Limit binary blocks and display only the required fragment.
- Disable targets that are not being used.
- Separate builds from development and production using XTRACE_ACTIVE and project configuration.

When XTRACE_ACTIVE is not defined, macros become empty operations or neutral values according to the compiler. This allows to keep instrumentation calls in the code without maintaining full cost in all settings.

Recommended use during a diagnostic session

- Start eXtended Trace Monitor and enable the necessary receivers.
- Configure the application target with the correct address, port or local mechanism.
- Assign name, version and ID before first messages.
- Check in the tree that appears the machine and application expected.
- Enable timemark and IP if the problem depends on order or provenance.
- First apply a source filter and then text/level to avoid hiding information prematurely.
- Post stable variables in the status panel and reserve the main panel for events.
- Pause when the bug is played and save the catch with a name that includes date and scenario.
- Also keep XLOG files if a persistent and complete record is required.

Security and deployment

Traces are a development and operation tool, not a business data channel. They should not include passwords, keys, tokens, personal data or full content of sensitive communications. Binary dumps deserve special attention because they can reveal more information than expected.

Interface filters do not protect transport: they only control what is shown. When the monitor receives from other machines, the network must be controlled by segmentation, VPN, firewall rules or a transport channel suitable for the environment. A diagnostic receiver should not be exposed to the Internet without an explicit security architecture.

In production it is appropriate to limit targets, frequency and content, and to define when they are enabled. In embedded devices the bandwidth of UART, the time of CPU and the effect on real-time tasks should also be considered.

Classes and related components

- XTRACE: central manager of destinations, metadata, packaging and issuance.
- XTRACE_TARGET: type, target, network resources and shipping counter.
- XTRACE_STATUS_MSG and XTRACE_STATUS_MSGS: tipped values and status panel collection.
- XWINDOWSTRACE, XLINUXTRACE and XANDROIDTRACE: desktop/mobile platform specializations.
- XESP32TRACE and XSTM32TRACE: Specializations for microcontrollers.
- DIOXTRACEINTERSTREAMS: processing traces through DataIO's streams.

- XLOGBASE and XLOG: persistent record that can double your entries to XTRACE.
- Common_DefaultXTraceCFG: default network destinations and local reference settings.

Practical summary

eXtended Trace Monitor centralizes GEN Framework traces from different applications, machines and devices. It can receive by network, local platform mechanisms, pipeline or series according to compilation; classifies messages by origin; filters by IP, text, level and color; displays status variables; pauses, cleans and saves captures.

XTRACE and XLOG have different functions. XTRACE offers immediate and multiple observation targets; XLOG offers persistence, filters, limits and backups. The `isdumpintrace` option unites both without confusing them: the log retains its file and the monitor receives a real-time copy.

The utility is part of the open ecosystem of GEN Framework and serves as a daily tool for remote development, support and diagnosis. Used with an appropriate volume and security policy, it allows you to understand what happens in a distributed system without relying on the debugger of a single machine.

Annex III: Frequently Asked Questions — Questions & Answers

This annex brings together brief answers to questions that usually appear when you discover, evaluate, or start using GEN Framework. It does not replace the previous chapters: it serves as a reference map to locate an idea, clarify a design decision, or remember where to continue.

The answers describe the overall approach of the framework. A specific function may depend on the enabled module, compiler, platform or available third-party libraries. When there is a difference, the project configuration, examples and source code are the final reference.

1. Fundamental concepts

P1. What exactly is GEN Framework?

GEN Framework is a development architecture in C/C++ to build native, modular and cross-platform applications. It gathers basic types, utilities, communications, graphics, input, sound, encryption, compression, databases, scripts, user interfaces and application services under common conventions. Its value is not only in the number of classes, but in how those classes collaborate and isolate the differences of each platform.

P2. Why should it not be understood as a set of independent libraries?

Because their modules share types, life cycles, factories, configurations, traces, routes and compilation criteria. A basic class can serve a more specialized class and several services can form a complete application. That deliberate relationship between parts allows reasoning about the system as a framework and not as a folder of unrelated utilities.

P3. What does it mean to develop in Lego mode?

It means atomizing functionalities in small classes, with clear responsibility and controlled dependencies. These parts can be tested, replaced and combined to create larger components. GEN Framework is built with that principle and allows it to be applied also in the projects that are developed on it.

P4. Why is C/C++ used?

C/C++ offers compilers for a wide variety of processors, operating systems and embedded environments. It allows you to generate efficient native code, work near the hardware when needed, and at the same time use modern abstractions. The existence of several compilers and toolchains avoids relying on a single company or virtual machine and provides a solid basis for portability.

P5. What does it mean to be designed as programmers for programmers?

It means that its interfaces are born of real development needs. The utilities try to reduce repetitive work without hiding what the programmer needs to control: memory, configuration, routes, communications, states, resources and diagnosis. The goal is to offer comfortable and effective tools, not to replace the technical criteria of the user.

P6. Do classes exist only because they seemed useful on paper?

No. The philosophy of the project is to incorporate functionalities that have been necessary in specific projects. tests help to verify them, but its design also captures the experience of actual compilations, deployments, devices and applications. That practical provenance does not eliminate the need to validate each new use, but avoids building a purely academic architecture.

2. First steps

P7. Where do you get GEN Framework?

The entry point is the GEN Framework website. Its download section provides scripts for Windows and Linux capable of obtaining the repository set. It is also possible to clone each repository using Git. It is only necessary to use the system procedure that will serve as a compilation base.

P8. Why is the project divided into several repositories?

The separation allows you to maintain clear responsibilities and download or update each block independently. The kernel, Common, ThirdPartyLibraries, Examples, Tests, Utilities, Resources and Documents have different rhythms and purposes. Download and build scripts coordinate that organization so that the user does not have to manually reconstruct the relationship between repositories.

P9. What tools are needed to get started?

Git, CMake, a construction system such as Ninja and a C/C++ compiler suitable for the platform are needed as a base. Windows is usually used in Visual Studio with MSVC; Linux is commonly used in GCC or Clang. Visual Studio Code can act as a common editor in both environments, and Docker or WSL2 are useful when the project needs reproducible environments or cross compilation.

P10. What should be compiled first?

You should start with a simple official example and use Common scripts. This only validates the location of the repositories, the compiler, CMake, the third-party libraries and the chosen configuration. When the example compiles and executes, there is already a known reference to compare a project of its own.

P11. Why do examples not always directly define the main function?

In the AppFlow modes, the boot is integrated into the framework infrastructure. The application implements its initialization, updating and closing within a common life cycle. Thus, it can share the same general logic between platforms whose input functions, windows or system services are different.

P12. Where should a project of its own be located?

The recommended organization places the project at the same level as the GEN_FrameWork folder within Projects. This allows you to access the necessary repository paths without mixing your code with the framework code. This separation also facilitates updating GEN Framework and keeping the project in its own repository.

P13. Is it necessary to know all classes before creating an application?

No. It is more effective to learn by layers. First the environment is prepared and an example is compiled; then AppFlow is understood, the configuration and the traces; finally the modules that solve specific needs are incorporated. XUtilities and official examples offer a good initial vocabulary, but it is not necessary to memorize the complete catalog.

3. Architecture, modules and applications

P14. What function do factories perform?

Factories create objects whose implementation can change according to the platform. The application requests a screen, a stream, an input device or a sound resource using a common interface; the factory selects the specific class. This confines the Windows, Linux, Android dependent code or embedded system and prevents it from being dispersed by business logic.

P15. Is it mandatory to use AppFlow?

No. GEN Framework supports base use of classes and services, console applications with AppFlowConsole and graphical applications with AppFlowGraphics. AppFlow is convenient when you want a homogeneous life cycle, integrated configuration, traces, routes and states. A small project can use only the modules you need, provided you properly manage your initialization and closure.

P16. What does XUtils contribute?

XUtils provides the common base: strings, buffers, paths, files, dates, timers, collections, events, state machines, threads, configuration, traces and other general services. Many modules use their types to maintain a coherent interface. Understanding XSTRING, XBUFFER, XPATH, XVECTOR, XEVENT and basic factories facilitates learning the rest of the framework.

P17. When should DataIO be used?

DataIO concentrates communications and data input or output: streams, sockets, serial ports, Bluetooth, USB, web protocols, notifications, hardware buses and external services. It is suitable when the application needs to communicate without being directly linked to a specific operating system or device API.

P18. How do Graphic, Input and UserInterface relate?

Graphic provides screens, canvas, bitmaps, fonts, and rendering. Input abstract buttons, cursors, and interaction devices. UserInterface combines both blocks to build interface layouts, widgets, styles, and events. An application can use Graphic without UserInterface, but a complete interface is usually supported on all three levels.

P19. Are all modules always included in the executable?

No. The construction system allows you to activate features using options and macros. A program without sound, databases or scripts should not load those dependencies. This selection reduces the size, compilation time and maintenance surface, which is especially important in embedded systems.

P20. How is it controlled that a module does not depend on the entire framework?

Dependence must move from basic parts to more specialized services and express itself in the construction configuration. Common interfaces, factories and types of XUtils help to avoid circular references. If a class needs to know internal details of many modules, it is usually appropriate to review its responsibility and divide it.

P21. What role do the configuration and folder assets have?

The configuration separates deployment decisions from compiled logic: ports, paths, targets traces, databases, or script lists can change without rewriting the kernel. The assets folder groups resources that accompany the application, such as images, sounds, layouts, certificates, or auxiliary files. XPATHMANAGER helps to locate them consistently.

4. Platforms, compilation and performance

P22. What does GEN Framework understand by platform?

A platform is the combination of hardware — microcontroller, microprocessor, PC, Raspberry Pi or other device — and operating system or running environment. There may also be a platform without operating system. The target build identifies the specific destination for which the appropriate implementations and tools should be selected.

P23. Multiplatform means that the same binary works everywhere?

No. It means that the same architecture and a very high part of the source code can be compiled for different destinations. Each platform generates its own binary and may require a specific toolchain, libraries or permissions. Portability is to confine those differences, not to pretend that hardware and operating systems are identical.

P24. Does the application code always compile without any change?

The goal is to keep the common logic unchanged, but some capabilities do not exist in all destinations. A screen, a camera, a file system or a specific connection may need alternatives. Portable design detects those differences in configuration and platform layers, preventing them from spreading unnecessarily to the rest of the application.

P25. Which compilers can be used?

GEN Framework is designed to work with several C/C++ compilers. GCC, Clang and MSVC are common, and there are additional environments depending on the platform. Accurate compatibility depends on the target, activated options and external libraries. Testing with more than one compiler helps to discover non-portable assumptions and improves code robustness.

P26. How is support added for a new platform?

First it identifies which common interfaces need the target. Then the specific classes are implemented within the Platform layer and connected to their factories. The application code should continue to use the existing interfaces. Changes in the common kernel are only necessary when the new platform reveals a general capability that was not yet modeled.

P27. Does abstractions reduce performance?

Any abstraction has a potential cost, but the design in C/C++ allows you to keep it visible and measure it. Many decisions are resolved during the compilation or using very thin layers. Actual performance depends on the algorithm, the use of memory, the input and output and the platform; that is why it is appropriate to measure the critical points in the actual target instead of assuming that an abstraction is free or expensive.

P28. Can GEN Framework be used in microcontrollers?

Yes, when there is a platform adaptation and a set of modules is selected according to the available resources. In a microcontroller unnecessary services are usually disabled, buffers are reduced and heavy dependencies are avoided. The same architecture can be shared with larger applications, but the configuration must respect the memory, storage and peripherals of the device.

5. Quality, diagnosis and safety

P29. What is the difference between a trace and a log?

A trace is aimed at observing the immediate behavior of an application and can be sent to different targets, including eXtended Trace Monitor. A log is designed to maintain a persistent record, apply rotation and allow further analysis. They can collaborate, but are not equivalent: the trace favors interaction during diagnosis and log favours persistence.

P30. Where do you start when a compilation fails?

You should play the error with a known configuration, identify the first relevant message and check target, compiler, paths and repositories. Then you will review the enabled options and the availability of ThirdPartyLibraries. Compare with an example that you do compile often quickly reveal if the problem belongs to the environment, a dependency or your own code.

P31. How does memory control help?

Memory control options allow you to record bookings and releases made using the mechanisms provided by the framework. They are useful for locating leaks, double releases or resources that survive the shutdown. They do not replace compiler tools or platform analyzers, but they provide an integrated vision with GEN Frameworkclasses and traces.

P32. Are tests enough to trust a class?

tests check contracts and regressions, but they must be complemented by examples, integration tests and actual platform execution. GEN Framework especially values the functionalities that have already solved specific project needs. However, each application must validate its own limits, loads, devices and failure conditions.

P33. Are all classes safe to use from multiple threads?

Interwire security depends on the responsibility and contract of each class. Containers, callbacks, factories or platform resources may require external synchronization. The programmer must review the documentation and protect the shared status, keeping the critical sections small and avoiding blocking within callbacks uncontrolled.

P34. Does using Cipher automatically make an application safe?

No. Cipher provides hashes, encryption, keys and certificates, but security also depends on protocol, key generation and custody, certificate validation, permissions, updates and error treatment. Existing algorithms and revised designs should be used; having cryptographic primitives does not replace a threat analysis.

P35. Should we keep the traces active in production?

Depending on the product. Operating traces can be very valuable, but they must limit volume, frequency and content. They should never display passwords, keys, tokens or personal data. It is advisable to separate development and production settings, control the enabled targets and document how long records are kept.

6. Third-party libraries, licences and maintenance

P36. Why are third-party libraries needed?

They allow to build larger systems on proven solutions for typography, images, audio, compression, databases, network, scripts or toolchains. Refilling each standard would consume resources and increase risk. The important decision is not to avoid all dependence, but to choose it for its functionality, portability, maintenance, integration and license.

P37. Why do you prefer to integrate the source code?

When license and technology allow, compiling the library next to the project reduces global installations and makes it easy to use the same code with different toolchains. It also allows you to set options, debug integration and play a construction. Manufacturer SDK, drivers or dynamic libraries may require a different strategy and be confined to the relevant platform.

P38. Can an open source library be used without checking anything?

No. Each license establishes conditions of attribution, redistribution, modification or link. The project must keep legal notices, identify the included version and check the compatibility with the form of distribution of the product. Permissive licenses, LGPL, GPL, public domain and specific licenses do not impose the same obligations.

P39. Why is the ThirdPartyLibraries inventory not linked to a specific version of GEN Framework?

Because it documents the audited copy of each dependency, its function, and its license as a reproducible photograph of the ThirdPartyLibraries tree. That information remains useful even if the general version of the framework changes. When updating a dependency, the revision of code, compilation, compatibility, and license should be repeated, without relying on references to specific commitments in the text of the book.

P40. Can one external library be replaced by another?

Yes, if the contract that GEN Framework exposes and validates the behavior on all affected platforms is maintained. The proper layer around the dependency exists precisely to prevent your API from extending through the application. Replacement should review formats, errors, performance, license, compilation options and compatibility with existing data.

P41. How is a third party dependency maintained?

You should register version, source, license, local patches and compilation options. An update should first be tested in isolation and then with the modules and examples that use it. You should also compare symbols, formats, behavior and legal notices. Updating by routine, without a need or clear validation, can introduce more risk than benefit.

P42. How do you contribute code while maintaining the style of the project?

Annex I summarizes the name rules, file structure, including, life cycle, memory, Doxygen, factories and platform code. Before creating a module, it is best to study two or three close classes of the same subsystem. Local consistency is more important than imposing an external style that makes the new code seem foreign.

7. Adoption and criteria for use**P43. Can GEN Framework be incorporated into an existing project?**

Yes, and it is usually best to do so gradually. It can start with types, configuration, traces or an isolated service, create a clear border and expand integration when the team understands life cycles and dependencies. A complete migration only makes sense if it brings sufficient benefits against the risk and cost of rewriting.

P44. Does it make sense to use it if the app only has one platform?

You can have it. Modularity, factories, configuration, traces and reuse bring order even in a single destination. However, the benefit must be compared with the size and expected life of the product. For a disposable utility of few lines, an extensive architecture may not be necessary.

P45. Can a complete graphical interface be built?

Yes. Graphic provides rendering and visual resources; Input provides interaction; UserInterface adds layouts, widgets, styles and events; and AppFlowGraphics coordinates the application cycle. The result can be adapted to different displays and devices, provided the interface design includes resolution, density, input and performance of the target.

P46. Is it possible to combine communications, databases and scripts?

Yes. That combination is one of the objectives of an integrated architecture. DataIO can receive information, Databases store it, Cipher protect it and Scripts apply configurable rules. AppFlow coordinates the service and XUtils provides types, events, routes and diagnostics. The application must keep clear boundaries so that integration does not turn all modules into mutual dependencies.

P47. When does GEN Framework fit especially well?

It fits into native products that must live for years, share code between platforms, communicate with devices or services and maintain control over performance and resources. It is also useful when an organization develops several related applications and wants to reuse an architecture, a diagnostic style and already tested modules.

P48. When may not be the best choice?

It may not be for ephemeral prototypes, projects whose ecosystem is entirely dependent on another high-level platform or equipment with no experience or maintenance capacity in C/C++. The decision should consider requirements, service life, tools, knowledge, licenses and total cost, not only the time needed to display the first screen.

P49. What is a recommended learning path?

First you should read the introduction and prepare the environment; then compile an example and understand AppFlow, settings and traces. The next step is to study XUtilities and the module closest to the problem you want to solve. Finally you can review ThirdPartyLibraries, the style guide and these questions. Learning through a small and executable application is usually more effective than going through the class catalog linearly.

Annex IV: Glossary of terms

This glossary brings together technical concepts, abbreviations, tool names and identifiers that appear throughout the book and may be unfamiliar outside its usual context.

The definitions describe the meaning with which each term is used in this work. They serve as a quick reference, but do not replace the documentation of a API, a library, a license or a specific platform.

1. A-C

AES: Advanced Encryption Standard. Block symmetric encryption algorithm used to protect data with the same key in encryption and decryption.

AGG: Anti-Grain Geometry. 2D rendering library that can act as backend to convert vector geometry into pixels with smooth edges.

Algorithm: A defined sequence of operations that transforms input data into a result. In the book it appears in compression, encryption, graphics, scripts and other areas.

Android: Platform made up of compatible devices and the Android operating system. Requires toolchains, libraries and specific platform classes.

API: Application Programming Interface. Public contract whereby a module offers types, functions or classes without necessarily exposing their internal implementation.

AppFlow: GEN Framework block that organizes the life cycle of an application: initialization, first updates, periodic execution and orderly closure.

AppFlowConsole: AppFlow mode oriented to console programs or services without full graphical interface.

AppFlowGraphics: AppFlow mode that integrates the application cycle with display, rendering, user input and updating a graphical interface.

AppProc_Ini / AppProc_Update: Usual methods of the AppFlow cycle. The first prepares the application and the second attends to its periodic update during execution.

ARM / ARM64: Families of widely spread processor architectures in embedded mobiles, plates and systems. ARM64 designates the 64-bit variant.

Backend: Concrete implementation located behind a common interface. The same graphical, sound or communications service can have different backends depending on the platform.

Database: System that stores and queries structured information. The Databases module abstracts engines such as SQLite, MySQL, MariaDB or PostgreSQL.

Binding: Linking that allows you to use from a language or API the functions offered by another component. It usually adapts types, calls and error management.

Bitmap: Image represented as a matrix of pixels. Its size, color format and depth determine how it is stored and rendered.

Bluetooth: Family of short-range wireless technologies. DataIO treats it as a means of communication dependent on the capabilities of the platform.

Buffer: Memory zone that temporarily retains data while producing, transforming or consuming. It is common in files, network, audio, compression and serialization.

Builder: Embarcadero development environment and compiler mentioned among compatible toolchains. It should not be confused with the design pattern of the same name.

Callback: A function or method that one component registers for another to invoke when an event occurs, terminates an operation, or information arrives.

Canvas: Logical surface of 2D drawing. Group operations to represent lines, shapes, text, bitmaps and other graphic elements.

Layer: Level of an architecture that groups responsibilities and limits dependencies. GEN Framework separates, among others, common, specialized and platform layers.

Checksum: Value calculated from some data to detect accidental changes or damage. It does not necessarily amount to a cryptographic hash.

Cipher: GEN Framework module dedicated to hashes, encryption, keys, certificates and related cryptographic operations.

Numbering: Transformation of readable information into data protected by an algorithm and one or more keys. Their security also depends on the protocol and management of those keys.

Asymmetric encryption: Encryption based on a related key pair: one public and one private. RSA and X25519 appear in the book within this field.

Symmetrical encryption: Encryption in which the same secret key protects and retrieves the information. AES is one of the algorithms cited.

Clang: Compiler for the C++ and C languages based on LLVM. Testing an application with Clang and other toolchains helps you discover non-portable assumptions.

Class: Type that gathers state and behavior. GEN Framework Lego mode design favors small classes, clear responsibilities and composition.

CMake: Build Project Generation System. Read the project settings and produce files suitable for Ninja, Visual Studio or other environments.

CMakeLists.txt: CMake Main Configuration File. Declares source, options, dependencies, definitions and targets construction.

Source code: Text written by the programmer before the compilation. Integrating a library into source code makes it easy to adapt its construction to different platforms.

Native code: Machine code generated for the processor and target system. It runs without relying on a mandatory virtual machine.

Compilation: Process that translates source code and resources into objects, libraries or executables. Includes analysis, code generation and, normally, link.

Cross compilation: Construction carried out on a host platform to produce an executable for another architecture or system.

Builder: Tool that translates a programming language to an executable or intermediate rendering. GCC, Clang, MSVC and Builder are examples cited.

Compress: GEN Framework module that unifies algorithms and compression formats, as well as operations on files and compressed buffers.

Settings: Set of values that modify the behavior without rewriting the main logic. You can reside in INI, XML, JSON files or structures created by the application.

Console: Textual interface based on standard input and output. It is suitable for tools, tests, services and applications without a graphical window.

Container: Structure that groups elements, such as a vector or a list. In Docker, the same term also designates an isolated and reproducible execution environment.

CRC: Cyclic Redundancy Check. Family of effective check codes to detect accidental errors in data blocks, communications or files.

2. D-F

DataIO: GEN Framework module that abstracts input, output and communications: streams, sockets, serial ports, hardware buses, web and other devices.

DEBUG / RELEASE: Construction settings with different targets. DEBUG prioritizes diagnostics and debugging information; RELEASE prioritizes distribution and optimization.

Dependency: Relationship by which one component needs types or services from another. Controlling them avoids circular couplings and makes it easy to select modules.

Docker: Container technology used to prepare reproducible compilation environments with known tools and dependencies.

Public domain: The legal situation in which a work is not protected by or has been expressly dedicated to exclusive rights must be checked for each remedy.

Driver: Component that allows the software to communicate with a particular device or service. It usually belongs to the platform dependent layer.

Doxygen: Tool that generates documentation from source code and structured comments associated with classes, functions and types.

DXF: Drawing Exchange Format. Vector format used to exchange technical drawing and geometry information.

EGL: Interface that connects graphical APIs such as OpenGL ES to the native windows and surfaces system. Manages contexts, buffers and presentation.

Embedded: System integrated into a device with a specific function and normally limited resources. It can work with, with or without an operating system.

Link: Phase combining compiled objects and libraries to solve symbols and produce an executable or a final library.

Event: Notification that something relevant has happened, allowing the producer to decouple information from objects that react to it.

eXtended Trace Monitor: External application that receives, organizes, filters and presents traces issued by applications built with GEN Framework.

Factory (Factory): Object or pattern responsible for creating specific instances behind a common interface. In GEN Framework you can select implementations according to platform and configuration.

Feedback: Information that an operation returns about its status, progress or result. It can be sent through events, traces or module-specific mechanisms.

Firmware: Software closely linked to the hardware of a device. It is usually compiled for a specific architecture and directly controlled peripherals.

FreeType: Open source library used to load and rasterize fonts on graphical systems.

Framework: Consistent infrastructure that defines architecture, life cycles, contracts and extension forms. It is not simply a collection of independent libraries.

3. G-I

GCC: GNU Compiler Collection. Set of multiplatform compilers that includes support for languages C and C++.

GEN_DELETE / GEN_NEW: GEN Framework macros that centralize release and dynamic reserve when memory control mechanisms are activated.

GEN Framework: Framework multiplatform in C/C++ that organizes modules, common classes and platform layers to build native applications of different sizes.

GKF: GEN Framework key file. Format used by Cipher to represent and preserve key information within framework utilities.

GPIO: General-Purpose Input/Output. Configurable digital pins of a microcontroller or plate that can act as inputs or outputs.

GPL: GNU General Public License. Free copyleft software license that imposes specific conditions when redistributing derivative works.

Graphic: GEN Framework module dedicated to screens, canvas, bitmaps, fonts, video, vector graphics and rendering backends.

GZip: Compression format that normally encapsulates a DEFLATE flow and adds header and integrity check.

Handler: Identifier or opaque reference representing a resource managed by another system, such as a window, file or connection.

Hash: Fixed length summary calculated from some data. It serves for testing, identification or cryptography according to the algorithm used.

Thread: Sequence of execution within a process. Sharing memory between threads requires synchronization and clear contracts on concurrent access.

HTTP / HTTPS: Application protocols used on the web. HTTPS adds TLS protection to authenticate and encrypt communication.

I2C: Inter-Integrated Circuit. Two-line synchronous bus series used to connect sensors, memories and nearby peripherals.

IDE: Integrated Development Environment. Application that brings together editing, navigation, compilation, debugging and other development tools.

Including: Preprocessor directive that incorporates statements from a header file. The term is also used to refer to the stated dependency.

INI: Textual configuration format normally organized into key-value sections and pairs. It is simple to read and edit.

Input: GEN Framework module that abstracts buttons, cursors and input devices so that the application does not depend directly on each native API.

Instance: Concrete object created from a class. Several instances of the same type can maintain different states.

Interface: Visible contract that separates what a component offers from how it implements it. It can be expressed by common classes, functions or types.

Graphical interface: Set of visual and interactive elements with which the user works: windows, layouts, controls, text, images and events.

IP: Internet Protocol. Network protocol that identifies destinations through addresses and allows you to route packets between systems.

4. J-M

JavaScript: Built-in scripting language as one of the options in the Scripts module to run logic loaded at runtime.

JPEG: Compressed image format, usually with loss, suitable for photographs and scenes with continuous color variation.

JSON: Textual format of data exchange based on objects, lists and values. It is common in configuration and web services.

Layout: Description of spatial organization of elements of an interface. Defines hierarchy, position, size and rules of adaptation.

LGPL: GNU Lesser General Public License. Limited copyleft license that allows certain library uses under specific link and redistribution conditions.

Permissive license: Open source license that allows wide reuse with obligations normally focused on retaining notices and attribution. MIT and BSD are common examples.

Log: Persistent recording of relevant facts for subsequent audit or analysis. Unlike an interactive trace, it is usually retained and rotated.

Lua: Small and embedded scripting language that can be integrated to run rules or extensions without recompiling the entire application.

LZW: Lempel-Ziv-Welch. Lossless compression algorithm based on a dictionary that expands while processing data.

Macro: Preprocessor-controlled replacement before compiling. GEN Framework uses them for options, platform, diagnostics and some common patterns.

State machine: A model that represents a system through states and transitions. It helps express life cycles and reactive behaviors explicitly.

MariaDB: Relational database engine is largely compatible with the MySQL ecosystem and accessible from the Databases block.

Dynamic memory: Memory requested during execution whose life cycle controls the program. It must be released in an orderly manner to avoid leaks.

Module: Functional framework block with a defined responsibility, its own classes and controlled dependencies.

MSVC: Microsoft Visual C++. Compiler and toolchain main Visual Studio for native applications in Windows.

Cross-platform: Ability to maintain an architecture and much of the code for several destinations, generating on each platform its binary and corresponding adaptations.

Mutex: Object of mutual exclusion that allows a single thread to access a critical section each time.

MySQL: Client-server relational database engine supported by the abstractions of the Databases module.

5. N-R

Namespace: Namespace that groups identifiers and avoids collisions between classes, functions or constants of different sources.

NDK: Native Development Kit. Set of Android tools to compile and integrate native C/C++ code.

Ninja: Lightweight construction system designed to quickly execute the rules generated, for example, by CMake.

Node: Element of a linked or hierarchical structure. In DataIO you can represent parts of external information along with values and handlers.

NULL: Traditional representation of a null pointer. In modern C++ nullptr is usually preferred because it expresses the type better.

Purpose: Instance with identity, status and behavior. Relationships between objects allow to compose greater functionalities.

OGG: Open container format commonly used for compressed audio and other media streams.

OpenAL: API multiplatform positional audio. Sound block can be used as backend playback block.

OpenGL: API graphical multiplatform for accelerated rendering. GEN Framework uses it through layers that isolate context creation and platform.

Open source: Software whose source code is available under a license that allows you to study and reuse it under defined conditions. It does not mean absence of obligations.

Package: Information unit transmitted or distributed. Depending on the context it can be a network block, an installable set or a group of resources.

PATH: Variable or list of paths where the system searches for executables. Path is also used as a general term for a file or directory path.

PEM: Textual format based on Base64 and readable headers used for keys, certificates and other cryptographic structures.

Pipeline: Chain of stages through which data or tasks pass. Each stage receives a result, transforms it and delivers it to the next.

Platform: Combination of hardware and running environment, usually a processor or microcontroller along with your operating system. It may exist without operating system.

Plugin: Loadable component that expands an application without being a mandatory part of its core. It needs a stable integration contract.

PNG: Raster image format without loss, with support for transparency and suitable for interfaces, icons and captures.

Portability: Ease with which a code can be compiled and behave correctly on different platforms with limited and controlled changes.

PostgreSQL: Client-server relational database engine supported through the interfaces of the Databases module.

Protocol: Set of rules, formats and sequences that allow two components to exchange information in a compatible way.

Pointer: Value referenced by a memory or object address. Its use requires control of validity, ownership and life cycle.

Serial port: Communication channel that transmits data sequentially. UART is a frequent technology to deploy it to hardware.

Point of entry (main): Function from which the system starts a traditional program. AppFlow can hide its platform differences and delegate the cycle to common methods.

RAII: Resource Acquisition Is Initiation. C++ technique that links the life of a resource to that of an object to release it automatically when destroyed.

Raspberry Pi: Single-plate computer family, usually with ARM processors, used as an example of a compact Linux platform.

Rendering: Process that converts a scene, geometry, text or visual resources into the final image displayed on screen or stored.

Repository: Versed warehouse of code and resources. GEN Framework is distributed in several repositories with different responsibilities.

Recourse: An element that an application needs and must manage, such as memory, files, images, sounds, connections or system identifiers.

RSA: Asymmetric cryptographic algorithm used for operations with public and private keys, signatures or exchange protected according to the protocol.

Runtime: Services and environment available while a program is running. It can include libraries, chargers, interpreters or system support.

6. S-U

Script: Program or fragment interpreted or compiled at runtime. Allows to incorporate rules and automation without rebuilding the entire application.

SDK: Software Development Kit. Set of headers, libraries, tools and documentation to develop on a platform or product.

Serialization: Conversion of an object or structure into a representation that can be stored or transmitted and subsequently reconstructed.

Server: A component that listens to requests and offers data or services to one or more clients through a defined protocol.

SHA: Secure Hash Algorithm. Family of cryptographic hash functions; the book uses specially variants of SHA-2.

Singleton: Pattern that provides a single accessible instance of a class. It is useful for certain managers, although it introduces global state that needs to be controlled.

Embedded system: Computer system integrated into a product and dedicated to specific functions, often with strict limits of memory, energy or time.

Operating system: **Software that manages processes, memory, devices, files and other resources of a platform.** GEN Framework confines its differences after specific layers.

Socket: Extreme of a network communication identified by protocol, address and port. You can use TCP, UDP or other transports.

Sound: GEN Framework module dedicated to resources, playback, buffers and audio backends.

SPI: Serial Peripheral Interface. Synchronous series bus used to communicate a controller with sensors, memories and other peripherals.

SQL: Structured Query Language. Language used to define, consult and modify data in relational databases.

SQLite: Relational database engine integrated into a file and linked within the application, without separate server.

Stream: Abstraction of an orderly stream of bytes or messages. Allows similar processing of files, sockets, serial ports and other channels.

SVG: Scalable Vector Graphics. Format XML to describe scalable vector graphics using shapes, layouts, text and styles.

Target: Concrete destination of a construction or execution. You can identify an architecture, a platform, a library, an executable or a trace channel.

TCP: Transmission Control Protocol. Connection-oriented transport that delivers an orderly byte flow and controls losses and retransmissions.

ThirdPartyLibraries: **Repository that brings together third-party libraries integrated or referenced by GEN Framework, along with their versions, licenses and adaptations.**

TLS: Transport Layer Security. Protocol that authenticates and numbers communications, used, for example, by HTTPS.

Toolchain: Coordinated set of construction tools: compiler, assembler, linker, libraries, debugger and associated utilities.

Trace: Diagnostic message designed to observe the behavior of an application during its execution. It can include level, color, origin and data blocks.

UDP: User Datagram Protocol. Connected transport that sends datagrams without guaranteeing itself delivery, order or retransmission.

UART: Universal Asynchronous Receiver-Transmitter. Peripheral hardware implementing asynchronous serial communication.

Unicode: Standard that assigns codes to characters from numerous writing systems. You need a specific encoding to store them.

URL: Uniform Resource Locator. Address that identifies the location and method of access to a resource, especially in web services.

USB: Universal Serial Bus. Standard to connect devices, transfer data and, in many cases, supply power.

UserInterface: GEN Framework module that builds layouts, widgets, styles and interface events on Graphic and Input.

UTF-8: Unicode encoding of variable length, compatible with ASCII and widely extended in files, communications and web services.

7. V-Z

Vector: Graphic representation based on geometry, curves and attributes, not on a fixed grid of pixels. It can be scaled without losing geometric definition.

Viewport: Region of a rendering surface in which a scene is projected. Defines the visible area and its transformation to screen coordinates.

WAV: Format audio container that usually stores samples PCM, although it supports other encodings.

Widget: Reusable element of a graphical interface, such as a button, tag, text field or list.

WSL2: Windows Subsystem for Linux 2. Linux environment integrated into Windows using lightweight virtualization, useful for compiling and testing Linux toolchains.

X.509: Digital certificate standard linking an identity to a public key and validity data signed by an authority.

X25519: Key exchange mechanism based on Curve25519. Allows two parties to obtain a shared secret through an unprotected channel.

XBUFFER: XUtils class that manages byte sequences and usual reading, writing, copying and conversion operations.

XBYTE / XWORD / XDWORD: Whole types of controlled size used to clearly express widths of 8, 16 and 32 bits in the framework code.

XEVENT: XUtils event object. Transports the type of notification, its origin and the information needed for observers.

XFACTORY: XUtils common infrastructure for creating and accessing factories that select concrete implementations.

XFILE: Abstraction of XUtils to work with files without dispersing the details of each operating system.

XFSMACHINE: Base class to implement finite state machines, their states, events and transitions.

XLOG: XUtils service aimed at maintaining persistent records and organizing them for further consultation.

XML: Extensible Markup Language. Hierarchical text format based on tags, attributes and content.

XPATH: XUtils class that represents and manipulates file paths and directories. It should not be confused with the XPath language of XML.

XPATHMANAGER: Manager that centralizes root and route sections, such as configuration, resources, graphics, scripts or data.

XSTRING: XUtils string class used for text, conversions, formatting and common operations in framework modules.

XTRACE: XUtils Subsystem that creates and sends traces to one or more targets diagnostics.

XUses: GEN FrameworkBase Module. It provides types, strings, buffers, files, routes, collections, time, events, threads, traces and other transversal utilities.

XVARIANT: Container capable of representing values of different types. It is useful when exchanging generic information, especially with databases.

XVECTOR: XUtils Dynamic Container for maintaining neat collections of elements with an interface common to the rest of the framework.

ZIP: Format compressed file container that allows you to gather multiple files and metadata into a single drive.